Instruction Sets, Programs, and Proofs

Instruction Sets, Programs, and Proofs

Semantics, Equivalence, and Optimization

Michael J. Bommarito II

BOMMARITO PRESS

2026

Instruction Sets, Programs, and Proofs: Semantics, Equivalence, and Optimization

First Edition.

Published by Bommarito Press.

Portions of this book were drafted and edited with the assistance of AI tools. All facts, citations, and final language were reviewed and verified by the author.

Printed in the United States of America.

Preface

> "[T]his arithmetic by 0 and 1 is found to contain the mystery."
> —Gottfried Wilhelm Leibniz, "Explanation of Binary Arithmetic" (Leibniz 1703)

Leibniz found mystery in an exact rule. With only zero and one, he could represent numbers and calculate with them. Three centuries later, the same two marks describe the words stored in a processor, the instructions that transform them, and the proofs that say those transformations are correct. The mystery did not disappear as the rules became precise. Precision gave it somewhere to live.

This book begins with a modest question: what does it take to establish a claim about machine code? The question leads farther than its size suggests. A compiler replaces one instruction sequence with another. A programmer says that a sequence is the shortest possible. An instruction-set manual prints a table found by exhaustive search. Each statement concerns a finite machine, yet words such as *every*, *equivalent*, and *shortest* reach beyond the examples anyone can inspect.

That reach is one source of the subject's beauty. A short program can still denote a function on every possible input state in its finite domain. A finite formula can ask whether any of those inputs makes the program fail. A finite refutation can sometimes rule out every cheaper program in a declared language. Small written objects can carry universal claims about machines. This book studies when they really do.

Testing remains indispensable. It can expose an error in a proposed program. It cannot by itself show that two programs agree on every input, or that no cheaper program exists. For those claims we need mathematics, and we need to know exactly what the machine evidence establishes.

The chapters first build a machine without choosing a commercial instruction set. They develop words, registers, memory, machine state, operand forms, and instructions as functions. This foundation lets us compare features often grouped under the labels reduced instruction set computer (RISC) and complex instruction set computer (CISC): explicit and implicit operands, load-store rules and memory operands, fixed and variable encodings, condition flags, and instruction extensions. RISC-V and x86-64 then become sustained contrasting companions. Optional vector extensions appear only when the scalar model's boundary is itself the subject.

After that foundation, the book turns to equivalence, lower bounds, cost models, proof certificates, and exhaustive search. The deeper subject is the passage from an executable example to a statement that ranges over every allowed case. No particular instruction set owns that passage.

Readers should know how programs and processors work. Experience with a compiler, assembler, or systems code is enough preparation. No prior work with satisfiability solvers, proof assistants, or proof certificates is assumed. The mathematics uses functions, finite sets, elementary algebra, logic, and proof by contradiction. Each piece of notation arrives with the problem that needs it.

The intended reader is willing to stop at a small example and ask what has actually been defined. That habit matters more than prior formal training. A compiler engineer may recognize every instruction in an example yet still need to state which flags a rewrite preserves. A verification student may know the solver vocabulary yet still need to follow an effective address into memory. The book brings those two kinds of care together.

It can be read in three ways. For a first pass, execute the Ao examples by hand, follow the paired RV64 and x86-64 comparisons, and attempt the exercises marked by a concrete verb such as Execute, Break, or Transfer. For a course in semantics, add the definitions and reader proofs, requiring each answer to name its state and observation. For a course or reading group in machine-assisted proof, reproduce the artifact routes and their negative controls. The paths differ in depth, but none should reverse the dependency order: words before states, states before steps, steps before programs, and program meaning before optimization.

Examples use small widths because a reader should be able to inspect every case that carries the explanation. The real architectures retain their real 64-bit widths where the rule depends on them. Moving between those scales is part of the method. A width-four table reveals a carry boundary; algebra says which positive widths share it; a bit-vector route checks a declared machine instance. The text identifies which step supplies which reach.

Figures and colored panels are working parts of the exposition. Definitions fix objects. Key-idea panels state distinctions worth carrying forward. Warnings isolate attractive mistakes. Proof kits expose the shape of an argument before its details. Artifact panels bind a claim to evidence. A reader should be able to cover any panel and recover its role from the surrounding prose; the panel organizes the explanation but never replaces it.

Two rules govern the book.

First, scope is part of every technical claim. A statement about an instruction sequence must name the word width, allowed instructions, operand forms, constants, and cost model that give the statement its meaning. Change one of them and the answer may change. Scope is not fine print around a theorem. It is part of the theorem.

Second, every main theorem has a reader proof and a machine proof. The reader proof is small enough to reconstruct with pencil and paper. It may use an

invariant, a case split, or a replay. The machine proof runs the complete stated instance against a stored artifact. One supplies understanding; the other supplies coverage. A solver output is not an explanation, and an explanation is not a complete check of a large search space.

The evidence is recorded in objects. Each object joins a statement to its scope, artifact, checking route, and evidence status. A negative control sends a nearby false claim or damaged artifact through the same route. The checker must reject it. This modest demand catches a surprisingly empty ceremony: a checker that reports success no matter what it receives.

Different objects justify different kinds of belief. Some rely on a solver verdict. Some carry a certificate that a separate program checks. Others reproduce a finite calculation or enter a proof-assistant kernel. The text keeps these routes distinct. A bounded calculation does not become a theorem for every width, and a solver verdict does not become a kernel proof because both happen to end successfully.

The exercises are part of the development. They ask the reader to reproduce a result, damage an artifact, alter a machine, find a boundary, or carry an argument one step farther. At several points the exercises stop where the present proof library stops. An honest boundary is not a defect in the subject. It is where the next piece of mathematics begins.

The accompanying repository contains the object ledger, evidence artifacts, checkers, and build instructions. The mathematics can be studied without running the software. Running it reveals another part of the beauty: a claim about every input eventually becomes particular bytes, read by a particular program, with a particular way to fail.

The Introduction starts with fixed-width addition and asks which surrounding state gives that operation its meaning. Chapter 1 then constructs one machine word. From that smallest exact object, the book builds state, instructions, memory, programs, two real architectures, and finally the claims we can prove.

Contents

Introduction

Put 15 in a four-bit register and add 1. The register now contains 0. Put the same two values in wider registers and the result may be 16. Ask an operation to keep a carry bit, and another part of the state changes as well. Ask it to store the result, and addresses, byte order, and possible failure enter the story. The written symbol (+) did not change. The machine around it did.

An instruction set makes these choices precise. It says which values exist, where they live, how instruction bytes are decoded, which parts of state an operation reads and writes, how control moves, and what happens when execution cannot continue normally. A processor implements those rules. An assembler gives them a readable spelling. A calling convention adds agreements between separately written programs. These layers meet in ordinary code, but they are not the same layer.

This book takes them apart and puts them back together.

An old ambition with modern consequences

The wish to make calculation mechanical is older than the electronic computer. Binary arithmetic offered a spare alphabet. Stored-program designs made instructions and data available to automatic control. Instruction-set manuals then became contracts between hardware and software: compact enough for programmers to use, detailed enough for implementations to share.

The present problem is not that those contracts have disappeared. It is that our claims about programs now travel through more layers. A compiler rewrites machine code. A superoptimizer searches for a shorter sequence. A binary translator moves work between architectures. A solver reports that no counterexample exists. A proof checker receives a certificate produced by yet another program. At every boundary, a correct calculation can answer the wrong question.

The superoptimizer made that tension explicit. Massalin searched instruction sequences for the smallest implementation of a function in 1987. (Massalin 1987) Later systems made the search broader and more systematic, from peephole superoptimization to parallel hierarchical search. (Bansal and Aiken 2006; Lu and Bodík 2025) The modern question is no longer only whether a machine can discover good

code. It is what the word *good* means, whether the search covered the declared language, and what evidence a skeptical reader can check afterward.

The book's central question

What must be defined, executed, related, and checked before a statement about machine code deserves belief?

This question includes ordinary correctness before it reaches optimization. If two programs are said to agree, we need their state, inputs, outcomes, and observation. If one is said to refine the other, we need the relation between different machine states. If one is said to be minimal, we need the candidate language and cost. The proof machinery comes last because it can only preserve the meaning supplied to it.

A machine small enough to finish

We begin with an abstract instruction set called Ao. It has fixed-width words, eight registers, byte-addressed memory, a program counter, four condition bits, and normal or exceptional outcomes. Its instructions move data, perform integer arithmetic and logic, access memory, compare values, branch, and halt.

Ao is deliberately artificial. No hardware vendor must preserve its past, and no software ecosystem depends on its encodings. We can therefore print its whole relevant state and give every core instruction a complete rule. When an example behaves unexpectedly, there is nowhere for a hidden architectural custom to hide.

The point is not to design an ideal processor. Ao gives us a common language for questions. What is a word? What counts as state? Does an operand name a value, a location, or both? When do two addresses overlap? Does a branch read registers directly or read condition state written earlier? What does it mean for an execution to end?

After answering each question in Ao, we ask it again of two real instruction sets: x86-64 and RISC-V. Their differences then become informative rather than bewildering. We already know the role that needs to be filled. We can inspect how each architecture fills it.

Why invent A0 instead of starting with a real manual?

A real manual must serve compatibility, implementation, and reference needs. Its definitions are distributed across chapters, modes, tables, and exception rules. Ao lets the reader see one complete semantic dependency chain first. The real manuals then become sources we can interrogate with precise questions, not forests in which we hope to recognize familiar mnemonics.

Ao also makes mistakes cheap. We can reverse byte order, omit a condition write, or move a branch target by four bytes and immediately ask which claim breaks. Those mutations become negative controls for the later executable model. A teaching machine should make both the right rule and its nearest wrong neighbors easy to inspect.

Two companions, not two stereotypes

RISC-V and x86-64 are often placed on opposite sides of a reduced-versus- complex instruction-set contrast. The labels are historically useful, but they cannot supply an instruction's meaning. They do not tell us which register changes or how many bytes an encoding occupies. Nor do they say whether arithmetic writes condition flags, which forms access memory, or what a call does before an application binary interface (ABI) gives the stack a role.

The book compares those dimensions directly. A base RISC-V instruction can name a destination register separately from two sources. A selected x86-64 arithmetic form can use its destination as an input and also change condition state. RISC-V commonly expresses memory traffic through load and store instructions. Selected x86-64 arithmetic forms can name memory as an operand. The base encoding structures differ. So do some of the special rules attached to registers and operand widths.

None of these observations declares a winner. Each one creates a semantic obligation. We have to record the right state, decode the right instruction form, and compare the right outcomes. The labels RISC and CISC can summarize a family resemblance after that work. They cannot perform it.

The real architectures in this book are also bounded slices. The name x86-64 will never silently mean every historical mode, optional extension, privileged operation, and processor timing table. The name RISC-V will never silently mean every standard or proposed extension. Each load-bearing example will name the instruction forms, state, source revision, privilege assumptions, and ABI convention it uses.

> **Family labels are not causal explanations**
>
> Do not explain an effect by saying only that an architecture is RISC or CISC. Name the actual dimension: operand role, instruction length, memory form, condition state, address calculation, call mechanism, or extension boundary. The label may summarize history after the rule is known.

The sustained comparison has a second purpose. An abstraction that seems natural after reading only one architecture may merely reproduce that architecture's habits. Ao places condition bits in state, but RV64 branch predicates remind us that control need not consume a separately stored flag. Ao uses fixed four-byte

encodings, but x86-64 reminds us that instruction length can itself be a decoded result. The companions pressure-test the foundation.

From execution to a claim

Suppose two short programs leave the same value in their designated output register. Are they equivalent?

Perhaps. One may also change a scratch register. One may change condition state that a later branch reads. One may touch memory, trap on an address, or stop after a different number of instruction bytes. Agreement on the selected output is a fact. Equivalence is a claim whose observation and preconditions must say whether the other differences matter.

The book therefore constructs state before it studies optimization. A local rewrite is not a relation between two lines of assembly alone. It is a relation between their behaviors for every allowed starting state. If the rewrite crosses architectures, equality is usually the wrong shape. The two machines have different register names and condition state. We instead relate their inputs and show that their executions produce related outputs.

The difference between a test and a proof begins here. Running both programs on ten inputs may reveal an error. Passing ten inputs does not settle a claim about every allowed state. For small domains we may enumerate all cases. For larger finite domains we may ask a solver for a **counterexample**: one input on which the programs differ. If none exists, we must still ask what formula the solver received, which semantics produced it, and what independently checkable evidence supports the result.

The machinery is valuable only after the statement is right. A perfect proof of a formula about the wrong flags or a reversed byte order proves the wrong thing with unusual confidence.

Consider two four-bit programs. Program P adds 1 to register r_0 and halts. Program Q adds 1 to r_0, clears r_1, and halts. If the only reported output is r_0, the programs agree on every input. If the caller may inspect r_1, they do not. Neither conclusion is a correction to the other. They answer different questions because they use different observations.

Now give addition a carry output. On an initial $r_0 = 15$, both programs leave 0 in r_0, but a version that forgets to update carry no longer agrees with one that follows the full addition rule. A test that starts at $r_0 = 3$ will miss the defect. The boundary input 15 exposes it. This small example contains the structure of much larger verification tasks: declare the input set, execute a complete state transition, select an observation, and quantify over every allowed start.

The order matters. If we first notice that the printed values of r_0 match, we may be tempted to name the programs equivalent and repair the definition afterward. The book uses the reverse discipline. We decide what a client may observe, then

ask whether the two executions agree under that observation. A claim is not a favorable output with a grander name.

Observation relative to a client

An observation is a declared function that extracts the parts of a machine outcome a particular client may distinguish. Two outcomes agree for that client when the observation returns the same result for both. Changing the client may change the observation and therefore change the claim.

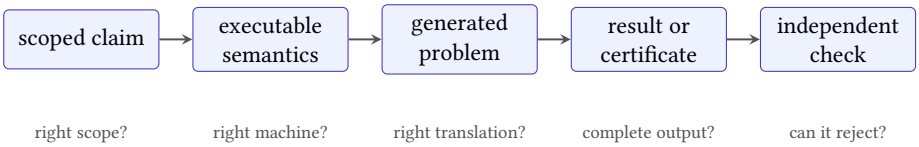


Figure 1: A machine-backed claim crosses several trust boundaries. Each arrow needs its own explanation and failure test.

Figure 1 shows why a final green check is not enough. The checker may faithfully validate an artifact for a generated problem. The generator may faithfully encode executable semantics. Those semantics may still omit a flag that the original claim meant to preserve. Assurance accumulates only when every connection is identified.

A correct proof of the wrong byte order

Suppose a generator reverses the weights used to join four memory bytes. A solver and certificate checker may correctly establish every property of that generated function. The result still does not prove a claim about Ao little-endian loads. The failure lies before the solver, at the connection between the printed semantics and the generated problem.

The two proofs

Every load-bearing theorem in this book needs two forms of support. The reader proof exposes the reason. It may use a width-four example, an invariant, a case split, a short simulation, or a contradiction that can be reconstructed at a blackboard. The machine proof covers the declared finite space through an artifact and a checker that can reject damaged evidence.

Neither can replace the other. A solver log does not explain why a branch rewrite works. A persuasive hand calculation does not cover every 64-bit input.

The hand argument tells us what the encoding should preserve. The machine route tells us whether the industrial-size instance contains an exception.

Computations keep a separate name. An exhaustive run can be valuable and reproducible without carrying a proof certificate. The book states which kind of evidence it has rather than promoting every successful command to the same rank.

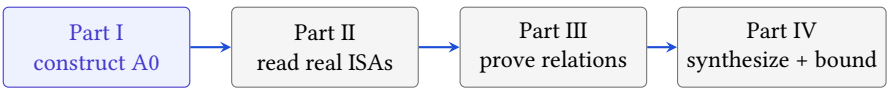
We also make the checker lose. A nearby false claim, mutated instruction, damaged trace, or truncated certificate must be rejected for the expected reason. This negative control cannot prove the original statement. It can show that the route distinguishes at least one precise error from success.

The two proofs also fail differently. A reader argument may omit a case or use an unstated premise. A machine route may encode the wrong operation, search an incomplete candidate language, accept a damaged certificate, or lose the provenance of its inputs. Placing them beside each other creates useful friction. The hand proof tells us which invariant and controls to expect. The machine result tells us where the friendly miniature stopped covering the real finite domain.

Three recurring proof shapes

Word and state laws often use a case split or direct algebra. Program equivalence often uses an invariant or simulation across steps. Minimality usually argues by contradiction: assume a cheaper candidate exists, encode that existential claim, and refute every candidate in the declared language. The shape of the proof follows the quantifier in the claim.

The route through the book



definitions before adapters before universal claims

Figure 2: The curricular order follows the dependency order of the eventual proofs. Later parts may not bypass a missing semantic layer.

Part I constructs Ao. Chapter 1 begins with words and the several meanings a fixed pattern can receive. Chapters 2 through 5 add state, operands, instructions, memory, programs, and control. By the end, the reader can execute a small Ao program by hand and say exactly why each state change occurs.

Part II keeps one concept fixed while placing RISC-V and x86-64 beside it. Encoding and decoding come first because instruction bytes and instruction meaning are different objects. Data movement, addresses, arithmetic, condition state, con-

trol transfer, procedures, stacks, and ABIs follow. The comparison stays paired: we do not teach one architecture as normal and the other as a collection of exceptions.

Part III asks what can be proved about programs. It develops observation, equivalence, refinement, cross-ISA simulation, and candidate program languages. It then asks about cost and **minimality**, the claim that no cheaper candidate exists. The proof system grows from the semantics already on the page.

Part IV carries one scalar algorithm through Ao, RV64, and x86-64. The final chapter then marks the edge of the model: floating point, packed and vector state, concurrency, privilege, virtual memory, self-modifying code, timing, and leakage all require more structure than the scalar sequential core.

The order is an argument. First say what a machine step means. Then compare real steps. Then relate programs. Then ask a finite artifact to support a universal sentence.

The chapters are not independent essays. A byte-order proof in Chapter 4 depends on the word decomposition in Chapter 1. A trace in Chapter 5 depends on the complete step in Chapter 3. A cross-ISA simulation in Chapter 12 will depend on both real-ISA adapters from Part II. A minimality statement in Chapter 13 will depend on all of those semantics plus a candidate language and cost. The dependency chain prevents a solver formula from quietly becoming a machine theorem.

How to work through a chapter

The book is meant to be used with pencil, terminal, and source manual near one another. New material normally arrives in five passes. First comes an object in ordinary language: a word, state, operand, instruction, trace, or relation. Next comes a mathematical definition precise enough to settle boundary cases. Then a small instance is executed or proved by hand. A real-architecture comparison asks which parts survive contact with RV64 and x86-64. Only then does an Axeyum route enlarge or check the claim.

Do not skip the hand execution because the code can run it. For a transition, write the old state on the left and the new state on the right. Circle every component the instruction reads. Mark every component it writes. State why the untouched components remain unchanged. This practice catches omitted state before a solver turns the omission into thousands of flawless but irrelevant cases.

Proofs deserve a similar first pass. Before following symbols, identify the quantifier. Is the claim about one execution, every word of a fixed width, every state satisfying a premise, or every candidate below a cost bound? Then identify the proposed reason: decomposition, invariant, simulation, or contradiction. The details become easier to judge once the burden is visible.

A six-question reading routine

For each load-bearing claim, ask:

1. What are the objects, and are their widths and state components fixed?
2. Which starting objects are allowed?
3. What outcome or observation is compared?
4. Is the claim about one case, a finite domain, or an unbounded family?
5. What is the human-scale reason it should hold?
6. What artifact, checker, and failing control cover the machine route?

If one answer is missing, pause there. Later machinery cannot supply an earlier definition.

Exercises carry part of the exposition. The early ones rehearse a definition on a new value. Middle exercises alter a premise or observation and ask which conclusion survives. Later ones ask for a proof, a counterexample, or a small checker. Some deliberately present an attractive false statement. Finding the smallest state that breaks it is often more instructive than reading another warning about hidden assumptions.

The figures also have jobs beyond decoration. A state diagram inventories what must be preserved. An encoding diagram separates bit fields from their decoded roles. A control-flow graph turns a program counter update into a visible fork. When a figure seems obvious, try to redraw it from the adjacent definition. Any edge or field that cannot be justified is a useful question for the text or model.

Readers may choose different depths without changing the logical order. A first course can execute Ao examples, study the paired ISA cases, and complete selected reader proofs. A verification course can add every artifact and negative control. An experienced compiler engineer may move quickly through basic arithmetic but should resist skipping the observation and state relations: those are where familiar optimization claims acquire exact scope.

How Axeyum enters the argument

Axeyum is the evidence stack used by the accompanying repository. It supplies bit-vector expressions, solver routes, certificate machinery, and other formal infrastructure. This book asks it to grow new architectural layers: Ao execution, bounded real-ISA slices, decoders, state relations, and replay. The software is introduced when a mathematical need appears rather than in a detached command tour.

The first use is deliberately modest. A reader constructs one fixed-width word, evaluates an operation, and asks whether a proposed identity has a counterexample. The next layer constructs complete Ao states and observations. Later layers execute instructions and traces, relate states from different machines, generate solver problems, and check evidence. Each addition should reuse the semantics already exercised by hand.

> **The four questions to ask of an Axeyum result**
>
> What exact statement entered the route? Which executable semantics generated it? What artifact or model came back? Which independent program checked or replayed the result, and what nearby false input did that route reject?

The chapter sequence is not an Axeyum reference manual. Commands and interfaces may change; the evidentiary questions should not. The repository guide pins the checkout, build steps, route, and limitations used by each machine-backed object. A reader interested only in the mathematics can follow the reader proofs. A reader interested in trust can reproduce the machine route and attack its controls.

At the current boundary, Axeyum has useful formal substrate but not the Ao, RV64, x86-64, and cross-ISA packages the full book requires. Planned examples remain labeled as obligations until those packages and their controls run. This prevents tutorial prose from becoming accidental evidence.

What the ledger is for

The project keeps a machine-readable record for its definitions, claims, and unresolved obligations. Chapters take scope and evidence descriptions from that record rather than inventing stronger wording in prose. Evidence files carry raw digests. Checkers return failure when a claim, artifact, or control does not meet its contract.

The ledger is not the subject of the first chapters. A reader should meet a word before meeting its object record. They should understand a state transition before meeting its formula. The record keeps the book honest when a claim moves between prose, executable semantics, solver input, and checked result.

Three distinctions in that record matter throughout the book. A definition fixes vocabulary and creates an implementation obligation; it does not become truer because software happens to represent it. A theorem makes a universal claim and needs an argument that covers its stated domain. A computation reports the result of a reproducible finite procedure. It may settle a bounded question, find a witness, or expose a counterexample without pretending to be a proof of something larger.

The printed artifact panels bind those distinctions to concrete evidence. An identifier points to the record. The scope names widths, instruction forms, starting

states, observations, and cost assumptions. The route names the generator or executor and the checker. The limitation says what the route does not establish. A digest detects a changed file; it does not establish that the file expresses the intended machine. That connection still needs semantic review and controls.

Every chapter makes the same promise

The ordinary prose will say what an object means and why a claim should hold. The formal statement will fix its scope. When machine evidence exists, the artifact route will say how to reproduce and challenge it. When a required route does not yet exist, the book will name the missing work instead of borrowing certainty from a nearby calculation.

This separation also lets the manuscript improve without rewriting its own history. A missing executable layer can later be implemented and checked. A computation can later gain a certificate. A false conjecture can be replaced by its smallest counterexample. The prose must then change where the evidence changes, but the reader never has to infer that a successful command silently upgraded the kind of claim being made.

At present, the mature solver machinery is ahead of the architectural layers this revised book requires. The missing work is stated plainly. Axeyum must gain reusable Ao state and execution, source-pinned RV64 and x86-64 slices, decoders, and cross-machine state relations before the later chapters can claim semantics-backed ISA proofs. A handwritten bit-vector identity is not a substitute for those layers.

That boundary is a useful place to begin. We know the machine we need to construct, the two real systems against which it must answer, and the kinds of evidence that would let a skeptical reader check the result. Chapter 1 starts with the smallest object any of them can hold: one fixed-width word.

CHAPTER ONE

Words and Their Meanings

The pattern `11111111` can be read as 255, as -1, as eight Boolean values, as a mask, or as one byte of an instruction. The pattern does not confess which reading is intended. The operation or convention supplies it.

That distinction is the first foundation of instruction semantics. A register stores a fixed number of bits. An operation supplies a reading and a rule for producing another fixed pattern. We begin with the finite set on which all those rules act.

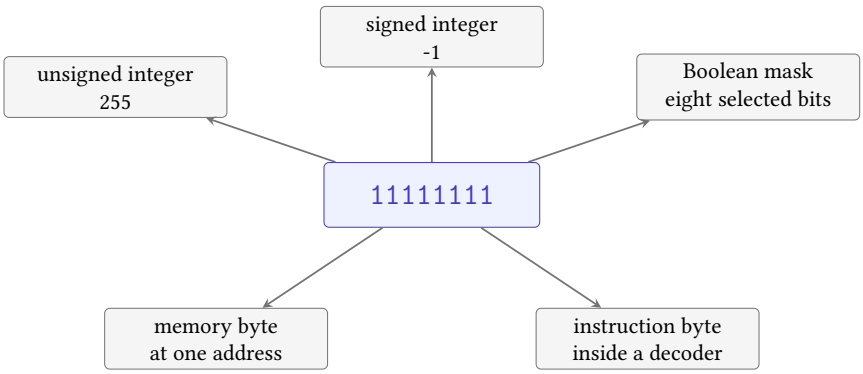


Figure 1.1: A stored pattern acquires a meaning only through a declared reading or use. The arrows are interpretations, not changes to the eight bits.

Figure 1.1 warns against a common shortcut. We may say that a register "contains -1" when the signed reading is already clear. The machine stores the pattern. The signed operation, comparison, conversion, or printing convention supplies the -1. This distinction may feel fussy on one byte. It becomes decisive when two instructions preserve the same bits but give them different roles.

Positional notation made rules reusable

A positional numeral assigns a weight to each place. In decimal, those weights are powers of ten. In binary, they are powers of two. The gain is not merely a shorter

alphabet. One small collection of carrying and place-value rules works at every position, so a finite procedure can act on numerals much longer than the examples used to teach it.

Mechanical and electronic computers inherited that advantage. A circuit can store two stable alternatives and combine them with Boolean operations. A register groups a fixed number of those positions. The instruction set then defines what operations may do to the group: add it, shift it, compare it, split it into bytes, or treat it as an address.

Binary storage did not remove interpretation. It multiplied the useful interpretations of one pattern. The same positions can support unsigned arithmetic, two's-complement arithmetic, logical masks, encodings, and pieces of larger structures. Modern verification must therefore bind the pattern to the intended width and operation before it can establish a claim.

Place value is already an algorithmic idea

The written digits are finite data. A rule for their weights tells us how to interpret every numeral of the admitted length. Machine words keep that compact bargain and add a fixed boundary: beyond the highest position, the operation must say what happens.

1.1 Writing patterns without losing the width

Binary notation writes one symbol for every bit, which makes bit positions visible and long words tiresome. Hexadecimal notation groups four bits into one digit. The digits 0 through 9 keep their usual values; A through F stand for 10 through 15. Thus

$$1111 = \texttt{0xf}, \qquad 1010\,0110 = \texttt{0xa6}.$$

The prefix `0x` tells us that the following digits are hexadecimal. It does not tell us the width. The written numeral `0xff` may be an 8-bit word, the low byte of a 32-bit word, or an integer literal that an assembler will later fit into an instruction field.

A value is not yet a width

The numeral 255 names a mathematical integer. A word names an element of a fixed finite set. Before applying a machine operation, state the width and the rule that maps the numeral into that set.

Leading zeroes carry this missing information when the context has not already fixed it. The patterns `1111` and `00001111` have the same unsigned reading, but

they are different-width words. They offer different high bits, different wraparound points, and different results under a width-sensitive operation such as bitwise not.

1.2 A finite circle

Begin with a four-bit counter. It visits the patterns for 0 through 15. Add one to 15 and the counter returns to 0 because there is no fifth bit in which to store 16. This return to the start of a fixed-width range is **wraparound**. Add two to 15 and the counter stops at 1.

A0 word

For $w \geq 1$, a **word of width** w is a residue modulo 2^w. We use the natural representatives $0, 1, \ldots, 2^w - 1$. Addition, subtraction, and multiplication take the ordinary integer result and keep its remainder modulo 2^w.

Object `A0.def.word` records this carrier and its readings. The quotient notation says that integers differing by a multiple of 2^w represent the same stored word. At width four, 16 and 0 therefore represent the same word, as do 17 and 1.

The reader proof of wraparound follows from division with remainder. For any width-four word x, ordinary addition gives $x + 1$. If $x < 15$, that number already lies between 1 and 15, so taking the remainder changes nothing. If $x = 15$, then $x + 1 = 16 = 1 \cdot 16 + 0$, whose remainder after division by 16 is 0. The apparent exceptional case is simply the boundary of the finite carrier.

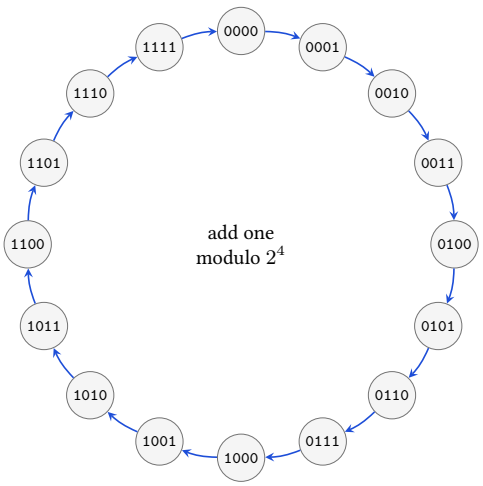


Figure 1.2: Addition by one visits every width-four word and returns to zero after `1111`. The circular layout shows the modular rule; it is not a picture of storage or time.

The cycle in Figure 1.2 gives the right shape but not the definition. A diagram might suggest that the counter physically travels around a ring. It does not. The remainder operation supplies a successor for each of the sixteen words, including the edge case `1111`. The circle is useful because the last arrow is governed by the same rule as every other arrow.

There are exactly 2^w words of width w. A unary instruction on one word is therefore a function on a finite set. At width 64 the set is too large to list usefully, but it still has a declared edge. One finite rule gives an answer for every one of those inputs.

A proof from one decomposition

To show that adding one wraps exactly at the largest width-w word, split the domain into two cases. If $0 \le x < 2^w - 1$, then $x + 1 < 2^w$, so its remainder is itself. If $x = 2^w - 1$, then $x + 1 = 2^w$, whose remainder is zero. The two cases cover every natural representative, and the boundary appears in only the second case.

This argument is more useful than listing all inputs. It identifies the load-bearing fact: every representative except the largest has room for one more, while the largest is one less than the modulus. The same split works at width 8 or width 64 without asking us to inspect 2^{64} rows.

1.2.1 Congruence explains why representatives may change

Write $a \equiv b \pmod{2^w}$ when $a - b$ is a multiple of 2^w. The two integers then name the same width-w word. At width four, $-1 \equiv 15 \pmod{16}$ and $17 \equiv 1 \pmod{16}$. Choosing the natural representative gives a convenient stored spelling; it does not erase the other integers in the same residue class.

Congruence is preserved by addition and multiplication. If a and b name the same word, then adding the same integer c to both produces another pair whose difference is still a multiple of 2^w. Multiplying by c does the same. This is why modular operations are well-defined on words rather than on one accidental choice of integer representative.

Subtraction needs no separate underflow object in the carrier. At width four, $0 - 1 = -1 \equiv 15 \pmod{16}$, so the stored result is `1111`. An instruction may also record a borrow or carry convention, but that condition is additional state about the mathematical subtraction. The word result itself comes from the same remainder rule.

The carrier and the condition answer different questions

Modular arithmetic always supplies a width-w result. Carry, borrow, and signed overflow report how an integer reading crossed a chosen boundary. They do not repair or replace the stored word.

Binary arithmetic before electronic computers

In 1703, Leibniz described arithmetic using only zero and one and connected the spare notation with rules for calculation. His machine was not a modern instruction set, and the essay does not supply our semantics. It records an older encounter with the same compression: a tiny alphabet can support a complete arithmetic once its rules are fixed.(Leibniz 1703)

1.3 Bits and Boolean operations

For $0 \le i < w$, bit i of a word x is

$$(x \operatorname{div} 2^i) \bmod 2.$$

Bit zero is the least significant bit. Bit $w - 1$ is the most significant. Bitwise and, or, exclusive or, and not apply one Boolean operation at every position.

For example,

$$1010 \operatorname{and} 1100 = 1000, \qquad 1010 \operatorname{xor} 1100 = 0110.$$

Each output position can be checked without consulting another position.

Bitwise operations become useful when one word carries several independent choices. A **mask** is a word used to select positions. If a mask has a one at position i, then and keeps bit i of the other operand; if the mask has a zero there, and clears it. With width eight,

$$10110110 \operatorname{and} 00001111 = 00000110.$$

The mask keeps the low four positions. Using or with 10000000 sets the high position without deciding the other seven. Using xor with the same mask toggles that position. These are three different state transformations even when a particular input happens to make two results agree.

Reading a field

Suppose bits 4 through 6 hold a three-bit field inside an eight-bit word x. First shift x right by four positions. Then apply the mask `00000111`. The shift moves the desired field to positions 0 through 2; the mask clears everything above it. Each step has a separate purpose, which makes a wrong shift distance or a wrong mask easy to attack.

A logical left shift by k multiplies by 2^k and then reduces modulo 2^w. A logical right shift divides by 2^k and keeps the whole-number quotient. Thus `1100` shifted logically right by one becomes `0110`. An arithmetic right shift has a different purpose: it repeats the high bit under the two's-complement reading, so the same input becomes `1110`. The operation, not the stored pattern, chooses the behavior.

We should not read meaning directly from bits. A high bit of one does not announce that a value is negative. A signed operation treats it as a sign; an unsigned operation does not.

A shift count also needs a boundary rule. What happens when $k \geq w$? Different languages and instruction forms may mask the count, reject it, or define a zero result for a logical shift. Ao must choose one rule before a shift theorem can range over all inputs. Until that choice appears in the executable word package, examples in this chapter keep $0 \leq k < w$.

1.4 Unsigned and signed readings

The unsigned reading of a word is its representative between 0 and $2^w - 1$. The signed two's-complement reading agrees with that value when the high bit is zero. When the high bit is one, it subtracts 2^w.

At width four, `0111` has unsigned and signed value 7. `1111` has unsigned value 15 and signed value $15 - 16 = -1$. No bit has moved. We changed the map from the stored word to an integer.

The high-bit-zero half contains 0 through $2^{w-1} - 1$. The high-bit-one half contains the representatives 2^{w-1} through $2^w - 1$; subtracting 2^w maps them to -2^{w-1} through -1. The signed range is therefore

$$-2^{w-1}, \ldots, 2^{w-1} - 1.$$

At width four it is -8 through 7. Adding `0001` to `0111` produces `1000`. Its unsigned reading is 8; its signed reading is -8. The stored addition is well-defined. Whether the mathematical interpretation crossed a signed or unsigned boundary is a separate fact that later condition-state rules may record. It is imprecise to say that `1000` simply is negative. It has signed reading -8 and unsigned reading 8.

Two boundary facts deserve separate names. An unsigned addition produces a **carry out** when the ordinary nonnegative sum is at least 2^w. A signed addition

has **signed overflow** when the sum of the two signed readings lies outside -2^{w-1} through $2^{w-1}-1$. At width four, 1111 plus 0001 has a carry out: 15 plus 1 is 16. Under the signed reading, -1 plus 1 is 0, so there is no signed overflow. By contrast, 0111 plus 0001 has no carry out, but 7 plus 1 lies outside the signed range. The stored result 1000 is the same kind of word in both calculations; the two conditions answer different questions about it.

One result, two overflow questions

Do not use *overflow* without saying whether the claim concerns the unsigned carry boundary or the signed range. The conditions can disagree on the same stored addition.

The signed-overflow rule can be read directly from signs. Adding operands with different signed signs cannot overflow: their mathematical sum lies between them. Adding two nonnegative values overflows exactly when the stored result has a high bit of one. Adding two negative values overflows exactly when the stored result has a high bit of zero. Chapter 3 will turn these observations into explicit Ao condition-state updates rather than leaving them as prose.

Two readings, one pattern

List all sixteen width-four patterns. Write the unsigned and signed reading of each. Identify the point at which the signed order wraps from 7 to -8.

1.5 Changing widths

Moving a value between widths requires another rule. Truncating from width w to a smaller width v keeps the remainder modulo 2^v. Truncating 00010001 to four bits gives 0001; the discarded high bit cannot be recovered from the result. Zero extension to a larger width inserts zeroes above the old high bit. Sign extension repeats the old high bit so that the two's-complement integer reading remains the same.

For 1111, zero extension from four bits to eight gives 00001111, whose signed reading is 15. Sign extension gives 11111111, whose signed reading remains -1. An instruction that merely says it produces eight bits has not yet told us which value-preservation rule it uses.

The preservation arguments are short. Zero extension adds only zero coefficients above the old high bit, so the unsigned sum of bit weights does not change. For sign extension, a nonnegative source also gains only zeroes. A negative source of width w, with unsigned representative u, becomes the wider unsigned repre-

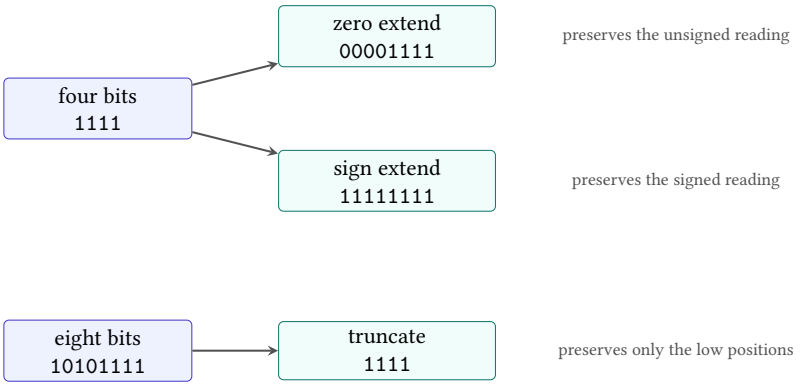


Figure 1.3: Zero extension, sign extension, and truncation are three different maps. Their names identify both the bit movement and the reading or positions they preserve.

sentative $u + 2^v - 2^w$ at width v. Subtracting 2^v gives $u - 2^w$, the original signed reading.

This distinction will return in both companion architectures. Their registers may hold 64-bit words while selected operations produce narrower results. The semantics must say whether upper bits are preserved, cleared, or supplied by sign extension.

The words *preserves the value* are incomplete until the reading is named. Zero extension preserves the unsigned reading. Sign extension preserves the signed reading. Neither operation promises to preserve both. For a source whose high bit is zero, they happen to produce the same wider pattern; that agreement on half the inputs is not a reason to treat the operations as interchangeable.

Why sign extension preserves a negative value

Let a negative width-w word have unsigned representative u. Its signed reading is $u - 2^w$. Extending it to width $v > w$ fills the new high positions with ones, adding $2^v - 2^w$ to the unsigned representative. The wider signed reading subtracts 2^v:

$$(u + 2^v - 2^w) - 2^v = u - 2^w.$$

The two large terms cancel, leaving the original signed integer.

Truncation has no such general preservation law. It is a many-to-one map: every pair of source representatives that differs by a multiple of 2^v has the same width-v result. A later proof may still use truncation under a premise that the

discarded bits are known to be zero, or known to repeat the sign. The premise, not the word *truncate*, earns the preservation claim.

Some compositions do have exact laws. Truncating a zero-extended word back to its source width returns the original pattern. The same holds after sign extension because both extensions preserve every original low position. Extending after truncation does not generally return the original wider word: the discarded positions are unavailable, so the extension rule must invent them from zero or from the retained sign bit.

These one-way identities are useful controls. A checker should accept truncate-after-extend for every source word and reject the reverse identity on a witness with nonzero discarded positions. The pair teaches more than two successful examples because it marks exactly where information is lost.

1.6 Words and bytes

A byte is a word of width eight. Memory will store bytes at addresses, so a wider word needs a declared relation to a sequence of bytes.

Little-endian split and join

Let the word width be a positive multiple of eight. Byte i is

$$b_i = (x \operatorname{div} 2^{8i}) \bmod 2^8.$$

A **little-endian** byte sequence puts the lowest-weight byte first in increasing address order: $b_0, b_1, \ldots$. Joining the sequence computes

$$\sum_i b_i 2^{8i}$$

at the original width.

Object `A0.def.byte` records this relation. It does not yet define memory. It says how one word can be taken apart and put back together.

The reader proof of the round trip is positional. Each b_i selects exactly bits $8i$ through $8i + 7$. Multiplying by 2^{8i} returns those bits to their original positions. Different bytes occupy **disjoint** positions, meaning that no position belongs to two bytes. Their sum reconstructs the original word.

For two bytes, write $x = b_0 + 2^8 b_1$, where both byte values lie between o and $2^8 - 1$. Splitting extracts $b_0 = x \bmod 2^8$. Dividing x by 2^8 discards that low remainder and exposes b_1. Joining returns $b_0 + 2^8 b_1 = x$. For more bytes, the same quotient-and-remainder step repeats. Nothing in this argument depends on the hexadecimal spelling of x.

Table 1.1: Word-level questions carried from Ao to each real instruction form.

Question	RV64 slice	x86-64 slice
Source width	named operand form	named operand form
Destination width	register or memory rule	register, subregister, or memory rule
Integer reading	operation and condition	operation and condition
Upper bits	explicit width rule	selected subregister rule
Byte order	declared memory premise	declared memory premise

A byte round trip with unequal digits

Split `0x00a17c03` into four little-endian bytes. Join them in the proper order, then join the reversed list. Unequal bytes make an order error visible; a test word such as `0x00000000` would let the wrong rule pass.

For `0x12345678`, the increasing-address byte sequence is `78`, `56`, `34`, `12`. If we instead attach increasing addresses to the list `12`, `34`, `56`, `78`, the same little-endian join produces `0x78563412`. The numeric word did not reverse by itself. We changed the relation between addresses and place values.

Width four was useful for the reader proofs because all sixteen cases fit on a page. A complete Ao state has width equal to a positive multiple of eight. Here lies the first firm boundary between a proof miniature and the executable teaching machine.

1.7 The two companion questions

Both later companion slices contain fixed-width integer values and access byte-addressed memory. That common description does not settle how a given instruction reads a word, changes its width, or treats its upper bits. Those rules belong to the selected instruction form.

The question for RV64 and x86-64 is therefore not whether either machine uses binary. It is which width each form reads, which width it writes, which integer reading its conditions use, and how its bytes meet memory. We will pin the exact forms and source revisions before answering those questions as architectural facts.

The paired questions can now be stated without slogans:

This table is a checklist, not a set of answers. Saying that one architecture has 64-bit registers does not settle the result of a narrower write. We need the exact instruction form and the governing manual revision. Part II will supply those source-pinned answers after Ao has given us the questions in a stable order.

1.8 The implementation boundary

The definitions in this chapter are book-level foundations. Axeyum already has fixed-width bit-vector terms and solver routes, but the redesigned book does not treat the former width-eight formulas as its word foundation. Object `OP.a0.word-package` records the actual missing layer: one reusable Ao package for words, readings, width changes, and byte split and join, with interfaces suitable for execution and proof.

The controls should attack the rules we have just used. A wrong modulus should fail at wraparound. A wrong sign extension should fail on a source whose high bit is one. Reversed byte weights should fail on a word with unequal bytes. A damaged round trip should return a concrete witness rather than a success message.

An introductory Axeyum workflow will eventually make this obligation concrete. The reader will construct a fixed-width term, state a relation such as split followed by join, ask for a counterexample within the declared width, and replay any returned model through executable Ao operations. If the solver reports that no counterexample exists, the route must still identify the generated formula and the independent checking boundary. Axeyum's current bit-vector machinery is useful substrate for that route; it is not yet the Ao word package named by this chapter.

The first Python-facing lesson should use a small, typed surface. Construct a word by giving its width and natural representative. Ask for its unsigned and signed readings. Apply one width change or byte split. The bound Rust type should reject a value outside the constructor contract rather than silently choosing a width. Python may format binary and hexadecimal views, but Rust must own the carrier and operations that later checks reuse.

The proof-facing lesson begins from the same objects. It quantifies over a declared width, applies the executable operation or its matched formula encoding, and requests either a separating input or evidence that the finite claim has no separating input. Replaying a returned word must reproduce both its stored pattern and the observations that made it a counterexample.

This progression keeps three activities distinct: calculating one example, searching the whole finite domain, and checking the evidence returned by that search. The commands may be close together in a notebook. Their claims are not interchangeable.

What Axeyum will add

The mathematics on these pages defines the intended word operations. The planned Axeyum package will add executable operations, formula generation, counterexample replay, checked evidence for selected finite claims, and negative controls. It will not turn an unstated width convention into a theorem.

Until that package exists, this chapter has reader proofs and exact definitions but no generated machine-proof box. A solver result about one handwritten formula would not establish the width-parametric library on which later Ao state and instruction semantics must depend.

One word now has a carrier, bit operations, several readings, width changes, and a relation to bytes. It still has no location. We do not know which register holds it, which memory address names one of its bytes, which instruction comes next, or whether execution has stopped. Chapter 2 constructs the state in which those questions have answers.

1.9 Exercises

1. **Execute.** Compute every width-four sum $x + 1$. Explain the wraparound once by enumeration and once by the remainder definition.

2. Find two width-four additions with the same stored result but different signed or unsigned interpretations.

3. Convert `0x9d` to eight binary digits. Use masks and shifts to extract its low three bits and its high three bits.

4. Zero-extend and sign-extend each width-four pattern whose high bit is one. State which integer reading each operation preserves.

5. For every pair among `0111`, `1111`, and `0001`, compute the width-four sum, unsigned carry out, and signed overflow. Explain each disagreement between the two conditions.

6. **Transfer.** Split `0x89abcdef` into little-endian bytes and join them again. Then reverse the byte list and calculate the different word it represents.

7. **Prove.** Prove that splitting and joining a two-byte word returns the original word. Identify the exact step that generalizes to any positive byte count.

8. **Break.** Design a false join rule by changing one exponent. Give a word whose round trip exposes the error. This will become the negative control for the future Ao word package.

9. **Prove.** Show that bitwise exclusive or with a fixed mask is its own inverse. State why the argument works independently at every bit position.

10. **Break.** Give a pair of different width-eight words that truncate to the same width-four word. Then state a premise under which truncation would preserve the unsigned reading.

CHAPTER TWO

State

Pause a machine between two instructions. The arithmetic has stopped, but its result is only one part of what remains. Registers contain words. Memory contains bytes. A program counter identifies the next instruction. Condition bits remember facts that a later branch may use. The machine is either able to continue, halted, or trapped for a reason.

Leave one of these parts out and two programs may appear to agree only because we forgot where to look. We will therefore build the complete state first. For a particular question, we may then state which parts of that state matter.

A stored program needs a place to resume

The 1945 EDVAC report divided a proposed computing system into organs for arithmetic, control, memory, input, and output. Its terminology is historical, but the design problem remains current: after one operation, the machine must retain enough information to determine the next. Modern architectural state is not copied from that report; it is one descendant of the need the report made explicit.(Neumann 1945)

The important word is *enough*. We could describe every charge, latch, queue entry, and temperature in a physical processor. Such a description would be too large for our purpose and still incomplete. We could describe only the destination register. That would be small, but too weak to predict a branch or a memory access. A state model earns its size by the questions it can answer.

State is therefore a boundary around relevant history. Once the state is fixed, the next architectural step should not need a secret account of how the machine arrived there. Two executions that reach the same Ao state face the same Ao successors under the same program. Their earlier paths may differ, but the transition rule receives the same input.

State sufficiency

A state description is sufficient for a transition model when the modeled next-step possibilities depend only on that state and the declared external inputs, not on an unrecorded execution history.

State sufficiency is a modeling requirement, not a physical claim that history vanishes entirely. A cache or branch predictor can retain effects of earlier execution. Ao omits them, so Ao cannot answer a question whose next result depends on those effects. A timing model may restore them as state. The right snapshot changes with the promised observation.

2.1 The teaching machine

The abstract machine used in this book is Ao. Its word width w is a positive multiple of eight. The width-four patterns in Chapter 1 remain useful on a blackboard, but they are not complete Ao machine states. Full Ao begins at one byte because its memory is byte-addressed.

A0 machine state

An Ao **machine state** is

$$S = (R, M, pc, Z, N, C, V, O).$$

The map R assigns a w-bit word to each of the eight registers $r_0, \ldots, r_7$. The finite map M assigns bytes to valid data addresses. The w-bit word pc is the program counter. The bits Z, N, C, V record zero, negative, carry, and overflow conditions. The outcome O is running, halted, or trapped with a reason.

The tuple is not a bag of unrelated fields. Its components constrain one another through instruction rules. A running state permits a step; a halted or trapped state does not. The program counter supplies the next fetch address. Register contents may supply operands or addresses. Condition bits may supply a branch predicate. Memory supplies bytes only at addresses in its finite domain. Chapter 3 will define the transformations that preserve these connections.

Object `A0.def.state` records this contract. Every register is ordinary: Ao has no hardwired zero register and no implicit stack pointer. The condition bits are named state, not comments about the previous line. If an instruction writes one of them, a later instruction can observe that write.

Program code is not part of M. An Ao program supplies a separate, finite map of immutable instruction bytes and an entry address. This choice rules out

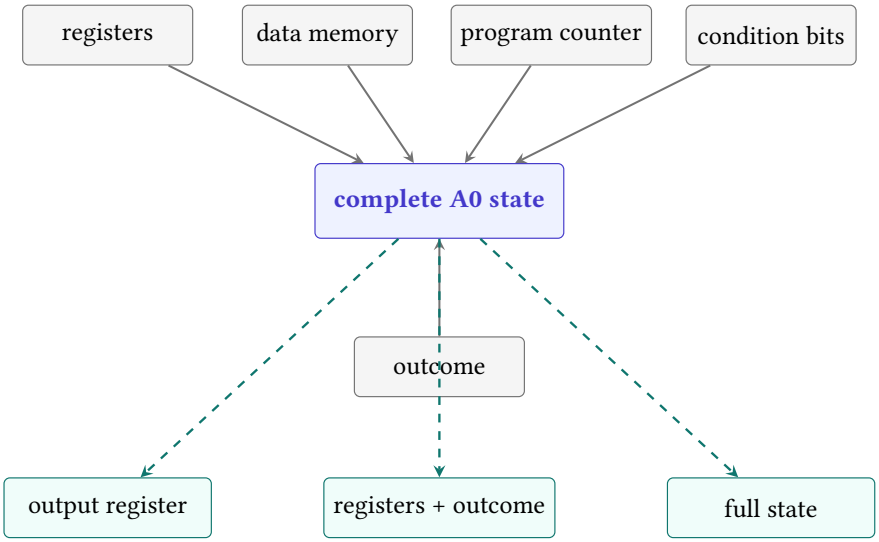


Figure 2.1: A0 gathers every modeled component into one complete state. An observation projects that state to the information a claim exposes.

self-modifying code in the first model. It also prevents a data store from silently changing the next instruction. Chapter 5 will add the program to the execution setting; for now we are describing the snapshot on which one step acts.

Separating code from data is a modeling decision, not a claim about all real machines. It gives the first A0 model a stable program while we learn to reason about data memory. A theorem under this separation says nothing about self-modifying programs unless a later model adds code memory to the writable state and repairs every affected definition.

The outcome needs three cases. A running state may take a step. A halted state has finished by executing `halt`. A trapped state records that execution could not continue under an A0 rule, such as an invalid memory range. Halt and trap both have no successor, but they do not mean the same thing. A program that finishes and a program that fails are not interchangeable merely because both stop.

Three stopped descriptions

Suppose the last destination contains 42 in three states. In the first, the outcome is halted. In the second, an invalid load has trapped. In the third, the outcome remains running but no further step has been requested. Looking at the destination alone collapses three different execution situations into one number. A claim about successful completion must include the outcome.

The reason attached to a trap also matters when a claim distinguishes failure modes. Ao will name reasons such as illegal encoding, incomplete fetch, and invalid data range. We may later choose an observation that retains only the fact of trapping, but the complete state does not discard the reason first.

2.1.1 Well-formed states and componentwise equality

Not every tuple of mathematical objects is an Ao state. The width must be a positive multiple of eight. Each register value and the program counter must fit that width. Every memory value must be a byte, and every memory key must be a width-w address. Condition fields are bits. The outcome must be one of the declared cases, with a recognized reason when it is trapped.

A constructor should enforce these requirements once. Later instruction rules can then assume they receive a well-formed state and must return another one. Without that invariant, every transition theorem carries distracting side cases about a nine-bit value in an eight-bit register or a memory entry that is not a byte.

Two Ao states are equal exactly when their corresponding components are equal: the same register map, memory map, program counter, four conditions, and outcome. This componentwise rule gives both proof directions. To establish state equality, establish every component. To refute it, one unequal component is enough.

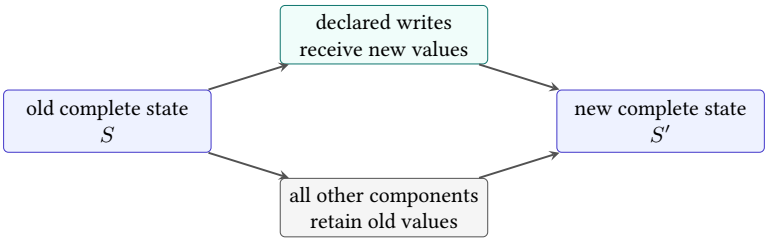


Figure 2.2: A complete transition combines declared new values with a frame condition that preserves every component outside the write set.

The preservation branch in Figure 2.2 is a **frame condition**. It says what the instruction leaves alone. A state diff is a convenient way to display the other branch, but a diff is complete only when it is interpreted against an old state and a rule that preserves everything omitted from it.

A destination-only test is therefore weak. It checks one changed component without checking conditions, control, memory, outcome, or the frame. A complete-state checker can still print a compact diff for the reader. It must derive that diff from states whose equality and preservation it actually checks. Compact presentation and complete checking can therefore coexist without weakening the state contract.

2.2 Which state belongs to the architecture?

The tuple above is an **architectural state**: it contains the parts that Ao instructions are defined to read or change. A physical processor needs much more. It may contain pipeline registers, cache tags, branch predictor entries, queues, and circuitry that has no name in an assembly program.

Whether those parts belong in a model depends on the question. If the question is whether two additions leave the same Ao state, cache replacement policy is irrelevant because Ao has no cache state. If the question is how long the two additions take on a particular processor, the Ao tuple is insufficient. A smaller model is not false merely because it omits timing; it is false when it is used to answer a timing question.

One practical test sharpens the word *architectural*. Ask whether the declared instruction semantics can read the component, write it, or expose it through a modeled outcome. If yes, the component belongs in the state for that semantic question. If it affects only how a particular implementation reaches the result, it may remain outside a functional model. The answer can change when the question changes: cache contents are outside Ao functional equivalence and inside many timing or leakage models.

An omitted component is a theorem premise

Every omitted kind of state limits the claims that follow. Leaving caches out does not assert that caches do not exist. It asserts that the present observation and transition rules do not speak about their effects.

The distinction cuts both ways. Different internal executions may have the same architectural result. The processor is free to use either route if its contract permits both. Yet equal values in one destination register need not mean equal architectural results. Memory, flags, the program counter, and the outcome can still differ.

Consider two paused Ao states. Both have 7 in r_0, 12 in r_1, the same memory, and a running outcome. In the first state $pc = 8$; in the second $pc = 12$. If we record only r_0, the difference disappears. At the next step it may decide which instruction executes. The missing component was not minor. It was merely unobserved.

State completeness is therefore relative to a declared machine contract. The Ao tuple is complete for Ao because every Ao instruction effect must appear in that tuple or its outcome. It is not complete for RV64, x86-64, or a physical processor merely because it is written with eight fields. Each later slice must perform its own inventory.

2.3 What we choose to observe

Now consider a less severe difference. Program p leaves 7 in r_0 and 19 in scratch register r_3. Program q also leaves 7 in r_0, but leaves 20 in r_3. A caller that receives only r_0 may be unable to tell. A comparison of all registers can tell at once.

Observation

An **observation** is a declared function from a complete state to the information exposed by a claim. Two states agree under observation A when

$$A(S_1) = A(S_2).$$

Object `A0.def.observation` fixes this role. An observation may select one register, all registers, a range of memory, the outcome, or a declared combination. The complete state still exists. The observation says which differences the claim exposes.

Figure 2.1 shows three observations, ordered from narrow to broad. This ordering is not always a straight line. One observation might expose a memory range and hide all registers; another might expose two registers and no memory. Neither necessarily contains the other. The useful relation is whether one observation can be derived from another. If it can, the first reveals no distinction that the second has already forgotten.

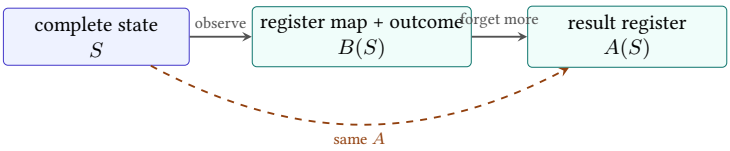


Figure 2.3: A narrower observation can factor through a broader one. Once the broader result is known, an additional projection obtains the narrower result without consulting the complete state again.

In the diagram, $A = f \circ B$ for a function f that selects the result register. Agreement under B then implies agreement under A: equal broad observations remain equal after applying f. The reverse may fail because f discards the other registers and the outcome.

An observation partitions the state space into groups that it cannot distinguish. Every state with the same observed result belongs to the same group, even if hidden components differ. A broader observation usually splits some of those groups. Full-state observation leaves only groups of equal states.

The partition view explains why an observation is neither correct nor incorrect by itself. It is adequate or inadequate for a context. If a caller can read only the

result register and outcome, grouping states that differ solely in a dead scratch register may be appropriate. If a later branch reads a condition bit, an observation that groups different condition values is too coarse for replacement before that branch.

Observation fixes what counts as a difference

Choose the observation from the contract of the surrounding context. Then ask whether the programs agree under it. Choosing the projection after seeing a counterexample changes the question instead of answering it.

The scratch-register example gives a small proof of an important limit. Let $A(S) = R[r_0]$. Both final states map to 7, so they agree under A. Let $B(S) = R$, the complete register map. The first map sends r_3 to 19 and the second sends it to 20, so $B(S_p) \neq B(S_q)$. Agreement after applying one observation does not imply equality of the underlying states.

The converse direction is safe. If two complete states are equal, every function maps them to equal observations. Observation can forget a difference; it cannot create one between equal inputs.

Equality survives every observation

Assume $S_1 = S_2$. An observation A is a function, so replacing an input by an equal input cannot change its output. Therefore $A(S_1) = A(S_2)$. The reverse implication needs more: it holds only when A is injective on the states under discussion. A projection to one register is not injective because many complete states share that register value.

The tiny argument will later carry a large burden. Full-state equivalence implies agreement under every declared observation. Observational equivalence does not imply full-state equality unless the observation retains enough information. When a proof uses the reverse direction, it must establish that extra premise rather than smuggle it in through the word *equivalent*.

The freedom is useful and dangerous. We must choose the observation as part of the claim, before seeing which components a proposed rewrite happens to disturb. Otherwise any difference can be dismissed after the fact by choosing a function that forgets it. That would not establish equivalence. It would only tailor the question to the answer.

We also need the observation to be stable across both programs. Comparing program p under one observation with program q under another can make any two results appear equal by choosing constant functions. One claim uses one declared observation, applied to both sides.

Design the smallest honest observation

A routine receives two words, uses r_3 as scratch space, writes its answer to r_0, and may trap on an invalid input. A caller may later read r_0 and distinguish normal return from a trap, but it promises not to read r_3. Write an observation that exposes exactly those facts. Then remove one component and describe the false replacement it would permit.

One result, three questions

Suppose two runs finish with the same r_0, different C, and different outcomes: one halted and one trapped. They agree under the observation $A(S) = R[r_0]$. They disagree under an observation of $(R[r_0], C)$, and they disagree under any observation that includes the outcome. All three statements are exact. Only the first would be useful to a context that cannot inspect the carry bit or distinguish success from failure.

2.3.1 Initial-state premises restrict the question honestly

A program theorem rarely concerns every well-formed state. It may require a running outcome, a particular entry program counter, valid input addresses, or argument words in selected registers. These facts form the **initial-state premise**. They identify the states from which the claim promises behavior.

The premise differs from the observation. A premise restricts which inputs must be considered. An observation selects which output differences count. Weakening the premise asks the program to handle more starting states. Broadening the observation asks it to preserve more visible output state. Both changes make a claim harder, but they act at opposite ends of execution.

One address premise prevents an expected trap

Suppose a routine loads four bytes beginning at the address in r_1. A premise that all four addresses exist removes invalid-range inputs from the claim. Without that premise, a correct theorem must include trapped outcomes or establish some other reason the range is valid. Hiding the outcome in the final observation would not make the load succeed.

Premises must be stated before testing candidate programs. Otherwise a failed input can be excluded merely because it is inconvenient, just as an output difference can be hidden by choosing a narrow observation afterward. Later objects will bind the input premise and output observation beside the program and machine model so neither can drift during search or replay.

For cross-ISA work, the premise may be relational. It can say that an Ao register and an RV64 register represent the same argument while their unused registers differ. The output observation then asks whether the related runs produce corresponding results and outcomes. Equality of the raw state tuples is neither expected nor required. What matters is the declared correspondence at entry, at matched control points, and at the observed exit.

2.4 State in the two companion machines

Ao now gives us an inventory to carry to a real instruction set. We ask which registers exist, what the program counter denotes, which condition state an instruction can read or write, how memory enters the model, and which exceptional outcomes the selected instruction forms admit.

For the later RV64 slice, the integer register named `x0` needs special treatment. A model that stores and later returns an arbitrary word from `x0` would describe a different machine. The slice must also name its program counter, ordinary integer registers, memory assumptions, and relevant exceptional outcomes.

For the later x86-64 slice, the model must name the selected general-purpose registers, `RIP`, memory, and exactly which `RFLAGS` bits matter to the included forms. An arithmetic replacement that preserves its destination but changes a later branch condition is not full-state equality. It may be valid under a narrower observation, but only if the surrounding program is unable to rely on the omitted flag.

These are state questions, not verdicts about RISC and CISC. One architecture has a special integer-register rule; the other example carries selected condition bits. The exact slices and source revisions must be fixed before either description supports an architectural claim.

The comparison also explains why cross-ISA states will not be equal. Ao has eight ordinary registers and four named condition bits. RV64 has a larger integer register file with a special zero-register rule in the planned slice. The selected x86-64 slice has its own register names, partial-register rules, and condition state. Later we will relate states by what they represent, not by demanding identical tuples.

A state relation is not an observation

An observation extracts information from one machine state. A state relation connects states from two possibly different machines. Cross-ISA proofs need both: a relation for corresponding inputs and intermediate states, and an observation for the outputs the claim exposes.

2.5 The implementation boundary

We can now say what an executable Ao state package must do. It must construct the entire tuple, enforce the width and memory conditions, represent all three outcomes, and apply observations without changing the state they inspect. Object `OP.a0.state-memory` records that implementation obligation for Axeyum.

The controls follow from the reader examples. A projection checker must reject a claim that two unequal full states are equal merely because r_0 matches. An outcome test must distinguish halt from trap. A state constructor must reject a width that is not a positive multiple of eight. A memory test must not allow an address outside the finite map.

The first Axeyum lesson at this layer should remain small. Construct two Ao states that differ only in r_3. Apply an output-register observation and a full-register observation. The first results should agree; the second should produce the differing register as a witness. Mutate the projection so that it silently omits a requested component. The negative control must reject the mutated route.

The serialized state should be canonical enough for an artifact digest to bind one meaning. Register order, address order, word width, outcome tags, and trap reasons must not depend on a hash-map iteration order or a Python display choice. A reader-facing table may choose a pleasant order, but replay should parse one explicit representation and reject duplicates, missing fields, and out-of-width values.

Observation descriptions also belong in the artifact. A label such as `result` is insufficient unless it resolves to a declared register, width, and outcome policy. The checker should rebuild the observation from that declaration and apply it to both full states. It should not trust stored observation outputs that could have been edited independently of their source states.

The Python layer can make this lesson direct. It constructs states through the bound Rust types, asks for named observations, and prints both the compact result and any separating component. Rust owns width checking, canonical serialization, projection, and comparison. Python owns presentation and the sequence of questions; it does not carry a second definition of state.

That exercise teaches the division of labor. Executable state constructors make malformed states hard to express. Observation functions state what the claim exposes. A solver may search for two states that a proposed implication handles incorrectly. A replay checks the returned states through the same executable definitions. None of those layers decides which observation is appropriate for the surrounding program; that remains part of the theorem we choose to state.

No such Axeyum package accompanies this draft yet. The tuple and the arguments in this chapter are definitions and reader reasoning; they are not a hidden machine-proof route. That boundary will remain visible until the executable state and its controls exist.

A state is a still picture. It tells us where every modeled value is, what may happen next, and whether another step is allowed. It does not move itself. Chapter 3 supplies the object that turns one such picture into the next.

2.6 Exercises

1. **Execute.** Write an Ao state of width eight with register $r_i = i$, memory bytes 10, 20, 30, 40 at addresses 0 through 3, $pc = 0$, cleared condition bits, and a running outcome. Evaluate observations that select r_0, all registers, and (pc, O).

2. **Break.** Construct two states that agree in every register but differ in memory. Give an observation that hides the difference and one that exposes it. Explain why neither observation changes the states.

3. **Prove.** Show that complete-state equality implies agreement under every observation. Then use a two-element example to show that the converse is false for a non-injective observation.

4. **Transfer.** For a proposed RV64 state and a proposed x86-64 state, list one special component or rule that has no direct Ao counterpart. State the functional question that makes it relevant.

5. A sequence changes C but no register or memory byte. State two replacement claims for which that fact leads to different answers.

6. Give one timing question for which Ao architectural state is too small. Name the missing physical information without adding it to Ao.

7. **Execute.** Starting from the state in the first exercise, change only pc, then only O, then one memory byte. For each change, list the observations in Figure 2.1 that can detect it.

8. **Prove.** Let observation A be computable from observation B. Show that agreement under B implies agreement under A. Give an example showing that the reverse implication can fail.

9. **Break.** Define an observation after inspecting two unequal states so that it hides their only difference. Explain why the resulting true equality does not support the original full-state claim.

10. **Transfer.** Sketch a relation between an Ao input state and a future real-ISA input state for a routine with two arguments and one result. Name what the relation deliberately leaves unmatched.

CHAPTER THREE

Instructions and Operands

The word add does not say enough. It does not name the width, the inputs, the destination, the condition bits, the next program counter, or the cases in which execution cannot continue. It is a useful name for a family of actions, not a complete account of any one action.

To move from one machine state to the next, we need an operation, an operand form, concrete operands, and a rule that exposes every architectural effect. The printed instruction is short. Its meaning cannot be.

An instruction meaning is a complete transition

An instruction does not merely calculate a value. It transforms one declared machine state into another, or produces a declared failure. Every component that can affect later execution belongs in that account.

This view separates three questions that assembly listings often place on one line. Which bytes were fetched? Which instruction instance did the decoder produce? What does that instance do to state? A correct answer to one does not repair an error in another.

From operation tables to transition rules

An instruction table is one of computing's durable forms. It gives a short name or bit pattern to an operation and tells a programmer which operands may accompany it. Early manuals needed such tables because a machine could perform only the actions supplied by its hardware. Modern architecture manuals still use them because compatibility requires precise answers about decades of accumulated forms.

The purpose has widened. Compilers use instruction descriptions to select and schedule code. Assemblers turn accepted forms into bytes. Disassemblers try to recover forms from bytes. Emulators perform the effects. Hardware verification compares an implementation with an architectural account. Binary translators

relate one machine's transition to another's. Each consumer needs more than a mnemonic, though not always the same more.

For this book, the central object is the architectural transition. It lies between notation and hardware. It says what state change software is entitled to observe while leaving circuit organization and timing to a later model. That position gives the instruction an unusual reach: a finite rule describes its action on every admitted state.

A short operation name can denote a vast table

At width sixty-four, one register addition rule covers more input pairs than could be listed. The rule earns that compression by stating the arithmetic, conditions, failures, and preserved state exactly. A short name without those clauses compresses nothing; it merely postpones the definition.

3.1 From a spelling to an action

Instruction instance

An **instruction instance** is an operation together with one allowed operand form and concrete operands. Its semantics gives the complete state change for a running input state, including the next program counter and any trapped outcome.

Object `A0.def.instruction` records this state-transformer view. A decoded instruction is not the same object as its printed assembly or its instruction bytes. Assembly is a human-readable notation. Decoding turns bytes into an instruction instance. Semantics says what that instance does. Chapter 6 will study the boundary between the last two.

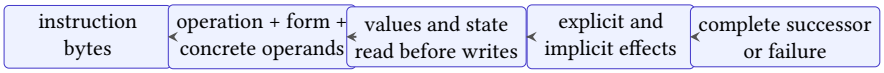


Figure 3.1: One printed instruction conceals several contracts. Bytes select an instance; the instance determines reads and effects; the semantics returns a complete successor or a declared failure.

For deterministic A0, a fixed instruction instance and running input state select at most one successor. We may therefore present its meaning as a function whose result includes either an ordinary state or a trapped state. Other machine models may use a relation and admit several successors: an underspecified value, an external event, or a concurrent action can make more than one next state possible. Calling both accounts semantics does not make their proof obligations identical.

Deterministic instruction semantics

An instruction semantics is deterministic when one instruction instance and one admitted input state never relate to two different successor states. Existence is separate: a halted state may have no instruction successor, and a partial model may leave some forms outside its domain.

Determinism is useful because a witness state can be replayed without making another choice. It does not imply termination of a program, successful execution, or simplicity of an instruction. A deterministic load may trap on one state and succeed on another. A deterministic variable-length decoder may need several bytes before it selects its one answer.

The separation matters even when the printed text looks familiar. Two assemblers may accept different spellings for the same decoded form. One sequence of bytes may be illegal even though a nearby legal sequence prints a recognizable mnemonic. A decoder can correctly identify an instruction whose transition function is wrong. The book will test each boundary with a different kind of control.

Consider the Ao instance `add r0, r1, r2`. It reads $R[r_1]$ and $R[r_2]$. It adds them modulo 2^w, writes the result to $R[r_0]$, sets Z, N, C, V by the Ao addition rules, and advances pc by four modulo 2^w. Other registers and data memory remain unchanged. Because the instruction has already been decoded, no data-range trap belongs to this particular operation.

Take width eight, $R[r_1] = 255$, and $R[r_2] = 1$. The result in r_0 is 0. The zero bit Z is one, the negative bit N is zero, and the carry bit C is one because the mathematical sum reached 256. Signed overflow V is zero: the signed inputs are -1 and 1, whose mathematical sum is representable. If $pc = 12$, the next sequential value is 16.

One complete A0 addition

The input facts are width eight, $R[r_1] = 255$, $R[r_2] = 1$, $pc = 12$, and a running outcome. The instruction reads r_1 and r_2. It writes 0 to r_0, writes $(Z, N, C, V) = (1, 0, 1, 0)$, advances pc to 16, preserves all other registers and data memory, and leaves the outcome running. Removing any one of those clauses produces an incomplete transition.

Preservation clauses deserve to be explicit. If a rule lists only the fields that change, a reader may infer that every unlisted field stays fixed. An executable semantics must enforce that inference. Otherwise an implementation could change r_7 while still satisfying a postcondition that mentions only r_0 and the flags.

Table 3.1: The four Ao operand forms and the work each form contributes.

Form	Value source	State access	Extra rule
register	named register	read or write	width w
immediate	instruction byte	no register read	sign extension
memory	base plus offset	register and bytes	range check
relative target	sequential pc plus offset	program counter	alignment and code range

This worked step shows why a destination value is not the whole result. A rule that writes 0 to r_0 but leaves $C = 0$ has performed a different Ao transition. So has a rule that leaves $pc = 12$.

3.2 What an operand does

An operand is often introduced as a place where an instruction gets a value. That account is too narrow. A destination operand names where a value goes. A memory operand also triggers address formation and can trap. A relative target contributes to the next program counter rather than to arithmetic data.

Operand form

An Ao **operand form** states how an operand is obtained, whether the instruction reads or writes it, its width, and any state effect required to use it. Ao has register operands, signed immediate values, base-plus-offset memory locations, and relative branch targets.

Object `A0.def.operand` records these four forms. An Ao immediate comes from the signed eight-bit immediate field and is sign-extended to w bits. A memory address adds that same signed offset to a base register. A relative target adds four times the signed offset to the sequential address $pc + 4$. The multiplication keeps branch targets on four-byte instruction boundaries.

The written operands need not list every state component the operation uses. The Ao `add` instance prints three registers yet also writes four condition bits and the program counter. These are **implicit effects**: architectural reads or writes fixed by the operation even though they are not printed as operands.

The distinction between a printed operand and a value role prevents another shortcut. A destination may also be a source. An address operand may read a base register, read an offset from the instruction, access memory, and expose a trap. A branch target may consume the current program counter even when the assembly does not print it. Counting commas does not count semantic inputs.

Explicit effects

A semantic rule must expose every architectural read, write, program-counter change, and exceptional outcome used by its instruction form.

Object `A0.prin.explicit-effects` records this rule. It is a discipline for the semantics, not a requirement that assembly syntax spell out every effect.

3.2.1 Well-formed operands are checked before arithmetic

An operation name does not accept every operand tuple. A register field must name an available register. A form may require distinct roles, a particular width, or an immediate that fits its encoded field. A memory form may permit only certain base and offset combinations. These restrictions define the set of legal instruction instances.

Keeping that check separate from execution gives illegal forms an honest home. If reserved bytes decode to no instruction, the failure belongs to decoding. If an external API tries to construct an operand tuple that no Ao encoding can denote, the typed constructor should reject it before the step function runs. The execution semantics need not invent behavior for an object the instruction set excludes.

Legality precedes effect

First establish that the bytes select one admitted instruction instance. Then apply the transition for that instance to state. A semantic equation for a legal addition says nothing about a reserved encoding that resembles it.

This distinction matters in testing. Random instruction bytes exercise the decoder's acceptance partition. Random legal instruction instances exercise the transition rules. Generating the second set by decoding the first is useful, but it can miss a legal form that the decoder never emits. A complete test plan checks the two domains and their connection. Neither collection can establish coverage of the other merely by producing many successful examples.

3.2.2 Read old values before committing new ones

An instruction may name the same location in more than one role. In `add r0,r0,r1`, the old value of r_0 is a source and r_0 is also the destination. The semantics must read the source before replacing the destination. Otherwise the result would depend on the order in which a prose description happened to mention its effects.

A clean rule takes a logical snapshot of every required input from the old state. It evaluates register sources, immediate values, addresses, and any required memory

bytes against that state. Only after those operations succeed does it commit the destination, implicit state, next program counter, and outcome together.

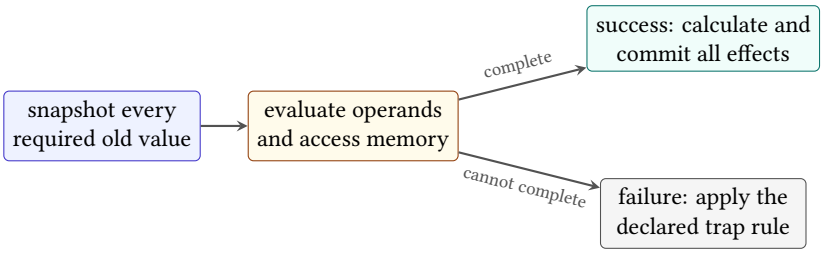


Figure 3.2: Operand evaluation uses the old state. The instruction commits its ordinary effects only after every required operand operation succeeds.

This snapshot is semantic, not a demand that hardware copy the full state. An implementation may forward a result, rename a register, or calculate several pieces in parallel. It implements the rule when its architectural successor is the same as if the inputs came from the old state and the effects were committed according to the declared outcome.

Memory makes failure ordering visible. Suppose an instruction reads a memory source and writes a register destination. If the address or range check fails, the destination write does not occur under the Ao rule. A faulty implementation that clears the destination first and discovers the bad address second has created a successor the semantics excludes.

Prose order is not effect order

The sentence "write the destination and advance the program counter" does not license an implementation to expose the first write when a later required operation fails. A semantic rule must state which effects commit together and which state accompanies each failure.

For instructions with two memory operands or implicit stack effects, the same issue becomes more demanding. The rule must identify every access, their ordering, and the state retained on failure. Ao avoids those forms in its first instruction set, but the method is already in place for the selected x86-64 cases that combine more work in one instruction.

3.3 One step

A running machine does not execute a mnemonic in isolation. It uses the program counter to fetch instruction bytes, decodes those bytes, and applies the decoded instruction's transition.

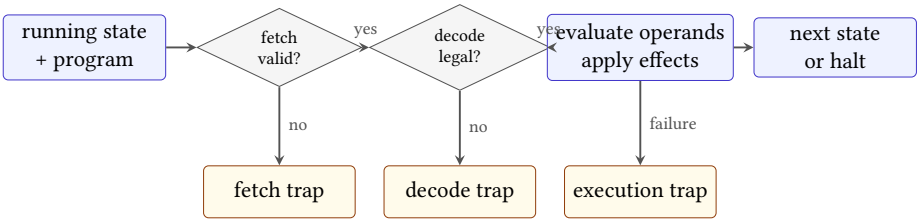


Figure 3.3: One Ao step keeps fetch failure, decode failure, and execution failure distinct. Only the successful path reaches the ordinary next state.

A0 single step

An Ao **single step** takes a running state and a program. It checks the program counter and fetch range, decodes one four-byte instruction, and applies that instruction's complete transition. A fetch or decoding failure produces a trapped state. A halted or trapped state has no successor.

Figure 3.3 is ordered. The machine cannot evaluate an operand from an instruction that did not decode. It cannot decode bytes that were not fetched from a valid code range. If operand evaluation traps, later writes in the instruction do not occur unless the instruction rule explicitly says otherwise. This order is part of the semantics, not an implementation optimization.

Three failures, three debugging questions

A fetch trap asks whether the program counter names available instruction bytes. A decode trap asks whether those bytes form one legal Ao encoding. An execution trap asks whether a legal decoded instruction can complete on this state. Collapsing all three into "bad instruction" hides which rule failed.

Object `A0.def.step` records this relation. The definition names decoding before Chapter 6 teaches the encoding because execution needs the boundary to exist. For hand traces in Part I, we will print both the four bytes and their decoded form so the decoding step is visible without becoming the lesson.

The ordinary next address is $pc + 4$ modulo 2^w. A branch or jump may replace it with a relative target. A trap records its reason and prevents a later instruction from running. These three cases are part of the transition, not annotations added after the register calculation.

Two simple attacks test whether a proposed step rule has hidden omissions. First, change the expected carry bit while leaving the destination correct. A complete-state check must reject the step. Second, leave the program counter

unchanged after a normal `add`. A checker that looks only at r_0 will miss the error; a checker for the whole Ao transition must not.

3.3.1 Read and write footprints summarize dependence

The **read footprint** of an instruction on a state is the architectural information needed to determine its successor. The **write footprint** is the set of components that may differ in that successor. For `add r0, r1, r2`, the read footprint contains the two source registers, the program counter, and enough outcome state to require a running input. Its write footprint contains r_0, the four condition bits, and the program counter.

Footprints are more exact than lists of printed operands and less cumbersome than repeating the complete transition. They expose data dependence: a later instruction depends on an earlier one when it reads a component the earlier one may write. They also support frame reasoning: everything outside the write footprint must retain its old value unless the failure rule says otherwise.

The footprint can depend on the state. A memory operand's byte addresses come from register values. A conditional branch reads the predicate needed to choose one path, while the successor address depends on that choice. For a conservative static analysis, one may use a larger footprint that contains every possible address or component. That is safe for detecting possible dependence, but it may report interactions that no admitted state can realize.

Equal reads give equal deterministic effects

Fix one deterministic instruction instance. If two admitted input states agree on its complete read footprint, operand evaluation receives the same values and makes the same success or failure decisions. The instruction then calculates equal values for its write footprint. Components outside that footprint retain their respective old values. The full successors are equal only if the old states also agree on those preserved components.

The last sentence matters. Agreement on what an instruction reads is enough to make its newly calculated effects agree. It is not enough to make complete successor states equal when unrelated old state already differed. Chapter 11 will turn that distinction into observations and state relations.

3.4 Three value roles, two printed operands

The two companion architectures make operand roles visible in different ways. A later RV64 integer addition will use a form such as `add rd, rs1, rs2`. The destination name is separate from the two source names. The selected base form does not write an x86-style condition-code register.

A later x86-64 register addition will use a two-operand form such as `add rax, rbx`. The destination `rax` is also the first source. The instruction reads the old values of `rax` and `rbx`, writes the sum to `rax`, and changes selected status flags.

One spelling therefore names three register roles with three operands. The other names three value roles with two operands and adds implicit condition writes. This is a concrete comparison of operand and state design. Calling one RISC and the other CISC may place the examples in historical families; it does not supply either transition rule.

Here is the reader argument that keeps the extra state honest. Suppose two instructions start from the same state and both leave 2 in their destination. One leaves $Z = 0$, while the other leaves $Z = 1$. Their output states differ in the Z component. They may agree under an observation of the destination alone, but they are not equal state transformers. A following branch that reads Z can expose the difference.

The comparison also warns against translating syntax directly. To relate the two additions, we must first decide where the three input and output values live, then account for every extra state component. A destination-only observation may permit a relation that a flag-sensitive context would reject. The choice belongs in the claim, not in an informal declaration that both lines are additions.

Instruction forms also differ in where they place constants and addresses. An RV64 immediate form may carry a constant in fixed instruction fields and write a destination distinct from its source. An x86-64 form may allow an immediate or memory operand in a position whose other role can be a register. These are encoding and operand choices. The arithmetic relation can still be the same after widths, source values, destinations, and extra effects are aligned.

Implicit operands deserve the same treatment as implicit conditions. A stack operation may read and write a stack pointer that the assembly does not print. A procedure-return instruction may obtain its target from a fixed register or memory location. Some multiply and divide forms use architecturally selected register pairs. These choices can make a short line depend on substantial state.

Fewer printed operands do not mean fewer inputs

Assembly syntax is a notation optimized for programmers who know the architecture's conventions. It may omit a fixed register, a condition-code write, a stack update, or the ordinary next address. Semantic comparison must restore those facts before counting sources, destinations, or effects.

Read sets before arithmetic

For each of these Ao forms—`add r0,r1,r2`, `movi r0,-1`, `load r0,[r1+4]`, and `branch.eq +2`—list the complete read set and write set before calculating any values. Include implicit state and possible outcomes. Which two forms can trap after successful decoding?

3.5 Instructions in sequence

If instruction i takes S_0 to a running state S_1, and instruction j takes S_1 to S_2, then the sequence $i; j$ has taken S_0 to S_2. This is ordinary function composition with an outcome check between the functions.

If i halts or traps, there is no S_1 on which j may run. A model that applies j anyway has not found an unusual execution of Ao. It has defined another machine. This small point will become important when bounded execution distinguishes a stopped trace from one that merely reached its step limit.

Memory operands add a second source of interruption. Before an arithmetic operation can use a memory value, the machine must form the address and complete the load. If that access traps, the main arithmetic write does not occur. The order of these effects belongs in the instruction rule.

Composition needs a running middle state

Assume instruction i takes S_0 to a running state S_1, and instruction j takes S_1 to S_2. Then the two-instruction sequence is defined by applying j to the output of i. If i instead returns a halted or trapped state, the premise needed to apply j is false. The sequence stops with the first outcome; it does not possess a hidden second transition.

The result is a partial form of composition. Each decoded instruction has a total rule over the running states admitted by its operands: it returns a next state or a trapped state. Program sequencing adds an outcome guard. Later trace definitions will make that guard visible at every adjacent pair.

When all required middle states are running, deterministic composition is associative. Grouping $(i; j); k$ or $i; (j; k)$ produces the same successive states because both groupings apply i, then j, then k. Parentheses organize the argument; they do not change the instruction order. The outcome guard must still be checked after each step.

Footprints explain when order can change. Two instructions with disjoint write footprints are not automatically interchangeable: one may write a component the other reads, both may trap under different premises, or their program-counter effects may select different code. For straight-line local reasoning, a sufficient

commutation argument must show that neither write meets the other's reads or writes and that both outcome conditions remain the same in either order.

These footprint intersections are the semantic root of familiar instruction hazards. A read-after-write dependence carries a value forward. A write-after-read dependence can make reordering destroy an old value before it is consumed. A write-after-write dependence makes final state depend on which write comes last. Hardware may execute internally in another schedule, but retirement must preserve the architectural result required by the machine model.

The statement is deliberately broader than registers. Condition bits, memory bytes, program counters, and outcomes can create the same dependencies. An optimizer that sees only named registers may move an addition across a flag-reading branch or move a load across an overlapping store. The assembly still looks plausible. The state transition has changed.

3.6 The implementation boundary

An Axeyum implementation must join the state, decoder, operand rules, and instruction transitions into one reusable Ao step. Object `OP.a0.step` records that obligation. Its controls must include the wrong destination, a missing condition write, a wrong sequential program counter, a wrong branch target, reserved encoding fields, and attempted execution after halt or trap.

The corresponding introductory Axeyum route should begin with execution, not with a solver. A reader should construct one width-eight state and one decoded `add`, run the public step function, and inspect the complete output. Only then should a relation ask whether a proposed shortcut agrees for every allowed input. A returned counterexample must replay through that same step function. A no-counterexample result must name the formula and checking route before it can support a machine-proof claim.

The public data model should keep the layers in Figure 3.1 separate. A decoded instance contains an operation and typed operands, not a closure supplied by Python. The Rust semantics matches that closed instruction type and returns a complete state result. Serialization records enough information to reconstruct both objects without accepting a printed mnemonic as authority.

One useful introductory display is a state diff paired with the full-state record. The diff teaches which components changed. The full record lets the checker establish preservation of everything else. If the artifact stores only the diff, a missing illicit write is indistinguishable from a preserved component.

Controls should attack one layer at a time. Change bytes while retaining the printed instance. Change an operand register while retaining the operation. Change one implicit condition while retaining the destination. Corrupt the next program counter while retaining all arithmetic. A route that rejects only the last mutation has not checked decoding or operand binding.

A useful first mutation

Change only the sequential update from $pc + 4$ to $pc + 1$. A test at $pc = 0$ catches the immediate error. A stronger route quantifies over every allowed running state and shows that destination arithmetic cannot conceal the wrong next address. The mutation is useful because it preserves the tempting part of the result.

The implementation does not accompany this draft. The worked addition is a reader execution of the printed Ao definition, not evidence that a general step function has run over every state. When the package exists, its output must agree with these rules and its controls must fail for the stated reasons.

Registers and immediates keep an instruction's values close at hand. A memory operand does not. It turns one step into a claim about addresses, byte order, valid ranges, overlap, and failure. We need to construct that part of the machine before we can safely compose programs that touch it. The next chapter therefore treats a memory access as a checked state movement, not as a bracketed spelling around an address.

3.7 Exercises

1. **Execute.** At width eight, run `add r0, r1, r2` from $R[r_1] = 127$, $R[r_2] = 1$, and $pc = 20$. Compute r_0, Z, N, C, V, and the next pc.

2. **Break.** Alter exactly one component of your answer to the first exercise. State an observation that would miss the error and the smallest observation that would catch it.

3. **Prove.** Show that if a first instruction traps, the two-step Ao sequence has no state produced by the second instruction. Identify the outcome rule on which the argument depends.

4. **Transfer.** Write the value roles for the three-operand RV64 addition pattern and the two-operand x86-64 pattern. Do not compare their complexity; compare what each form reads and writes.

5. For `movi r3, -1`, state where the immediate originates and how it becomes a width-w word. Explain why it is part of the instruction instance but not a register in the input state.

6. Design a negative control for a step implementation that incorrectly advances pc by one byte instead of four. Give the starting pc, expected result, and bad result.

7. **Execute.** Give four legal instruction bytes and a matching decoded Ao `add`. Walk through every box in Figure 3.3, including the checks that succeed.

8. **Break.** Keep the decoded operation and all explicit operands correct, but omit one implicit write. Construct a following instruction that can expose the omission.

9. **Prove.** Show that two state transformers equal on every complete running state remain equal after composition with the same deterministic suffix, provided both produce running middle states.

10. **Transfer.** For one paired addition form, state the weakest observation under which it could refine Ao addition and one surrounding context that requires a stronger observation.

CHAPTER FOUR

Memory

Store the 32-bit word `0x12345678` at address 8. What does address 8 contain afterward? The answer is not the whole word. Ao memory gives each address one byte, so the store must distribute the four bytes across four addresses. To define that one instruction, we need an address, a width, a byte order, a valid range, and a rule for failure.

Memory makes order matter twice. The bytes of one value have an order across addresses, and separate accesses have an order through time.

It also makes absence observable. A register read always names one of Ao's eight registers. A memory access can form a word address for which some needed byte does not exist. The semantics must decide that failure before it can state what the instruction writes.

4.1 A finite map of bytes

A0 data memory

Ao **data memory** is a finite map from w-bit addresses to bytes. A load reads $w/8$ consecutive addresses and joins their bytes in little-endian order. A store splits a w-bit word and writes its bytes to consecutive addresses in that order.

Object `A0.def.memory` records the complete contract. This is data memory, not the immutable program map. An Ao store cannot overwrite instruction bytes in the first model.

4.1.1 Addressable memory separated place from meaning

Early calculating machines had storage, but not every store presented the same abstraction. A value might live in a physical register, a delay line, a drum position, or a location selected by wiring. The stored-program idea made numbered locations serve both instructions and data. Later machines varied the physical technology

while preserving a stable architectural question: what value does a load obtain from a named address?

That question contains two separations. First, an address names a place, not a type. The byte at address 8 does not announce whether it belongs to an integer, a character, a pointer, or an instruction. The operation that reads the byte supplies the width and interpretation. Second, architectural memory is not a photograph of the hardware. Caches, translation structures, buses, and storage cells may participate in an access without appearing as separate components in the instruction-set state.

Ao begins below both companion architectures with a finite byte map. The map is deliberately small enough to draw and complete enough to define loads, stores, overlap, and failure. It has no page table, cache, permission bit, memory-mapped device, or concurrent writer. Those omissions let us establish the sequential byte rules first. Chapter 16 will ask what must change when a later model admits the missing mechanisms.

The durable abstraction is a numbered place

Memory technology has changed repeatedly. The architectural contract survives by naming locations and the effects of accessing them, rather than requiring software to describe the physical device that retains each bit. That abstraction is powerful precisely because it has a boundary: timing and concurrent visibility need additional rules.

An access of $n = w/8$ bytes beginning at address a is valid only when all addresses $a, a+1, \ldots, a+n-1$ belong to the finite data-memory range. The first Ao model permits an access to begin at any address; it imposes no data alignment restriction. That is a design choice, not a claim about either companion architecture.

Address arithmetic still uses words. The machine forms a memory address by adding the sign-extended eight-bit offset to a base register modulo 2^w. It then checks the resulting sequence of byte addresses against the finite range. Wraparound arithmetic does not make an out-of-range byte valid.

Address formation and access validity are different rules

Word arithmetic always produces an effective-address word. A memory access succeeds only when every byte in its declared range exists. The first fact does not imply the second.

The requirement that every byte exist prevents a dangerous half-definition. If a four-byte access begins at a valid address but ends beyond memory, Ao does not improvise a shorter access. The requested width belongs to the instruction form. Either the whole access completes or the state records a trap.

4.1.2 Memory equality is checked one address at a time

Two finite memories are equal when they have the same domain and return the same byte at every address in that domain. Their construction history does not matter. One memory may have reached its present contents through a store; another may have begun with those bytes. If every lookup agrees, the Ao state cannot distinguish them through memory.

This pointwise definition gives memory proofs a reliable shape. To show that two stores produce equal memories, choose an arbitrary address and divide the argument according to whether it lies inside or outside each write footprint. To show that memories differ, one address with unequal bytes is enough. The first task is universal; the second needs a witness.

One byte separates two memories

Let memories M_1 and M_2 share the domain 0 through 15. Suppose they agree everywhere except address 9, where $M_1(9) = 00$ and $M_2(9) = 01$. An eight-bit load at 9 distinguishes them directly. A thirty-two-bit little-endian load at 8 also distinguishes them because the unequal byte receives weight 2^8.

Preservation claims use the same definition. Saying that a store changes only its requested range means that every address outside that range returns its old byte. The phrase *everything else is unchanged* is therefore not a gesture toward the rest of memory. It abbreviates a quantified clause that a checker must test or derive.

4.2 Store, then load

Return to `0x12345678`. Its little-endian split is `78`, `56`, `34`, `12`. A store at address 8 writes `78` at 8, `56` at 9, `34` at 10, and `12` at 11. A load from address 8 joins those four bytes with weights $2^0, 2^8, 2^{16}, 2^{24}$.

The word has not been numerically reversed. The low byte receives the low address. Endianness describes the relation between byte addresses and place values.

4.2.1 Bytes acquire a value through an access rule

Memory stores bytes. A load instruction decides how many adjacent bytes to read and how to place them in its result. The four locations in Figure 4.2 therefore support several valid readings. An eight-bit load uses one byte. A sixteen-bit load uses the first two. A thirty-two-bit load uses all four. None changes the stored bytes.

The destination width creates a further question. A narrow load into a wider register must say what fills the upper bits. Zero extension fills them with zero. Sign

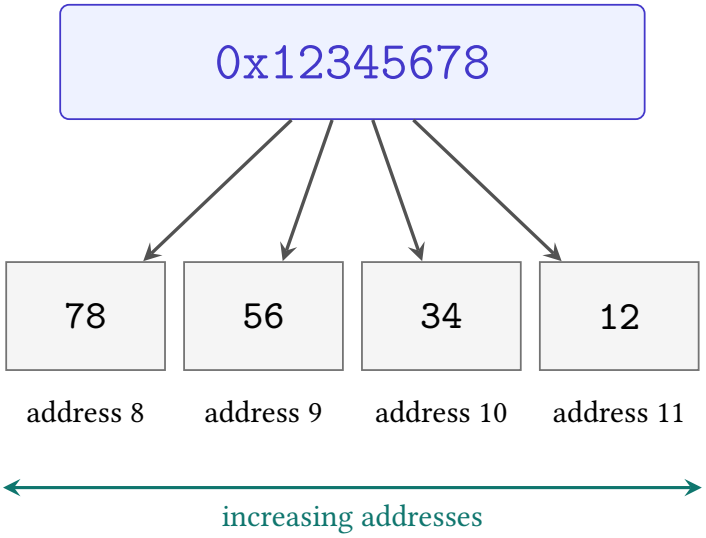


Figure 4.1: A little-endian store places the low-weight byte at the lowest address. The word's printed hexadecimal order and memory's address order run in opposite visual directions here.

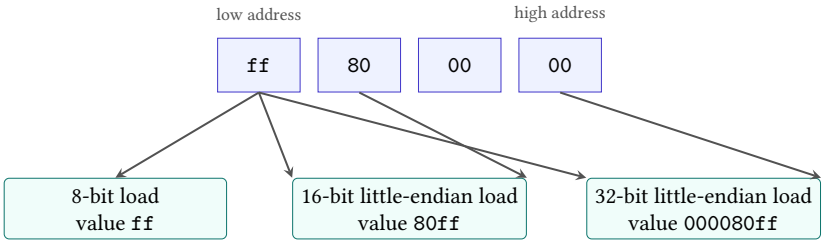


Figure 4.2: A byte sequence does not carry one compulsory width. The selected load form determines the range and the little-endian place values.

extension repeats the narrow value's high bit. Loading the byte `ff` into a thirty-two-bit destination can therefore produce `000000ff` or `ffffffff`. Endianness alone chooses neither result.

Load interpretation

A load interpretation consists of a starting address, a byte count, a byte order, and a rule for placing the assembled value in the destination. The last rule includes any zero extension, sign extension, truncation, or preservation of destination bits.

This definition prevents a common shortcut. Saying that a machine is little-endian settles the weights of bytes within a multi-byte value. It does not settle the access width, destination width, signed extension, alignment, or failure behavior. Two instruction forms can read the same first byte and still produce different register values without disagreeing about byte order.

Stores reverse the construction but have their own width. A byte store writes one low byte even when its source register is wider. A word store writes the declared sequence of bytes. Unwritten neighboring bytes retain their old values. This preservation fact is the memory version of the frame conditions from Chapter 2, and later proofs will rely on it as heavily as on the bytes that change.

The round-trip proof now has a machine premise. Let x be a w-bit word and suppose every address in the $w/8$-byte range beginning at a is valid. The store writes

$$b_i = (x \operatorname{div} 2^{8i}) \bmod 2^8$$

at address $a + i$. The following load reads that same b_i and computes

$$\sum_{i=0}^{w/8-1} b_i 2^{8i}.$$

Chapter 1 showed that these disjoint byte positions join to x. No other memory byte enters the sum. The load therefore returns x.

Store then load

The proof has three premises: the store and load use the same width, the same starting address, and a wholly valid range. The store writes byte b_i at $a + i$. The load reads that same location and multiplies the byte by 2^{8i}. Chapter 1's split/join identity reconstructs x. If any premise changes, the conclusion needs a new argument.

Notice what the proof does not require. It does not require the starting address to be a multiple of four, because the first Ao memory model permits unaligned

data access. It does not require unrelated bytes to have any particular value. It does require the store to be atomic on failure and no intervening write to change the loaded range.

The valid-range premise is load-bearing. Suppose a four-byte store begins at the last address of memory. Only its first target address exists. Ao traps and leaves data memory unchanged; it does not perform a one-byte fragment of the store. A later load cannot appeal to the round-trip argument because the required store never completed.

Partial failure changes the theorem

A faulty implementation might write each valid byte and trap only when it reaches the missing address. That route changes memory before reporting failure. It does not implement Ao's atomic store rule, and a later observation of the valid prefix can expose the difference.

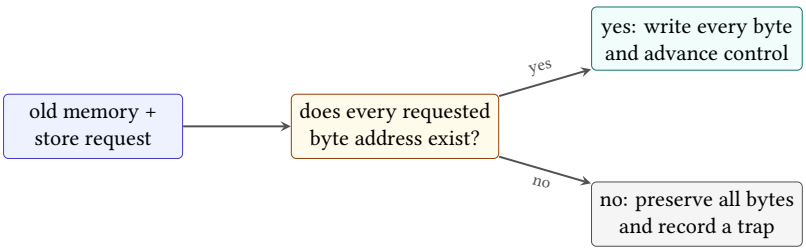


Figure 4.3: Ao checks the complete store range before selecting either a full memory update or a trapped state with memory preserved. There is no partial commit between them.

Atomic here describes one transition in the sequential Ao semantics. It means that a failing store has no partly updated successor. It does not claim the stronger hardware property that another processor can never observe pieces of a successful multi-byte store. Ao has no second processor or concurrent observer. Reusing the word outside that scope would quietly add a theorem the model cannot even state.

The success case has a frame rule. Let A be the set of addresses written by a valid store, let M be the old memory, and let M' be the result. For every address d outside A,

$$M'(d) = M(d).$$

Inside A, the corresponding source byte determines $M'(d)$. Together, the inside and outside clauses define the entire new finite map. A checker that compares only the requested range verifies the first clause and misses corruption elsewhere.

The trap case has an even stronger frame rule: every memory address retains its old byte. Other state changes also require an explicit decision. Ao records

the trapped outcome and does not advance into another instruction. The source and destination registers remain as the step rule states. A real architecture may record exception metadata in additional state; that state is outside Ao rather than implicitly preserved by it.

A control that reverses the weights

A faulty load joins the bytes from addresses 8 through 11 with address 8 at weight 2^{24} instead of 2^{0}. It returns `0x78563412`, not `0x12345678`. The stored bytes are individually correct. The relation between address and weight is wrong.

4.3 Effective addresses and range

For the Ao operand $[r_1 - 4]$, the base is the word $R[r_1]$, and the offset is the sign extension of the immediate byte `fc`. The effective address is their width-w sum. Only after forming that word does the machine check whether the whole access range exists.

This order separates two ideas that are easy to blur. Word arithmetic always produces a word, even when it wraps. Memory validity is a predicate on the resulting addresses. If an eight-bit base contains 2 and the offset is -4, the effective address is 254. A four-byte access still traps in a memory whose valid range is 0 through 63.

The address expression also does not identify a fixed location by itself. Changing $R[r_1]$ changes the address denoted by $[r_1 - 4]$. Operand syntax and state work together to select bytes.

The valid range may itself wrap in word arithmetic. A width-eight access of four bytes starting at 254 names the word addresses 254, 255, 0, and 1 if we apply modular addition to each address. Ao does not infer validity from that cycle. Its finite memory domain must contain every address the access rule names. This makes the finite map, rather than the maximum word value, the authority for presence.

Nor must a finite domain be one uninterrupted interval. A model may contain addresses 0 through 31 and 48 through 63 while leaving a hole between them. A four-byte access beginning at 30 then finds its first two bytes and misses its last two. Checking that 30 and 33 fit within the minimum and maximum domain addresses would accept the access incorrectly. The semantics asks membership for each requested address.

That generality is useful even when an implementation later chooses a dense array. The mathematical interface describes which addresses exist. A dense representation can add a lemma that its bounds check is equivalent to per-address membership. A sparse representation can check the map directly. The theorem

belongs to the interface; the faster check earns its place by establishing equivalence to that theorem.

Form first, check second

At width eight, evaluate base 250 with offsets 5, 6, and 10. For each result, state the effective-address word and then test a four-byte access against a memory domain containing addresses 0 through 255. Repeat with a domain containing only addresses 0 through 63.

4.3.1 A range is a set of byte addresses, not two unchecked endpoints

For a non-wrapping access, it is convenient to write the requested range as the right-exclusive interval $[a, a + n)$. It contains a through $a + n - 1$ and contains exactly n addresses. Right-exclusive intervals compose cleanly: an access ending at b and one beginning at b are adjacent rather than overlapping.

The notation must not hide word wrap. If $a + n$ is calculated modulo 2^w, the two endpoints alone no longer reveal whether the access crossed the maximum word. A valid-range checker should enumerate the declared byte positions or use an arithmetic condition that detects overflow before relying on an interval comparison. Testing only that the first and last words belong to a domain can also fail when the finite domain has a hole between them.

Alignment adds an independent predicate. A rule may require $a \bmod n = 0$, a weaker alignment, or none. Passing that test does not make the range present. Failing it may trap even when every byte exists. Thus a complete access rule has at least three stages: form the effective-address word, check address-policy predicates such as alignment, and check every byte needed by the transfer.

This separation becomes useful when relating machines. Ao permits an unaligned access that a selected real-ISA environment may reject. The real operation cannot refine the Ao operation on all Ao states without either restricting the input relation to aligned addresses or representing the additional failure in the abstract model. Matching successful examples does not repair the missing case.

4.4 When two accesses meet

Two memory accesses **alias** when their byte ranges overlap in the same state. Equality of their starting addresses is sufficient but not necessary. A four-byte access at a and another at $a + 2$ share addresses $a + 2$ and $a + 3$.

Let the first store write `0x11111111` at a, and let the second write `0x22222222` at $a + 2$. In that order, the bytes at a through $a + 5$ end as

```
11  11  22  22  22  22.
```

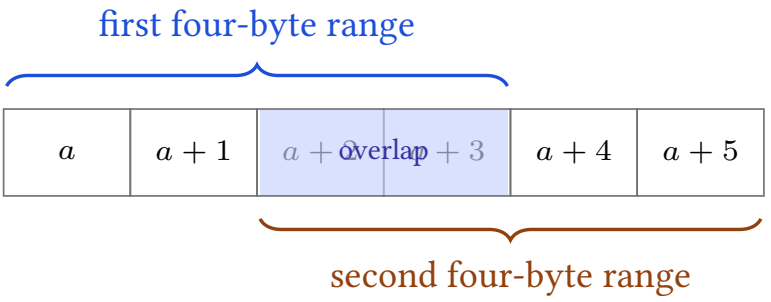


Figure 4.4: Equal starting addresses are not required for aliasing. These four-byte ranges share two addresses.

Reverse the stores and the middle two bytes become `11`. The final memory differs even though each store, considered alone, writes the same source word.

Disjoint stores have a simpler argument. If their address ranges share no byte, the first store changes no address read or written by the second, and the second changes no address written by the first. At every address, either one designated store supplies the final byte or the original byte remains. The final data memories are therefore equal after either order, provided both accesses are valid and neither traps.

Overlap breaks the disjointness step. A proof that exchanges memory operations must establish non-aliasing, account for the shared bytes, or accept a narrower claim. A register identity alone supplies none of these.

Why disjoint stores commute

Choose any address d. If d lies in the first range, disjointness says the second store does not change it; the first store supplies the same final byte in either order. The symmetric case handles the second range. If d lies in neither range, both stores preserve its original byte. The cases cover the memory domain, so the final maps are equal. Validity is required so that both orders perform both complete stores.

The address-by-address proof explains exactly why the theorem stops at overlap. An address in both ranges has two possible final writers, and their order decides which one wins. We may recover commutation when the overlapping bytes happen to be equal, but that is a different premise and a narrower theorem.

4.4.1 Loads turn aliasing into a data dependence

Place a load beside a store. If their ranges are disjoint and both operations succeed, moving the load before the store preserves the loaded value. The store changes no byte read by the load. If the ranges overlap, the loaded value may contain old bytes in one order and new bytes in the other. This is a memory dependence even when the instructions name different starting addresses or use different widths.

The proof again works address by address. For every byte position read by the load, non-aliasing says the store preserves that memory entry. The join rule therefore receives the same byte at every weight in both executions. Equal input bytes give an equal loaded word. The argument also needs the same outcome in both orders: moving an access across a trap can change whether the other access occurs at all.

Aliasing is state-dependent when addresses come from registers. The printed operands $[r_1]$ and $[r_2+4]$ neither guarantee overlap nor rule it out. A compiler may establish non-aliasing from an input contract, an allocation rule, or arithmetic facts about the registers. Without such evidence, treating different syntax as different memory is wishful typesetting.

Widths make the question asymmetric. A one-byte load inside a four-byte store range aliases the store, while a neighboring one-byte load may not. A proof must compare byte footprints rather than the source-language types or the number of registers written. This is why later instruction semantics will record read and write footprints separately.

Memory dependence follows byte footprints

Two accesses interact when the bytes read or written by one meet the bytes written by the other. Mnemonics, operand spellings, and unequal starting addresses are not enough to decide the question.

4.5 Memory in the two companion machines

The later RV64 slice will use distinct load and store forms to move values between integer registers and memory. Arithmetic in the selected base forms uses registers. An effective address commonly combines one base register with an immediate.

The later x86-64 slice will include selected arithmetic forms that name a memory source or destination. Such a form can combine address calculation, memory access, and arithmetic in one decoded instruction. Its semantic rule must still expose the address, bytes, value width, state effects, and possible exception.

This comparison holds one question fixed: which included instruction forms may access memory? It does not say that every instruction in one architecture has one character or that a shorter assembly spelling has fewer semantic obligations.

Table 4.1: Memory questions that syntax alone cannot answer.

Question	Required semantic fact
Where?	effective-address calculation and width
How much?	access width and byte range
Which value?	byte order and extension or truncation
What changes?	destination, memory, program counter, and other state
What can fail?	alignment, range, permission, or selected architectural outcome

For either slice, a usable memory rule must answer the same checklist. It must name the address inputs, byte range, value construction, alignment premise, successful state change, and failure outcome. The answers may differ. The questions do not.

Both real architectures live inside systems that add translation and access policy. A program commonly supplies a virtual address. Hardware and operating system state determine whether that address names storage, whether the access is permitted, and which exception follows a failure. Device mappings may give a read or write effects unlike ordinary memory. None of this turns the byte order question into a page-table question; it adds layers around the transfer.

The book's selected scalar slices will make those layers explicit by premise or exclusion. A theorem about an ordinary mapped region should say so. A decoder theorem need not model translation because it concerns bytes already supplied to decoding. A program theorem that loads from memory cannot borrow that simplification without naming how the bytes become available.

This discipline prevents an architectural manual from doing more work than it actually does. The manual specifies instruction behavior and exceptions in a declared mode. An application binary interface may add stack and object-layout conventions. An operating system supplies mappings and policies. A language implementation adds rules about valid objects and undefined behavior. A proof crossing these layers needs a relation between them, not a single citation to the lowest one.

4.6 The implementation boundary

The Axeyum state and memory package must implement the finite map, valid-range check, atomic load and store outcomes, little-endian split and join, wrapped effective-address calculation, and observations of changed and unchanged bytes. Object `OP.a0.state-memory` records that obligation.

Its negative controls are already suggested by the chapter. Reverse the byte weights. Permit a range whose last byte is absent. Partly modify memory before a

trapped store. Declare two overlapping ranges disjoint. Each mutation must be rejected by the corresponding check.

The introductory Axeyum route should make memory visible rather than burying it in a solver formula. A reader constructs a finite memory map, performs one store, inspects the changed bytes, and loads the word back. The route then asks for a counterexample to the round trip under the printed validity premise. Any model returned by the solver must replay through those executable store and load functions.

The controls should travel through the same route. Reversing the join weights must yield a concrete unequal-byte witness. Removing the last address from the domain must produce a trap and unchanged memory. A partial-write mutation must fail an observation of the valid prefix. These checks test different boundaries; one passing control cannot stand in for the others.

The artifact for one memory step should bind the complete old state, decoded instruction, effective address, requested footprint, outcome, and complete new state. A compact list of changed bytes is useful for display, but the checker must reconstruct the full successor and confirm the frame rule. Otherwise an artifact can omit an illicit change and still look like a plausible diff.

At the Python layer, the introductory exercise should expose values a reader can inspect without duplicating semantics. The reader supplies bytes and an initial state, calls the bound Rust implementation, and receives a structured step record. Python may print the address calculation and memory table. It must not recompute the successor with a second informal implementation and call agreement with itself a check.

The first solver question can stay finite and sharp: for every width-eight word and every valid four-byte starting address in a declared finite domain, does storing and then loading return the word while preserving every byte outside the range? A counterexample must replay through executable memory operations. A no-counterexample result needs an evidence route that covers the whole stated domain. Until that package exists in Axeyum, the object records an implementation obligation rather than a machine-backed result.

> **Why a memory formula is not yet memory semantics**
>
> A formula that expands four byte variables can establish a relation among those variables. It does not by itself establish address formation, finite range checking, atomic failure, or preservation of unrelated bytes. The Ao memory package must connect all of those rules before later instruction proofs may depend on it.

The store/load result in this chapter is a reader proof under a printed premise. It does not stand in for the absent executable memory model. Once that model

exists, the round trip and each bad variant must run through the same memory semantics used by later instruction proofs.

Memory has made sequence visible. Two individually valid stores can leave different final states when their order changes. A list of instructions still does not tell us which one runs next. For that we need a program counter, a program, and a trace through both.

4.7 Exercises

1. **Execute.** Store `0x89abcdef` at address 4 in little-endian order. List the four writes and reconstruct the loaded word by hand.

2. **Break.** Reverse the byte weights in the load rule. Give the smallest two-byte word for which the faulty round trip differs from the original, and calculate both results.

3. **Prove.** Prove that two valid stores to disjoint byte ranges commute. State exactly where the argument fails for overlapping ranges.

4. **Transfer.** For one later RV64 load form and one selected x86-64 memory-source form, list the address inputs, value width, destination, and possible exceptional outcome that their pinned semantics must supply.

5. In eight-bit address arithmetic, compute the effective address for base 2 and offset -4. Explain separately why the word result exists and why a four-byte access may still be invalid.

6. Design an atomicity control for a four-byte store whose final address is outside memory. State which bytes a faulty partial store would change and what Ao requires instead.

7. **Execute.** Draw the two ranges in Figure 4.4. Perform the two example stores in both orders and list the final byte at every address from a through $a + 5$.

8. **Break.** Change Ao so a failing store writes its valid prefix. Give a pre-state and an observation that distinguish the faulty result from the required atomic trap.

9. **Prove.** Strengthen the disjoint-store argument to one load and one store: state a non-aliasing premise under which moving the load before the store preserves the loaded value.

10. **Transfer.** Explain why a real-ISA memory form needs both a source-pinned address rule and a source-pinned failure model before it can refine the Ao access, even when a sample address calculation matches.

CHAPTER FIVE

Programs and Control

Write two instructions on adjacent lines. Why should the second run after the first? Position on the page is an assembly convention. The machine follows its program counter. An ordinary instruction advances that counter. A branch may replace the ordinary next address. A trap or halt permits no next step at all.

Control turns a list of instructions into a path through states. That path, not the layout of the listing, is the execution we will reason about.

A program listing is not an execution

A listing arranges instructions for a reader. An execution follows program counter values through complete states. Sequential layout matters only because the instruction semantics assigns it an address rule.

5.1 A program supplies the bytes

A0 program

An Ao **program** is a finite immutable map of instruction bytes, together with an entry program counter and a declared code-address range. Every Ao instruction occupies four bytes and begins at an address divisible by four.

Object `A0.def.program` records this object. The data memory in machine state and the program's code map are separate. The first Ao model cannot change its own instructions by storing to data memory.

The program counter selects four bytes from the code map. A normal non-control instruction advances pc by four modulo 2^w. A fetch traps if pc is not divisible by four or if the four-byte range is incomplete. The fact that another instruction happens to be printed below the current one does not repair either failure.

The code range also prevents a valid four-byte pattern from becoming an instruction merely because the bytes exist somewhere. A fetch must begin at a

declared code address, satisfy the four-byte alignment rule, and find all four bytes. Decoding begins only after those conditions hold.

For now, a hand trace will show both bytes and decoded instructions. Chapter 6 will explain how the decoder obtains the instruction instance and why illegal encodings are part of program behavior.

5.1.1 Stored programs made control a data-dependent process

Before stored programs, changing a machine's task could mean rewiring, reconfiguring switches, or replacing an external sequence of control media. Placing instructions in addressable storage made the next operation depend on a small part of machine state: the program counter. Conditional transfer then made that counter depend on values produced during the calculation.

The historical gain was not merely convenience. A finite body of code could repeat, select among cases, and call reusable routines. The same bytes could process inputs of many sizes because the path and number of visits were chosen while the program ran. A short program became a description of a potentially long computation.

That foundation remains visible today beneath compilers, operating systems, interpreters, and virtual machines. Optimizers rearrange control while trying to preserve observed behavior. Security mechanisms restrict indirect targets. Debuggers reconstruct paths from partial evidence. Formal verification asks whether every path from an allowed entry respects a contract. Each task begins with the same modest question: which successor states does the program admit?

A finite description can govern unbounded time

A program map is finite, but execution may revisit its addresses without a fixed limit. The mystery is not an infinite program hidden in memory. It is iteration: a finite transition rule applied again to each new state.

5.2 A trace through states

A0 execution trace

An Ao **execution trace** is a finite sequence

$$S_0, S_1, \ldots, S_k$$

in which each adjacent pair is linked by one Ao step. A bounded runner reports whether the last state halted, trapped, exhausted the step bound, or remains able to continue.

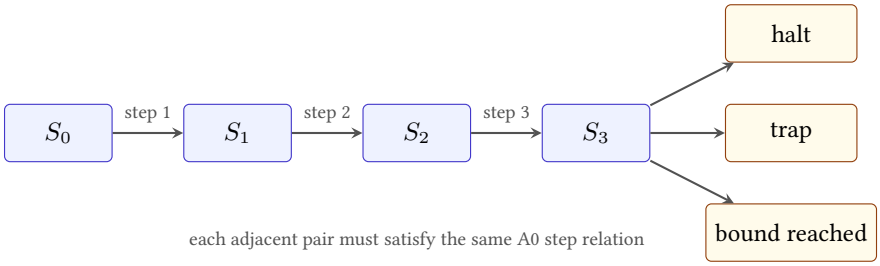


Figure 5.1: A bounded trace is a sequence of checked adjacent steps. Reaching a bound is distinct from reaching a terminal machine outcome.

The diagram's three exits do not mean that one state has three outcomes. A particular run produces one result. The fan-out records the cases the runner must distinguish. If the last state is running when the requested bound is used up, the runner reports that fact without changing the machine outcome to halted or trapped.

Object `A0.def.trace` records this relation. A trace contains its initial state. A three-instruction execution that finishes normally therefore contains four states: the state before the first step and one state after each step.

Consider this width-eight decoded program, with instructions at addresses 0, 4, 8, and 12:

```
movi r1, 5
movi r2, 3
add  r0, r1, r2
halt
```

Listing 5.1: A short A0 program

Begin with $pc = 0$, a running outcome, and arbitrary old values in the three registers. The first step writes 5 to r_1 and sets $pc = 4$. The second writes 3 to r_2 and sets $pc = 8$. The third writes 8 to r_0, sets $Z = 0, N = 0, C = 0, V = 0$, and sets $pc = 12$. The fourth changes the outcome to halted. It preserves the registers, memory, program counter, and conditions. There is no fifth step.

The complete four-instruction trace

The trace contains five states, not four. S_0 is the input. S_1 has $r_1 = 5$ and $pc = 4$. S_2 also has $r_2 = 3$ and $pc = 8$. S_3 also has $r_0 = 8$, cleared conditions, and $pc = 12$. S_4 changes only the outcome to halted. Every omitted component is preserved at each named step.

Writing the states rather than only the instruction outputs exposes two common counting errors. The initial state belongs to the trace, and halt is an executed transition rather than the absence of one. The statement "four instructions ran" therefore corresponds to four edges and five vertices.

This trace is straight-line because every non-halt step uses its ordinary next address. The definition still comes from the program counter. That distinction will let a branch choose another path without changing what a trace is.

5.2.1 Multi-step execution is the closure of one-step execution

The instruction semantics defines one edge $S \to S'$. Program theorems usually concern zero or more edges. Write $S \to^k T$ when there is a trace of exactly k steps from S to T. The zero-step case relates every state to itself. The successor case composes one step with a k-step execution.

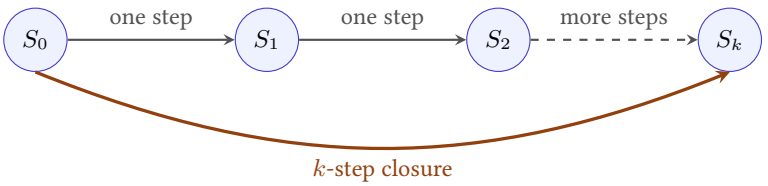


Figure 5.2: Multi-step execution is built from checked adjacent one-step transitions. It does not introduce another instruction semantics.

This definition supplies a composition law. If $S \to^m T$ and $T \to^n U$, concatenating the traces gives $S \to^{m+n} U$. The shared state T appears once in the combined state sequence even though it closes the first trace and opens the second.

Concatenating execution traces

Take the state sequence from S to T and append the second sequence after omitting its first copy of T. Every adjacent pair comes from one of the original traces, including the join because both use exactly the same state T. The edge count is $m + n$, while the state count is $m + n + 1$.

Determinism gives uniqueness at a fixed step count. If each running state has at most one successor, two k-step traces from the same initial state have the same states at every position. That fact justifies speaking of *the* bounded Ao run after the program, initial state, and bound are fixed. It does not guarantee that a successor exists: halt, trap, or invalid fetch can end the trace earlier.

Prefix closure is equally important. Every initial segment of a valid trace is a valid trace. A checker can therefore validate a long execution one adjacent pair at a time. The converse requires matching endpoints: two separately valid

traces cannot be joined merely because their printed instruction addresses look compatible.

5.3 A branch asks state a question

Ao separates comparison from conditional control. The instruction `cmp r1, r2` computes the width-w subtraction $R[r_1] - R[r_2]$ to set Z, N, C, V; it does not write the subtraction result to a register. A later conditional branch reads selected condition bits.

Suppose r_1 and r_2 both contain 5. After `cmp r1, r2`, $Z = 1$. The instruction `branch.eq 2` then takes its relative target:

$$pc + 4 + 4 \cdot 2.$$

If the branch itself begins at address 4, the target is 16. If the two registers differ, $Z = 0$, and the same branch advances to address 8.

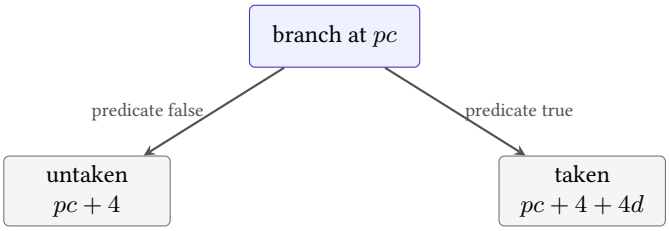


Figure 5.3: An Ao conditional branch chooses between the sequential address and a target measured from that sequential address.

The offset is relative to the sequential address, not to the current address. A faulty rule that uses $pc + 4 \cdot 2$ lands at 12 in this example. That near miss is useful: the wrong trace still reaches an aligned instruction, so a test must check the target rule rather than merely alignment.

The displacement d is a signed eight-bit value before extension to the word width. Multiplication by four scales instruction units to byte addresses. Adding it to $pc + 4$ makes displacement zero name the next instruction, not the branch itself. A backward branch uses a negative displacement. All three steps—sign extension, scaling, and choice of base—belong to the target rule.

Taken, untaken, and backward

Place a branch at address 20. Compute both successors for displacements 0, 2, and -3. For each displacement, separate the sequential address from the taken target before deciding which predicate value chooses it.

The paired architectures place the predicate differently. A later RV64 conditional branch will compare register values as part of the branch instruction. A common selected x86-64 pattern will have one instruction set condition flags and a later conditional jump read them. Ao deliberately uses the second broad shape. The comparison dimension is where the branch predicate lives, not which family receives the simpler label.

5.3.1 Control-flow graphs summarize possible local edges

A **basic block** is a sequence entered at its first instruction and left only at its end under the chosen graph construction. Ordinary fallthrough keeps execution inside the block. A branch, jump, halt, trap, or declared entry point creates a boundary. The exact partition depends on which targets and failures the analysis includes.

A control-flow graph places these blocks or instruction addresses at nodes and adds possible successor edges. For a direct conditional branch, the local graph usually includes the fallthrough and taken target even if one predicate is impossible under the program's input premise. A trace selects one of those edges in one state. A state invariant can later remove an infeasible edge.

The distinctions prevent three common overclaims:

1. absence from sampled traces does not make a graph edge unreachable;
2. presence in a syntactic graph does not prove that an allowed state can take the edge; and
3. local reachability of an address does not prove that fetching and decoding there succeeds.

Reachability begins at the declared entry state or entry set. Inductively, the entry is reachable, and a successor of a reachable running state is reachable. This definition is semantic because it mentions states and steps. Graph reachability over a conservative edge set is an overapproximation unless every edge has also been shown feasible.

> **Control point**
>
> A control point is an address or phase at which a proof states a relation or invariant. It need not be every instruction boundary. Loop heads, procedure entries, call returns, and final exits are common choices. The proof must still justify the intervening steps between consecutive control points.

Control points let two machines take different numbers of instructions while supporting the same higher-level argument. They also keep loop invariants readable: the invariant can describe the state at the head rather than every temporary state inside the body.

5.4 Four ways a bounded run can end

A halted trace has executed `halt`. A trapped trace has encountered a declared failure, such as an invalid fetch or memory range. Both have a final state with no successor, but one records normal completion and the other records failure.

A third run may consume its requested step bound while its last state is still running. The runner has stopped looking; the program has not necessarily stopped. This is **bound exhaustion**. A fourth result can return a running state before a larger calling procedure decides whether to take another step. Neither result proves termination or looping.

The difference matters even for a one-instruction loop. Run an unconditional branch back to itself for one hundred steps. The bounded trace contains one hundred valid transitions and a running final state. It is evidence about that prefix. To show that execution never halts requires an argument about every later step, such as an invariant that the program counter always returns to the same instruction and no state component can cause another path.

A finite state description can thus give rise to an unbounded question. The machine is small; time need not be.

> **A bound is a resource limit, not a semantic verdict**
>
> Reaching 1,000 valid steps establishes a 1,000-step prefix. It does not show that the program halts at step 1,001, runs forever, or eventually traps. Each of those conclusions needs evidence beyond the exhausted run.

A loop proof supplies that further evidence by identifying an invariant. For the unconditional self-branch, the program counter returns to the same address, the branch preserves every other state component, and the outcome remains running. Applying the same rule again recreates the same state. An induction over the number of steps then covers every finite prefix. The proof uses the transition rule, not the patience of the runner.

5.4.1 Invariants prove safety; variants can prove termination

An invariant is a predicate intended to hold whenever execution reaches a chosen control point. The proof has two obligations. Initialization shows that the entry path establishes the predicate. Preservation assumes it at one visit, executes the loop segment, and shows it again at the next visit. By induction, the invariant holds at every reached visit.

The invariant need not describe all state. It should carry exactly the facts needed for the next iteration and final theorem: counter bounds, address validity, preserved memory, relationships among registers, or a condition connecting machine words to mathematical values. A weak invariant may be true but unable to prove safety. An

unnecessarily strong one may fail because it mentions scratch state that legitimately changes.

Loop-invariant induction

Let $I(S)$ be the invariant at loop head h. Prove that the entry execution reaches a state S_0 with $pc = h$ and $I(S_0)$. Next, for an arbitrary reachable S with $pc = h$ and $I(S)$, prove that one loop iteration either exits under the intended postcondition or reaches a new head state S' satisfying $I(S')$. Induction covers every finite number of completed iterations.

This establishes preservation properties but not necessarily termination. A **variant** adds a value in a well-founded order, commonly a natural number, that strictly decreases on every returning iteration. If the body cannot get stuck and the variant cannot decrease forever, execution must eventually leave the loop.

Safety and termination can therefore split. An infinite self-branch has the invariant that the complete state repeats, which proves it never reaches bad state under the model. It has no decreasing natural variant and does not terminate. A countdown loop can have both an invariant keeping its counter in range and a variant equal to the remaining count.

The exit theorem uses the invariant plus the negated continuation condition. For example, if an invariant states $0 \leq r \leq n$ and the loop continues exactly while $r > 0$, reaching the ordinary exit yields $r = 0$. The control predicate is part of the mathematical conclusion, not merely a jump detail.

5.4.2 Partial and total correctness answer different questions

Suppose the four-instruction example has the postcondition $r_0 = 8$. We can ask two different questions. First: whenever the program reaches its normal exit, does the final state satisfy that postcondition? Second: from every allowed initial state, does the program actually reach that exit? The first question concerns **partial correctness**. The second question, joined to the first, gives **total correctness**.

For the straight-line example, the distinction is easy to miss. Fetch and decode succeed at four consecutive addresses, each instruction has one successor, and the fourth instruction halts. The same trace that establishes the result also establishes arrival at the result. Loops separate the two obligations. An invariant may say exactly what would be true at the exit while providing no reason that the exit will ever be reached.

A true implication may say nothing about arrival

The implication *if the program halts, then* $r_0 = 8$ is true for a program that never halts and never writes 8. Partial correctness is useful, but it must not be read as a termination claim. To obtain total correctness, add a termination argument under the same initial-state premise.

Traps require another separation. A contract may permit them, rule them out, or assign them their own postcondition. If normal exit and trap are both terminal outcomes, a claim about halted states alone says nothing about trapped states. A robust program theorem therefore partitions the possible outcomes before stating what must hold in each part.

A bounded runner does not collapse this distinction. If the bound is reached, the runner has established a finite prefix. A sufficiently large bound may cover every run only when another argument supplies a uniform upper bound on execution length. That bound is a theorem premise or conclusion external to the step relation; it is not hidden inside the number entered at the command line.

This vocabulary will matter when later chapters compare two programs. One may produce the right value whenever it exits but diverge on an input where the other program halts. Agreement on final register values then fails to capture the difference in behavior.

5.5 Replacement inside a context

Suppose programs p and q leave the same value in r_0 for every allowed initial state. Can one replace the other before any suffix r?

No. Let p also leave $r_3 = 0$, while q leaves $r_3 = 1$. They agree under an observation of r_0. Choose a suffix that copies r_3 into the final output. The suffix exposes the difference that the earlier observation discarded.

The same counterexample works with condition state. Two prefixes may agree on registers while setting Z differently. A suffix beginning with `branch.eq` can choose different paths. Agreement on an output before the branch is not enough.

The repair is exact. Suppose p and q, from every allowed initial state, finish with the same outcome and agree on every state component that the deterministic suffix can read. The first suffix instruction then receives indistinguishable inputs. It produces matching values for the components needed by the second instruction, and the same argument repeats through the suffix. The final states agree under the declared observation.

The argument is a reader proof of a contextual rule, not yet a general theorem about all programs. Its premises need a precise read set, termination account, and observation. Later chapters will formalize those conditions as equivalence and refinement. For now, the counterexample has done its job: local replacement depends on context unless the state relation covers what the context can see.

Replacement through a deterministic suffix

Let two prefixes finish in running states that agree on every component the first suffix instruction can read. Determinism gives matching relevant outputs after that instruction. Repeat the argument for each later suffix instruction. If both executions reach the end under the same outcomes, their final states agree under the declared observation. The proof fails when a suffix reads a component on which the prefix states disagree.

The premise must follow the path actually taken. A suffix containing a branch may read one set of components on one path and another set on a second path. Agreement sufficient to choose the same first branch can simplify the rest of the proof. Agreement only at the final output of the prefixes cannot.

We can describe the required agreement with a projection π. The projection retains the components a context may observe: perhaps selected registers and the outcome, or perhaps registers, conditions, reachable memory, and program bytes. If $\pi(S_p) = \pi(S_q)$ after the two prefixes, replacement is sound only when every permitted context treats states with the same projection alike. Choosing π is therefore part of the claim, not a formatting choice made after the executions have been compared.

A context can observe more than its final arithmetic result. It can branch on a condition, load from an address changed by the prefix, encounter a fault on one path, or use an indirect target obtained from a register. In a machine with writable code, it might even fetch different instruction bytes. A proof that forgets any such input licenses a suffix designed to reveal the forgotten difference.

Closed-program agreement can be weaker

Two complete programs may agree under a final-output observation even though their intermediate states differ. That can be enough when nothing will be appended and no external observer sees the intermediate states. Replacement inside an arbitrary context is stronger: the context becomes an observer and may turn a hidden difference into a visible one.

The useful projection is rarely the whole physical machine. It is the state named by the model and exposed to the admitted contexts. Ao excludes caches, timing, interrupts, and self-modifying code, so an Ao contextual result cannot settle whether two real implementations leak different timing information. Conversely, retaining every Ao register when the suffix reads only one may be safe but needlessly strict. Later equivalence proofs will balance these two errors: forgetting an observable component and preserving irrelevant detail.

A context exposes one forgotten bit

Prefix p and prefix q both leave $r_0 = 7$, but they leave different values of Z. Append a suffix that branches to write 0 or 1 into r_0 according to Z. The composed programs now disagree on their visible output. The suffix did not create the earlier difference; it converted hidden state into an observed register.

5.6 The implementation boundary

An Axeyum runner for Ao must fetch, decode, and step through the program while keeping halt, trap, bound exhaustion, and continued execution distinct. Object `OP.a0.run` records that obligation.

The introductory Axeyum run should print a trace a person can compare with the book. Each row needs the step number, program counter, decoded instruction, changed components, and outcome. A replay command should consume the same initial state and program bytes, not a second handwritten formula that merely resembles them.

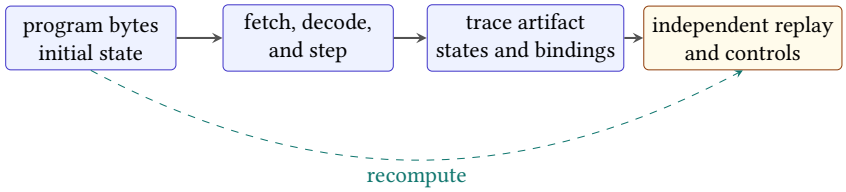


Figure 5.4: A future Axeyum trace route binds program bytes and initial state to an artifact, then independently replays every derived transition and control.

The artifact should not trust its own change summary. It stores complete states or a lossless initial state plus checked deltas; the replayer reconstructs each successor and compares every component in scope. It also decodes from the bound program bytes at the recorded program counter. A printed mnemonic is a display field, not semantic input.

Termination labels need independent derivation. The replayer inspects the last machine outcome and the requested bound. Halt and trap must arise from actual steps. Bound exhaustion requires a running final state and exactly the allowed number of steps. A continued-execution result may expose a shorter prefix without claiming that the machine stopped.

A solver-backed question comes later. It may ask whether two short Ao programs can finish with different observations under an input premise. If the answer is yes, the model must decode into an initial state and replay into two traces. If the answer is no, the route needs the generated formula, scope, and independent

checking boundary. The trace printer remains useful in both cases because it connects the verdict to executable steps.

The controls attack control itself. Mutate a relative target by four bytes. Try to execute after a trapped state. Try to execute after halt. Report a running state at a finite bound as if the program had terminated. Each error can preserve the arithmetic destination for a while, which is why trace checking must inspect more than the final register.

Add structural controls as the implementation grows. Remove one intermediate state while leaving the endpoints. Duplicate a state without a matching step. Change the program digest after producing the trace. Alter an unobserved register in one successor. Each mutation attacks a different binding: adjacency, step count, program identity, or full-state preservation.

For a first Python lesson, the public surface can stay small: construct an Ao program from bytes, construct a complete initial state, request a bounded run, print the returned trace, and invoke verification. The Rust layer should own fetch, decode, step, serialization, and checking. Python organizes the lesson and renders the explanation; it must not supply an alternate transition rule.

This runner does not accompany the draft. The traces in this chapter are hand executions of the stated Ao rules. They establish how the future runner must behave on these examples; they do not claim that all Ao traces have already been checked.

5.7 One complete Part I trace

We can now name every layer in the four-instruction example. The immediate bytes denote width-eight words. Registers and conditions belong to complete state. Each decoded instruction has explicit operands and effects. The program counter selects immutable code bytes and advances by four. The add uses modular arithmetic and records facts about the mathematical sum. The `halt` outcome prevents another step. An observation may expose r_0 alone, but the trace itself retains the rest.

Nothing in that account depends on calling Ao a RISC or CISC machine. Those labels would add less information than the rules we have printed. When the book turns to real architectures, the same inventory will let us compare one choice at a time.

One boundary remains even in our tiny example. We have written decoded instructions beside the trace. A processor receives bytes. Before an instruction can transform state, those bytes must select one legal operation, operand form, and length—or cause a decoding failure. Part II begins at that boundary.

5.8 Exercises

1. **Execute.** Run the four-instruction program in this chapter from $pc = 0$, with every register initially zero. List every complete state component that changes at each step.

2. **Break.** Replace the taken branch target rule $pc + 4 + 4d$ with $pc + 4d$. For a branch at address 4 with displacement 2, show the two traces diverging at their next program counters.

3. **Prove.** Give an invariant showing that an unconditional zero-displacement branch returns to the same instruction under the Ao target rule. Explain why any finite bounded run alone would be weaker.

4. **Transfer.** Write one register-predicate branch pattern for the later RV64 slice and one compare-then-jump pattern for the later x86-64 slice. List the state read by each branch instruction itself.

5. Construct two one-instruction prefixes that agree on r_0 but differ on a component read by a suffix. Give the suffix and trace both composed programs.

6. Act as the runner for a program that traps on its second instruction. Show why appending a valid third instruction creates no third transition.

7. State the exact result of running a still-running program at bounds 0, 1, and 2. Do not use the words halt or loop unless the trace warrants them.

8. **Execute.** For the program in this chapter, list the five states as rows with pc, changed registers, condition bits, and outcome. Confirm that each adjacent pair satisfies exactly one step.

9. **Break.** Make a runner label bound exhaustion as halt. Give a one-instruction self-loop on which the faulty label changes the observable outcome.

10. **Prove.** Formalize the self-loop argument by induction on the number of transitions. State the invariant and the base and induction cases.

11. **Transfer.** For one planned RV64 branch and one planned x86-64 conditional jump, list the state needed to select the path and the rule needed to calculate both target addresses.

CHAPTER SIX

Encoding and Decoding

Four bytes lie in memory at the program counter. They are values until a decoder treats them as an instruction. The decoder may produce an operation and operands, reject the pattern, or consume a different number of bytes than a neighboring architecture would. None of those choices yet says what the decoded instruction does.

This boundary is where software becomes unusually literal. A compiler may intend an addition. An assembler may print an addition. A disassembler may report an addition. The processor receives bytes. If the route from those bytes to the intended instruction is wrong, every later proof begins with the wrong object.

Bytes, instructions, and effects are different objects

An encoding is a byte string. Decoding maps that string, in a declared mode, to an instruction instance or a rejection. Execution maps the instruction instance and a state to a successor state. Assembly is notation for humans. Evidence about one map does not automatically establish the other.

6.1 Why instruction encodings exist

An early stored-program machine needed a way to place operations and their arguments in the same finite storage used for numbers. That choice made a program inspectable, movable, and mechanically fetchable. It also created a permanent design problem: which parts of a finite word name the operation, which parts name operands, and what happens to patterns that have no assigned meaning?

A regular format gives the decoder fixed places to look. It can make hardware, teaching, and formalization easier. It may spend bits on fields an instruction does not need. A variable format can give common instructions short spellings and retain compatibility with older ones. It makes the first question—how many bytes belong to this instruction?—part of decoding itself.

Neither choice is simply “modern” or “old.” RISC-V’s base integer format uses regular 32-bit instructions, while its optional compressed extension adds 16-bit

forms. x86-64 retains a variable-length family shaped by decades of compatible extension. The useful comparison is not a slogan about RISC and CISC. It is a list of exact decoder obligations.

Compatibility leaves marks in the byte stream

Instruction formats are historical records. A field may exist because an earlier machine needed it. A prefix may acquire a new role in a later mode. An opcode region may be reserved so that a future extension has somewhere to grow. Reading an encoding is therefore partly mathematics and partly careful historical interpretation of a versioned specification.

The rest of this chapter uses one addition on three machines. Ao makes every field visible. RV64I shows a regular commercial-scale design. x86-64 shows a short variable-length instruction whose meaning depends on a prefix and an operand byte. The arithmetic result is familiar. The path used to select that arithmetic is not.

6.2 Four objects, not one

We will keep four representations separate.

Assembly spelling. Text such as `add r3, r5, r2`. It depends on a syntax and may contain aliases.

Instruction bytes. A finite sequence such as `10 2B 02 00`. Byte order and length are explicit.

Decoded instance. A structured value such as $\mathrm{Add}(rd = 3, rs1 = 5, rs2 = 2)$.

Transition rule. The rule that reads two registers, writes their fixed-width sum, updates declared conditions, advances the program counter, and preserves everything else.

An assembler normally maps spelling to bytes. A decoder maps bytes to an instance. A disassembler maps an instance or bytes back to a chosen spelling. The reverse maps need not recover the original text because several spellings can denote the same instruction. A proof should therefore use the structured instance as its semantic hinge rather than requiring printed text to round trip character for character.

A successful disassembly is not an execution proof

A disassembler can correctly identify an instruction while an executor gives that instruction the wrong effect. An executor can implement addition correctly while its decoder assigns addition to the wrong byte pattern. Test and prove the two maps separately before testing their composition.

The mode belongs to the decoder input. The same x86 byte sequence can be interpreted differently in different execution modes. The enabled RISC-V extensions determine whether some patterns are ordinary instructions, compressed instructions, hints, reserved encodings, or illegal encodings. A decoder whose type is merely *bytes* $\to$ *instruction* hides information needed to state its contract.

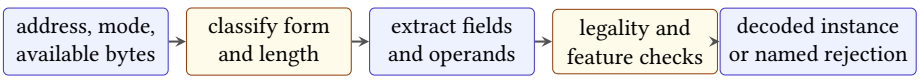
A more honest decoder accepts a source pin, mode, feature set, address, and byte sequence. Its result has the shape

$$\text{Decoded}(i, n) \quad \text{or} \quad \text{Reject}(r),$$

where n is the number of bytes consumed and r is a declared rejection reason. Ao fixes most of these parameters, but keeping the full shape in view prevents the small model from teaching an accidental universal rule.

6.2.1 A decoder is total when rejection is a result

A useful decoder function returns an answer for every finite input supplied to its declared interface. That answer need not be an instruction. It may say that the bytes are incomplete, the address is misaligned, the pattern is reserved, the requested feature is disabled, or the form lies outside the implemented slice. Making these cases explicit avoids host-language exceptions and silent fallbacks becoming accidental architecture semantics.



The execution rule begins only after this pipeline returns a legal instance.

Figure 6.1: A staged decoder returns either a structured instruction or a named rejection. Execution begins only after decoding succeeds.

Incomplete input differs from an illegal complete encoding. A fixed-four-byte decoder given only three available bytes cannot yet inspect all reserved fields. A streaming decoder may request another byte. A fetch model may trap because the fourth address is unreadable. By contrast, four available Ao bytes with a set reserved bit form a complete pattern that the legality rule rejects. The surrounding step semantics decides how each decoder result becomes a machine outcome.

Address belongs in the input even for fixed-length formats. Ao and the selected RV64I subset require aligned instruction addresses. For variable-length x86, the address also determines which byte begins the parse. Starting one byte later can produce a different legal instruction sequence from the same memory. A decoder over an isolated byte array should therefore state that element zero is already the chosen instruction boundary.

Decoder result and fetch result

A decoder result classifies supplied bytes under a mode and feature set. A fetch result determines which bytes are available from machine memory at a program counter and whether reading them is permitted. Keeping the two relations separate allows the same pure decoder to be tested without claiming that every supplied byte string is fetchable in a running machine.

This separation also improves controls. Truncating the buffer should exercise an incomplete-input path. Setting a reserved bit should exercise legality. Changing the start address should exercise alignment or boundary selection. One generic rejection loses the evidence that all three checks actually fire.

6.3 Ao: every byte has a job

Every Ao instruction occupies four bytes and begins at an address divisible by four. The program map is read in increasing address order.

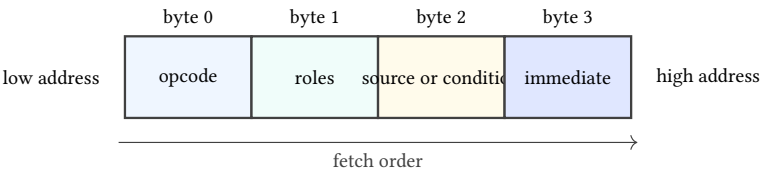


Figure 6.2: The four-byte Ao envelope. The detailed meaning of bytes 1–3 depends on the opcode, but their locations and the total length never change.

Byte 0 is the opcode. In byte 1, bits 0–2 name rd, bits 3–5 name $rs1$, and bits 6–7 are reserved zero. In byte 2, bits 0–2 name $rs2$, bits 3–5 hold a branch condition, and bits 6–7 are reserved zero. Byte 3 holds a signed eight-bit immediate when the instruction uses one. An unused field must be zero. This last rule makes Ao stricter than a decoder that merely ignores irrelevant bits.

Consider

```
add r3, r5, r2
```

Listing 6.1: One Ao register addition

The opcode is 10. Packing $rd = 3$ and $rs1 = 5$ into byte 1 gives

$$3 + (5 \ll 3) = 3 + 40 = 43 = 2\text{B}.$$

Byte 2 contains $rs2 = 2$, so it is 02. Addition has no immediate or branch condition, so byte 3 and every unused field are zero. The complete encoding is

10 2B 02 00.

The decoder performs the argument in reverse. It checks the fetch address and length, reads opcode 10, rejects any set reserved bit, extracts the three register numbers, checks that the condition and immediate fields are zero, and returns $\text{Add}(3, 5, 2)$.

A0 addition round trip

Let E be the encoder for a legal Ao addition and D the decoder. To show $D(E(rd, rs1, rs2)) = \text{Add}(rd, rs1, rs2)$, use the fact that each register number is below eight. Masking the packed byte by 07 recovers rd. Shifting it right by three and masking by 07 recovers $rs1$. The same mask recovers $rs2$ from byte 2. The encoder writes zero to every reserved and unused field, so all legality checks pass.

That proof has a converse worth stating carefully. If the decoder accepts an Ao addition byte string b, encoding the returned instance reproduces b. Acceptance matters. Without it, a byte string with a reserved high bit could decode to the same three registers after masking but fail to reproduce the original byte. The reserved-bit check is what makes accepted encodings canonical.

Break the A0 encoding one field at a time

Begin with 10 2B 02 00. Change byte 1 to 6B, which sets a reserved bit. Change byte 2 to 0A, which leaves $rs2 = 2$ but sets an unused condition bit. Change byte 3 to 01. A strict decoder rejects all three. A decoder that only extracts operands accepts them, revealing three different omissions in one small test family.

Illegal encodings are not gaps outside the semantics. If a running Ao state fetches a complete but illegal instruction, its next state is trapped with an illegal-encoding reason. Thus the decoder's rejection becomes an architectural event through the step relation.

6.3.1 Canonical encoding is stronger than successful decoding

A decoder could ignore every unused Ao field and still recover the intended operation and operands from encodings produced by the official encoder. Such a

decoder satisfies the forward round trip on encoder output. It fails the canonicality contract because many byte strings with nonzero unused fields would be accepted as the same instruction.

Canonical encodings provide one byte representation for each legal Ao instance. This makes byte equality and instruction equality coincide on accepted inputs:

$$D(b_1) = D(b_2) \quad \text{implies} \quad b_1 = b_2.$$

The implication relies on reserved and unused field checks. It would be false in an architecture that intentionally permits several encodings of one instruction.

Accepted A0 encodings are injective

Suppose two accepted Ao byte strings decode to the same instruction instance. The opcode and every used operand field must match that instance. Acceptance forces all reserved and unused fields to their canonical zero values. Each byte is therefore determined by the common instance, so the byte strings are equal. The same argument would fail if the decoder merely masked away an unused set bit.

Canonicality helps evidence handling. A manifest can store the legal bytes and the decoded object without choosing among aliases. It does not eliminate the need to store both: the bytes bind the claim to the program image, while the instance binds execution to structured operands. A digest of only one layer cannot detect every substitution in the other.

Real ISAs may admit aliases at the assembly or encoding level. A disassembler can also choose a preferred spelling that differs from the source text. Their correct round-trip property may therefore use normalization: decode, re-encode to a designated canonical form, and compare meanings rather than demand reproduction of the original spelling.

6.4 RV64I: one word, several fields

The RV64I base integer ISA uses 32-bit base instructions. The optional compressed extension can add 16-bit instructions, but we exclude that extension in this example. The source pin is the ratified unprivileged architecture, version 20260120; RV64I itself is version 2.1 (RISC-V International 2026b).

An R-type instruction divides the 32-bit word into six fields.

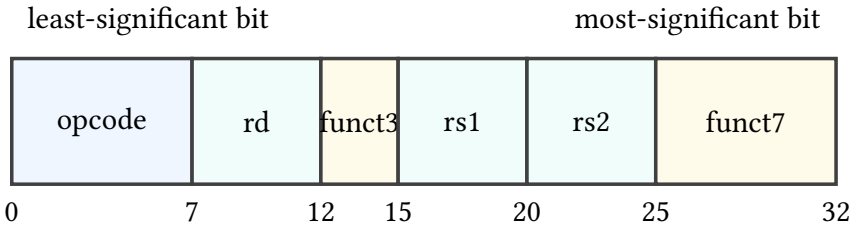


Figure 6.3: The RV64I R-type format, drawn from least-significant bit at left to most-significant bit at right. Printed manuals often reverse that direction.

For `add x3, x5, x2`, the fields are

field	value	role
opcode	`0110011`	integer register operation
rd	3	destination $x3$
funct3	`000`	operation selector
rs1	5	first source $x5$
rs2	2	second source $x2$
funct7	`0000000`	operation selector

Packing these fields gives the word `002281B3`. In little-endian memory, the four bytes at increasing addresses are

```
B3 81 22 00.
```

The field diagram describes bit positions in the 32-bit instruction word. The byte sequence describes memory addresses. Confusing those two views is a common source of reversed examples.

The opcode alone does not identify `add`. It identifies the broad register-operation class. The *funct3* and *funct7* fields complete the selection. Changing *funct7* from `0000000` to `0100000` while leaving the other fields fixed changes this instance to `sub x3, x5, x2`. Changing only *funct3* can select another operation in the same broad class.

Decode from bytes without reading the mnemonic

Read `B3 81 22 00` as a little-endian 32-bit word. Bits 0–6 are `0110011`. Bits 7–11 give 3; bits 12–14 give 0; bits 15–19 give 5; bits 20–24 give 2; and bits 25–31 give 0. The table for the pinned RV64I version assigns that complete field combination to `add x3,x5,x2`. Only after this selection should the execution rule read $x5$, read $x2$, write their modulo-2^{64} sum to $x3$, and advance the program counter.

RV64I adds another semantic condition absent from Ao: register $x0$ reads as zero, and writes to it have no architectural effect. The encoding still has a five-bit destination field when its value is zero. Decoding `add x0,x5,x2` must retain that destination in the instruction instance; the execution rule supplies the discard behavior. Erasing the operand during decode would mix a state rule into a format rule.

Reserved and hint encodings require the same care. A pattern's status depends on the exact ISA and extension set. "My current processor does nothing visible" does not establish that a pattern is universally a no-operation. The book will name the enabled subset before assigning meaning.

6.4.1 Regular length does not mean one field layout

RV64I base instructions in this window share a 32-bit length, but their fields do not all play the same roles. R-type arithmetic has two source registers and one destination. I-type arithmetic and loads replace the second register field with immediate bits. Stores need two registers but no destination, so their immediate is split around the other fields. Conditional branches also split and rearrange a signed displacement.

The decoder must reconstruct a logical immediate before sign extension. For a split field, concatenating pieces in memory-byte order is wrong; the source specification assigns each instruction bit to a particular result bit. Once the pieces are placed, the high immediate bit determines sign extension to the architectural calculation width. Scaling or an implicit zero low bit is a separate step for forms that require it.

> **Field extraction precedes interpretation**
>
> Suppose a 12-bit I-type immediate field is FFF. Extraction first returns the 12-bit pattern with value 4095 under an unsigned reading. The instruction rule then sign-extends that pattern, producing -1 under its signed reading and an all-ones 64-bit word. Calling the field itself negative before naming its width and extension rule compresses two operations into one.

Instruction selection and operand construction are likewise distinct. The opcode and function fields may select a load class; another field selects the load width and extension policy; register and immediate fields construct its operands. A decoder result should preserve those decisions explicitly so the executor does not inspect raw instruction bits again and create a second, possibly inconsistent decoder.

Do not sign-extend before reassembling a split field

Sign-extending each fragment separately invents high bits that were never part of the immediate. Place every fragment into the declared logical field, then apply the one extension rule to the complete field.

The feature set remains load-bearing even at fixed length. A 32-bit pattern reserved in RV64I may acquire meaning under an enabled extension. Soundness is therefore relative to the declared ISA string, not merely to XLEN and the word value. A decoder that accepts extension forms while claiming the base subset is too permissive; one that rejects enabled forms may be sound but incomplete for the larger scope.

6.5 x86-64: length is part of the answer

In 64-bit mode, an x86 instruction can contain legacy prefixes, a REX prefix, an opcode, a ModR/M byte, an optional SIB byte, an optional displacement, and an optional immediate. Not every instruction contains every part. The decoder must decide which parts are present as it moves through the byte stream. Our source is Intel's version 092 instruction reference (Intel Corporation 2026a).

Consider Intel-syntax `add rax, rbx`. One encoding is

`48 01 D8`.

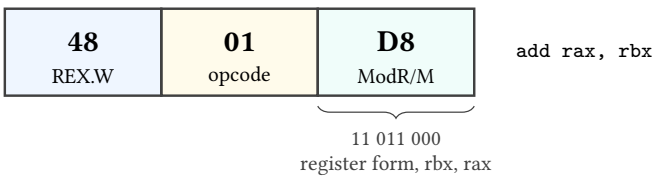


Figure 6.4: One three-byte x86-64 encoding of `add rax, rbx`. The byte boundaries are regular here; the instruction family as a whole is not fixed at three bytes.

Byte `48` is a REX prefix with the W bit set, selecting a 64-bit operand size for this form. Opcode `01` selects ADD with a register source and a register-or-memory destination. ModR/M byte `D8` is `11 011 000` in binary. The `11` *mod* field selects a register destination rather than memory. The `011` *reg* field names `rbx`. The `000` *r/m* field names `rax`.

Remove byte `48` and `01 D8` is still a valid instruction, but the ordinary operand size is 32 bits: `add eax, ebx`. Changing or removing a prefix need not make the input illegal. It can produce a nearby legal instruction with a different state effect. That makes prefix mutations strong negative controls.

Change ModR/M from `D8` to `18`. Its high two bits are now `00`, so the destination is memory addressed through `rax`, not the register `rax`. The decoder may then need more bytes for an addressing form. A one-bit change has crossed from register arithmetic into a memory operation and altered the possible failure behavior.

Do not infer an x86 instruction boundary from a line break

The next byte in a hex dump is not necessarily the next instruction. It may be a SIB byte, displacement, or immediate belonging to the current instruction. Conversely, a prefix-like byte may begin a separate instruction when parsing starts at another address. Instruction boundaries come from the decoder and the chosen starting address.

This property makes a damaged byte stream especially instructive. If one decoder consumes three bytes while another consumes two, their disagreement does not end at the first instruction. They begin the next decode at different addresses. A length bug can change the interpretation of the entire suffix.

6.5.1 x86 decoding is a constrained walk through the bytes

A practical x86-64 decoder cannot split every instruction into the same fixed slots. It walks through a bounded sequence of possible components. Prefix bytes alter mode-dependent attributes or extend fields. The opcode determines which later components are required. A ModR/M byte can select registers or a memory-addressing form. That form may require a SIB byte and displacement. The selected operation may finally require an immediate.

Later choices depend on earlier ones. The same byte value can be an opcode at one position and data for a displacement at another. A byte that resembles a prefix has that role only while the parser is still in a prefix-accepting position and the current mode gives it that meaning. Consequently, scanning for familiar byte values without decoder state cannot establish boundaries.

The walk should return both a structured instance and its consumed length. Execution uses the structured operands. The fetch loop uses the length to calculate the following instruction address. A disassembler uses both to print one line and advance through a buffer. If these consumers recalculate length independently, the system contains several decoders whose disagreements can desynchronize the program.

A memory form expands the parse

In the chapter's `48 01 D8` example, ModR/M mode bits `11` finish operand selection with registers and no displacement. Changing those bits to a memory mode can make the remaining ModR/M fields request a SIB byte, a displacement, or both. The opcode still belongs to ADD, but the instruction length, operand kind, address calculation, and possible faults can all change.

Incomplete x86 input is therefore position-sensitive. Ending after an opcode that requires ModR/M differs from ending inside a four-byte displacement. Both need more bytes, but a diagnostic can name the missing component. Such diagnostics make truncation controls much stronger than one undifferentiated end-of-buffer error.

Length is part of decoder soundness

A decoder that selects the right operation and operands but reports the wrong length is unsound for program execution. The current step may appear correct, yet fallthrough begins at the wrong byte and changes the decoded suffix.

6.6 Three additions, three decoder shapes

The examples perform the same broad arithmetic but do not have the same architectural contract.

Question	A0	RV64I	x86-64 example
Length	always four bytes	four bytes in the selected base subset	three bytes for this form
Operation selector	one opcode byte	opcode plus funct3 and funct7	prefix, opcode, and ModR/M
Operand names	three-bit fields	five-bit fields	ModR/M fields extended by REX bits
Arithmetic width	parameter w	64 bits for `add`	64 bits because REX.W is set
Condition writes	Z, N, C, V	none	selected status flags
Ordinary next PC	$pc + 4$	$pc + 4$	$pc + 3$ for this instance

The table blocks a tempting but false abstraction: "ADD means the same thing everywhere." The destination sum may correspond under a state relation, yet the

condition state, zero register, program-counter increment, legal inputs, and failure modes differ. Cross-ISA reasoning begins by naming those differences, not by hiding them behind a shared mnemonic.

A useful common decoded form

A comparison tool can translate all three instances into a book-owned form such as AddReg(*width*, dst, $left$, $right$, *condition policy*, *length*). That form is not a new universal ISA. It is an explicit bridge carrying the facts later relations need. If the translation drops instruction length or condition behavior, it cannot support proofs that observe those facts.

6.7 What decoder correctness can mean

There is no single useful sentence called "the decoder is correct" until we choose a reference and a scope. Four properties matter.

1. *Soundness:* every accepted byte string is assigned the instruction instance specified by the pinned source in the declared mode.
2. *Completeness:* every in-scope legal encoding is accepted.
3. *Length correctness:* the decoder consumes exactly the bytes belonging to the instruction.
4. *Rejection correctness:* reserved, illegal, incomplete, and out-of-scope patterns receive the declared result rather than an invented instruction.

Round-trip tests help but cover narrower statements. If $D(E(i)) = i$ for every supported instance i, the encoder does not produce an input that this decoder misreads. The property says nothing about byte strings the encoder never emits. Conversely, if $E(D(b)) = b$ for every accepted canonical byte string b, accepted inputs reproduce their bytes. Aliases and multiple legal encodings can require a normalization relation instead of literal equality.

An independent decoder gives another view of the same bytes. Agreement over a large corpus is valuable defect evidence. It is not proof against a shared misreading of the specification, nor does it establish execution semantics. The strongest practical route combines source-derived cases, independent implementations, generated boundary inputs, and small proofs for regular field operations.

6.7.1 Decoder tests should cover partitions, not only examples

Random byte strings often spend most of their time in common rejection paths. A more informative generator partitions the input space by form, operand boundary, reserved field, feature gate, length, and rejection class. It then draws legal and

nearby illegal cases from every partition. Coverage means that each declared decision fired, not merely that many bytes were processed.

Mutation tests start from a source-derived legal encoding and alter one load-bearing feature. Set each reserved bit. Move every register field across its minimum and maximum. Change an opcode selector while holding operands fixed. Truncate at every byte boundary. For x86, change prefix presence, ModR/M mode, SIB demand, displacement size, and immediate length separately. For RV64, mutate each fragment of a split immediate before checking the reassembled value.

A decoder control matrix

A control matrix maps each decoder obligation to at least one accepted case and one mutation expected to change its result. Rows name obligations such as length, field extraction, reserved-bit rejection, feature selection, and canonicality. Columns record the witness bytes, expected class, actual class, and replay result.

Differential decoding belongs in the matrix but does not replace it. When two decoders disagree, retain the bytes, modes, versions, structured results, and consumed lengths. Resolve the witness against the pinned primary source. Silently choosing the majority result would turn implementation agreement into an authority it does not possess.

The corpus should also retain boring boundary successes. A decoder that rejects every input can pass an illegal-only suite. Legal minimum and maximum field values, shortest and longest in-scope forms, and enabled feature cases prove that the acceptance paths remain live.

Prove the regular pieces; test the tables

Field extraction is algebra. For a field of width k beginning at bit j,

$$\mathrm{field}_{j,k}(x) = (x \gg j)\ \&\ (2^k - 1).$$

One can prove that packing non-overlapping bounded fields and extracting them returns the inputs. Opcode assignments and mode-specific legality come from a versioned table. Keep the algebraic lemma separate from the source-dependent table check so a later manual revision does not masquerade as a change in bit arithmetic.

6.8 The introductory Axeyum route

Axeyum does not yet contain the architectural packages used in this chapter. At sibling revision `a9991fdad6c1e4b2bda596b46d2c8c715556ceae`, it provides

reusable bit-vector expressions, solver routes, SAT and DRAT machinery, and kernel reconstruction infrastructure. It does not provide a reusable Ao decoder, RV64 decoder and semantics, or x86-64 decoder and semantics. This boundary is recorded in the book's Axeyum guide and must remain visible until code and evidence replace the implementation obligations.

The first Axeyum exercise is therefore constructive. Define a small Ao decoded instruction type. Write a total decoder returning either a legal instance or a named rejection. Add an encoder for legal instances. Prove or exhaustively check the round trip at a small word width, then run controls that set every reserved or unused field.

The next layer can reuse Axeyum's word machinery to express field extraction and packing. A solver can search for a byte string that passes the decoder but fails re-encoding, or two legal instances that encode to the same bytes. A SAT or SMT result is a search result. Promotion to book evidence still requires a manifest, exact source revision, replay route, and negative control.

For RV64I and x86-64, do not begin by implementing the whole architecture. Pin the manual, select the chapter's finite instruction forms, record the mode and feature set, and build a decoder whose public result includes length. Compare against an independent mature decoder, but keep the book-owned rule as the object under test. Execution semantics come afterward.

> **Four questions to ask of every decoder run**
>
> Which exact manual revision supplies the rule? Which mode and extensions are enabled? How many bytes did the decoder consume? What changed when a reserved bit, prefix, field, or final byte was mutated? A transcript without those answers is difficult to interpret and too weak for a load-bearing claim.

6.9 Where the model stops

This chapter does not define all RV64I or x86-64 encodings. It does not cover RISC-V compressed instructions, vectors, privilege, or every extension. It does not cover the full x86 prefix history, VEX, EVEX, APX, every addressing form, or the maximum instruction length. Later chapters add selected forms only when a proof needs them.

It also does not claim that equal decoded arithmetic establishes equal program behavior. The next chapters add data movement, address formation, arithmetic conditions, control transfer, and calling conventions. Only after those effects are explicit can Parts III and IV define equivalence and refinement between machines.

The working discipline is now fixed: preserve the original bytes, state the mode and enabled features, return length explicitly, distinguish every rejection class, and connect execution only to a legal structured instance. That discipline scales from

Ao's four transparent bytes to a variable-length x86 stream without pretending their decoder problems are identical.

6.10 Exercises

1. **Execute.** Decode `10 2B 02 00` under Ao. List every legality check, the structured result, the number of bytes consumed, and the next program counter after execution.

2. **Break.** Construct four mutations of that Ao encoding: wrong opcode, reserved bit, unused condition, and nonzero immediate. State the required rejection for each.

3. **Prove.** Prove that packing and extracting Ao's rd and $rs1$ fields round trips for register numbers below eight. Identify exactly where the range premise is used.

4. Pack RV64I `sub x3, x5, x2`. Give the 32-bit word and its four little-endian memory bytes. Explain which one field differs from the addition example.

5. Starting from RV64I word `002281B3`, change only rd to zero. Separate what changes in the decoded instance from what changes in execution.

6. Decode x86-64 bytes `48 01 D8` by naming the contribution of every byte. Then remove `48`; explain why the remaining sequence is not merely an incomplete version of the same instruction.

7. Change ModR/M `D8` to `18`. Which field changed, and why can the new form involve memory and additional addressing bytes?

8. Give an example showing why $D(E(i)) = i$ does not establish that the decoder rejects all illegal byte strings.

9. **Transfer.** Design a common decoded addition object rich enough to represent the three chapter examples. Mark which fields belong to decoding and which belong only to execution.

10. Design a differential test manifest for one RV64I form or one x86-64 form. Include the source pin, mode, generator domain, independent decoder, expected agreement, and at least two negative controls.

CHAPTER SEVEN

Data Movement and Address Formation

A move that appears to copy one value may also extend it, truncate it, discard it, or obtain it through an address calculation. To describe data movement we must name the source value, destination location, width, and every special rule attached to either operand.

Many machine-code mistakes hide here. The assembly line looks quiet: load, store, move, address. Yet a narrow load can decide whether the high bits become zeros or copies of a sign bit. A register name can decide whether an old upper half survives. An address expression can wrap before memory is checked. The instruction moves data only after these choices have determined what the data and location are.

A movement has a value rule and a location rule

The value rule says which bits reach the destination and how its full width is formed. The location rule says which register or memory byte range changes. Both rules belong to the instruction's meaning. Neither can be recovered from the mnemonic alone.

7.1 Why machines spend instructions on movement

Arithmetic circuits act on values already presented in the right places. Programs therefore spend much of their time arranging operands: bringing a constant into a register, loading a field from memory, saving a result, or forming the address of the next object. Early stored-program designs already had to distinguish transfers from arithmetic because storage locations and working registers played different roles.

The distinction remains, although modern machines blur it in different ways. RISC-V gives arithmetic instructions register operands and uses separate load and store instructions for memory. A selected x86-64 arithmetic form can name a memory operand directly. This difference affects encoding and intermediate

state. It does not by itself determine which program can be related to which. A two-instruction load-then-add and a one-instruction memory add may implement the same observed transformation under suitable fault and alias conditions.

The phrase “data movement” also covers operations that compute no memory access at all. An immediate instruction constructs a value from instruction bits. An address-generation instruction computes a number that may later be used as an address. A move to a special destination can discard the value. What unites the family is not literal copying. It is the controlled production or placement of a word.

The load/store boundary is a design choice

Instruction-set families have repeatedly chosen different places to draw the line between computation and memory access. The useful historical lesson is not that one side is pure and the other compromised. It is that an instruction set exposes one contract while implementations remain free to realize that contract with very different internal work.

This chapter holds the semantic questions steady across three machines. What value is produced? Where is it written? How is an effective address formed? Which failure prevents a write? What state is preserved? Those questions are more durable than a classification label.

7.2 Values, locations, and transfers

A **location** is a place named by the architecture that can hold or yield a value. In our models, ordinary register names and valid memory byte ranges are locations. An immediate is not a location. It is a value encoded in the instruction. An effective address is also a value until a memory operation uses it to select bytes.

We can describe a transfer with five items:

1. the source location or value-producing rule;
2. the source width;
3. the rule that forms the destination-width value;
4. the destination location; and
5. the effect on every other state component, including failure.

Consider an eight-bit source `80`. Read as unsigned, it is `128`. Read as a signed two’s-complement byte, it is `-128`. Zero extension to 16 bits produces `0080`. Sign extension produces `FF80`. The low eight bits agree; the destination words do not.

One byte, two full-width results

Suppose memory byte 12 is 80. A zero-extending byte load returns the 16-bit word 0080. A sign-extending byte load returns FF80. An observation restricted to the low byte cannot distinguish them. A later comparison, shift, store of the full destination, or wide addition can.

Truncation goes the other way. Writing an eight-bit location from a 16-bit source keeps the low eight bits and drops the rest. This is not arithmetic remainder calculated after a signed reading; it is a bit selection. The discarded bits may still survive somewhere else if the destination is part of a larger architectural register. That last possibility is architecture-specific.

The word "move" is therefore too weak for a proof statement. A useful claim says, for example, "copy the low eight bits of register 2 to memory address a, preserving all other bytes," or "load four bytes, assemble them in little-endian order, and sign-extend bit 31 through a 64-bit destination."

7.2.1 Locations may overlap without being independent

Two names can denote overlapping storage. The clearest example is the x86-64 RAX family, but memory ranges overlap too. A four-byte store at address 100 and an eight-byte store at address 96 both include bytes 100 through 103. Treating named locations as unrelated map keys would miss the interaction.

It is often safer to model one underlying object and define views into it. A subregister read extracts selected bits from the containing register. A subregister write replaces selected bits and then applies any architectural rule for the rest, such as zeroing after a 32-bit general-purpose write. Likewise, a memory load extracts a byte interval from one byte-addressed map, and a store updates that interval while preserving every address outside it.

Read footprint and write footprint

The read footprint of a step is the set of state components whose old values can affect the successor. The write footprint is the set of components the step may change. For memory, a footprint includes byte intervals rather than a single abstract memory name. For overlapping registers, it is expressed in the containing architectural storage.

Footprints support composition. Two steps with disjoint write footprints and no write-to-read conflict are candidates to commute. If either writes a byte or register portion the other reads, swapping them can change the result. The footprint test is a necessary structural check; faults and outcome ordering still need semantic reasoning.

A narrow write changes a later wide read

Start with RAX equal to `1122334455667788`. Writing AA to AL changes the containing register to `11223344556677AA`. A later read of RAX observes the narrow write even though its destination name was only AL. A model with independent AL and RAX cells would wrongly preserve the old 64-bit value.

7.3 Ao movement is deliberately explicit

Ao separates register moves, immediate construction, loads, and stores. Every ordinary register has width w, so `mov rd,rs1` copies one complete word. It preserves memory and conditions, writes only the destination register, and advances the program counter by four. If source and destination are the same register, the architectural register map is unchanged, but the step still advances the program counter.

`movi rd,imm` sign-extends byte 3 to width w. At width 16, immediate `7F` produces `007F`; immediate `80` produces `FF80`; immediate `FF` produces `FFFF`. The instruction does not load a byte from data memory. The byte belongs to the immutable program encoding.

For `load rd,[rs1+imm]`, Ao first forms

$$a = R[rs1] + \mathrm{sext}_w(imm8) \pmod{2^w}.$$

It then requires the entire $w/8$-byte range beginning at a to be valid. If so, it assembles those bytes in little-endian order and writes the result to rd. If not, it traps without a partial register write.

The store form uses the same address rule. It writes all $w/8$ low-to-high bytes of $R[rs2]$ atomically in the architectural model. A failed range check changes the outcome to trapped and preserves memory.

A0 move preservation

For a legal `mov rd,rs1` step, prove that every register other than rd is unchanged, memory is unchanged, all four condition bits are unchanged, the outcome remains running, and the program counter advances by four. The proof is mostly projection of the transition definition. Writing the preservation clauses is the point: a destination equation alone does not state a complete instruction theorem.

The first Ao model has no byte load, halfword load, or partial register write. This omission is useful. Extension and truncation can be introduced as explicit bridge operations when we relate a real instruction to Ao rather than being hidden in the small machine's basic move.

7.4 Effective addresses are calculated words

An **effective address** is the offset produced by an instruction's address calculation before the memory system determines whether the requested access is valid. In the flat portions of our models, that offset names the memory byte. More complete machines can add translation, segment bases, privilege checks, and other layers after it.

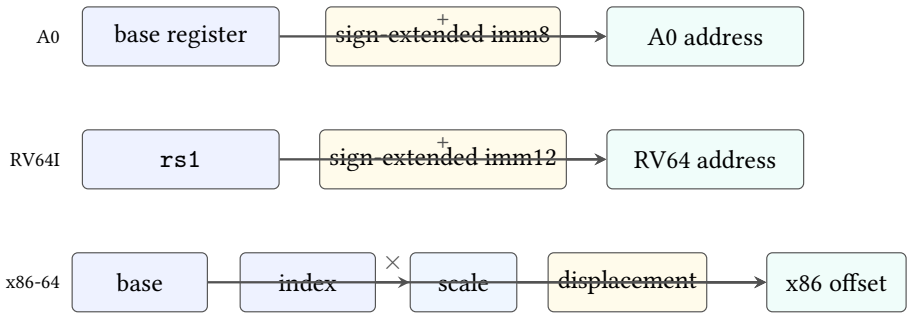


Figure 7.1: Selected effective-address shapes. A0 and RV64I use one base and a sign-extended immediate. A selected x86-64 form can combine base, scaled index, and displacement. Every result still needs a validity check before access.

A0 and the selected RV64I loads share the broad equation

$$a = \mathit{base} + \mathit{signExtendedOffset} \pmod{2^{64}}$$

when A0 is instantiated at width 64. The immediate widths differ: eight bits for A0 and twelve bits for the selected RV64I form. Their encodings and failure environments also differ.

A selected x86-64 memory operand can compute

$$a = \mathit{base} + s \cdot \mathit{index} + \mathit{displacement} \pmod{2^{64}},$$

where the scale s is 1, 2, 4, or 8. A form need not use every component. The encoding can omit a base, omit an index, or use instruction-pointer-relative addressing. This chapter initially uses the ordinary 64-bit address size and a flat address space without nonzero FS or GS bases. Intel's current basic architecture manual supplies the complete mode-dependent rules (Intel Corporation 2026b).

Suppose `rbx=0x1000`, `rcx=3`, the scale is 8, and the displacement is 16. The selected x86-64 address is

$$1000 + 3 \cdot 8 + 16 = 1028.$$

The scale naturally matches an eight-byte array element, while the displacement can locate a field within or beyond the indexed object. The hardware does not know that story. It performs the declared arithmetic and access checks.

Address arithmetic is not an object-bounds proof

An effective address can be numerically valid while pointing outside the programmer's intended array or object. The ISA supplies address arithmetic and architectural faults. Language-level provenance and object bounds require additional models and premises.

Address wrapping and range validity are distinct. If the word-sized addition wraps, the resulting address may still fall inside the finite modeled memory. A theorem that intends to exclude wrap must say so as a precondition. Merely checking the final byte range does not establish that the mathematical sum fit.

7.4.1 Address formation and memory access are separate stages

A load or store proof is clearer when decomposed into stages. First read every old operand needed for the source and address. Next calculate the effective address as a word. Then check the complete byte range and any selected alignment, permission, or address-validity policy. Only after those checks does a load assemble bytes or a store split its source into bytes. Finally, commit one successor state or one declared failure outcome.

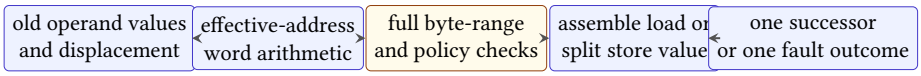


Figure 7.2: A memory transfer crosses several semantic gates. Separating them makes failure ordering, preservation, and negative controls explicit.

This order prevents self-dependence errors. In a load whose destination register also supplies the base, the address uses the old register value. Likewise, a store reads its source word before updating any overlapping memory byte in the atomic architectural step. A functional state update naturally expresses this order when all right-hand sides are evaluated from the entry state.

Range checking must include width. For an n-byte access at word address a, the intended mathematical interval is $[a, a + n)$. Under a finite word address model, proving that interval valid requires more than checking $a + n - 1$ after modular arithmetic. The addition might wrap to a small final address. One safe premise is

$$\langle a \rangle_u + n \leq 2^w$$

together with membership of every address in the modeled readable or writable region.

Full-range validity from endpoint inequalities

Assume ordinary address arithmetic does not wrap, and a permitted memory region is the half-open interval $[b, e)$. An n-byte access beginning at a lies inside it exactly when $b \leq a$ and $a + n \leq e$. The first inequality protects the low endpoint. The second protects every byte through $a + n - 1$. Half-open intervals make an access ending exactly at e valid without admitting byte e.

Alignment is another policy, not a consequence of range validity. An eight-byte access at address 3 can lie wholly within mapped memory while failing a model that requires divisibility by eight. Other selected environments may permit the access, split it internally, or report a different failure. A machine-code theorem must pin the policy actually used rather than infer it from the operand width.

Do not merge all memory failures

An out-of-range address, an alignment violation, a permission failure, and a later translation fault can preserve the same destination while arising at different stages. If the model distinguishes them or their priority, the trace and refinement relation must distinguish them too.

7.5 RV64I movement: widths are in the instruction

Our RV64I window uses the base integer ISA at XLEN 64. Register `x0` always reads as zero, and writes to it have no architectural effect. The other 31 integer registers hold 64-bit words. Loads and stores form addresses by adding a sign-extended 12-bit immediate to `rs1` (RISC-V International 2026b).

`ld x5,16(x6)` reads eight bytes at `x6+16`, assembles a 64-bit little-endian word, and writes it to `x5`. `sd x5,16(x6)` writes the low 64 bits of `x5` to the same byte range. These are the closest selected counterparts to width-64 Ao load and store.

Narrow loads make the value rule visible. `lw` loads four bytes and sign-extends bit 31 through the 64-bit destination. `lwu` loads the same four bytes and fills bits 63–32 with zeros. Likewise, `lb` and `lbu` differ only in how they form the high 56 destination bits; the memory range and low byte can be identical.

Stores do not extend. `sb`, `sh`, and `sw` write the low 8, 16, or 32 bits of `rs2`. The high source bits are ignored for that access. Neighboring bytes outside the selected range remain unchanged.

One RV64 word load, two destination values

Let bytes at addresses a through $a+3$ be `00 00 00 80`. Little-endian assembly gives the 32-bit word `80000000`. An `lw` produces `FFFFFFFF80000000`; an `lwu` produces `0000000080000000`. Both read the same bytes. The instruction's extension rule creates the difference.

7.5.1 Extension is a theorem about all destination bits

Let u be the unsigned reading of an n-bit source and let the destination width be $m > n$. Zero extension preserves the same nonnegative reading:

$$\langle \mathsf{zext}_m(x) \rangle_u = u.$$

Sign extension instead preserves the source's signed two's-complement reading. If the source high bit is zero, both extensions agree. If it is one, sign extension fills the new high bits with ones while zero extension fills them with zeros.

This gives an exact agreement condition useful in optimization proofs:

$$\mathsf{zext}_m(x) = \mathsf{sext}_m(x) \quad \text{iff} \quad \mathsf{bit}_{n-1}(x) = 0.$$

The forward direction follows because the new high bits differ when the source high bit is one. The reverse direction follows because both operations then copy the low n bits and fill every new bit with zero.

A narrow load has two independent obligations

First prove that the instruction reads exactly the selected $n/8$ bytes and assembles the intended source word in the declared byte order. Then prove that its extension rule forms the full m-bit destination. Matching the low source bits does not discharge the high-bit obligation, and matching the final word does not by itself prove that the correct memory interval was read.

Narrow stores have the dual shape but no extension. They select the low source bits, split them according to byte order, and update exactly the destination interval. A round trip through a narrow store and wider sign-extending load does not generally recover the original wide register; it recovers the signed extension of the stored low portion.

An immediate arithmetic instruction can also serve movement. `addi x5,x6,0` copies `x6` to `x5` as an assembler pseudoinstruction commonly spelled `mv x5,x6`. The hardware-visible instruction is still ADDI. If the destination is `x0`, the calculated result is discarded. Decoding must retain `rd=0`; execution enforces the special destination behavior.

Pseudoinstructions belong to the notation layer

A pseudoinstruction can make source code easier to read without adding an ISA operation. Proof artifacts should record the decoded base instruction and its operands. The chapter may print the convenient spelling beside it, but should not count the alias as another machine instruction.

7.6 x86-64 movement: register names carry width

x86-64 names overlapping portions of a general-purpose register. In the RAX family, `rax` names 64 bits, `eax` the low 32, `ax` the low 16, and `al` the low 8. These are not four independent storage cells. The destination name and instruction form determine what happens to the whole architectural register.

In 64-bit mode, a 32-bit write to a general-purpose register zeros its upper 32 bits. Thus `mov eax, ebx` writes the low 32 bits from EBX and leaves RAX with a zero high half. A 16-bit write to AX or an 8-bit write to AL does not receive that automatic full-register zeroing rule; the unselected higher bits of RAX remain as specified by the instruction form.

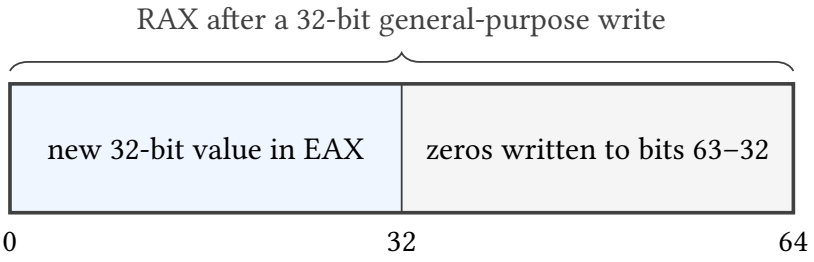


Figure 7.3: A selected 32-bit general-purpose destination write in 64-bit mode defines all 64 bits of the containing register: the high half becomes zero.

Let RAX initially be `1122334455667788`. After writing `AABBCCDD` to EAX, RAX is `00000000AABBCCDD`. After instead writing `EEFF` to AX, RAX is `112233445566EEFF`. A proof that observes only the named destination width misses the difference in the containing register.

x86-64 also distinguishes moves that extend a smaller source. A selected zero-extending move fills the new high bits with zero; a selected sign-extending move copies the narrow sign bit. Ordinary MOV between equal-width register or memory operands does neither. The exact opcode and operand sizes must be named before the word "move" has a complete value rule. The pinned Intel instruction reference provides those form-specific contracts (Intel Corporation 2026a).

The LEA instruction is a useful boundary case. For a selected 64-bit form, it computes the offset described by a memory-style addressing expression and

writes that number to a register. It does not read the memory bytes at that address. Therefore a bad pointer can be harmless under LEA and fault under a MOV load that uses the same printed brackets.

Address calculation is not memory movement

Choose base, index, scale, and displacement so the calculated offset lies outside the chapter's valid memory. Compare a selected LEA-like address computation with a load from that address. The first can still produce the numeric word; the second must pass an access check before writing its destination.

Instruction-pointer-relative addressing adds a timing detail to the address equation. In a selected x86-64 form, the base is the address following the instruction, not its first byte. Because instruction length is determined by decode, address formation depends on faithful length decoding. Relocating the instruction and its referenced object together can preserve the displacement, but changing the encoding length at the same address changes the base and may require displacement repair.

The scale field also has limits. It multiplies the index by 1, 2, 4, or 8; it does not encode an arbitrary element size. An array of 12-byte records may need an earlier arithmetic instruction or a combination such as three times an index followed by scale four. The effective-address theorem should describe the actual arithmetic sequence rather than attribute source-level array semantics to one addressing form.

An address expression has no inherent object type

The same base-plus-index expression can locate an array element, a structure field, a table entry, or bytes outside every intended object. Machine semantics provides the numeric word and access behavior. Object identity and provenance enter only through a stronger language or compiler relation.

7.7 Relating selected movements

A real instruction refines an Ao movement when related starting states lead to related results under printed preconditions. The relation can translate register names and instruction addresses. It cannot discard a destination-width effect that a later observation can reveal.

For width-64 Ao load and RV64I `ld`, a simple relation can map one Ao register to `rs1`, another to `rd`, and equate their finite memory windows byte for byte. The immediate relation must equate Ao's sign-extended eight-bit offset with RV64I's sign-extended twelve-bit offset for the selected values. Both accesses must be

valid. The post-relation equates destination words, preserved memory, running outcomes, and logical successor positions.

The same direct relation does not cover `lw`. Ao's width-64 load reads eight bytes, while `lw` reads four and sign-extends. We can instead use a width-32 Ao instance and add an explicit sign-extension bridge, or define a more abstract operation that reads four bytes and produces a 64-bit extended result. The model should change visibly rather than pretending the widths match.

For x86-64 `mov eax, ebx`, an Ao width-32 move can match the low result. To relate full 64-bit states, the bridge must also require the x86 destination's upper half to be zero after the step. If the Ao state has only 32-bit registers, that fact may live in a representation invariant rather than a destination equality.

Fault agreement is part of movement refinement

Two instructions do not refine the same operation merely because their successful destination values agree. If one can trap on an address where the other succeeds, either exclude that input, relate the fault outcomes, or weaken the claim. Silently comparing only successful runs changes the theorem.

Aliasing adds another premise. Suppose the destination register contributes to the address. An architectural instruction reads the old operand values needed for its address and source before committing the destination write. A symbolic implementation that updates the register first can compute the address from the new value and still look plausible on non-aliasing tests. Read sets and write order should be explicit even when the mathematical transition is presented as one atomic step.

7.7.1 When two memory movements may commute

Reordering independent loads and stores is central to compilers and processors, but independence is a semantic claim. Two stores to disjoint byte intervals commute in this chapter's atomic memory model when both addresses and source values are already fixed by state neither store changes. Applying either store first yields the same final byte map because each update preserves every byte written by the other.

Overlap breaks the argument. If one store writes $[a, a + 4)$ and another writes $[a + 2, a + 6)$, their final values at bytes $a + 2$ and $a + 3$ depend on order. A load and store also fail to commute when the load reads a byte the store changes. Even disjoint memory intervals are insufficient if a destination register of the first load supplies the second instruction's address.

Disjoint stores commute

Let store A update interval I with byte function v_A, and store B update disjoint interval J with v_B. Consider an arbitrary address q. If $q \in I$, both orders leave $v_A(q)$ because $q \notin J$. If $q \in J$, both leave $v_B(q)$. Outside $I \cup J$, both preserve the entry byte. The final memories are therefore equal pointwise. The argument also requires both steps to have the same successful outcomes in either order.

Faults make the last sentence load-bearing. Suppose the first store succeeds and the second faults. In an architecture with one committed instruction per step, swapping them changes whether the first store becomes visible before the fault. A refinement that observes only final successful runs hides this difference. Reordering needs a premise that both accesses succeed, or a richer argument about permitted fault behavior.

The same footprint reasoning explains why a single x86-64 memory-source arithmetic instruction cannot always be replaced by a load followed by register arithmetic. The expanded sequence introduces an intermediate state and perhaps a different fault boundary. Under a no-fault premise and an observation only at the sequence endpoints, the replacement may still be valid; without those conditions, instruction count is not the only change.

7.8 How Axeyum enters this chapter

The live Axeyum checkout still lacks Ao, RV64I, and x86-64 architectural packages. This chapter therefore defines the first data-movement obligations rather than reporting completed machine proofs.

Start with an executable Ao transfer package. Represent the register map, finite byte memory, and running/trapped outcome. Implement `mov`, `movi`, `load`, and `store`. Reuse Axeyum's bit-vector machinery for extension, truncation, word addition, and byte extraction. Every operation should return a complete successor state rather than only its main result.

The smallest useful property family includes immediate sign extension, store-then-load reconstruction, preservation of untouched registers and bytes, range failure without partial writes, and wrapped-address controls. Run small widths exhaustively where possible. Use solver search at larger widths, but replay any model through the executable transition.

Then add thin adapters for the selected real forms. The RV64 adapter needs `x0`, 12-bit offsets, narrow load extension, and narrow stores. The x86-64 adapter needs selected subregister writes and the chosen effective-address form. Each adapter should name its manual revision and refuse every form outside its declared slice.

Build the checker around the staged pipeline in Figure 7.2. Record the old operands separately from the effective address so a mutation that substitutes the

new destination value can be detected. Record the requested width and full interval so a first-byte-only range check cannot pass. Reconstruct loaded values and store bytes inside the checker rather than trusting either derived field in the artifact.

The initial control set should change one fact at a time: immediate sign extension, address wrap, access width, byte order, load extension, store truncation, x86 subregister preservation, and fault atomicity. Each control needs a small witness whose failure class is known in advance. That structure turns an Axeyum example into a lesson about the contract rather than a single opaque pass result.

A good first solver question

Ask whether zero-extending and sign-extending an n-bit value to a wider word ever agree when the source high bit is one. The solver should report no such value. Flip the premise to high bit zero and prove they always agree. The pair tests extension definitions, premise handling, and the negative-control route before any full instruction semantics depends on them.

An artifact for this chapter will eventually record inputs, initial memory, decoded instruction, calculated effective address, bytes read or written, full successor state, source pins, Axeyum revision, and control results. Until such an artifact is active, the examples remain reader-checkable specification work.

7.9 Where the model stops

This chapter uses finite byte memory and single-threaded atomic instruction steps. It does not model caches, translation lookaside buffers, page tables, weak memory, concurrency, memory-mapped devices, or speculative execution. It does not claim the full RV64I or x86-64 addressing repertoire.

The x86-64 discussion assumes ordinary 64-bit address size and a flat segment base for the selected examples. The RV64I discussion leaves access-fault and misalignment policy to a later execution-environment pin where the base unprivileged ISA does. These limits are not footnotes to the theorem. They say which machine the theorem is about.

Within that machine, something broad has become exact. Ao, RV64I, and x86-64 can spell movement differently, yet the comparison reduces to finite words, named locations, address equations, byte ranges, and observations. Once those objects are written down, resemblance can give way to a relation that either holds or produces a counterexample.

The durable lesson is to resist treating movement as the easy part between interesting calculations. A value has a width and interpretation; a location has a footprint; an address has an arithmetic derivation; an access has a range and failure policy; and a destination write has a rule for the rest of its containing storage.

Those facts determine whether later arithmetic and control flow receive the state the programmer intended.

This decomposition also gives a practical proof order. Establish operand reads, calculate the address, justify the complete access, construct the destination value, and finally prove preservation. When a witness disagrees, the failed layer tells the reader what kind of movement error occurred.

7.10 Exercises

1. **Execute.** At Ao width 16, trace `movi r2,0x80`. Give the complete destination word, next program counter, preserved state, and outcome.

2. Trace a width-16 Ao store followed by a load at the same address. List the two bytes after the store and reconstruct the loaded word.

3. **Break.** Change the Ao load implementation so it checks only the first byte of the range. Construct a final-address example that exposes a partial or out-of-range read.

4. Give an eight-bit source for which zero extension and sign extension to 64 bits differ. Give a second source for which they agree, and explain why.

5. Compute the selected x86-64 effective address for base `0x2000`, index 5, scale 4, and displacement -8. Show the word arithmetic before any memory validity decision.

6. Starting from RAX `FFEEDDCCBBAA9988`, compute full RAX after writing `12345678` to EAX, `1234` to AX, and `12` to AL in three separate initial states.

7. Compare RV64I `lw` and `lwu` on bytes `FF FF FF FF`. Construct one observation that distinguishes the results and one that does not.

8. Explain why `addi x5,x6,0` may be printed as a move while its decoded instruction remains ADDI. What should an evidence manifest record?

9. **Prove.** State and prove the preservation clauses for an Ao register move. Include the case where source and destination are equal.

10. Prove that two disjoint Ao stores commute under the chapter's atomic memory model. Identify the premise that fails when the byte ranges overlap.

11. **Transfer.** Write a state relation for one width-64 Ao load and RV64I `ld`. Name the register map, memory agreement, offset condition, validity premise, and post-state observation.

12. Extend that relation to a selected x86-64 load using base plus scaled index plus displacement. State which address-size and segment assumptions are required.

13. Design an Axeyum negative control that changes sign extension to zero extension. Give one concrete witness it must find and the failure class the checker should report.

CHAPTER EIGHT

Arithmetic, Logic, and Condition State

At fixed width, addition produces a stored word even when the mathematical sum lies outside the word's unsigned or signed range. Carry and signed overflow describe different facts about that event. Confusing them is easy; making both rules explicit is easier.

Arithmetic also changes more than numbers. One machine writes condition bits that a later branch can read. Another writes only a destination register and asks a branch to compare registers directly. A third can materialize a Boolean answer as an ordinary word. The calculation and the place where its facts are recorded must both enter the proof.

A result word does not contain every arithmetic fact

The low w bits determine the stored result. They do not by themselves say whether the unsigned sum crossed 2^w, whether the signed sum left its range, or which condition state an instruction promised to update. Those are separate parts of the transition.

8.1 Why machines record arithmetic conditions

A program rarely computes only to keep the result. It also asks whether the result was zero, whether one value was less than another, whether an unsigned addition carried, or whether a signed calculation left its representable range. Early arithmetic units exposed some of these facts as condition state because the next instruction could branch on them without storing another ordinary word.

That design remains visible in x86-64. Selected arithmetic and compare forms write named status flags, and later conditional instructions read Boolean formulas over those flags. RISC-V's base integer design usually keeps the condition in the instruction that needs it or writes an explicit zero-or-one comparison result. Ao uses four condition bits so that the hidden-state style can be printed in full before we compare it with the alternatives.

Neither state shape is intrinsically more semantic. A flag is architectural state if later instructions can observe it. A Boolean register is ordinary state with a convention about its value. A direct branch comparison can avoid materializing either. What matters is whether the surrounding program receives the information it needs.

Condition state is a compact memory of a calculation

A condition code preserves selected facts while discarding the operands and, for a compare instruction, often the arithmetic result itself. It is a small summary of a larger event. The summary is useful precisely because it is incomplete; proofs must therefore say which later questions it can answer.

8.2 One addition, four separate questions

Let x and y be width-w words. Their ordinary nonnegative readings lie from zero through $2^w - 1$. Write

$$s = \langle x \rangle_u + \langle y \rangle_u$$

for the mathematical unsigned sum and

$$r = s \bmod 2^w$$

for the stored word. Four questions now have distinct answers:

1. What is the stored result r?
2. Is r zero?
3. Is the high bit of r one?
4. Did the unsigned or signed mathematical reading leave its range?

Ao names the corresponding bits Z, N, C, V. Zero and negative depend only on the stored result:

$$Z \quad \text{iff} \quad r = 0, \qquad N = \mathrm{bit}_{w-1}(r).$$

Carry records an unsigned boundary:

$$C \quad \text{iff} \quad s \geq 2^w.$$

Signed overflow records a signed boundary. It holds exactly when the operands have the same high bit and the result has the other high bit.

The upper-left example in the figure adds `1111` and `0001`. The unsigned readings are 15 and 1, so the mathematical sum reaches 16 and sets carry. The signed readings are -1 and 1, whose sum is representable, so signed overflow is clear. The lower-right example adds 7 and 1. It has no unsigned carry, but the signed result 8 is outside the width-four signed range and sets overflow.

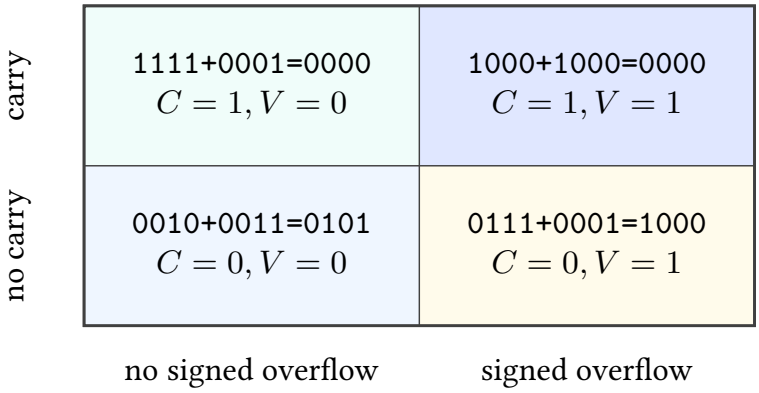


Figure 8.1: Width-four addition supplies examples in all four carry and signed-overflow combinations. Neither condition implies the other.

8.2.1 The widened identity separates storage from range

Because two unsigned width-w readings sum to at most $2^{w+1} - 2$, one extra bit is enough to hold their exact sum. There are unique values $C \in \{0, 1\}$ and $0 \le r < 2^w$ such that

$$\langle x \rangle_u + \langle y \rangle_u = C2^w + r.$$

The low part r is the stored word. The coefficient C is the carry. No approximation or particular circuit is involved. The equation is Euclidean division by 2^w.

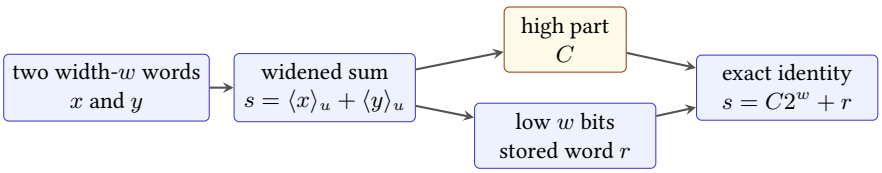


Figure 8.2: Widening makes the unsigned arithmetic exact. Truncation stores the low word, while the high bit records whether the unsigned range was crossed.

The identity gives two interchangeable definitions:

$$r = s \bmod 2^w, \qquad C = 1 \quad \text{iff} \quad s \ge 2^w.$$

It also yields a useful test that avoids a wider host type. For width-w unsigned addition, carry occurs exactly when the stored result is below either operand. If no carry occurs, $r = x + y$ is at least each operand. If carry occurs, subtracting 2^w makes r smaller than each positive contributor. A proof assistant or solver can check this equivalence directly over bit vectors.

The stored word has another algebraic meaning. Width-w words under addition form arithmetic modulo 2^w. Addition is associative and has zero as an identity

even when intermediate mathematical sums cross the unsigned boundary. Carry is not part of that modular result; it is extra information about the chosen unsigned representatives. This distinction lets one theorem establish a modular calculation while another establishes that no range boundary was crossed.

One result, two claims

At width eight, `FA + 0A` stores `04`. The modular equation $250 + 10 \equiv 4 \pmod{256}$ holds without a precondition. The ordinary integer equation $250 + 10 = 4$ is false, while the widened identity $250 + 10 = 1(256) + 4$ is exact. A result theorem must say which of these meanings it promises.

Say which overflow you mean

The word *overflow* is incomplete in a fixed-width claim. Unsigned carry and signed overflow answer different range questions and can disagree. Name the reading, range, and condition bit.

8.2.2 Zero and negative are readings of the stored word

The zero condition has no signedness ambiguity. A width-w word has unsigned reading zero exactly when all bits are zero, and its signed reading is then also zero. Thus Z can support equality-to-zero tests for either reading. What produced the word is irrelevant once the instruction policy says that Z is defined from its result.

The negative condition is different. N copies the high bit, so it says that the stored word has a negative two's-complement reading. It does not say that an unsigned result is below zero, and it does not by itself say that a signed mathematical calculation produced a negative value. If signed overflow occurred, the stored signed reading has wrapped away from the mathematical result.

These facts explain several branch formulas. After a subtraction that does not overflow, N alone agrees with signed less-than. Without that premise, $N \neq V$ repairs the wrapped cases. Equality uses Z without a range premise. Unsigned order uses the borrow convention rather than N, because the high bit is simply part of the unsigned magnitude.

Zero after subtraction means equal words

Modular subtraction stores zero exactly when $x - y \equiv 0 \pmod{2^w}$. Both unsigned readings lie in an interval of length 2^w, so their difference lies strictly between -2^w and 2^w. The only multiple of 2^w in that range is zero. Therefore the readings, and hence the words, are equal. The reverse direction is immediate.

Condition names still need an instruction policy. A move might leave Z unchanged even when it writes a zero word; a logic instruction might define Z from its result; a compare might define Z from a subtraction it does not store. The mathematical predicate and the architectural decision to record it are separate claims.

8.3 Deriving the signed-overflow rule

The sign rule is not a hardware incantation. It follows from the signed range. If one operand is nonnegative and the other negative, their sum lies between them. It cannot exceed both ends of the representable interval. Therefore opposite-sign addition cannot overflow.

If both operands are nonnegative, signed overflow occurs exactly when the mathematical sum is at least 2^{w-1}. The stored high bit is then one, so the result appears negative. If both operands are negative, their sum can fall below -2^{w-1}. Modular reduction then yields a stored word whose high bit is zero. These are precisely the same-input-sign, different-result-sign cases.

Writing $h(z)$ for the high bit of word z, the rule is

$$V = \neg(h(x) \operatorname{xor} h(y)) \operatorname{and} (h(x) \operatorname{xor} h(r)).$$

A reader proof of addition overflow

Split on whether the two input high bits agree. When they differ, one signed operand is nonnegative and the other negative, so the sum stays within range. When both are zero, overflow means the nonnegative sum reaches the signed upper boundary, which makes the stored high bit one. When both are one, overflow means the negative sum crosses the signed lower boundary, which makes the stored high bit zero. These cases match the Boolean formula.

This proof handles every width $w > 0$. Exhausting all 256 pairs at width four is still useful, but it checks an instance. The case argument explains why the same rule holds at width 8, 32, 64, or another positive width.

The boundary cases are worth naming. Adding one to the greatest signed positive word sets V and produces the least signed word. Adding negative one to the least signed word may set carry while clearing V, because the signed sum remains representable. Adding the two least signed words sets both conditions at widths above one. These examples defeat the vague rule that overflow means the high bit changed: many legitimate sums change the high bit, and some overflowing sums wrap to a word with the same high bit as one operand.

8.4 Subtraction, borrow, and comparison

Subtraction stores

$$r = x - y \bmod 2^w.$$

Ao defines C as *no unsigned borrow*: it is one exactly when the unsigned reading of x is at least that of y. This polarity must be printed because other conventions may name or invert the condition differently.

At width four, `0011-0101` stores `1110`. Since unsigned 3 is below 5, a borrow was needed and Ao clears C. Conversely, `0101-0011` stores `0010` and sets C. In both cases, Z and N come from the stored result.

Signed subtraction overflows exactly when the input signs differ and the result sign differs from the minuend's sign. Subtracting a negative number from a nonnegative one can become too positive. Subtracting a nonnegative number from a negative one can become too negative. Same-sign subtraction cannot cross a signed boundary in that way.

Ao's `cmp rs1,rs2` performs the subtraction condition calculation without writing the result to an ordinary register. Its later signed less-than predicate is $N \neq V$. Its unsigned lower-than predicate is $\neg C$. Equality is Z. These formulas recover comparisons from the retained summary.

> **Why the sign bit alone is not signed less-than**
>
> At width four, compare `0111` with `1111`. The signed readings are 7 and -1, so 7 is not less. The stored subtraction is `0111-1111=1000`, whose high bit sets N. Signed overflow also sets V, so $N \neq V$ is false. Reading N alone would give the wrong answer.

The correction by V is the compact trace of wraparound. When signed subtraction stays in range, the result sign tells the ordering. When it overflows, that sign is reversed from the mathematical ordering, and the overflow bit reverses the decision back.

8.4.1 Subtraction can be read three ways

The same stored subtraction supports three claims. As a word equation,

$$r \equiv x - y \pmod{2^w}$$

always holds. As an unsigned natural subtraction, $r = x - y$ holds without wrap exactly when $\langle x \rangle_u \geq \langle y \rangle_u$. As a signed integer subtraction, the stored signed reading equals the mathematical difference exactly when $V = 0$. A theorem that promises correct subtraction must choose among these meanings.

Subtraction may also be implemented algebraically as addition of a two's-complement inverse:

$$x - y \equiv x + (\sim y) + 1 \pmod{2^w}.$$

This identity explains why an arithmetic unit can reuse addition machinery, but it does not settle flag polarity. The final carry-out of this addition is one exactly when no unsigned borrow is needed under Ao's convention. An ISA that defines a borrow flag with the opposite polarity must invert that fact at its architectural boundary.

Unsigned comparison from subtraction

Let $a = \langle x \rangle_u$ and $b = \langle y \rangle_u$. If $a \geq b$, the ordinary difference lies in $[0, 2^w - 1]$, so modular reduction changes nothing and Ao sets no-borrow C. If $a < b$, the negative difference is represented by adding 2^w, which is precisely the wrap associated with a borrow, and Ao clears C. Thus $\neg C$ is equivalent to unsigned less-than and C to unsigned greater-than-or-equal.

Equality needs less retained information. Modular subtraction stores zero exactly when the two width-w words are identical, so Z recovers word equality without a signedness choice. Signed and unsigned order disagree on some pairs, but equality does not: both readings are one-to-one over the same finite set of words.

Negation reveals a special signed boundary. The least signed word represents -2^{w-1}. Its mathematical negation 2^{w-1} is not representable, so modular negation returns the same word and sets signed overflow under a policy that reports it. Every other signed word has a representable negation. This is why an absolute-value routine needs an explicit policy for the least signed input.

One subtraction, two orderings

At width eight, compare FF with 01. Unsigned 255 is greater than 1, so no borrow is needed and $C = 1$. Signed -1 is less than 1, and the signed predicate $N \neq V$ is true. The same subtraction summary answers both questions because the branch chooses a different formula.

8.5 Logic and shifts need condition policies too

Ao's AND, OR, XOR, and NOT instructions set Z and N from the result and clear C and V. The clearing is a policy, not a theorem about Boolean algebra. Another ISA may preserve, clear, define, or leave selected condition bits undefined for a corresponding operation.

The distinction matters after a sequence. Suppose an addition sets carry, a logical operation produces the same destination in two candidate programs, and a later branch reads carry. If one logical semantics clears carry while another preserves it, destination-only tests miss the difference.

Shifts add a further question: which bit was last shifted out? Ao uses the low unsigned value of $rs2$ modulo w as the count. A zero-count shift preserves the source result and clears C. A nonzero shift writes the last bit removed into C, sets Z, N, and clears V.

For width eight, shifting `10110001` left by three produces `10001000`. The last bit removed is the original bit 5, which is one, so $C = 1$. Logical right shift fills from the high side with zeros. Arithmetic right shift repeats the original high bit. The two right shifts agree for a source high bit of zero and can differ when it is one.

Follow the bit that becomes carry

Write the source bits above positions 7 through 0. For left shifts by 1, 2, and 7, circle the last bit that leaves the word. Repeat for logical right shifts. Then test count zero separately; do not infer its condition rule from a diagram showing movement.

Large and zero shift counts are frequent semantic fault lines. Ao's modulo-width rule is part of its model. RV64I and x86-64 selected forms have their own count rules. A cross-ISA theorem must either relate those rules on a restricted domain or expose the difference.

8.5.1 A shift theorem needs a count domain

For a logical left shift by a mathematical count k with $0 \le k < w$, the stored result has unsigned reading

$$(\langle x \rangle_u 2^k) \bmod 2^w.$$

For a logical right shift in the same count range, its reading is

$$\left\lfloor \frac{\langle x \rangle_u}{2^k} \right\rfloor.$$

These clean formulas do not define what happens for $k = w$, larger counts, or a negative mathematical count. The instruction semantics must first map its encoded or register-supplied count into an effective count domain.

Ao chooses the low unsigned count modulo w. At width eight, raw counts 0, 8, and 16 therefore all have effective count zero. A theorem restricted to raw counts below eight can use the simple formulas directly. A theorem over every input word must include the modulo operation. Confusing raw and effective counts is

especially dangerous in cross-ISA work because two machines can agree on small counts and diverge exactly at the boundary.

Arithmetic right shift needs a signed reading. For an effective count below the width, it behaves like division by 2^k rounded toward negative infinity under the usual sign-extension rule. That rounding matters: arithmetic shift of -3 by one produces -2, whereas integer division specified to truncate toward zero would produce -1. Replacing a division with a shift therefore needs a premise or correction term, not just a claim that both operations divide by a power of two.

The carry bit from a nonzero shift is yet another output. For left shift by k, A0 records original bit $w - k$; for right shift it records original bit $k - 1$. The formulas apply only after proving $1 \leq k < w$. Count zero follows its own rule and cannot index either expression safely.

Normalize the count before indexing a bit

An implementation that calculates the shifted-out bit from the raw count may index outside the word even when the result shift masks that count correctly. Derive the effective count once, branch on zero, and use the same value for both the result and condition calculation.

8.6 Three ways to carry a condition forward

The state paths differ across our machines.

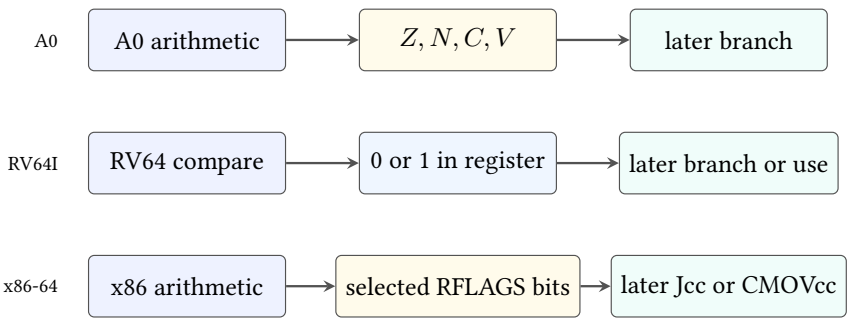


Figure 8.3: Selected ways to preserve a decision for later use. A0 and x86-64 write condition state; RV64I can write an explicit Boolean register.

RV64I base ADD and SUB write the destination register but no x86-style arithmetic flags. `slt rd,rs1,rs2` writes one when the signed reading of `rs1` is less than that of `rs2`, otherwise zero. `sltu` uses unsigned readings. Branch instructions can compare registers directly. These rules come from the pinned unprivileged specification (RISC-V International 2026b).

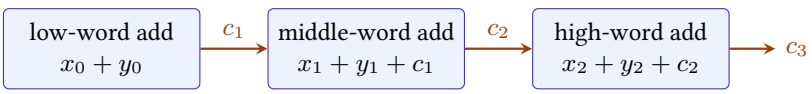
x86-64 selected ADD, SUB, and CMP forms update arithmetic status flags including CF, ZF, SF, and OF. CF records the unsigned carry or borrow convention of the selected operation; OF records signed overflow; ZF records zero; SF records the result high bit. Later conditional forms interpret combinations of those bits. The exact affected and undefined flags remain form-specific in Intel's pinned instruction reference (Intel Corporation 2026a).

Ao's four names resemble that x86 subset, but resemblance is not identity. Ao deliberately defines one no-borrow polarity for subtraction and explicit policies for logic and zero-count shifts. A relation must map predicates after checking these definitions, not map letters because they look familiar.

> **Compare information, not storage shapes**
>
> An Ao condition bit, an RV64 zero-or-one register, an RV64 direct branch predicate, and an x86-64 flag formula can represent the same decision. A cross-machine relation should equate that decision on the declared states. It need not invent identical condition registers on all three machines.

8.7 Carry chains turn condition state into data

A word is often only one limb of a larger integer. Let a three-word unsigned number use little-indexed limbs x_0, x_1, x_2, each of width w. Add the low limbs, store result limb r_0, and pass carry c_1 into the next addition. Continue until the high limb produces final carry c_3.



Each carry is data for the next word, whether stored in a flag or an ordinary register.

Figure 8.4: Multiprecision addition makes each carry a live input to the next word. The storage location differs by ISA, but the mathematical recurrence is the same.

For limb i, define the exact widened sum

$$s_i = \langle x_i \rangle_u + \langle y_i \rangle_u + c_i = c_{i+1} 2^w + \langle r_i \rangle_u,$$

with $c_0 = 0$. Multiplying the equation for limb i by 2^{wi} and summing makes every internal carry cancel. The result is

$$X + Y = c_3 2^{3w} + R,$$

where X, Y, R are the three-word unsigned readings. The local widened identity therefore composes into a whole-number theorem.

This use makes condition liveness concrete. On a flag-based machine, an instruction inserted between two limbs must not destroy the carry before the next add-with-carry consumes it. On a machine without an architectural carry flag, code may calculate the carry into an ordinary register and add it to the next limb. The state shapes differ, but both proofs need the same recurrence and must protect c_{i+1} across the intervening steps.

Signed multiprecision addition usually uses the same limb operations. Signed overflow is judged at the high end from the full-width operand and result signs, not by combining the per-limb overflow bits. Internal limbs are pieces of one large representation; interpreting each as a separate signed integer answers the wrong question.

Two-limb carry composition

Write the low identity as $x_0 + y_0 = c_1 2^w + r_0$ and the high identity as $x_1 + y_1 + c_1 = c_2 2^w + r_1$. Multiply the high equation by 2^w and add the low equation. The terms $c_1 2^w$ cancel from opposite sides, leaving the exact two-limb sum with low limbs r_0, r_1 and final carry c_2.

8.8 Observed and unobserved conditions

Consider two instruction sequences that always leave the same destination word. One writes the correct conditions; the other leaves old conditions in place. Under an observation of the destination alone, they can be equivalent. Before a condition-reading branch, they need not be.

Take width-eight addition of `FF` and `01`. Both sequences store `00`. The correct Ao conditions include $Z = 1, C = 1, V = 0$. If the faulty sequence leaves $C = 0$, a later `branch.hs` chooses another path. The destination-only claim and the flag-sensitive counterexample are both valid because they use different observations and contexts.

The distinction determines legal optimization as well as good testing. A compiler may replace an instruction with a flag-different sequence only when the surrounding program cannot observe those flags before they are overwritten. That fact can come from liveness analysis, a typed context, or a theorem whose observation omits the condition state.

Dead conditions require evidence

Conditions are not unobservable merely because the current basic block does not print them as outputs. A successor branch, conditional move, carry chain, or surrounding context may read them. Prove that the relevant condition state is dead or preserve it.

The same reasoning applies to explicit Boolean registers. If an RV64 comparison result is stored in a scratch register that the observation omits, changing it may be harmless in one closed routine. If later code reads the register, the difference is visible. Hidden versus explicit state changes the analysis tools, not the need for a scope.

8.8.1 Condition liveness is ordinary backward reasoning

To decide whether a condition write matters, begin at the later use and reason backward. A branch that reads Z makes the most recent reaching definition of Z live. Earlier writes are dead if every path to the branch overwrites Z first. If one path preserves an earlier value, that definition remains live on that path. The same analysis applies separately to C, V, and N; treating the whole condition register as one indivisible value can be safe but unnecessarily conservative.

Partial condition writes complicate the picture. If an instruction defines Z, N while preserving C, a later unsigned branch may depend on a carry created several instructions earlier. If the architecture leaves a condition undefined, a theorem cannot retain its old value unless the selected semantics explicitly chooses that behavior. Undefined, preserved, and cleared are three different policies.

An optimization proof can encode liveness as an observation boundary. Show that both sequences agree on every location read before its next common overwrite. Conditions absent from that set may differ. This local relation is strong enough to justify replacement in the chosen context without making the false global claim that the instruction sequences have identical complete states.

> **A carry survives an unrelated-looking operation**
>
> Suppose an addition creates a carry for the next limb. A candidate replacement inserts a logical instruction before the add-with-carry. If that logical form clears C, the arithmetic destinations produced so far can still match, yet the next limb differs. Backward liveness from the add-with-carry identifies C as a required observation across the inserted instruction.

8.9 How Axeyum can check the arithmetic layer

The current Axeyum checkout provides reusable bit-vector and solver machinery but not the Ao or real-ISA transition packages. The first implementation goal for this chapter is a width-parametric arithmetic record containing the result word and declared conditions. It should not begin with a full processor.

Implement pure functions for addition, subtraction, logic, and shifts. At small widths, enumerate every input pair and compare the implementation with independent mathematical definitions. At larger widths, ask a solver for a counterexample

to each equivalence: carry versus extended unsigned sum, signed overflow versus the sign formula, subtraction no-borrow versus unsigned order, and compare predicates versus signed or unsigned readings.

Every solver model should replay through the executable function. Negative controls should invert carry, confuse carry with overflow, use N instead of $N \neq V$ for signed comparison, preserve a logic flag that Ao clears, and mis-handle count zero. Each mutation has a small concrete witness, so a green route that cannot reject it is not exercising the promised rule.

The first solver obligations can be small and independent of instruction decoding. For arbitrary width-w bit vectors x, y, build an extended $(w + 1)$-bit sum by zero-extending both operands. Compare its low w bits with the executable result and its high bit with executable carry. For signed overflow, compare the executable bit with the high-bit Boolean formula. Ask the solver for a state where either equality fails.

Subtraction needs its own obligation rather than reuse by name. Compare executable no-borrow with unsigned $x \geq y$, and executable signed less-than with the signed bit-vector predicate. Shift checks quantify over a raw count, derive Ao's effective count, and split zero from nonzero before stating the shifted-out-bit formula. These separate formulas make a failed control diagnostic: a carry mutation should not be reported as an unrelated signed-overflow failure.

At a teaching width, a census should retain counts as well as a pass bit. For addition, partition every ordered pair by (C, V), then retain one witness from each nonempty class. Requiring all four classes prevents a checker from passing only easy rows. A second census for subtraction should cover equality, unsigned less-than, signed less-than, and the cases where the two orderings disagree.

The future Python layer should expose these as explicit objects rather than a single Boolean helper: an input domain, mathematical reference, executable function, observation fields, enumerated census, solver query, replayed witness, and semantic digest. The Rust layer should own the width-safe word operations and serialized evidence checks. Python can assemble lessons and display tables without becoming an unrecorded semantic authority.

From exhaustive rows to a universal formula

Width four has 256 ordered addition pairs. Exhaustive checking can classify all of them by C and V. The Boolean overflow proof covers every positive width. Axeyum can connect the two: enumerate the teaching instance, solve the general bit-vector negation at selected widths, and later reconstruct a width-parametric theorem in a stronger kernel route. Keep those assurance levels separate in the evidence record.

Thin RV64I and x86-64 adapters come after the pure laws. The RV64 adapter maps selected comparison results and arithmetic destinations. The x86 adapter

maps selected flags for exact operand widths and forms. An evidence manifest must name the observation: destination only, selected conditions, or complete in-scope state.

8.10 Where the model stops

This chapter treats integer words. It excludes floating-point exceptions, saturation, packed lanes, decimal arithmetic, and multiprecision conventions beyond a small carry-chain example. It does not claim that one flag policy covers every x86-64 instruction form or optional RISC-V extension.

It also treats an instruction as an atomic state transition. Hardware may compute condition information through internal circuits unlike these formulas. The architectural theorem concerns the visible result, not the microarchitectural route that produced it.

The reward for this restraint is a reusable core. A few finite-word identities explain wraparound, comparisons, branches, and the legality of later program transformations. The machine keeps only bits. A proof can still recover the mathematical boundary those bits crossed.

The chapter's recurring method is deliberately uniform: widen when the exact unsigned result matters, interpret high bits when signed range matters, and name every condition-state policy that later code can observe. That method scales from a four-bit teaching table to a multiprecision routine and from a reader proof to a solver obligation without changing the meaning of the claim.

8.11 Exercises

1. **Execute.** For every ordered pair of width-two words, compute the stored sum and Ao bits Z, N, C, V. Group the rows by the four (C, V) combinations.

2. Give width-eight additions with carry but no signed overflow, signed overflow but no carry, both conditions, and neither condition. Explain each using mathematical ranges.

3. **Prove.** Prove that opposite-sign signed addition cannot overflow. State the signed interval used by the proof.

4. Derive the Boolean high-bit formula for signed addition overflow and check it on all four examples in Figure 8.1.

5. For width four, compute Ao subtraction conditions for `0011-0101`, `0101-0011`, `0111-1111`, and `1000-0001`.

6. Show with a concrete pair why N alone does not implement signed less-than after subtraction. Show how $N \neq V$ repairs the case.

7. Compare logical and arithmetic right shift of `10010000` by counts 0, 1, 3, and 7 under Ao. Include the last shifted-out bit.

8. **Break.** Replace signed overflow with carry in an Ao addition implementation. Give the smallest-width counterexamples you can find in both directions.

9. Construct two sequences with the same destination result but different carry bits. Supply a suffix that observes the difference.

10. Explain how an RV64I `slt` result and an Ao $N \neq V$ predicate can represent the same signed comparison without identical machine states.

11. **Transfer.** State a relation between one selected x86-64 CMP form and Ao `cmp`. Include operand width, subtraction polarity, mapped conditions, program-counter relation, and excluded flags.

12. Design an Axeyum manifest for the width-four carry/overflow census. Include the independent definition, row count, digest, replay check, and a negative control.

13. Extend a destination-only arithmetic equivalence claim to a condition- sensitive one. List the additional postconditions and tests required.

CHAPTER NINE

Control Transfer

Most instructions continue with the next instruction in memory. Control- transfer instructions make another future possible. A conditional branch asks a question about the current state and chooses one of two addresses. A jump selects a destination without asking. A call also leaves behind an address from which execution can later continue.

These operations are small enough to fit in one line of assembly, yet they connect nearly every part of the machine model. Their predicates read registers or condition state. Their targets depend on instruction addresses, lengths, and signed displacements. Their repeated execution makes loops. A single error in the target convention can preserve every data result in a one-step test and still send the program into the wrong block.

Control flow is arithmetic over addresses plus a predicate

A direct conditional branch needs three facts: the address used as its base, the signed displacement or absolute destination, and the Boolean predicate that selects the target instead of the fallthrough address. State all three before proving where execution goes.

9.1 Why programs learned to choose

The earliest stored-program machines already needed more than a straight list of arithmetic operations. Repetition made a short program perform a long calculation. Conditional transfer let that repetition stop when a counter, remainder, or comparison reached the desired state. Subroutines then made one piece of code serve many callers. The program counter turned stored instructions from a static list into a path selected during execution.

Modern machines retain that foundation even when processors predict several paths, fetch ahead, or execute instructions out of order. Architectural semantics describe the committed path. A proof about a scalar instruction set therefore needs

no branch predictor, but it cannot omit the program counter, the branch predicate, or the possibility of a bad target.

Control transfer serves several present-day purposes:

1. a conditional statement chooses between two computations;
2. a loop returns to a test or body;
3. a jump enters another block without creating a return link;
4. a call records a continuation address and enters a procedure;
5. an indirect transfer obtains its destination from a register or memory;
6. a trap or return crosses a boundary whose rules exceed ordinary scalar control flow.

This chapter handles direct conditional branches, direct jumps, and the link values needed to prepare for procedures. Chapter 10 adds stacks and calling agreements. Indirect calls, privilege changes, exceptions, and speculation remain outside the present model unless a later section names them.

A loop compresses time into an address

A backward edge says that the same finite instruction bytes may act on a new state. The code need not grow with the number of iterations. What grows is the trace: one state after another, each justified by the same small transition rules.

9.2 Fallthrough, target, and link

Suppose an instruction begins at address p and occupies L bytes. Its **fallthrough address** is

$$f = p + L.$$

Execution continues there when an ordinary instruction completes or a conditional branch is not taken. For a direct relative transfer, a signed displacement d is combined with a specified base b:

$$t = b + d.$$

The architecture decides whether b is the current instruction address, the fallthrough address, or another quantity, and whether the encoded displacement is scaled before addition.

For a predicate $P(S)$ over state S, the next program counter is

$$pc' = \begin{cases} t & \text{when } P(S), \\ f & \text{when } \neg P(S). \end{cases}$$

A direct jump uses the first line without a predicate. A linking jump also writes a **link value**, normally an address at which a later return can resume. The link is an architectural data result, not merely an annotation on the control-flow graph.

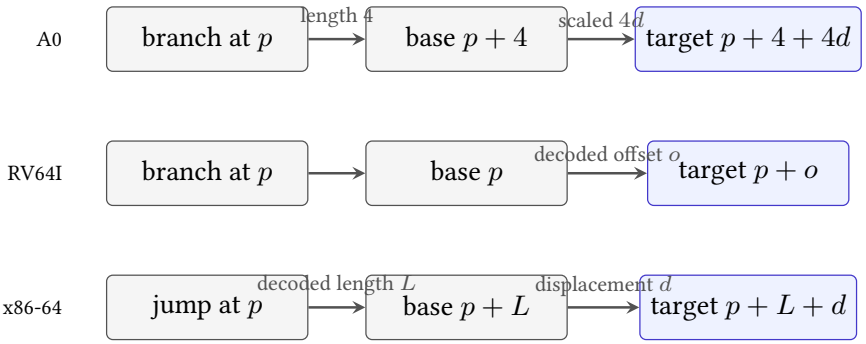


Figure 9.1: Three selected relative-target conventions. Similar assembly displacements do not imply the same base, scale, or instruction length.

Figure 9.1 exposes a common source of off-by-one-instruction errors. A0 deliberately uses $p + 4 + 4d$. Selected RV64I conditional branches add their sign-extended, encoded offset to the address of the branch instruction. Selected x86-64 relative jumps add a sign-extended displacement to the instruction pointer for the following instruction. RV64I instructions in this chapter are four bytes, while the x86-64 instruction length depends on the actual encoding. The source-pinned rules appear in the RISC-V unprivileged specification and Intel instruction reference (RISC-V International 2026b; Intel Corporation 2026a).

Never infer the target base from the printed operand

An assembler may print a destination label, a byte displacement, or a symbolic expression. The architectural rule operates on encoded fields and a defined program-counter base. Decode the instruction and name that base before doing target arithmetic.

The same address, three calculations

Place a transfer at address 100. An Ao branch with encoded displacement 3 targets $100 + 4 + 4(3) = 116$. An RV64I branch whose decoded offset is 16 bytes targets $100 + 16 = 116$. An x86-64 near conditional jump of length 6 with displacement 10 targets $(100 + 6) + 10 = 116$. All three reach the same address, but no pair uses identical encoded numbers and conventions.

A target calculation that survives relocation

Let an instruction and its target both move by the same address delta k. If the relative displacement is target minus the architecture's base, that base also moves by k. The new displacement is $(t + k) - (b + k) = t - b$. Thus the encoded relative distance is unchanged, provided the moved target remains representable and alignment and encoding constraints still hold. The cancellation explains why relative branches help relocatable code; it does not prove that every relocation stays in range.

9.3 Ao makes both edges explicit

Ao instructions are four bytes and begin at four-byte-aligned addresses. An ordinary instruction advances from p to $p + 4$. A conditional branch with signed eight-bit immediate d selects $p + 4 + 4d$ when its condition is true and $p + 4$ otherwise. The scale of four ensures that a target obtained from an aligned branch is aligned. It does not ensure that the target contains a complete legal instruction; fetch and decode still have to succeed.

Ao supplies eight branch predicates over Z, N, C, V. Equality and inequality read Z. Signed less-than and greater-than-or-equal test whether N and V differ or agree. Unsigned lower and higher conditions use the no-borrow bit C, with Z distinguishing strict from non-strict comparisons. The branch preserves registers, memory, and conditions. Its visible write is the program counter, unless the selected target causes a later fetch trap.

Consider this width-eight loop, with each line occupying four bytes:

```
00: movi r1, 1
04: cmp  r0, r1
08: branch.lo exit
0c: sub  r0, r0, r1
10: branch.ne loop
14: halt
```

Listing 9.1: Ao counts ro down to zero

The first comparison protects the subtraction when the initial unsigned value is zero. For a positive initial r_0, subtraction decreases the unsigned reading by one without wrapping. The subtraction also updates Z, so `branch.ne` returns to the loop body exactly while the result is nonzero. The listing uses labels for reading; an encoder must replace them with signed displacements that satisfy Ao's target formula.

Encode both A0 branch displacements

Using the addresses in the listing, let `exit` denote address 20 and `loop` denote address 12. Solve $p + 4 + 4d = t$ for each branch. Check that both signed values fit in byte 3, then recompute the destinations from the encoded values.

The branch at address 8 needs $d = 2$, because $8+4+4(2) = 20$. The backward branch at address 16 needs $d = -2$, because $16 + 4 + 4(-2) = 12$. Treating the current address rather than the fallthrough address as the base would send both branches four bytes early.

9.4 RV64I compares registers at the branch

The selected RV64I conditional branches compare two integer registers as part of the branch instruction. BEQ and BNE test equality. BLT and BGE use signed readings; BLTU and BGEU use unsigned readings. A taken branch adds its decoded, sign-extended B-type offset to the address of the branch instruction. An untaken branch proceeds to the following instruction (RISC-V International 2026b).

```
    beq  a0, x0, exit
loop:
    addi a0, a0, -1
    bne  a0, x0, loop
exit:
```

Listing 9.2: RV64I counts ao down to zero

Here `x0` supplies the constant zero. The branch itself reads both named registers. The `addi` instruction changes `a0` but creates no x86-style zero flag; `bne` performs the next comparison directly. Assembler register names such as `a0` carry ABI meaning, while the base ISA semantics identify an integer register. Chapter 10 separates those two layers.

RV64I also provides direct jumps with links. `JAL rd,offset` writes the address of the following instruction to `rd` and transfers to a PC-relative target. Writing the link to `x0` discards it and yields a plain direct jump. `JALR` obtains a target from a register plus immediate, writes a link, and clears the target's low bit

under its architectural rule. The direct loop above needs neither instruction, but procedures will.

> **A pseudoinstruction is not another semantic primitive**
>
> An assembler may print `j label`, `ret`, or a branch-against-zero mnemonic that expands to a base instruction. Proofs over machine code must use the decoded base instruction and its operands. Pseudoinstructions remain valuable explanations of programmer intent, but they do not add encodings.

The conditional-branch immediate is assembled from noncontiguous instruction bits and has alignment constraints. A chapter proof may begin with the decoded offset when encoding is not under study, but the evidence manifest must say so. A decoder theorem and a step theorem are different claims: the first recovers the offset; the second uses it to update the program counter.

9.5 x86-64 reads flags and counts variable-length bytes

Selected x86-64 compare instructions perform a subtraction for flags without storing the arithmetic result. A following `Jcc` reads the condition formula associated with its suffix. For example, `JE` reads ZF; `JNE` reads its negation; signed less-than uses $SF \neq OF$; and unsigned below uses CF. The exact flag effects and accepted encodings belong to the selected instruction forms in Intel's reference, not to the mnemonic name in isolation (Intel Corporation 2026a).

```
    test rdi, rdi
    je   exit
loop:
    sub  rdi, 1
    jne  loop
exit:
```

Listing 9.3: x86-64 counts RDI down to zero

`test rdi,rdi` computes a bitwise conjunction for flags and leaves `rdi` unchanged. It sets ZF exactly when the selected value is zero. `sub` then decrements the register and supplies the ZF consumed by `jne`. This dependency is real even though it is absent from the register operands printed on the jump.

x86-64 relative jump targets use the instruction pointer for the following instruction as their base. Because instruction lengths vary, the decoder must first determine that next address. Short and near conditional-jump encodings can express different displacement widths while implementing the same predicate. Replacing one encoding with another changes instruction length and therefore changes both the base and the displacement required for a fixed destination.

A flag-reading jump has a hidden-looking input

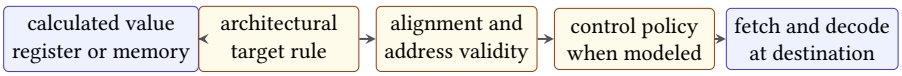
The condition suffix identifies which flags the jump reads. Register equality at the jump is irrelevant unless preceding instructions established the required flags. Moving, replacing, or inserting a flag-writing instruction between compare and jump can change the path without changing any explicit jump operand.

The x86-64 loop, the RV64I loop, and the Ao loop implement the same small mathematical recurrence only under declared input and observation relations. They do not have identical registers, instruction counts, condition state, or program-counter values. A proof must relate those differences instead of erasing them.

9.6 Direct and indirect transfers owe different proofs

A direct transfer carries a destination relative to the instruction or names one through relocation. Once its bytes and address are fixed, target arithmetic usually determines one destination. An **indirect transfer** obtains at least part of its destination from current machine state. The same instruction bytes may therefore reach many addresses on different executions.

This difference changes the proof. For a direct branch, faithful decode and correct target arithmetic can identify the destination locally. For an indirect jump, local instruction semantics only explain how a register or memory value becomes a candidate program counter. A surrounding invariant must restrict that value to an allowed target set. A compiler might derive the set from a switch table, a procedure proof from a saved continuation, or a control-flow integrity policy from approved entry points.



Computing an address does not by itself justify execution from that address.

Figure 9.2: An indirect destination passes through several proof gates before the next instruction may be justified. Some machines omit or add policy gates, but the target value alone is never the whole argument.

RV64I JALR, for example, adds a sign-extended immediate to a register value and clears the least significant bit of the result. A theorem must use that transformed target rather than equating the new program counter with the raw register. Selected x86-64 indirect branches obtain their destinations from register or memory operands under their own mode and operand rules. Core Ao has no indirect jump,

so a proof must not manufacture one from a register that happens to contain an address.

Target validity is a second question. Depending on the model, the destination may need suitable alignment, a canonical address, mapped executable memory, an instruction boundary, and bytes that decode legally. A hardware or operating-system policy may add another restriction. These checks need not all occur in the transfer instruction itself. A model may update the program counter first and report a fetch failure on the next step. The trace should preserve that ordering instead of replacing it with one generic failure.

A finite target set needs provenance

Listing the destinations seen in tests does not establish the allowed target set. Derive the set from a table bound, type invariant, saved-link contract, or other stated premise. Then show that every calculated destination belongs to it and that every member is a valid entry under the selected semantics.

Return instructions are important indirect transfers. Their intended target comes from a prior call, but that history is not automatic evidence. The proof must connect the current link register or stacked word to the continuation created at the matching call and show that intervening execution preserved it. Chapter 10 turns that historical connection into a procedure contract.

Target soundness and target completeness

For an indirect transfer under a precondition, target soundness says that every destination the machine can select belongs to the declared target set. Target completeness says that every destination claimed as possible can actually be selected by some allowed entry state. Soundness protects execution from an omitted bad destination. Completeness prevents a coarse analysis from adding spurious edges that may obscure invariants or make a later proof needlessly hard.

The two directions need different witnesses. A soundness failure is one entry state whose selected destination lies outside the set. A completeness claim needs, for each listed destination, an allowed state that reaches it. Many safety arguments require soundness but not completeness. An exact control-flow graph requires both, which is why a graph inferred from a few runs is generally neither a safe overapproximation nor an exact account.

9.7 One loop, three state shapes

Let the input be a nonnegative mathematical integer n that fits in the selected word without wrapping during the loop. Each program maintains a machine word whose unsigned reading is the number of decrements remaining. The common control-flow graph has an entry test, a decrementing body, a back edge, and an exit.

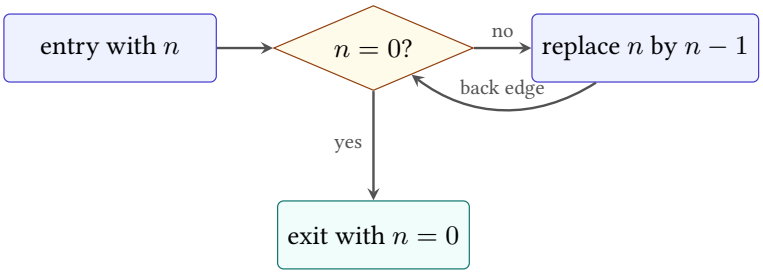


Figure 9.3: The common mathematical control shape beneath the three listings. The machine-specific predicates and target conventions implement these edges.

At the head of the body, use the invariant

$$0 < \langle r \rangle_u \leq n,$$

where r denotes the machine's related counter register. The body changes the reading from $k > 0$ to $k - 1$. The value therefore remains in range and strictly decreases as a natural number. When it becomes zero, the back-edge predicate is false and execution reaches the exit.

Termination and final value of the counted loop

If the input is zero, the entry test reaches the exit without executing the body. Otherwise let the counter reading at the first body entry be $n > 0$. Each body execution subtracts one from a positive reading, so modular subtraction agrees with natural subtraction and cannot wrap. The natural measure decreases from n through $n - 1, \ldots, 1, 0$. A strictly decreasing natural measure cannot decrease forever. At zero, each machine's related not-equal predicate is false, so the program exits with counter value zero.

The premise that subtraction occurs only at a positive value is load-bearing. Delete the entry test and start at zero: fixed-width subtraction produces the maximum unsigned word, after which the loop may take $2^w - 1$ further iterations before returning to zero. The original termination conclusion may still hold for a finite width, but the proof, trace length, and intended cost claim have changed.

The invariant is also not a complete cross-ISA relation. Such a relation must identify Ao r_0, RV64I a0, and x86-64 RDI; relate the values at chosen control points; state what memory must agree on; map Ao's Z and x86-64's ZF when their branches depend on them; and permit the RV64I state to lack that flag. Chapter 12 develops this style of relation.

Stuttering between related control points

One machine may need an extra compare or test instruction before a branch. The other may combine comparison and transfer. A simulation can relate states only at loop heads and exits, allowing several steps on one side for one or several on the other. This is stuttering, not an error, provided the relation is restored at the declared observation points.

9.7.1 Safety, progress, and termination are separate

The counted-loop argument combines three claims that should remain visible. *Safety* says every visited instruction and memory access is defined and the subtraction does not wrap under the intended natural-number reading. *Progress* says a nonterminal state can take another justified step. *Termination* says the run eventually reaches the exit. A decreasing natural measure helps with termination only after safety and progress connect each iteration to the next loop head.

The separation becomes useful when a proof fails. A counter may decrease on every completed iteration while an invalid branch target traps before the next head. The measure argument is then true but irrelevant to normal termination. A loop may remain safe forever while its measure fails to decrease. Or it may reach the exit with the expected counter while corrupting an observed memory location. No one of these facts substitutes for the others.

Iteration count is another conclusion, not a side effect of termination. Add a ghost natural number i counting completed decrements and strengthen the invariant at the loop head to

$$i + \langle r \rangle_u = n.$$

Initially $i = 0$ and the counter reads n. One body step increases i by one and decreases the counter reading by one, preserving the sum. At exit the counter is zero, so $i = n$. The ghost value is proof bookkeeping; it need not exist in machine state.

Why the variant must be checked at one control point

A variant can rise inside an iteration and still decrease from one loop head to the next. Choose a recurring control point, prove that every return to it has a smaller natural variant, and prove that execution cannot avoid either that

return or an exit forever. Comparing values at arbitrary adjacent instructions can reject correct loops or accept a cycle that bypasses the measurement point.

9.8 From traces to control-flow claims

A trace records the actual successor at each step. For a chosen input, a checker can decode the instruction at the recorded program counter, recompute its predicate and target, and compare the complete successor with the next state in the trace. This catches a wrong edge, a wrong decrement, a stale condition bit, or execution after a terminal outcome.

One trace does not prove a general loop theorem. Running the loop from inputs zero through 255 proves facts about those chosen executions under a trusted enumeration and checker. The invariant proof covers every input in its stated domain because it reasons about an arbitrary positive counter and a decreasing measure. Machine evidence can support that argument by checking the step lemmas or refuting their negations, but a bounded replay cannot silently become an unbounded theorem.

A bounded run has three distinct endings

A bounded interpreter may halt, trap, or exhaust its step bound while the machine is still running. Bound exhaustion is not a machine trap and not proof of nontermination. It says only that this run prefix did not reach a terminal outcome within the supplied budget.

A control-flow graph also differs from a trace. The graph contains possible edges admitted by local branch analysis. A trace contains the one edge selected in each visited state. Proving that an edge is unreachable needs a state argument, not merely its absence from a few traces.

The graph can be checked at several strengths. A syntactic graph connects each decoded direct transfer to its calculated destination and adds fallthrough where appropriate. An indirect edge may conservatively point to many possible targets. A state-restricted graph removes edges whose predicates or target values are impossible under a supplied invariant. Reachability from the entry then removes disconnected code. Each reduction needs evidence: a visually tidy graph is not a proof that the omitted edge cannot execute.

Loops arise from cycles in this reachable graph, but a cycle alone does not identify an induction invariant or termination measure. Conversely, an unrolled trace can revisit an address without revealing every possible cycle. Graph structure organizes the proof obligations; semantic reasoning over states discharges them.

9.9 An introductory Axeyum route

The current Axeyum checkout supplies reusable bit-vector, solver, certificate, and kernel components. It does not yet supply executable Ao, RV64I, or x86-64 instruction semantics. For this chapter, Axeyum use is therefore a construction route with a precise boundary, not a claim that the three loops already run there.

Start with Ao because its complete state and fixed-width encoding are owned by this book. Define a decoded branch containing a condition code and signed displacement. Define the predicate as a pure function of Z, N, C, V, then define the two successor-PC formulas. Only after unit tests cover every condition and both outcomes should the branch enter the general Ao step function.

The first evidence object should contain an initial state, instruction bytes, decoded branch, selected predicate value, expected successor state, and the semantic-package digest. Its checker should decode again and recompute the step rather than trusting recorded derived fields. A small command-line wrapper can then print a teaching trace while its exit status depends on full agreement.

> **Controls for the first branch checker**
>
> Use a positive case whose taken target differs from fallthrough. Pair it with mutations that invert the predicate, omit the displacement scale, use pc instead of $pc + 4$ as the Ao base, or change one reserved encoding bit. Each mutation must make the checker fail for the intended reason. Also retain a valid untaken case so a checker that always chooses the target cannot pass.

The RV64I and x86-64 routes come later. Each needs a source-pinned decoder slice before its transition function can carry machine-code claims. The RV64I control should mutate a split immediate field or confuse signed and unsigned predicates. The x86-64 control should alter a prefix, opcode, displacement, or decoded length. A handwritten branch formula can test arithmetic; it cannot stand in for those decoders.

For the counted-loop theorem, separate four evidence levels:

1. replay one complete trace for a small input;
2. enumerate every input at a teaching width;
3. ask for a counterexample to invariant preservation at a named width and replay any model;
4. reconstruct a width-parametric decreasing-measure proof through a kernel route when that route exists.

The manifest must label which level was achieved. Solver UNSAT at width 64 is a strong finite-width result, but it is not automatically a theorem quantified over

every positive width. A proof certificate can reduce trust in the solver verdict; it still proves only the encoded formula.

9.10 What a branch proof must establish

Branch correctness is often compressed into the phrase "the target is right." That phrase hides several separate obligations. Keeping them separate makes both reader proofs and negative controls more useful.

First comes *fetch and decode*. The current program counter must identify a complete legal instruction. The decoder must recover the intended condition, register fields, and signed displacement. A proof that starts from an already decoded branch may assume this layer, but it must say so.

Second comes *predicate agreement*. The implementation must ask the question named by the instruction. Signed and unsigned comparisons can choose different edges for the same word pair. Ao and selected x86-64 forms must read the condition state produced by the preceding instruction; an RV64I branch must read the specified register values. Preserving the right target while inverting the predicate is still a wrong branch.

Third comes *target arithmetic*. The proof must identify the base, extension rule, scale, and arithmetic width. It must also distinguish the taken target from fallthrough. Alignment, canonical-address, and mapped-code requirements belong here when the selected machine model includes them.

Fourth comes *state preservation*. A conditional branch usually changes the program counter and preserves most other architectural state. "Reached the right block" does not justify an unnoticed register, memory, flag, or outcome change. A linking jump is different because its link destination is an intended additional write.

Finally comes *context*. The successor address must lead to code whose execution is covered by the surrounding claim. A locally correct edge into an illegal byte sequence traps on the next step. A locally correct loop edge does not prove termination. Those are composition properties beyond the branch's single transition.

Five obligations for a direct conditional branch

Check, in order: legal fetch and faithful decode; the exact predicate over the declared input state; taken and untaken program-counter formulas; preservation of every other observed component; and the premise needed to continue from the chosen successor. A proof may discharge the layers separately, but the whole-program claim needs all five.

This decomposition locates counterexamples. A wrong immediate field is a decode failure. A stale ZF is a predicate failure. A missing Ao scale factor is a target failure. A jump that clears an unrelated register is a preservation failure. A

correct backward edge in a loop without a decreasing measure is a context failure. Merely reporting that "the branch test failed" throws away the lesson carried by the witness.

9.11 Where the control model stops

This chapter treats architectural steps as atomic and sequential. It excludes branch prediction, speculative execution, pipeline recovery, timing, caches, and side channels. Those mechanisms can make two architecturally equivalent paths observably different to a timing adversary, but they require another state and observation model.

We also exclude interrupts, exceptions beyond Ao's declared traps, privilege transfers, self-modifying code, concurrent modification, and weak memory. Indirect control flow receives only the link and target facts needed for the next chapter. A full account must also specify target validity, canonical addresses, control-flow enforcement, and the memory from which an indirect destination may be loaded.

Within the boundary, the central fact is exact and surprisingly broad. A finite program becomes an indefinitely extensible process because the program counter may revisit an address. The mystery does not lie in an unspecified machine. It lies in the consequence of a rule small enough to write on one line: choose a successor, then apply the same semantics again.

9.12 Exercises

1. **Execute.** Trace both outcomes of an Ao `branch.eq` at address 40 with displacement -3. Give the successor program counter and all preserved state components.

2. Compute the Ao displacement needed to branch from addresses 0, 12, and 100 to address 40. State which values fit the signed byte and which targets meet the alignment rule.

3. **Break.** Replace Ao's taken-target rule $pc + 4 + 4d$ with $pc + 4d$. Give a branch for which the mutation accidentally agrees and one for which it reaches a different legal instruction.

4. A four-byte RV64I branch lies at address 100 and has decoded offset -20. A six-byte x86-64 near jump lies at address 100 and has displacement -20. Compute both targets and explain the difference.

5. Explain why changing an x86-64 conditional jump from a short encoding to a longer encoding requires displacement repair even when its address and destination label appear unchanged.

6. For each Ao predicate `eq`, `lt`, `lo`, and `hi`, give values of Z, N, C, V that take and do not take the branch. Identify condition combinations that cannot arise from one selected comparison, if your claim assumes such a comparison.

7. Rewrite the RV64I counted loop to count upward from zero to an input bound. State whether its comparison is signed or unsigned and give an input premise that prevents wraparound.

8. **Prove.** Strengthen the counted-loop invariant to say exactly how many body executions have occurred. Use it to prove both final value and iteration count.

9. Delete the entry test from each loop and start the counter at zero at width four. Trace enough steps to identify the new iteration count. Explain why the original natural-number proof no longer applies at the first step.

10. **Transfer.** Define relation checkpoints for the three loop listings. Map counter values, control locations, outcomes, and the condition information required for the next branch; identify scratch or flag state that the final observation may omit.

11. Give two instruction sequences that leave the same counter register but different x86-64 flags immediately before `jne`. Supply initial values for which the chosen edges differ.

12. Explain the difference among a trace, a control-flow graph, and a proof that a graph edge is unreachable. Give one fact each can establish.

13. Design a bounded-run result type that distinguishes halt, trap, and bound exhaustion. Give a negative test for an implementation that reports bound exhaustion as nontermination.

14. Design the first Ao branch artifact for Axeyum. Include bytes, decoded instruction, initial state, expected successor, semantic digests, checker command, and four target or predicate mutations.

15. **Transfer.** State the obligations for replacing an Ao condition-reading branch with an RV64I register-comparing branch. Include the state relation before the branch and the relation required after both taken and untaken outcomes.

CHAPTER TEN

Procedures, Stacks, and ABIs

A machine can transfer control and preserve a continuation without knowing what a function is. A compiler can place an argument in a register without changing that register's instruction-set meaning. A procedure emerges when separately written pieces of code agree about entry, return, values, scratch space, and preservation.

That agreement is an **application binary interface**, or ABI. It sits between source-language intention and instruction semantics. Confusing the layers produces brittle explanations: an ISA register gets called an argument register in every context, a stack-growth convention is mistaken for a hardware law, or a successful return is credited to RET while the saved address and stack discipline go unproved.

A call mechanism is not a calling convention

The ISA explains how control reaches a target and how a continuation is recorded. The ABI explains where arguments and results live, which locations survive the call, how the stack is aligned, and who restores each resource. Procedure correctness needs both layers and must name each separately.

10.1 Why procedures became a machine-level agreement

Early programmers quickly learned that repeating instruction sequences by copying them was costly and error-prone. A reusable subroutine stored one body and let many sites transfer to it. Reuse created a new problem: the body had to know where to return, where to find its inputs, where to place its result, and which working locations belonged to its caller.

Assemblers and languages supplied conventions before those conventions became large platform documents. Linkers then had to connect separately assembled objects. Debuggers needed to recover call chains. Operating systems and shared libraries needed independently produced code to agree for years. The modern ABI records that durable binary agreement. It covers much more than calls, but this chapter uses its procedure-calling slice.

The same need exists today in compilers, foreign-function interfaces, dynamic instrumentation, binary translation, and formal verification. A compiler may optimize freely inside a procedure while still owing a precise boundary to every caller. A proof that two function bodies compute the same number is insufficient if one corrupts a preserved register, misaligns the stack before a nested call, or returns through the wrong address.

A procedure boundary makes separate construction possible

The caller need not know how the callee computes. The callee need not know why the caller wants the result. Both must know the narrow contract at the entry and exit. That controlled ignorance is what permits separate compilation.

10.2 Six parts of a procedure contract

For one scalar procedure, write the contract as six declarations:

1. *entry control*: the address and machine state from which the body begins;
2. *inputs*: registers or memory locations containing arguments;
3. *outputs*: locations containing results on normal return;
4. *volatile state*: scratch locations the caller must assume may change;
5. *preserved state*: locations the callee must restore before return;
6. *return control*: the saved continuation and rule that resumes it.

Stack bounds, alignment, allowed traps, and memory ownership refine these parts. The contract also needs an observation: perhaps the result, preserved registers, stack pointer, reachable memory, and normal-return outcome. It need not require temporary registers to recover their entry values.

Caller-saved and callee-saved name obligations

A caller-saved location may be overwritten by a conforming callee, so a caller with a live value there must save it before the call. A callee-saved location must have its entry value again when the callee returns normally, so a callee that uses it must save and restore it. Neither term says who physically owns a register; each assigns a proof obligation at the boundary.

The two disciplines can implement the same preservation fact. If a value must survive a call, the caller may move it out of volatile state, or the callee may promise

to preserve its chosen location. Performance, register pressure, and code size influence the division. Semantics requires only that the printed contract be met.

> **A value that is dead needs no ceremony**
>
> Suppose the caller places a temporary in a volatile register and never reads it after the call. The callee may overwrite it without a save. If a later edit makes the value live across the call, the old sequence becomes wrong unless the caller saves it or moves it to a preserved location. Liveness turns an ABI permission into a concrete save obligation.

10.2.1 A procedure theorem has four layers

A useful theorem about a procedure is not merely a relation between two snapshots. It joins four layers of reasoning. The *entry relation* connects the caller's abstract arguments and resources to concrete machine locations. A *body invariant* explains why every reachable internal state remains safe and still has enough information to finish. Each *call checkpoint* establishes the entry contract of a nested callee and records what the current procedure must keep across that call. Finally, the *exit relation* connects the machine state back to the promised result, preserved resources, and continuation.

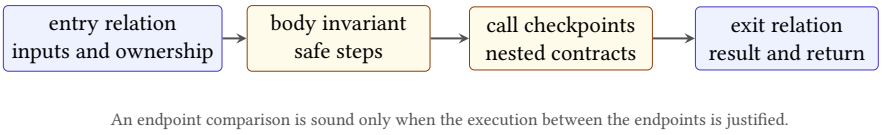


Figure 10.1: A procedure proof crosses four layers. Skipping the middle two turns an endpoint comparison into an assumption about the body.

This division prevents a common circular argument. Suppose a test begins in a valid entry state and happens to finish in a valid exit state. That observation does not show that all valid entries finish, that intermediate memory accesses are permitted, or that the body cannot jump somewhere else. The trace or inductive proof must connect the endpoints. Conversely, a safe trace that halts at the wrong continuation does not establish the procedure contract.

For a straight-line leaf, the body invariant may be no more than a symbolic expression for each changed register and a claim that the next instruction is defined. A loop needs a genuine loop invariant. A non-leaf body needs a checkpoint before every nested call. The theorem should become larger only when the control structure becomes larger; the boundary vocabulary remains the same.

Normal-return procedure theorem schema

Assume an entry state satisfies the procedure precondition and entry relation. Show that each instruction step is defined and preserves the body invariant. At a nested call, derive the callee's precondition, apply its procedure theorem, and reestablish the caller's invariant from the callee's exit relation. At a normal return, derive the result relation, restored stack pointer, preserved locations, allowed memory effect, and saved continuation. This establishes only the normal-return behavior named by the theorem; a trap, divergence, or exceptional exit needs a separate clause.

10.3 A stack is memory governed by an invariant

An architectural stack pointer is an ordinary register with special uses or conventions. Ao has none. RV64I base instructions do not force x2 to be a stack pointer; the RISC-V ABI names it sp. In x86-64, RSP has instruction-level roles in CALL, RET, PUSH, and POP, while the ABI adds alignment, frame, and preservation rules.

A downward-growing stack allocates a frame by subtracting from the stack pointer and releases it by adding the same size. "Downward" refers to numerical addresses, not to the way a diagram is drawn. The allocated frame is a byte range, and every load or store still needs an address, width, byte order, alignment policy, and range premise.

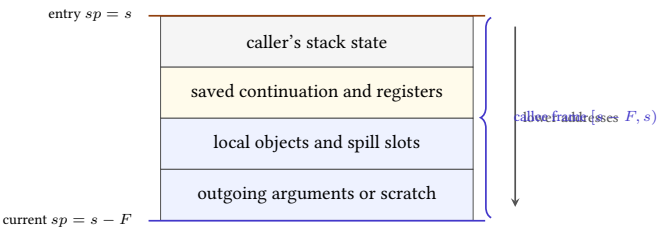


Figure 10.2: A downward-growing frame after allocation. The current stack pointer marks its low address; the caller's stack state lies at higher addresses.

Let s be the entry stack pointer and let a callee allocate F bytes. After $sp' = s - F$, its frame occupies the half-open interval $[sp', s)$. A store of k bytes at address a stays within the frame exactly when

$$sp' \le a \quad \text{and} \quad a + k \le s.$$

The half-open form handles adjacent objects without overlap and makes the upper-bound proof explicit.

Balanced allocation restores the stack pointer

Assume address arithmetic does not wrap and the entry pointer is s. Allocating F bytes produces $s - F$. Releasing the same frame produces $(s - F) + F = s$. This proves pointer restoration, not frame correctness. A separate argument must show that every access stayed within allocated and permitted memory and that every saved value was restored before release.

Alignment is a modular invariant. If the ABI requires alignment A, then a required checkpoint satisfies $sp \bmod A = 0$. Subtracting a frame size that is a multiple of A preserves alignment. A call mechanism that pushes a return address may deliberately change the congruence observed at callee entry, so the checkpoint and mechanism must be stated together.

Balanced arithmetic does not prove a safe frame

A prologue and epilogue can restore the same stack-pointer value while writing outside the frame, overwriting the saved return address, restoring the wrong register value, or violating alignment at an intervening call. Check the whole interval and every required checkpoint.

10.3.1 Nested frames are interval arithmetic

Suppose a procedure enters with stack pointer (s), allocates (F) bytes, and calls a second procedure that allocates (G) bytes. Immediately before the nested allocation, the first frame is ([s-F,s)). After it, the nested frame is ([s-F-G,s-F)). The intervals touch at (s-F) but do not overlap. This simple calculation is the spatial reason one procedure may keep saved values in its frame while another procedure uses the stack below it.

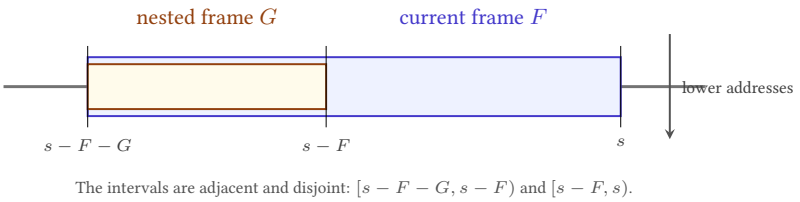


Figure 10.3: Nested downward-growing frames as half-open byte intervals. The orientation of the drawing is secondary; the address inequalities are the claim.

The argument depends on more than subtraction. Neither computation may wrap. Both intervals must lie in mapped, writable memory owned by the stack. The nested callee must respect its own frame bound rather than store upward into

the caller's frame. If it receives a pointer into the caller's frame, any write through that pointer must be separately allowed by the contract. Stack discipline supplies disjoint default regions; explicit pointer sharing can weaken that separation.

A frame proof is therefore a small memory-safety proof. For each slot, record an offset, width, alignment, and lifetime. Prove that its interval is inside the frame and disjoint from every simultaneously live slot unless intentional aliasing is part of the design. Then prove that the frame as a whole is inside the permitted stack region. Checking only the largest printed offset misses the width of the access: an eight-byte store beginning four bytes below the upper boundary crosses it.

10.4 Ao exposes the missing mechanism

Core Ao has direct relative `jump`, conditional branches, and eight ordinary registers. It has no call instruction, register-indirect jump, implicit stack pointer, or ABI. That is a useful boundary rather than an omission to conceal.

A fixed-call-site teaching program can jump directly to a body and let that body jump directly back to one known continuation. It can designate r_6 as a stack pointer and r_7 as a saved link by convention. Yet core Ao cannot use an arbitrary value in r_7 as the next program counter. A body shared by multiple callers therefore needs duplicated return blocks, a dispatch scheme, self-modifying code outside the model, or an explicit instruction-set extension.

> **Do not smuggle a return instruction into A0**
>
> Writing a return address into r_7 does not make `jump r7` legal when the Ao specification defines only a relative-immediate jump. Examples must either use a fixed direct return or declare an Ao extension with its own encoding, semantics, traps, and evidence controls.

This limitation clarifies the construction order. First define a procedure contract independently of any particular call opcode. Then show a fixed Ao instance satisfying it. If the book later adds an indirect-jump extension, prove that its general call/return sequence refines the same contract. The contract survives even as the mechanism becomes richer.

For a fixed-return Ao leaf, choose conventions only for the example: r_0 holds one input and the result, r_6 is an aligned stack pointer, r_4 and r_5 are preserved, and r_1, r_2, r_3, r_7 are volatile. The body may compute without a frame. Its proof must show the result relation, preservation of r_4, r_5, r_6, permitted scratch changes, unchanged memory, and a direct jump to the unique continuation.

10.5 RV64I separates link register from stack memory

RV64I JAL writes $pc + 4$ to a destination register and transfers to a PC-relative target. The common call convention writes the link to x1, named ra. JALR can transfer to an address computed from a register and immediate while writing another link. The assembler spelling ret denotes the appropriate base-instruction form that jumps through ra; it is not a new architectural instruction (RISC-V International 2026b).

The RV64 integer ABI gives selected registers procedure roles. It names x2 as sp, uses a0–a7 for integer arguments, uses a0 and a1 for integer return values, classifies t0–t6 as temporaries, and requires s0–s11 to be preserved. The standard stack grows toward lower addresses and is aligned to a 128-bit boundary at procedure entry; the standard ABI requires that alignment through procedure execution (RISC-V International 2026a).

Consider a leaf function that adds one to its first integer argument:

```
add_one:
    addi a0, a0, 1
    jalr x0, 0(ra)
```

Listing 10.1: An RV64I leaf under the selected integer ABI

The ISA explains both instructions. The ABI explains why entry a0 is the argument, exit a0 is the result, ra contains the caller's continuation, and no preserved register needs saving. The mathematical precondition decides whether the desired result is modular addition or ordinary integer addition without wrap.

A non-leaf function cannot assume ra will survive its own nested call: the nested linking jump writes a new continuation there. It must preserve its incoming return address, often in its stack frame, before making the call and restore it before returning. That need follows from liveness plus the call mechanism, not from a rule that every function must have the same prologue.

Find the first overwritten continuation

Trace a function entered with ra=100 that immediately calls another function using `jal ra,target`. Record ra after the nested call. Explain why a later `jalr x0,0(ra)` cannot return to 100 unless the original value was kept elsewhere and restored.

10.5.1 Leaf is a control-flow fact, not a size class

A **leaf procedure** makes no procedure call on the path under discussion. A **non-leaf procedure** does. The distinction does not say that a leaf is short, uses no stack, or cannot loop. A leaf may allocate a large frame, spill registers, and run for millions of steps. A non-leaf may contain only a few instructions around one call.

The distinction matters because a leaf can often keep its incoming continuation in the location established by its caller. On RV64, a nested `jal ra,target` replaces `ra`, so a non-leaf procedure with a live incoming continuation must keep it elsewhere. On x86-64, a nested CALL pushes another return address below the current one. The original continuation remains in memory, but its address relative to the moving stack pointer changes with frame allocation and additional pushes. These are different mechanisms producing the same proof obligation: the current procedure must still be able to reach its caller after the nested procedure returns.

Leaf status can be path-sensitive. A fast path may return without calling, while a slow path invokes a helper. The whole procedure is ordinarily treated as non-leaf for frame design unless the paths have separately justified prologues and epilogues. An optimizer that removes the last call may remove a now-unneeded continuation save, but it must prove that no remaining path or indirect transfer invokes a callee.

10.6 x86-64 puts the continuation on the stack

In 64-bit mode, a selected near CALL records the address of the following instruction on the stack and transfers to its target. A matching near RET obtains the return instruction pointer from the stack and advances the stack pointer. The exact operand, width, canonical-address, and fault rules belong to the pinned instruction forms in Intel's reference (Intel Corporation 2026a).

The System V AMD64 ABI assigns the first six integer or pointer arguments to RDI, RSI, RDX, RCX, R8, and R9 when their classifications permit. It uses RAX for the first ordinary integer return value. Among the general-purpose registers, RBX, RBP, and R12–R15 are preserved across ordinary calls; many others are volatile. Immediately before an ordinary call, the selected stack alignment rule makes the end of the input argument area a multiple of 16 bytes. After the call pushes an eight-byte return address, $RSP + 8$ has that alignment at function entry (x86 psABIs Project 2023).

```
add_one:
    lea rax, [rdi + 1]
    ret
```

Listing 10.2: An x86-64 System V leaf function

The selected LEA form computes the result without changing arithmetic flags. That detail is not required by the basic integer return contract, but it can matter to a stronger context. The function uses no preserved register and allocates no frame. RET nevertheless reads stack memory and changes RSP; calling it a register-only leaf would hide those architectural effects.

The System V red zone is a further ABI rule: qualifying leaf code may use a specified region below the current stack pointer without first adjusting RSP, subject to the ABI's interruption rule. It is not a general x86-64 hardware property

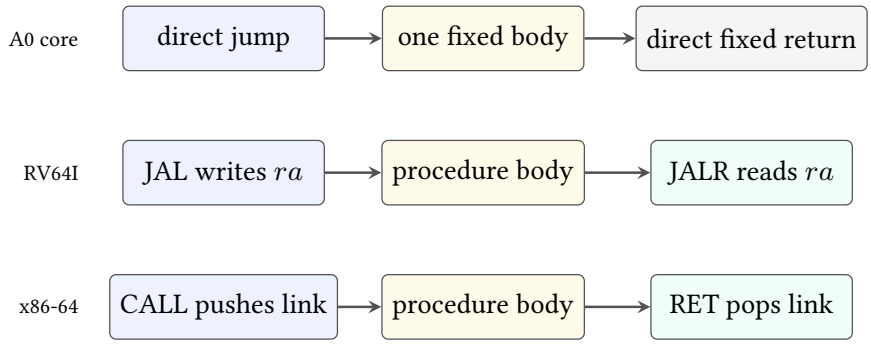


Figure 10.4: Selected call and return mechanisms. The ABI assigns meanings to the link location, stack pointer, argument, result, and preserved state.

and is not shared by every x86-64 calling convention. This chapter's worked proof does not rely on it.

10.6.1 The ISA supplies effects; the ABI selects a composition

It is tempting to divide the layers by vocabulary: instructions belong to the ISA, while register names belong to the ABI. The real boundary is functional. The ISA defines possible state transitions. The ABI selects some of those transitions and adds conditions under which separately built components may compose.

For example, RV64 JAL can place a link in many destination registers; the procedure convention commonly selects x1. An x86-64 near CALL has an architectural stack effect, but the ABI determines the alignment and ownership facts that must hold around it. A direct jump is an ordinary ISA transfer; it becomes a tail call only when the outgoing argument state satisfies the next procedure's entry contract and the current procedure's obligations have already been discharged.

This gives a useful proof order. First apply the instruction rule to establish the exact link, program-counter, register, and memory effects. Next apply the ABI lemmas that interpret the affected locations as a continuation, argument, result, stack area, or preserved resource. Finally apply the procedure theorem that connects the abstract caller and callee. Reversing that order risks using the word "call" as if it already guaranteed a valid target, valid stack, and eventual return.

The distinction also explains why an ABI violation need not be an illegal instruction execution. A body can follow every ISA rule while entering with a misaligned stack, returning a value in the wrong register, or damaging a callee-saved register. Architectural validity is necessary for ABI conformance, but it is not sufficient.

Name the x86-64 ABI, not merely the ISA

System V AMD64 and Microsoft x64 use the same broad instruction set but differ in argument registers, stack-area rules, and preservation details. A theorem about one calling convention must not be labeled only "x86-64." Name the ABI and the selected platform assumptions.

10.7 The same leaf contract in three machines

Define a mathematical procedure I that accepts a width-w word x and returns $x + 1 \bmod 2^w$. Its boundary contract observes the result, normal return to the caller's continuation, restored stack pointer, preserved registers, and all memory outside declared scratch. It permits named volatile registers and condition state to differ.

The three instances relate their input and result locations:

Table 10.1: A narrow procedure relation for the add-one leaf examples.

Machine	Input/result	Continuation	Preserved slice
A0 fixed case	r_0	known direct target	r_4, r_5, r_6
RV64 ABI	`a0`	`ra`	`sp`, `s0–s11`
SysV AMD64	`RDI/RAX`	top stack word	`RSP`, `RBX`, `RBP`, `R12–R15`

The table is deliberately a slice. The full ABIs classify floating-point, vector, aggregate, variadic, and memory arguments; define more registers and process state; and interact with linking and unwind information. A proof of these leaf examples imports only the rows it uses.

Reader proof for the shared add-one contract

Relate the three entry argument words to one arbitrary x. Each body stores $x + 1 \bmod 2^w$ in its declared result location. None writes memory or a declared preserved data register. A0 reaches its one fixed continuation by a direct jump. RV64I reads the unchanged incoming `ra` in `JALR` and leaves `sp` unchanged. x86-64 `RET` reads the continuation written by `CALL` and restores `RSP` to its pre-call value after consuming that word. Therefore the related observations agree on result, normal return, preserved slice, stack restoration, and nonscratch memory.

This proof contains machine-specific premises. The x86-64 case assumes the return-address load is valid and contains the caller's continuation. The RV64 case assumes `ra` names a valid aligned return target under the selected semantics. The

Ao case has only one statically fixed caller. If any premise is removed, the common mathematical result does not rescue the control claim.

10.8 Nested calls and frame preservation

A nested call makes preservation visible. Suppose a function receives x, computes an intermediate u, calls another function, and then combines its result with u. The value u is live across the call. If it occupies a volatile register, the caller must save it. If it occupies a callee-saved register, the current function must restore that register for its own caller before returning.

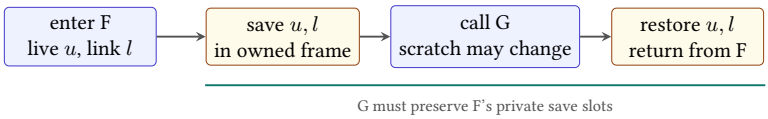


Figure 10.5: Preservation obligations cross call boundaries. A live intermediate and the caller's continuation must survive the nested call by declared means.

A conventional frame may contain a saved continuation, saved preserved registers, local objects, spill slots, and outgoing stack arguments. Their layout is not established merely by drawing boxes. Each slot needs a byte interval; distinct live objects need disjoint intervals unless sharing is intentional; each access needs the right width; and the epilogue must load the same saved values before deallocation.

Preservation by save, call, and restore

Let location q contain entry value v, and suppose the callee may change q. Before the nested call, store v in a private frame slot m. The call may change q but must not change m under the nested contract. After return, load m into q. The final value of q is then v. The proof depends on slot validity, nonaliasing with nested writes, correct width and byte order, and normal return to the restore instruction.

10.8.1 Normal return is only one exit class

Unwinding after exceptions is harder because the ordinary epilogue may not run. A procedure may also trap on an invalid access, fail a checked arithmetic operation, receive an asynchronous signal, diverge, or terminate the process. Those outcomes cannot be silently folded into a theorem whose conclusion says that control returns to the saved continuation.

One clean specification partitions behavior. On a normal exit, the result and ABI preservation clauses hold. On a named exceptional exit, a different relation describes the transferred control, visible state, and resources that must remain recover-

able. Divergence is a trace property rather than an exit state. A partial-correctness theorem may say what follows *if* normal return occurs; a total-correctness theorem must additionally rule out every other behavior permitted by its model.

Platform unwind metadata and language exception rules provide a route to recover saved registers and older frames without executing the ordinary epilogue. The metadata is then part of the binary contract and deserves its own validation: its description of frame changes must agree with the actual instructions at each covered program point. Debug information that merely prints a plausible backtrace is weaker evidence than a checked correspondence between unwind rows and instruction semantics. Full unwinding remains outside this first scalar model, but the boundary is now explicit.

Fault ordering also matters. If a prologue adjusts the stack pointer and then faults while saving a register, the machine is neither at the entry boundary nor at the normal exit boundary. A proof system that models faults needs the intermediate state and the architectural rule selecting the reported fault. If the theorem excludes faults, its precondition must be strong enough to make every prologue, body, and epilogue access valid.

10.9 Observing ABI conformance

A function need not restore volatile registers, temporary flags, or bytes in its private frame after release when the contract makes them unobservable. It must restore the stack pointer and every callee-saved register on normal return. It must not damage caller-owned memory. The return value must appear in the declared location, and control must reach the saved continuation.

The rule is observational equivalence with a structured boundary. Two bodies may use different frame sizes, instruction counts, scratch registers, and internal control flow while conforming to the same contract. Conversely, identical return values do not establish conformance if a preserved component differs.

The smallest useful ABI counterexample

Run two add-one bodies from related entries. Both return the expected result. One also replaces a preserved register value 7 with 8. The result-only observation reports agreement; the ABI observation reports the concrete register mismatch. Appending a caller that reads that register turns the boundary failure into a visible program difference.

Tail calls alter the return path. A proper tail transfer may reuse the current caller's continuation instead of creating another one, but only after the current frame and preservation obligations have been discharged. Calling a jump a tail call does not prove that the ABI state at the target is valid.

10.10 An introductory Axeyum route

The present Axeyum checkout still lacks executable Ao and real-ISA procedure semantics. The appropriate first implementation is therefore a small, machine-neutral procedure-contract checker layered over future step traces. It should not begin by encoding an entire platform ABI.

Represent an entry observation, a normal-return observation, input and result maps, volatile and preserved location sets, stack-pointer checkpoints, owned memory intervals, and an expected continuation. A trace adapter supplies the first and last related states. The contract checker compares every promised component and reports the first mismatch by class.

Then connect one Ao fixed-return trace. Its positive case computes add-one and preserves the declared slice. Controls change the result, one preserved register, the final stack pointer, a caller-owned byte, and the continuation. Every mutation must fail through the corresponding contract clause.

RV64I and x86-64 adapters require source-pinned decoder and step packages before their traces count as machine-code evidence. RV64 controls should overwrite `ra` across a nested call, omit a saved `s` register, or misalign `sp`. x86-64 controls should damage the stacked return address, restore the wrong RSP, omit an RBX restore, or enter a nested call at the wrong alignment.

Contract checking and semantic checking are different

A contract checker can prove that two supplied endpoint states satisfy an ABI relation. It does not prove that instruction bytes generated those endpoints. A trace checker connects every adjacent state to executable semantics. A complete evidence route needs both, plus decoder provenance and controls that fail in each layer.

For a solver-backed preservation theorem, symbolically execute the selected body from arbitrary contract-satisfying inputs and ask whether any promised exit component can differ. Replay a satisfiable counterexample through the executable semantics. For an unsatisfiable result, retain the exact formula, semantic digests, and the strongest available independent evidence. Keep the theorem limited to the body, ABI slice, memory region, and normal-return path actually encoded.

10.11 Where the procedure model stops

This chapter covers ordinary scalar calls and normal returns. It excludes floating-point and vector argument classes, aggregate classification, variadic functions, structure returns, thread-local state, dynamic linking, position-independent linkage sequences, system calls, signals, interrupts, exceptions, unwinding, coroutines, privilege changes, and control-flow enforcement.

It also excludes stack exhaustion, guard pages, asynchronous modification, concurrency, and timing. A valid architectural frame can still leak secrets through access patterns or speculative behavior under a stronger observation. Those claims require another model.

The narrow result remains powerful. A procedure boundary lets code written at different times, in different languages, and by different people compose through a finite agreement. The machine sees words, addresses, and control transfers. The ABI turns those objects into a promise one component can keep for another.

10.12 Exercises

1. Separate ISA facts, assembler spellings, and ABI rules in the RV64I `add_one` listing.

2. Do the same for the x86-64 listing. Include RDI, RAX, RSP, the stacked continuation, and RET.

3. **Execute.** Let an x86-64 call begin with RSP=0x1000 and return address 0x402006. Trace the stack-pointer and return-address memory effects of the selected near CALL and matching RET.

4. Explain why core Ao cannot implement a reusable return through r_7. State the minimum new semantic capability an extension would need.

5. **Prove.** For nonwrapping address arithmetic, prove that allocating and releasing an F-byte frame restores the entry stack pointer. Then give three frame errors that this equality does not exclude.

6. For a 32-byte downward-growing frame at entry pointer 256, list its byte interval. Decide whether 8-byte stores at 224, 248, and 252 stay inside it.

7. Give frame sizes that preserve 16-byte alignment and sizes that break it. State the checkpoint at which your claim applies.

8. Mark each selected RISC-V ABI register as argument/result, temporary, preserved, link, or stack pointer: `a0`, `a7`, `t2`, `s3`, `ra`, and `sp`.

9. A live value occupies `t0` across a nested RV64 call. Give two correct preservation strategies and identify who owes each save and restore.

10. **Break.** Remove the save of incoming `ra` from a non-leaf RV64 function. Construct the shortest trace that returns to the wrong address.

11. Give a System V AMD64 body that returns the right value but violates one callee-saved-register obligation. Append a caller that observes the damage.

12. Explain why a leaf x86-64 function containing only `lea` and `ret` still has a memory-reading return step.

13. **Transfer.** Write the complete narrow state relation used by the three add-one examples. Include input, result, continuation, preserved state, stack state, memory, outcome, and permitted scratch differences.

14. Extend that relation to a nested call with one live intermediate. State the frame-slot nonaliasing premise needed by the save/restore proof.

15. Design the first procedure-contract artifact for Axeyum. Include endpoint states, trace reference, observation sets, owned memory, ABI source pin, and five independently firing negative controls.

16. Distinguish a decoder proof, trace proof, endpoint contract check, and cross-ISA procedure refinement. Explain what evidence each still lacks.

17. Explain why a tail jump is ABI-correct only after the current function's frame and preservation obligations have been discharged.

CHAPTER ELEVEN

Equivalence, Observation, and Refinement

Two programs can produce the same answer and still disagree about memory, conditions, traps, running time, or whether they return at all. They can agree for one caller and fail for another. They can implement the same mathematical operation while inhabiting machine states with no matching register names. The word *equivalent* becomes useful only after these choices are fixed.

This chapter turns agreement into a statement with visible parts. A precondition selects inputs. Program semantics produces behavior. An observation selects what the claim exposes. Equivalence compares programs in one state space. Refinement relates programs whose states or permitted behaviors differ. A counterexample supplies an allowed input and an inspectable execution that breaks one required clause.

Equivalence is always relative to a contract

Name the initial states, termination policy, trap policy, observation, and program semantics. If a surrounding context will use the replacement, name what that context may read. Without those parts, "same program" is an intuition rather than a theorem statement.

11.1 Why program agreement became a central problem

Programmers have replaced instruction sequences for as long as they have optimized code. A hand optimizer shortened an address calculation. A compiler reordered operations or allocated another register. A **linker**, the tool that combines object files, relaxed a long transfer into a shorter one. Each transformation carried an implicit promise: the change would not alter what mattered to the rest of the program.

Modern systems make that promise at larger scale. Optimizing compilers perform thousands of rewrites. Binary translators move code between instruction sets. Cryptographic implementers replace clear scalar formulas with sequences

chosen for speed or side-channel discipline. Superoptimizers search for a least-cost program within a declared language. Formal tools compare implementations with specifications. All depend on a defensible meaning of agreement.

The strongest meaning is often unnecessarily strict. A scratch register may differ after a closed function. Temporary condition bits may be dead. A source machine may have no counterpart for a target machine's flags. The weakest meaning is unsafe: comparing only the desired numeric result can hide a trap, wrong return address, or caller-owned memory corruption. The task is to choose the smallest observation that still protects every permitted use.

Optimization is a promise about the future

A replacement is not justified merely because two isolated executions look alike. It is justified when every allowed future use receives the behavior the contract promised. Context gives local equality its program-wide meaning.

11.2 Programs produce behaviors, not only final values

Let P be a deterministic program and S an allowed initial state. Its execution may reach a normally halted state, reach a trapped state with a reason, or continue without a terminal state. Write

$$\mathit{Beh}(P, S)$$

for that behavior. A finite bounded runner may also stop because its external step budget is exhausted, but bound exhaustion is evidence about a prefix, not a fourth architectural behavior.

For a terminating behavior, let $\mathit{final}(P, S)$ denote the terminal state. A normal-result observation A is applied only where its domain is defined. A total behavior observation can instead map normal return, trap, and divergence to distinct forms while applying A to a normal final state.

Termination-sensitive observed behavior

Fix an observation A. The observed behavior $B_A(P, S)$ reports one of: normal return together with A of the final state; trap together with the declared trap observation; or divergence. Two executions agree only when these outer behavior classes agree and their exposed contents agree.

Divergence is a mathematical property of an unbounded execution. A thousand-step prefix does not establish it. Conversely, a termination argument may use an invariant and decreasing measure without enumerating the entire trace. Chapter 9's counted loop supplied exactly that form of reasoning.

Some claims intentionally use **partial correctness**: if both programs return normally, their results agree, but termination is handled elsewhere. That statement can be valuable, yet it does not authorize replacing a terminating program with a diverging one. The theorem and its use must retain the weaker termination policy in the name or scope.

A result observation must not erase behavior class

If one execution returns $r_0 = 7$ and another traps after previously writing $r_0 = 7$, projecting both states to r_0 hides the difference. Compare normal, trapped, and divergent behavior before comparing a result value.

11.2.1 When one input can have several behaviors

The selected A0, RV64I, and x86-64 slices are deterministic: a legal running state has at most one next architectural state. Larger models may admit several behaviors from one input. A source language may leave evaluation order unspecified. A concurrent machine may allow several event orders. An abstract specification may intentionally permit several implementation choices.

Then $\mathit{Beh}(P, S)$ is a set rather than one item. Agreement can mean set equality, inclusion, or agreement on selected possibilities. A refinement often requires every implementation behavior to be allowed by the specification:

$$\mathit{Beh}(Q, T) \subseteq \mathit{Beh}(P, S)$$

for related inputs. The implementation may resolve choices, but it may not add an outcome forbidden by the specification.

Two common questions differ. A **may** property asks whether at least one allowed execution has an observation. A **must** property asks whether every allowed execution has it. Showing that two programs can both return 7 does not show that they must return 7; one may also trap. A checker must retain the quantifier over behaviors.

This chapter keeps the executable examples deterministic so that one trace can explain a witness. The set view is still useful because it clarifies the direction of refinement and prepares the boundary discussion in Chapter 16. It also prevents an endpoint equation from being generalized beyond the model that made it single-valued.

11.3 Preconditions select the promised domain

A **precondition** $\Phi(S)$ is a predicate on initial state. It can require valid memory ranges, aligned code, nonaliasing buffers, a bounded counter, related ABI state, or a specific operand width. Equivalence under Φ quantifies over every initial state satisfying it, not merely the examples used to discover the claim.

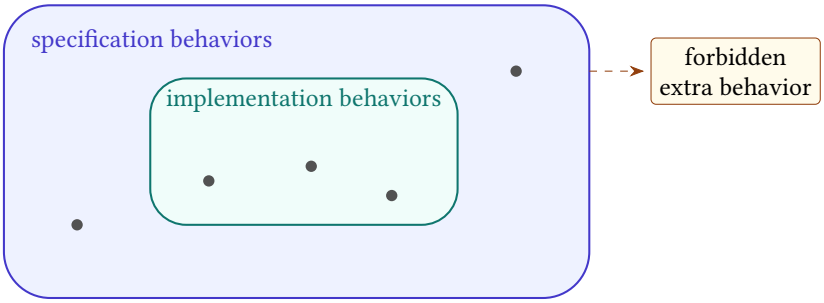


Figure 11.1: A deterministic behavior is one path. A nondeterministic specification admits a set; an implementation refines it when its behaviors remain inside that set.

For Ao, compare these one-instruction fragments at a running aligned program counter:

```
P: movi r0, 0
Q: xor  r0, r0, r0
```

Listing 11.1: Two candidate ways to clear r0

Both leave $r_0 = 0$ and advance by four on a legal fetch. They differ in condition state: `movi` preserves Z, N, C, V, whereas Ao `xor` sets Z, N from its zero result and clears C, V. Under an observation of r_0, next pc, and running outcome, the fragments agree. Under full state they generally do not.

Add the precondition that entry conditions already equal $(Z, N, C, V) = (1, 0, 0, 0)$. Now the two successors agree on those condition bits as well. With unchanged remaining registers and memory, they reach the same complete successor state. The precondition did not change either instruction; it selected the inputs on which their complete effects coincide.

Removing one premise produces a witness

Remove the entry-condition premise and choose $C = 1$. Program P preserves $C = 1$; program Q clears it. The initial state is a concrete witness against full-state equivalence. Appending `branch.hs` turns the hidden condition difference into different control flow.

Preconditions must be usable by the intended caller. A theorem requiring the two programs' outputs to be equal as an input premise is circular. A theorem requiring a buffer range to be valid may be useful when the caller already establishes that range. Proof systems can validate implications among preconditions, but the book must still explain why the chosen premise matches the application.

Strengthen and weaken a claim

For P and Q, write three contracts: result-only agreement with no condition premise; full-state agreement with the exact condition premise; and a false full-state claim with one premise removed. Give the witness for the false claim before asking a solver to find it.

11.3.1 Where preconditions come from

A precondition may be inherited from a caller, derived from earlier instructions, imposed by an ABI, or chosen to make a local optimization true. Those sources have different proof obligations. An inherited premise must be part of the public interface. A derived premise needs a proof from the prior state. An ABI premise needs a pinned profile. An optimization restriction must not exclude inputs the original program promised to handle.

For a candidate rewrite, one can ask for a **weakest precondition**: the largest set of initial states on which the desired agreement holds. In the clear-register example under full-state observation, the condition tuple must already equal the tuple written by XOR. That exact equality is the weakest condition-state premise when all other legal-fetch premises are fixed.

Weakest does not automatically mean best. A complicated disjunction may be hard for callers to establish or readers to understand. A slightly stronger aligned-range premise may match a public interface better than an exact bit-level formula. The book should state both the sufficient premise used and, when important, whether it is known to be necessary.

A caller must discharge the premise

Suppose a rewrite is valid only when carry is zero. If the preceding instruction clears carry by its declared semantics, sequential composition can establish the premise. If carry is merely dead after the rewrite, that does not show it was zero before the rewrite. Entry and exit facts must not be exchanged.

Solver-assisted precondition discovery can suggest a condition by analyzing counterexamples. The resulting formula remains a hypothesis until checked against the executable semantics and simplified without changing its domain. Discovery and theorem admission are separate routes.

11.4 A hierarchy of agreement

Let Φ be a precondition and A an observation. For programs over the same deterministic state space, define termination-sensitive observational equivalence by

$$P \equiv_{\Phi,A} Q \quad \text{when} \quad \Phi(S) \Longrightarrow B_A(P,S) = B_A(Q,S)$$

for every initial state S.

Full-state equivalence is the special case in which the observation retains the complete terminal state and behavior class. Trace equivalence is stronger again if it requires agreement after each corresponding step. Equal final states do not imply equal traces: one program may use a temporary register, take more steps, or visit another control location and later restore agreement.

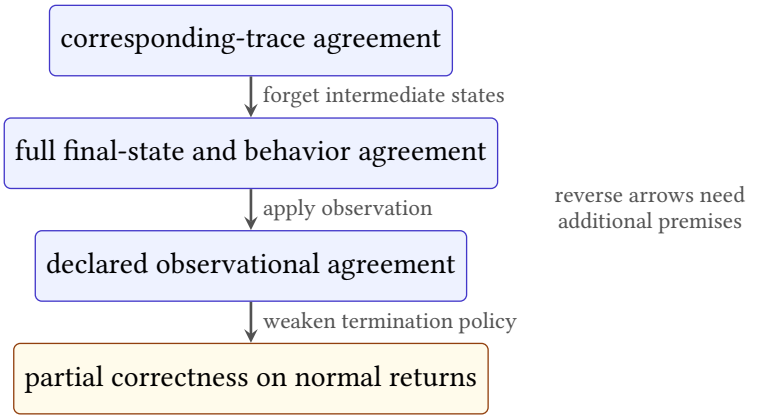


Figure 11.2: Common agreement notions retain different information. Downward implications hold for fixed compatible definitions; upward implications need additional premises and usually fail.

If observation A can be calculated from observation B, then agreement under B implies agreement under A. The converse can fail because A forgets information. This order lets a proof library reuse a strong result for weaker clients without pretending that every client needs full state.

Full-state equivalence implies observational equivalence

Assume both programs have the same behavior class and, on normal return, the same complete final state for every S satisfying Φ. An observation is a function. Applying the same function A to equal final states yields equal results. Trap and divergence agreement is retained by the outer behavior comparison. Therefore $P \equiv_{\Phi,A} Q$ for every declared A.

The converse fails whenever A is not injective over reachable final states. The clear-register example supplies two unequal condition states that map to the same result observation. No philosophical argument is needed; the two states and projection are enough.

11.4.1 Observations form a design space

An observation is not limited to selecting fields. It can calculate a mathematical reading, normalize several machine outcomes, record an address trace, or combine smaller observations. If A_1 observes the result and A_2 observes memory, their product

$$(A_1 \times A_2)(S) = (A_1(S), A_2(S))$$

retains both. Agreement under the product implies agreement under either projection.

One observation is **finer** than another when the coarser observation can be calculated from it. Full state is finer than a register projection. A complete trace is finer than its final state. Trap class plus reason is finer than trap class alone. This ordering is about retained distinctions, not about which theorem is more valuable.

The coarsest useful observation depends on the client. For a closed arithmetic expression, one result word may suffice. For a procedure, continuation and frame state matter. For constant-time code, an address or branch trace may matter even when results agree. Choosing too fine an observation rejects safe optimizations; choosing too coarse an observation accepts unsafe ones.

Table 11.1: Sample observations and distinctions they retain.

Observation	Retains	Forgets
Result word	one declared output	scratch, memory, conditions
Procedure result	result, outcome, continuation, frame	caller-saved scratch
Complete final state	every terminal component	intermediate movement
Address trace	selected memory locations and order	data values unless added
Complete trace	every selected architectural state	omitted microarchitecture

Observation conversion should be explicit in a proof library. If a theorem uses complete final state and a client needs only a result, apply the result projection as a lemma. Do not copy the theorem and silently give its scope a different name.

11.5 Refinement relates different state spaces

Equivalence starts both programs from one state. **refinement** uses an input relation $R_{in}(S, T)$ between source state S and implementation state T, plus an output

relation R_{out} between their behaviors or terminal states. One useful deterministic form is

$$R_{in}(S,T) \implies R_{out}(\mathit{Beh}(P,S), \mathit{Beh}(Q,T)).$$

The relation may map Ao r_0 to RV64I `a0`, ignore an x86-64 scratch register, relate little-endian memory regions at different base addresses, and connect different program-counter locations by control points. It does not make the state spaces identical.

Refinement is directional. An implementation may allow fewer behaviors than an abstract specification while satisfying every abstract requirement. For our deterministic scalar slices, direction often appears only in the chosen input and output relations. It becomes essential with **nondeterminism**, where one input can permit several behaviors, underspecification, concurrency, or implementation-defined choices.

A relation carries meaning across machines

An observation projects one state into visible information. A state relation states when two differently shaped states represent the same abstract situation. Cross-ISA refinement needs the relation before execution and after execution; observation alone cannot align the machines.

Chapter 12 develops stepwise simulations. For now, notice that endpoint refinement can be true even when instruction counts differ. Ao may require a comparison followed by a branch, RV64I may compare registers in the branch, and x86-64 may read flags set earlier. The relation needs to hold at selected entry and exit points, not after every unmatched instruction.

11.6 Contexts can expose forgotten state

A **context** is surrounding program structure with a hole into which a fragment is placed. Write $C[P]$ for context C containing fragment P. A local result observation does not automatically justify $C[P] \equiv C[Q]$, because the suffix may read state that the local observation forgot.

Return to `movi r0,0` and `xor r0,r0,r0`. A suffix that immediately overwrites all conditions before reading them cannot expose their difference. A suffix beginning with `branch.hs` can. Contextual replacement therefore needs either stronger local agreement or a restriction on what the context may observe before forgotten state is overwritten.

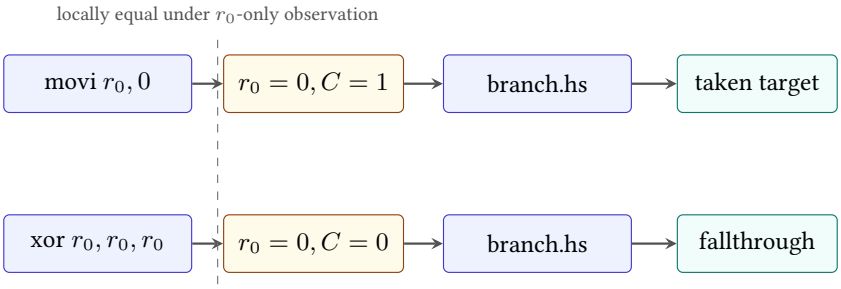


Figure 11.3: A narrow local observation can hide a condition difference. A suffix that reads carry converts it into a distinct control path.

A sufficient contextual-replacement rule

Suppose fragments P and Q terminate from every allowed entry and leave states related by R. Suppose the context's next step preserves R, and every later pair of related steps preserves it until the final observation. By induction over the context steps, the executions remain related. If R-related terminal states have equal final observations and behavior classes, then $C[P]$ and $C[Q]$ agree under that observation.

This rule reveals the work hidden by "the context cannot tell." We need a relation after the fragment, a preservation argument for the suffix, and an observation compatible with that relation. A read/write or liveness analysis can establish a simpler special case when all differing locations are dead until overwritten.

Dead at one point does not mean absent from the state

A condition bit or scratch register may be irrelevant to the current output and live again in the next instruction. Deadness is a property of a location relative to a program point and future uses. It is not a permanent semantic property of the location.

11.6.1 Read and write sets give a practical sufficient rule

Let D be the set of locations on which fragments P and Q may differ at their common exit. If every path in the suffix overwrites each location in D before reading it, and the overwrite does not itself depend on another difference, then those exit differences cannot affect the later observation. This is a semantic form of dead-state elimination.

The order matters. A suffix that reads carry and then overwrites all flags can still distinguish the clear-register fragments. A suffix that first executes an

instruction with fully defined condition outputs and only then branches may erase the difference. The proof follows paths, not just unordered read and write sets.

Memory makes the rule relational. A store to $[p, p + 8)$ overwrites a differing byte only when the suffix's effective address is known to alias that byte. A load through another symbolic pointer may expose it. Nonaliasing or must-alias facts therefore become premises of the contextual proof.

Overwrite-before-read replacement rule

Assume P and Q terminate with states equal outside D. Along every allowed suffix path, each location in D is overwritten with equal values before any instruction or final observation reads it. All other instructions receive equal inputs and have deterministic equal effects. Induct over suffix steps. After the last location in D is overwritten, the complete states agree; the remaining execution and observation therefore agree.

This rule is sufficient, not necessary. Two contexts may read unequal values and nevertheless calculate equal outputs. A relational invariant can prove that more general case. The overwrite rule is valuable because a compiler's live-variable and effect analyses can often establish it automatically.

11.7 Counterexamples should become traces

The negation of a finite-width observational-equivalence claim asks for an allowed initial state whose observed behaviors differ:

$$\Phi(S) \text{ and } B_A(P, S) \neq B_A(Q, S).$$

A solver may return assignments for thousands of internal formula variables. Those assignments are not yet a reader-facing counterexample. Decode the initial architectural state, run both programs through the same executable semantics used by the claim, and print the first meaningful divergence.

Figure 11.4: A useful solver counterexample returns to the semantic level. Formula assignments become an initial state, two checked traces, and a visible failed observation.

For the clear-register pair with unconstrained carry, a compact report can say: entry $C = 1$; P executes `movi` and preserves $C = 1$; Q executes `xor` and writes

$C = 0$; full-state comparison first differs at carry. If a contextual claim is under test, append the branch and show the two successor program counters as well.

The **first divergence** depends on the comparison. Traces of unequal length may have no step-for-step alignment. For identical one-step fragments, the first unequal successor is clear. For optimized routines, compare related control points or report the first violated relation rather than matching step numbers mechanically.

Three counterexample classes

An unequal final register with matching normal returns is a value witness. A normal return on one side and a trap on the other is an outcome witness. A pair with equal final observations but a differing intermediate secret- dependent address is a trace witness under an address-trace observation. The claim decides which class matters.

Counterexample minimization can improve explanation. Prefer small word values, short traces, few changed memory bytes, and the earliest failing component, provided minimization reruns the semantic checker. A smaller printed model that no longer satisfies the formula is not evidence.

Minimization should preserve a named witness predicate. Start from a replayed failure. Try setting irrelevant registers to zero, shrinking the initialized memory region, simplifying word values, or lowering a loop count. After every change, rerun the precondition, both executions, and the same observation. Keep the change only if the same failure class remains.

The smallest numeric value is not always the clearest witness. For signed overflow, boundary values explain the rule better than a solver's arbitrary large assignment. For aliasing, two short visibly overlapping ranges are better than unrelated high addresses. The minimizer can use a lexicographic cost: behavior-class mismatch first, earliest divergence second, number of nonzero state fields third, then magnitude.

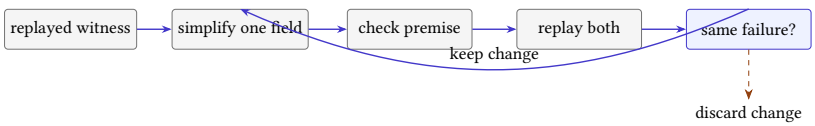


Figure 11.5: Counterexample minimization is a checked loop. A proposed simplification is kept only when semantic replay preserves the named failure.

A minimized trace should retain provenance to the original solver model. The report records which fields changed and which checker reconfirmed the witness. This keeps the explanatory example connected to the machine-derived result without pretending that the minimized state was the solver's original output.

11.8 Traps, undefined domains, and stuck states

A precondition can exclude a trap only when the theorem says so. For example, a load/store equivalence may require both byte ranges to be valid. Under that premise, no range trap occurs. Without it, the programs may trap differently because they compute different addresses or access different widths.

Do not confuse an undefined mathematical helper with an architectural trap. A partial specification function may be undefined outside its domain. An Ao running step instead produces a declared trapped outcome for an invalid data range or illegal instruction. A semantics can also become accidentally *stuck* if no rule applies and no trap was defined. Total executable semantics should distinguish legal successor, declared trap, terminal input, and malformed external data.

Trap equivalence itself has choices. A narrow claim may require only that both sides trap. A stronger claim may require the same reason and preserved state. An even stronger platform claim may expose fault addresses or exception codes. Print which trap observation is used; "both fail" is not a complete rule.

Choose a trap observation

Construct two Ao loads that both access outside finite memory but begin from different destination-register values. Compare them under three policies: trap class only; trap class and reason; complete trapped state. Identify which differences each policy can expose.

11.9 Composing equivalence claims

Equivalence is reflexive, symmetric, and transitive when its precondition, behavior policy, semantics, and observation are fixed appropriately. These properties permit chains of validated rewrites, but changing a scope in the middle can break the chain.

Suppose $P \equiv_{\Phi,A} Q$ and $Q \equiv_{\Phi,A} R$. For every allowed state, equality of observed behavior gives $B_A(P,S) = B_A(Q,S)$ and $B_A(Q,S) = B_A(R,S)$. Transitivity of equality yields $B_A(P,S) = B_A(R,S)$. If the first claim instead uses precondition Φ_1 and the second Φ_2, the composed claim needs at least their conjunction or another premise implying both.

Sequential composition also needs an intermediate-domain argument. If fragments P_1 and Q_1 leave related states, and P_2 and Q_2 agree only for states satisfying Ψ, then the first pair must establish Ψ for the second. Matching endpoint observations from the first pair may be too weak to establish that premise.

Congruence is equivalence that survives construction

An equivalence relation is a congruence for a program constructor when replacing related parts inside that constructor preserves the relation. Contextual equivalence asks for agreement in every allowed context and is therefore designed for replacement. Directly proving all contexts is often hard; simulations, logical relations, type systems, and effect analyses supply tractable sufficient conditions.

11.9.1 Refinement relations compose through an intermediate state

Suppose P refines to Q through relations R_{in}^{PQ} and R_{out}^{PQ}, and Q refines to R through corresponding relations for QR. Composition needs an intermediate witness: for every related P- and R-entry pair, there must be a Q entry satisfying both input relations. The two output relations must likewise imply the desired direct PR relation.

Writing this witness explicitly prevents a common gap. One proof may interpret a buffer at base a; the next may expect it at base b. One may expose normal-versus-trap outcome; the next may forget trap reason. One may require a preserved register that the next treats as scratch. The shared program name Q does not guarantee that its two contracts meet.

Relational composition pattern

Choose intermediate entry T such that $R_{in}^{PQ}(S, T)$ and $R_{in}^{QR}(T, U)$. Apply the first refinement to relate the behavior of P from S to Q from T. Apply the second to relate that same Q behavior to R from U. Finally use a relation- composition lemma showing that the two output relations imply R_{out}^{PR}. Quantify over every direct input pair for which such a compatible T is guaranteed.

In a compiler pipeline, the intermediate witness is often a translated program plus related state. In a cross-ISA proof, it may be an Ao state that both real machines represent. In an optimization chain, it may simply be the common machine state. The pattern is the same; only the relations change.

Optimization pipelines should record the exact contract of each rewrite. Combining destination-only, flag-sensitive, memory-sensitive, and termination-insensitive facts without conversion lemmas can produce an unsupported final claim even when every individual checker returns success.

11.10 Four ways to check a finite equivalence claim

Direct testing runs selected examples. It is fast and excellent for debugging, but its conclusion covers only those examples. Exhaustive enumeration covers every state in a genuinely finite declared domain; it becomes expensive as width, registers, memory, or program length grows.

Symbolic execution represents many inputs with expressions and accumulates path conditions. It can expose how a disagreement depends on an input and can feed a solver. A direct solver encoding instead asks whether the precondition and observation mismatch are jointly satisfiable. Both routes rely on the sound connection between machine semantics and generated formulas.

A reader proof can establish a parametric law, such as XOR clearing a register at every width, when the definitions admit algebraic reasoning. A proof assistant can check that argument against formal definitions. These methods are complementary: tests diagnose, enumeration calibrates tiny domains, solvers search wide finite spaces, and proofs explain universal structure.

Table 11.2: Checking routes support different scopes.

Route	Strongest direct scope	Central risk
Examples	named initial states	untested inputs
Enumeration	complete finite declared domain	omitted states or transitions
Solver query	generated finite formula	encoding and verdict trust
Formal proof	stated theorem over formal definitions	definition and assumption boundary

Agreement among routes is valuable but does not merge their scopes. If a direct-enumeration tool and solver agree at width eight, that cross-checks two implementations on one finite domain. A width-parametric proof needs its own argument. Conversely, a proof over an abstract step rule does not validate a decoder until bytes are connected to that rule.

The practical sequence mirrors the book's pedagogy. Start with a hand-chosen witness and its complete trace. Enumerate a miniature width to test the state inventory. Generalize the disagreement formula and ask a solver for a model. Replay every returned model. Only then invest in a certificate or parametric proof whose exact statement has survived those cheaper attacks. Each stage sharpens the claim carried by the next.

11.11 An introductory Axeyum equivalence route

The current Axeyum checkout supplies bit-vector and solver infrastructure but does not yet contain the executable Ao program layer required by this chapter.

The first sound route begins only after `a0-state`, `a0-step`, and `a0-run` exist with their negative controls.

Represent an equivalence query with semantic-package digests, two decoded programs, word width, precondition, behavior policy, observation, and a bound or termination argument. Generate the negation of the exact reader statement. For a satisfiable result, decode the model into a complete Ao initial state and replay both programs. Reject the evidence if replay does not reproduce the claimed mismatch.

For an unsatisfiable result, preserve the formula and solver configuration. The strongest available route should export independently checkable evidence or reconstruct the theorem in a kernel. The assurance label must still say whether the result covers one width, a finite width set, or a width-parametric statement.

The first A0 equivalence query

Compare `movi r0,0` with `xor r0,r0,r0`. Query result-only equivalence first; a deliberate destination mutation must produce a replayed witness. Then query full-state equivalence without the condition premise; the tool should find a condition-state witness. Finally add the exact premise and check the stronger claim. The sequence tests both proof and counterexample paths without requiring a long program.

Negative controls must target every layer. Mutate a destination or condition effect in Ao semantics. Omit one observed component. Invert the precondition. Corrupt one decoded model field before replay. Change the formula digest after the solver runs. A route that rejects only false programs but accepts a stale formula or nonreplaying model has not established the advertised claim.

Why replay is load-bearing

The solver establishes a result about the generated formula. Replay checks that a satisfying assignment decodes to an allowed architectural state and that executable semantics produces the reported difference. Without replay, a bug in formula generation or model decoding can manufacture a persuasive but irrelevant assignment. Replay does not validate an unsatisfiable verdict; that needs formula provenance and a certificate or stronger checking route.

The Python or PyO3 surface should make the teaching path small: construct two programs, choose a named observation, state a precondition, request a check, and receive either bounded evidence or a typed counterexample trace. The lower Rust layer owns semantic definitions, digests, solver translation, and replay. The convenient interface must not invent a broader result than the evidence returned by that layer.

11.12 Where the equivalence model stops

This chapter uses deterministic sequential architectural behavior. It does not cover concurrency, weak memory, probabilistic behavior, external input during execution, interactive systems, fairness, timing, energy, cache traces, speculation, or quantitative leakage. Each can require sets or distributions of behaviors and richer observations.

It also introduces refinement without yet proving a cross-ISA simulation. Chapter 12 supplies control-point relations, stuttering, and forward-simulation reasoning. Chapter 13 adds program languages and costs; agreement alone says nothing about minimality or speed.

Within the boundary, a precise equivalence statement turns optimization from hope into a refutable claim. The counterexample is not an embarrassment. It is the smallest state in which our promised sameness breaks, and therefore one of the clearest explanations the machine can give us. Preserving that explanation requires the report to retain the starting state, both outcomes, the selected observation, and the first component that violates the relation. A bare Boolean loses the lesson at the moment it finds the bug.

11.13 Exercises

1. Give two Ao final states related by an r_0-only observation but separated by a full-register observation, a condition observation, and an outcome observation.

2. **Execute.** Trace `movi r0,0` and `xor r0,r0,r0` from entry conditions $(0, 1, 1, 1)$. List the complete successors and their first differing components.

3. State the weakest observation in which the two clear-register fragments agree without a condition precondition. Explain one context that makes that observation insufficient for replacement.

4. **Prove.** Prove their full-state equivalence under the exact entry- condition premise printed in this chapter. Include unchanged registers, memory, program counter, and outcome.

5. **Break.** Remove each condition conjunct from that premise in turn. Give a concrete entry state when removal makes the full-state claim false, or explain why a conjunct was redundant.

6. Give a pair of programs with equal final states but different trace lengths. State a final-state observation that equates them and a trace observation that separates them.

7. Distinguish normal return, trap, divergence, and bound exhaustion. Which are architectural behaviors in the model, and which is a runner result?

8. Write a partial-correctness statement that permits termination mismatch. Explain why it cannot alone authorize replacement in a terminating context.

9. Let observation A be computable from B. Prove that agreement under B implies agreement under A. Give a noninjective counterexample to the converse.

10. Define input and output relations between the Ao, RV64I, and x86-64 add-one procedures from Chapter 10. Do not require identical register names or program-counter values.

11. **Transfer.** State endpoint refinement for those three procedures, including behavior class, result, continuation, stack restoration, preserved state, and memory.

12. Build a suffix that observes the carry difference between the two Ao clear-register fragments. Trace the first control-flow divergence.

13. Prove the sufficient contextual-replacement rule for a suffix of n steps by induction on n.

14. Give two trapping Ao executions that agree under trap-class observation but differ under a complete trapped-state observation.

15. Show how preconditions compose when $P \equiv_{\Phi_1,A} Q$ and $Q \equiv_{\Phi_2,A} R$. State a sufficient premise for relating P and R.

16. Design a counterexample report containing initial state, semantic digests, both traces, first violated relation, and final observations. Explain which fields a reader needs and which solver-internal fields may remain archived.

17. Design the first Axeyum Ao equivalence manifest. Include the exact query, width, precondition, observation, behavior policy, expected result, replay command, formula digest, and five negative controls.

18. A solver returns a model that fails executable replay. Give at least three possible fault classes and explain why the original equivalence verdict cannot use that model as evidence.

CHAPTER TWELVE

Relating Different Instruction Sets

An Ao state cannot equal an RV64 state or an x86-64 state. Their register files have different names and sizes. Their program counters visit different addresses. Their instructions occupy different byte sequences. Ao and x86-64 expose condition state that base RV64I does not possess. Literal state equality would reject every useful comparison before execution began.

Cross-ISA reasoning replaces equality with a relation. The relation says which machine objects represent the same input, where corresponding results must appear, which memory regions match, how control locations line up, and which state may differ. A simulation then shows that execution preserves this shared meaning even when the machines take different numbers and shapes of steps.

Relate meaning, not register names

A cross-machine theorem needs an input relation, control-point relation, memory relation, outcome relation, and output relation. It may deliberately ignore scratch registers or condition state, but only after showing that the rest of the allowed computation cannot expose those differences.

12.1 Why translation needs more than matching answers

Programs have crossed machine boundaries since assemblers, compilers, and emulators first separated a programmer's operation from one machine's code. Retargeting a compiler asks whether different instructions implement the same source operation. Emulation asks whether one machine can reproduce another's steps. Binary translation asks whether already encoded code can move between architectures. Hardware verification often compares an implementation with a simpler reference machine.

Today the same question appears in verified compilers, portable cryptography, instruction lifting, migration tools, and superoptimization. A translator can produce the expected result on test inputs while mishandling a fault, high address, condition

bit, calling convention, or loop exit. A destination-only comparison is a useful test; it is not a cross-machine correctness theorem.

The abstraction level also matters. A source-language integer may correspond to a word in each machine. A source function may correspond to entire machine routines. One abstract step may require several target instructions, while one target instruction may combine several abstract operations. The proof should choose a level at which the relation is both true and useful.

A relation is a small bilingual dictionary

The relation does not translate every fact in either machine. It records the facts needed for one shared computation: this register carries the input, these bytes represent the same array, these locations are corresponding loop heads, and these outcomes mean the same kind of completion.

12.1.1 Three translation questions that should not be collapsed

Compiler correctness, instruction lifting, and binary translation all cross semantic boundaries, but they begin with different objects. A compiler starts from a source-language program and usually relates whole behaviors through several intermediate languages. A lifter translates decoded machine instructions into an intermediate form for analysis. A binary translator turns one encoded machine program into another executable program.

In compiler verification, the source may leave implementation choices unspecified or declare some executions outside its guarantees. The target has concrete words, registers, and faults. A semantic-preservation theorem must state how source behaviors correspond to target behaviors; CompCert is a major demonstration of this style at realistic scale (Leroy 2009).

A lifter has a more local initial question: does executing the lifted intermediate statement reproduce the decoded instruction's architectural effect under a state relation? If the lifter drops flags, faults, or address width, its intermediate language may be too weak for later analyses. A useful lifting theorem therefore names both the translated effect and the observation under which omitted state is safe.

A binary translator owns control layout as well as local operations. Expanding one source instruction into several target instructions moves branch targets and may require scratch storage. Calls and returns must respect a target ABI. Code addresses embedded in data or calculated at run time can make relocation observable. Local instruction lemmas are necessary, but a whole-program control relation is what composes them.

The Ao-to-real-ISA direction in this book resembles a small reference-machine refinement. It does not claim that Ao is a source language or that either real machine implements every Ao behavior. The candidate slices and relations are chosen per theorem. Keeping that scope narrow is what makes the proof inspectable.

Table 12.1: Related translation tasks begin and end at different semantic levels.

Task	Source and target	Central proof question
Compiler	source language to machine code	are allowed source behaviors preserved?
Lifter	decoded instruction to intermediate statement	does one lifted movement match one machine movement?
Binary translator	encoded machine program to encoded machine program	do local matches compose through control and procedure context?
Emulator	guest states to host execution	does each guest movement have a faithful finite host realization?

12.1.2 RISC and CISC are shapes, not proof shortcuts

RV64I often exposes operations through regular fixed-width instructions and explicit registers. Selected x86-64 instructions may combine a memory access with arithmetic, use a destination as an implicit source, or update a rich status register. These broad RISC and CISC tendencies change useful cut points. They do not determine which proof is easier.

Consider adding a memory word to an **accumulator**, a register that holds the running result. An RV64 implementation may use a load followed by register addition. One selected x86 form may add a memory operand directly. The relation can compare the RV64 intermediate loaded word with the internal value consumed by the x86 instruction, or it can place cut points before and after the whole movement. The latter avoids inventing an architectural x86 state that does not exist between load and add.

Conversely, a single architectural instruction can have many cases. An x86 memory operand may require variable-length decode, address calculation, segment and mode assumptions, range checks, a memory read, arithmetic, status updates, and instruction-length advancement. Two RV64 instructions may give the proof more visible intermediate states even though the program has more steps. Step count and semantic complexity are different measures.

Choose a relation boundary that both machines possess

Relate the states immediately before an RV64 load-plus-add pair and an x86 memory-source add. Execute each complete segment. At the endpoints, require equal accumulator words, corresponding next control points, preserved related memory, and normal outcomes. Do not require a fictional x86 state halfway through its atomic architectural instruction.

The same principle applies in reverse when one architecture exposes an effect through a register and another through flags. Relate the Boolean or arithmetic

meaning at a point where both are available. Architecture-family labels help predict where mismatches may appear; only decoded semantics and a stated relation settle the theorem.

This view also explains why the book studies both families from one scalar foundation. Ao supplies a compact semantic vocabulary, not a claim that every real instruction decomposes into one canonical RISC-like sequence. Sometimes one Ao movement matches several real instructions. Sometimes one real instruction matches several Ao movements at once. The relation may choose either grouping when the endpoints, intermediate fault policy, and progress argument are explicit. Universality comes from the relational method, not from making one architecture imitate the surface form of another.

That flexibility is disciplined rather than arbitrary: every chosen grouping must still preserve the declared observation, expose its freedom clauses, and survive a control aimed at the semantic difference it intentionally crosses.

12.2 Anatomy of a cross-machine state relation

Let S_A, S_R, and S_X range over Ao, RV64, and x86-64 states. A relation for one routine can be written as a conjunction of smaller clauses.

1. *Value relation*: selected registers or memory objects carry equal width-w words or related mathematical readings.
2. *Memory relation*: corresponding byte intervals contain equal or translated bytes, with declared base addresses and ownership.
3. *Control relation*: each machine's program counter belongs to a named corresponding control point.
4. *Condition relation*: required predicates agree, even if one machine stores them in flags and another recomputes them.
5. *Outcome relation*: running, normal return, and selected traps correspond under the theorem's policy.
6. *Frame relation*: stack pointers, saved locations where execution continues, arguments, results, and preserved state satisfy the chosen ABI slice when procedures are in scope.
7. *Freedom clause*: scratch registers, dead flags, code addresses, and private bytes not mentioned above may differ.

The freedom clause is as important as the required clauses. Without it, readers may assume either that every omitted component must match or that every omission is harmless. A typed relation should list ignored and scratch state explicitly and prevent the final observation from reading it.

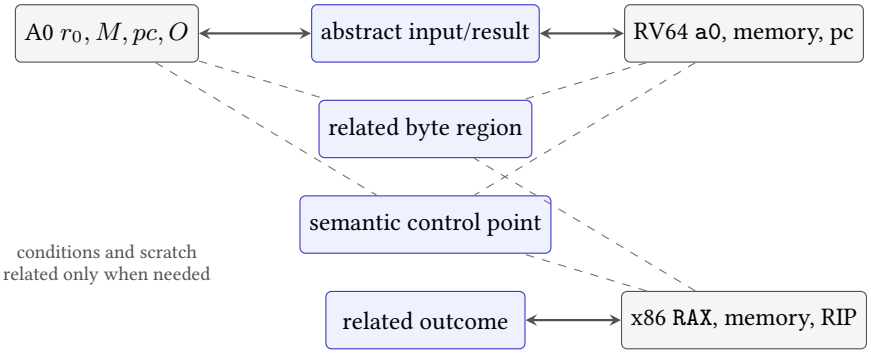


Figure 12.1: A relation aligns selected meanings across three unlike states. Gray components may differ only when the theorem and its contexts leave them free.

Control points name related moments

A control point is a semantic location such as routine entry, branch decision, loop head, memory update, or normal exit. It maps to one or more concrete program-counter values in each implementation. States at matching control points may be related even when states between them have no direct partner.

Numeric program-counter equality is usually irrelevant across ISAs. What matters is that A0 at its *decision* label corresponds to RV64 at its branch and x86-64 after the flags used by its conditional jump have been established. The relation can pair those distinct addresses through one control-point tag.

12.2.1 Design the observation before the relation

A relation can become needlessly strong when its author starts by pairing registers. Begin instead with the client-visible contract. Does the caller observe one result word, a returned memory region, normal versus trapped termination, preserved registers, or the next continuation? The relation then needs enough state to carry those observations through the computation.

Suppose a routine receives two words and returns their sum. A result-only test might compare A0 r_0, RV64 `a0`, and x86 `rax` at exit. A procedure theorem also relates entry arguments, stack discipline, continuations, and preserved registers. A contextual-replacement theorem may need condition state if a surrounding private convention reads it. These are three different claims about the same instruction sequences.

After choosing the observation, write a control-point table. Each row names a logical moment. Each machine column gives the concrete address or decoded instruction associated with that moment. The last column states the relation that

Table 12.2: Observation strength determines relation strength.

Claim	Required relation	May remain free
Result computation	inputs, result, normal outcome	scratch and continuation
Callable routine	arguments, result, frame, continuation	caller-saved scratch
Contextual replacement	every state later read by allowed contexts	state unreachable to those contexts

must hold there. Gaps in this table often reveal that one machine performs an operation earlier or later than the others.

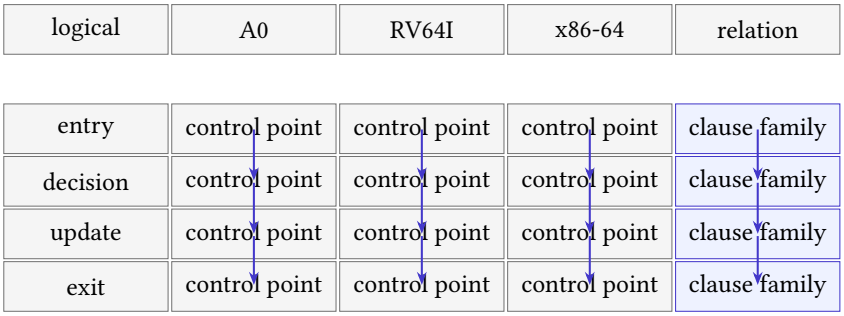


Figure 12.2: A control-point table turns three address spaces into one sequence of logical moments. Local proofs connect adjacent rows.

The table is not itself a proof. It is a proof plan. For every adjacent pair, ask which instructions execute, which relation clauses they read, which clauses they restore, and which intermediate states remain intentionally unrelated. This turns a broad cross-ISA claim into finite local obligations.

Make freedom executable

Choose one scratch register omitted at exit. Change its initial value while holding every related component fixed. Then append a context that reads it. Either the context must be excluded, or the relation must be strengthened.

12.3 Memory correspondence needs addresses and ownership

Equal bytes at numerically equal addresses are only one possible memory relation. A translated routine may place the source array at base a and the target array at base b. For a length n, a simple relation is

$$M_A[a+i] = M_T[b+i] \quad \text{for every } 0 \le i < n.$$

Outside those intervals, the theorem may require equality, preservation, or nothing, depending on ownership and observation.

Loads also need address correspondence. If Ao's effective address is $R_A[r_1] + d_A$ and RV64's is $R_R[x_5] + d_R$, the input relation must imply that both select corresponding offsets within the related regions. Equal loaded words then follow from byte agreement, width, and matching little-endian join rules.

Related bytes give related little-endian words

Assume width $w = 8k$, corresponding bases a, b, and $M_A[a+i] = M_T[b+i]$ for $0 \le i < k$. Each load joins byte i with weight 2^{8i}. Term-by-term equality gives equal sums modulo 2^w, so the loaded words are equal. The proof fails if widths, byte order, or accessed offsets differ.

Stores require a frame condition. Corresponding stores should write related bytes inside their owned intervals, and every caller-owned observed byte outside those intervals should remain unchanged. Merely comparing the written result misses collateral memory damage.

A memory relation must survive aliasing

Two symbolic base registers may identify overlapping ranges. If a proof treats them as disjoint, nonaliasing belongs in the input relation. If overlap is allowed, the simulation must account for the shared bytes and write order.

12.4 Forward simulation permits unequal step counts

A **forward simulation** begins with related states. Whenever the source makes a matched step or segment, the target makes zero or more steps to a state related to the source successor. Repeating this argument follows the source execution forward.

Allowing zero target steps is **stuttering**. It handles an abstract state change that the target already represents, bookkeeping steps ignored by the relation, or different placement of control points. To rule out a target that stutters forever

while the source progresses, a simulation may require a well-founded rank that decreases on zero-progress matches.

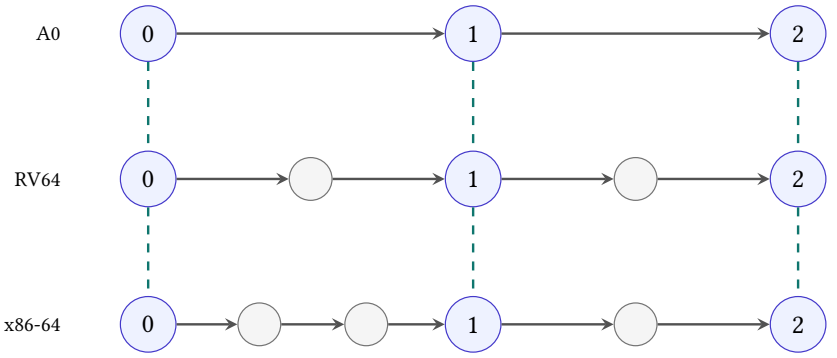


Figure 12.3: Forward simulation relates synchronization points, not every row. One A0 segment can match several RV64 or x86-64 instructions.

For finite straight-line routines, the proof can divide code into segments between control points. For loops, the relation must be invariant at every revisit, and progress or termination needs a separate argument. For branches, predicate agreement ensures that related states select corresponding edges.

From local simulation to an endpoint relation

Assume entry states are related. For each source segment between control points, the target has a finite matching segment whose endpoint restores the relation. Induct over the source segments. The base case is entry. The local simulation supplies the next related pair in the inductive step. At a terminal source point, the outcome clause and output relation therefore hold for the target endpoint. Looping executions additionally require a coinductive or invariant argument rather than finite induction over a fixed segment count.

Forward simulation is directional. It shows that target behavior can be matched by the specification under the selected orientation only if the step clause is stated that way. With nondeterminism, reverse simulation may be needed to exclude extra target behaviors. Our selected scalar examples are deterministic, but the direction should still appear in the theorem.

12.4.1 Safety, progress, and termination are separate transfers

A local simulation usually establishes a safety shape: if related execution reaches the next cut point, the new states are related. That conditional does not yet say that the target reaches the point. A target might trap, diverge in an internal loop, or stop because a runner exhausted its bound. Progress adds the fact that a legal

matched segment can take a next movement. Termination adds a global argument that repeated movements reach an exit.

For a deterministic finite straight-line segment, progress often follows by executing each decoded instruction and showing that no premise for a trap is violated. For a loop, a rank or source termination theorem is also needed. If one source iteration matches a finite target segment of at least one step, source termination can transfer. If zero-step matches are allowed, the proof must show that they cannot postpone progress forever.

Outcome refinement needs the same separation. One policy may require normal return to match normal return and every selected trap to match its counterpart. Another may prove only that normal source executions have normal target matches. The second theorem is weaker and can still be useful, but it cannot be quoted as universal behavioral equivalence.

Segment simulation obligation

For related source and target states at control point q, a segment obligation names a source path, a finite target path, their required entry premises, the next control point q', and the relation restored at their endpoints. It also names every failure outcome permitted before q'.

The word “finite” needs evidence. In a hand proof, the target path may be a fixed list of instructions. In a symbolic proof, a rank or bounded internal semantics may establish that the path is finite. A trace bound chosen by the checker is only a search resource until the theorem connects it to program structure.

12.4.2 When a reverse direction is required

Imagine a target program that returns the correct result for every related source run but also has an extra path that leaks into an unrelated successful result. A source-to-target existence statement can ignore the extra path. Behavioral equivalence cannot. A reverse simulation, determinism argument, or set-inclusion proof must rule it out.

Our three scalar machines are deterministic once bytes, state, and fault rules are fixed. This makes a forward simulation stronger: the target does not get to choose another successor. The theorem should still cite determinism rather than inherit it from intuition. Later models with concurrency, interrupts, or underspecified behavior lose that shortcut.

12.5 A three-machine absolute-value routine

Let x be a width-w word and $\langle x \rangle_s$ its signed reading. The mathematical absolute value lies outside the signed width-w range when x is the minimum signed value -2^{w-1}. We therefore choose one of two honest specifications:

1. modular absolute value, which returns x when nonnegative and $-x \bmod 2^w$ otherwise; or
2. mathematical absolute value under precondition $\langle x \rangle_s \neq -2^{w-1}$.

The worked proof uses the second. The precondition is not a decoder fact or an ABI rule. It is the domain on which a positive signed result represents the mathematical absolute value.

Use these schematic implementations:

```
cmp       r0, r1
branch.ge done
sub       r0, r1, r0
done:
```

Listing 12.1: Ao absolute value, with r1 initialized to zero

```
bge  a0, x0, done
sub  a0, x0, a0
done:
```

Listing 12.2: RV64I absolute value

```
mov  rax, rdi
test rax, rax
jns  done
neg  rax
done:
```

Listing 12.3: x86-64 absolute value in the selected local contract

The x86 listing is schematic Intel syntax for selected 64-bit forms. The chapter compares decoded semantics, not exact bytes. Machine-code evidence would additionally require source-pinned encodings and instruction lengths (RISC-V International 2026b; Intel Corporation 2026a).

At entry, relate Ao r_0, RV64 `a0`, and x86-64 `RDI` to the same input word x. Require Ao $r_1 = 0$. The x86 destination `RAX` is initially free because its first instruction copies the input. Relate running outcomes, valid code, and routine-entry control points. Memory is unchanged by all three routines and may be required equal or separately preserved under the local contract.

At the decision point, Ao's comparison establishes $N = V$ exactly for signed $x \geq 0$ in this comparison. x86 `test` establishes the sign predicate read by `jns`. RV64 `bge` compares `a0` directly with `x0`. Thus all three choose their done edge exactly when the common signed reading is nonnegative.

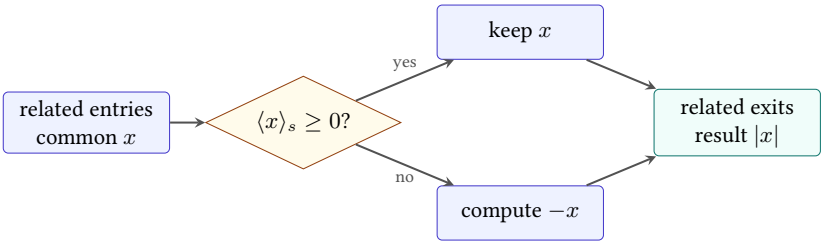


Figure 12.4: The absolute-value simulation splits on one shared signed predicate. Each path restores a common result relation at *done*.

Reader simulation for absolute value

Take arbitrary related entries satisfying the minimum-value exclusion. Split on the common signed reading. If $x \geq 0$, predicate agreement sends all three routines directly to *done*; their result locations contain x, which equals $|x|$. If $x < 0$, all take the negation path. Ao and RV64 subtract x from zero; x86-64 negates its copied word. Fixed-width arithmetic gives the same word $-x \bmod 2^w$. The precondition makes its signed reading positive and equal to $|x|$. At *done*, the result words, outcomes, and preserved memory are related. Scratch condition state may differ.

For the excluded minimum word, each modular negation returns the same bit pattern as the input. The modular-refinement claim still holds. The mathematical-positive-result claim fails because 2^{w-1} has no positive signed width-w representation. This counterexample shows why the input domain belongs beside the relation.

12.5.1 Expand the branch proof into local movements

The reader proof above compresses setup, decision, and update. Expanding those movements shows exactly where architecture-specific facts enter.

At Ao entry, $r_1 = 0$ is a premise supplied by the harness. The comparison writes condition state without changing r_0. Its signed predicate follows from the Ao subtraction rule. At RV64 entry, `x0` supplies zero and the branch compares register readings directly; there is no setup condition state. At x86 entry, `mov rax, rdi` copies the input, then `test rax, rax` sets the sign and zero information read by `jns`.

The decision relation can therefore be written as

$$D(x, S_A, S_R, S_X) \quad \text{iff} \quad \begin{array}{l} R_A[r_0] = R_R[a0] = R_X[rax] = x, \\ (N_A = V_A) \iff \langle x \rangle_s \geq 0, \\ SF_X = 0 \iff \langle x \rangle_s \geq 0, \end{array}$$

with the RV64 branch evaluating the last mathematical predicate from its two source registers. The formula does not equate Ao N with x86 SF: Ao's signed comparison uses $N = V$, while x86 `test` clears overflow and records the input high bit in SF. It equates the question they answer.

On the negative path, the update lemma assumes $\langle x \rangle_s < 0$. Ao and RV64 compute $0 - x$ by subtraction. x86 `neg` computes the same modular word. The minimum-value exclusion is not needed for word equality; it is needed only when converting that word to the positive mathematical value $-\langle x \rangle_s$. Keeping those proof steps separate shows that the modular theorem is strictly broader.

Table 12.3: Local obligations in the absolute-value simulation.

Movement	Main fact
Entry setup	all three result paths begin from the common word x
Predicate	each machine selects the nonnegative edge exactly for $\langle x \rangle_s \geq 0$
Keep path	result locations retain x and frames remain intact
Negate path	all result locations receive $-x \bmod 2^w$
Exit	outcome, result observation, continuation policy, and memory agree

Each row admits a focused negative control. Remove the x86 move to break entry setup. Change one signed predicate to unsigned to break the decision. Add an unexpected write to break a frame. Negate the wrong register to break update. Redirect the return to break exit. These controls diagnose more precisely than one end-to-end expected result.

Trace both paths at width four

Use inputs `0011`, `1101`, and `1000`. Record the shared signed predicate, each path, and final word. Identify which input satisfies modular absolute value but violates the mathematical-range specification.

12.6 Condition state can be related and then forgotten

Ao and x86-64 need condition information at their branches. RV64I does not retain an arithmetic-flags register. The simulation relation should therefore be control-point specific. At the decision point, relate the Boolean question "is the common signed input nonnegative?" to Ao's $N = V$, RV64's register comparison, and

x86-64's selected sign condition. At the routine exit, omit conditions when the output contract does not expose them.

Forgetting flags at exit is sound only for contexts that establish their own required condition state before reading it. If the x86 caller branches on flags immediately after the routine under a private convention, those flags belong in the output relation. Ordinary ABI rules and a custom local contract can expose different state.

Abstraction can change across control points

The entry relation may ignore destination scratch. The decision relation may require predicate agreement. The exit relation may forget conditions but require a result and continuation. One monolithic equality is less useful than a family of relations indexed by semantic control point.

Undefined or form-specific flags require special care. A relation must not read an architectural value the selected instruction leaves undefined. It may instead relate only the predicate whose source rules define it, choose another instruction form, or restrict the context so that the undefined component is unobserved.

12.7 Faults and outcomes must correspond

The absolute-value routine uses registers only, but instruction fetch, illegal encoding, invalid targets, and terminal input states can still affect execution in a complete model. A cross-ISA theorem normally restricts entry to valid code and running outcomes, then proves corresponding normal exits.

Memory routines need stronger fault agreement. One ISA may permit an unaligned access that another traps or decomposes. A theorem can require aligned valid ranges, relate both trap classes, or refine a source operation into a checked target sequence. Silently discarding inputs on which only one side traps changes the precondition and must be visible.

Shared normal results do not repair unequal faults

If one implementation traps before producing its result and another returns, their normal-result formulas may still be identical. The behavior relation fails unless the theorem explicitly uses a weaker policy that is appropriate for its context.

Likewise, related termination is not established by checking a fixed trace bound. A simulation can transfer termination from a source to a target when each source progress segment has a finite target match and stuttering cannot continue forever.

If the target may take an unbounded internal loop between control points, endpoint tests do not exclude divergence.

12.8 Building the Axeyum cross-ISA route

The current Axeyum checkout does not yet contain reusable A0, RV64I, or x86-64 decoder-and-step packages or typed cross-machine relations. This chapter therefore specifies the required route without presenting the absolute-value simulation as machine-established evidence.

Implementation should proceed in dependency order:

1. complete and test A0 state, decoder, step, and bounded-run packages;
2. implement a source-pinned RV64I slice for the exact branch, subtraction, and jump forms used by the selected routine;
3. implement a source-pinned x86-64 slice for the exact move, test, conditional jump, negation, and return forms used;
4. define typed control-point relations with register, memory, condition, program-counter, outcome, and freedom clauses;
5. check each local simulation segment and compose the endpoint theorem;
6. decode and replay every satisfiable counterexample through all affected executable semantics.

The relation should be data, not an informal callback hidden in a test. A manifest names semantic-package digests, source revisions, program bytes or decoded programs, control-point map, precondition, relation schema, observation, and expected evidence class. The checker reports which clause first failed.

Negative controls for the absolute-value simulation

Swap signed `bge` for unsigned `bgeu`; invert one branch; negate the wrong register; omit the x86 copy; relate `RDI` rather than `RAX` at exit; admit the minimum signed word while claiming a positive mathematical result; or let one implementation write an observed memory byte. Each mutation has a small witness and should fail a distinct relation or semantic clause.

At small widths, exhaustive enumeration can check every related entry and both paths. At width 64, a solver can search for a violation of each segment or the composed endpoint relation. A model must become three concrete traces and a failed relation clause. An unsatisfiable formula needs its exact digest and the

strongest available certificate or kernel route; a solver message alone does not create a cross-ISA theorem.

Counterexample triage should move from the outside inward. First validate that the model satisfies the serialized precondition. Then construct concrete machine states and check the entry relation. Decode the bound program bytes. Replay each segment. Finally compare the reported failed clause with the concrete endpoint values. A failure at an earlier stage invalidates the later interpretation.

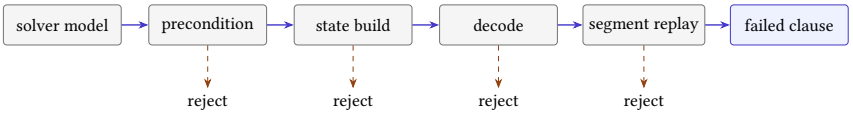


Figure 12.5: A solver assignment becomes a cross-ISA counterexample only after it survives construction, decoding, replay, and relation checking.

This order distinguishes a genuine semantic disagreement from a generator bug. For example, a symbolic model may assign an out-of-range register index that a real decoder rejects. It may describe code bytes different from those in the manifest. It may satisfy a formula whose branch target uses the wrong base. None is evidence that the two actual machine programs disagree; each is evidence that a binding layer needs repair.

Why two matching formulas are not enough

Encoding $y = x$ for nonnegative inputs and $y = -x$ otherwise on both sides checks a shared mathematical formula. It does not establish decoding, instruction effects, branch targets, scratch writes, memory preservation, traps, or control-point progress. The formula becomes useful evidence only after executable source-pinned semantics connect machine bytes to it.

The introductory Python or PyO3 API can expose named routines, widths, relations, observations, and returned counterexample traces. Rust should own the typed semantic objects, digests, solver encoding, and replay. Convenience at the boundary should reduce setup work, not erase which route and scope supported the result.

12.8.1 A beginner route should expose the relation

An introductory interface should not begin with a Boolean function named `equivalent`. It should let the reader construct the three entry states, inspect the control-point map, execute one segment, and ask which relation clauses hold. Only then should it offer a composed comparison.

```
case = abs_case(width=8, value=0xfd)
```

```
states = case.entry_states()

print(case.relation("entry").explain(states))
segments = case.step_to("decision", states)
print(case.relation("decision").explain(segments.ends))

report = case.compare_to_exit()
print(report.first_failed_clause())
```

Listing 12.4: Illustrative future relation-first interface

The sketch is not a current Axeyum import. Its design requirement is important: `explain` reports satisfied and failed clauses with concrete values. A negative control that changes RV64 `bge` to `bgeu` should point to predicate or control disagreement, not return an undifferentiated false value.

At the Rust layer, a control point can be an enumeration and each relation a typed structure containing value, memory, control, condition, outcome, frame, and freedom clauses. Segment checkers consume decoded traces and return their endpoint states. Solver formulas are derived from the same structures, and any model is replayed by those executable segment checkers.

Chapter 15 provides the next scale test. Its XOR loop reuses this machinery but adds memory, a loop invariant, and repeated cut points. Building the small absolute-value route first would therefore create reusable semantics rather than a disposable demonstration.

The relation-first interface also changes how a failed comparison is taught. Suppose the terminal values disagree. A flat comparator can report only the two numbers. A structured route asks where the relation last held, which segment broke it, and which clause failed first. If value correspondence fails after the decision point while control correspondence still holds, inspect the selected arithmetic step. If control fails while values remain related, inspect the predicate and target. If the frame clause fails, look for an unexpected scratch or memory write even when the selected result is correct.

That diagnosis is more than a convenience. It mirrors the composition proof. Each segment has premises, a local movement, and an endpoint relation. The first broken endpoint identifies the first local lemma that cannot be reused. A minimized counterexample can then retain only the state components named by that lemma, making the difference small enough to replay by hand.

A counterexample should name its control point

For cross-machine work, an input alone is rarely an adequate counterexample. Retain the three entry states, the last related control point, the first failed relation clause, and the decoded steps that crossed the gap. Those facts turn disagreement into a repairable semantic question.

12.9 Where the cross-ISA model stops

This chapter covers deterministic scalar architectural routines over selected integer instructions. It excludes floating point, vector state, atomics, concurrency, weak memory, privilege, interrupts, exceptions across frames, self-modifying code, dynamic linking, timing, microarchitectural state, speculation, and side-channel observations.

The worked routine is a semantic teaching instance, not evidence that current Axeyum can decode or execute its real machine code. It also does not claim that the three instruction sequences have equal cost. Chapter 13 defines candidate languages and costs; Chapter 15 develops a richer scalar routine through the whole evidence chain.

Within this boundary, a state relation lets unlike machines share one exact story. Their bytes, registers, flags, and step counts remain different. The simulation shows why those differences still converge on the same declared meaning.

12.10 Exercises

1. List the seven clauses of a cross-machine state relation and give one absolute-value fact belonging to each.

2. Relate Ao r_0, RV64 `a0`, and x86-64 `RDI` at routine entry. State the width and signed-reading convention.

3. Explain why x86-64 `RAX` may be unconstrained at entry but must be related at exit in the worked routine.

4. **Execute.** Trace all three absolute-value routines at width four for `0011`, `1101`, and `1000`. Include predicate and result relations at every control point.

5. **Prove.** Prove the nonnegative branch of the reader simulation, including program-counter, result, memory, and outcome clauses.

6. Prove the negative branch under the minimum-value exclusion. Identify the exact step that uses the precondition.

7. **Break.** Replace RV64 `bge` with `bgeu`. Find the smallest-width witness and replay the path difference.

8. State a modular absolute-value specification that includes the minimum signed word. Explain why its final bit-vector relation holds while a positive signed-result claim fails.

9. Draw control-point matches for one Ao segment corresponding to two or more x86-64 steps. Identify the intermediate x86 state intentionally omitted from the relation.

10. Give a stuttering simulation that would incorrectly permit infinite target stuttering. Add a natural-valued rank that rules it out.

11. **Transfer.** Define a memory relation for equal 16-byte arrays at different base addresses. Include bounds, byte correspondence, ownership, and outside-memory preservation.

12. Prove that corresponding little-endian bytes produce equal width-32 loads. Then break the proof by reversing one join order.

13. Give an aliasing counterexample to a relation that treats two overlapping output ranges as independent. State a sufficient nonaliasing premise.

14. Relate Ao and x86 condition state only at the absolute-value decision point. Explain why the exit relation may omit it under one context and must retain it under another.

15. Construct one pair of implementations with equal normal results but different fault behavior. State three possible outcome relations and which ones accept the pair.

16. **Prove.** Formalize the local-to-endpoint simulation argument by induction over a finite list of control-point segments.

17. Design an Axeyum manifest for the three-machine absolute-value routine. Include source pins, semantic digests, decoded programs, control points, precondition, relation clauses, observation, and seven negative controls.

18. A returned solver model violates the relation but fails RV64 executable replay. List possible generator, decoder, and semantic fault classes. Explain why the model cannot support the claimed counterexample.

CHAPTER THIRTEEN

Program Languages, Costs, and Minimality

Two programmers replace a six-instruction sequence with four instructions. The replacement passes every test they run. One programmer says that the new sequence is shorter. The other says that it is the shortest possible. The first statement needs a count. The second needs a universe.

That universe is easy to leave implicit. Perhaps an instruction may use any register, or only registers that are dead at the replacement site. Perhaps a constant may appear as an immediate, arrive in a register, or be loaded from memory. Perhaps two instructions with different byte encodings count as the same operation. On x86-64, a sequence with fewer instructions can occupy more bytes. On either real machine, a sequence that looks cheap in the ISA manual can be slow on one processor and fast on another. "No cheaper program exists" has no stable meaning until these choices have been printed beside the claim.

This chapter constructs that missing universe. We will first make a small candidate language, then choose a cost, and finally rule out every cheaper program in it. The worked case is deliberately small enough to check with a pencil. Its value is not the difficulty of adding two. Its value is that no premise can hide in the machinery.

A minimality claim has three named parts

A program is minimal only relative to a candidate language, an equivalence contract, and a cost order. Change any one of the three and the claim changes. The ISA name by itself supplies none of them.

13.1 From clever sequences to complete searches

Programmers have always looked for shorter or faster ways to express a fixed calculation. A compiler performs the same work mechanically when it replaces an expensive instruction pattern with a cheaper one. The old name *peephole optimization* describes the scale: look through a small window at a few adjacent

instructions, recognize a pattern, and substitute another. The window is small, but the question inside it is already mathematical. Do the old and new programs agree for every allowed input?

In 1987 Henry Massalin described a **superoptimizer**: a system that searches for the smallest program implementing a function rather than relying only on a catalog of known rewrites.(Massalin 1987) Later work made peephole superoptimization more systematic and made complete searches scale through better decomposition and parallelism.(Bansal and Aiken 2006; Lu and Bodík 2025) These systems differ in language, search method, and cost. They share a pressure that matters here: the word *smallest* forces the search space into the statement.

A search can serve at least three different goals.

1. It can find any program that meets a specification.

2. It can improve on a known program under a chosen cost.

3. It can show that no cheaper candidate meets the specification.

The first two goals need a witness. The third needs coverage. If a search finds a three-instruction program, then a three-instruction program exists. It does not follow that two instructions are impossible. To establish that lower bound, every candidate of cost zero, one, and two must be represented and rejected, or a separate argument must rule out the same space.

That asymmetry explains why optimization practice often stops after finding a good sequence. A witness is compact and cheap to replay. A failed search may mean that the tool timed out, its grammar omitted the needed operation, its encoding was wrong, or there truly is no candidate. Only the last reading is a lower bound, and it is the reading that requires the most care.

13.2 Print the language before searching it

A **candidate language** is the exact set of programs allowed to compete with the reference program. For a bounded search, we want this set to be finite and mechanically enumerable. An ISA manual is an input to its design, not the language itself. The manual may describe privileged instructions, optional extensions, hundreds of operand forms, implementation-dependent features, and behaviors outside the replacement site. A theorem about a selected scalar fragment must say selected scalar fragment.

The most useful language declaration answers eight questions.

Candidate-language contract

A bounded candidate-language contract declares:

1. the decoded instruction forms that may occur;
2. the operand forms and widths for each instruction;
3. the registers, constants, and temporary storage available at entry;
4. the memory regions candidates may read or write;
5. the legal starting addresses and decoding policy;
6. the precondition on initial states;
7. the maximum program length or cost bound; and
8. the observation used to compare final behavior.

Together these clauses determine which byte strings or decoded sequences are candidates and what it means for one to work.

The order matters less than the completeness. A register restriction without an operand restriction may still admit an implicit accumulator. A ban on stores does not ban loads. A list of decoded operations does not say whether all their encodings are admitted. A maximum of four instructions does not bound four x86-64 instructions to one fixed byte length. Each clause closes a different door.

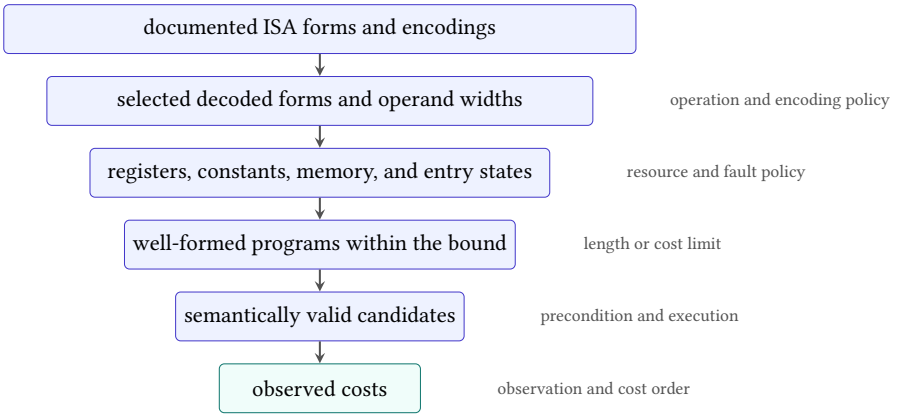


Figure 13.1: A bounded search does not range over an ISA name. Each declared choice narrows the source universe until a finite set of observed candidate behaviors remains.

13.2.1 Decoded forms and byte encodings

There are two sound ways to start. An *extensional* language prints every allowed decoded instruction instance. This is pleasant for a dozen candidates and impossible for a million. An *intensional* language gives a finite grammar: choose an operation from this list, a destination from this set, and each source from its permitted set. The grammar must expand to the same concrete instances the encoder or symbolic translator uses.

Decoded instructions and byte strings are not interchangeable. Ao assigns one four-byte encoding to each legal decoded instance, so the bridge is simple. RV64I uses fixed 32-bit base instructions, but the compressed extension adds 16-bit encodings and its own operand restrictions. x86-64 may encode closely related operations with different prefixes, immediate widths, or addressing forms. A search over meanings can quotient duplicate encodings when cost ignores bytes. A byte-minimal search cannot.

Canonicalization can reduce work without weakening a claim, but only when the discarded candidates are genuinely redundant. Suppose addition is commutative and both sources are pure registers. Keeping only `add r0, r0, r1` and dropping `add r0, r1, r0` preserves the possible result functions. If the two forms have different encodings, fault behavior, or costs, that quotient is no longer harmless. The proof obligation for a search-space reduction is the same as for any other rewrite: every removed candidate must have a retained representative with the same observed behavior and no greater cost.

13.2.2 Constants are resources

The expression $x + 1$ looks as though it contains a free constant. Machines do not receive constants from typography. A one may be encoded in an immediate field, held in an entry register, constructed from zero, read from a literal pool, or obtained through a special instruction. Those routes consume different instructions, bytes, registers, and memory permissions.

Consider a language with only register-register addition. If $r_1 = 1$ is an entry premise, then `add r0, r0, r1` computes $x + 1$ in one instruction. Remove that premise and the same text reads an arbitrary second input. Add an immediate form and the register is unnecessary. The function has not changed; the candidate language has.

Do not smuggle a constant through the harness

A test harness that initializes a register or memory cell has granted every candidate access to a resource. State that resource in the precondition and language contract. Otherwise the search and the printed claim concern different machines.

Temporaries work the same way. A dead register at a compiler replacement site may be available as scratch. A live register may be used only if the candidate restores it and the observation checks that restoration. Stack space, flags, and vector registers are not free merely because a calling convention makes some of them caller-saved. The local context decides which changes are observable.

13.2.3 Memory policy is more than yes or no

A candidate that may use memory needs a region, permissions, alignment rule, aliasing policy, and fault policy. "One temporary stack slot" should name its address relation to the stack pointer, its width, and whether it can overlap any input or output. If a reference program never faults but a candidate can fault on an allowed state, normal-result agreement does not rescue the candidate.

Memory also threatens finiteness. If an address immediate may be any word, the grammar has 2^w choices at width w. That is finite in mathematics and useless as a naive enumeration. A language may restrict addresses to a declared set of symbolic regions, or it may encode address choices in a solver. Either choice belongs in the scope. "Memory instructions permitted" is not enough information to reproduce the set.

13.3 The equivalence contract inside the search

Chapter 11 defined program agreement relative to a precondition and an observation. A minimality search inherits that entire contract. Let (S) be the specification program or function, (P) a candidate, (A(s)) the allowed-input predicate, and (O) the observation. The candidate works when

$$\forall s, \quad A(s) \Longrightarrow O(P(s)) = O(S(s)).$$

If execution can trap, halt, or diverge, (O) must retain whichever behavior classes the replacement context can detect. A result-only projection may be appropriate for a pure straight-line exercise. It is too weak for a compiler rewrite when the continuation can read flags or observe a fault.

This universal condition is where testing stops. Tests sample states. The candidate must work on every state allowed by (A). For a tiny width, direct enumeration can check all inputs. At width 64, symbolic bit-vector reasoning can represent all 2^{64} inputs without listing them one by one. The symbolic formula is still accountable to the instruction semantics and the observation that produced it.

The language and equivalence contract interact. If candidates may use a temporary register, then the observation must say whether its final value matters. If candidates may access memory, the precondition must supply valid memory and the observation must compare the permitted effects. If a candidate may be shorter than the reference, the harness must define where execution stops rather than letting it run into whatever bytes happen to follow.

One omitted flag changes the winner

Suppose an Ao sequence clears r_0 with arithmetic and leaves $Z = 1$, while a shorter move also writes zero but leaves different condition bits. Under an observation containing only r_0, the move may win. Under an observation that also contains (Z,N,C,V), it does not. Neither answer corrects the other. They answer different replacement questions.

13.4 Cost is a function, not an adjective

A **cost model** maps a candidate program and its declared context to an ordered value. The simplest model counts instructions:

$$C_{\text{inst}}(P) = |P|.$$

For Ao's fixed-width encoding, byte length is four times instruction count, so the two costs induce the same order. This convenience is a feature of the teaching machine, not a universal property of instruction sets.

For a variable-length encoding, define

$$C_{\text{bytes}}(P) = \sum_{i=1}^{|P|} b(P_i),$$

where $b(P_i)$ is the encoded length of instruction P_i. A two-instruction x86-64 sequence may occupy fewer bytes than one instruction with a long immediate and addressing fields. An RV64 search that admits the compressed extension can similarly prefer two 16-bit instructions over one 32-bit instruction only if the cost order permits a tie or includes another component; it cannot pretend that instruction count and byte count are the same measurement.

An abstract weighted model assigns a declared weight to each instruction instance or class:

$$C_{\text{weight}}(P) = \sum_{i} W(P_i).$$

Weights can represent an engineering preference, but their interpretation must be named. A multiply might receive weight four and an add weight one. That does not say the multiply takes four cycles on a real processor. It says the theorem uses the printed table.

The table has three different winners: P_A by instruction count, P_B by bytes and declared weight, and no universal winner independent of the question. Calling one sequence "better" would discard exactly the information a minimality statement needs.

Cost can also depend on the replacement site. A sequence that needs one dead temporary may be admissible where that register is free and inadmissible where it

Table 13.1: Three cost models can order the same candidates differently. The numbers are illustrative, not measurements of a processor.

Candidate	Instructions	Bytes	Declared weight
P_A: one rich instruction	1	7	4
P_B: two compact instructions	2	4	2
P_C: three simple instructions	3	6	3

carries a live value. Saving and restoring the register changes both the candidate and its cost. Alignment can change encoded padding or the placement of an x86-64 instruction across a fetch boundary. A literal already present in nearby memory is a different resource from a literal that the replacement must introduce. For this reason, the most general cost has the shape $C(P, K)$, where K records the declared context. A context-independent model is still useful; it simply fixes or ignores those terms and says so.

13.4.1 Latency and throughput need a processor

The ISA defines architectural behavior. It does not fully define how many cycles a concrete processor spends on a sequence. Latency can depend on operand values, cache state, branch prediction, instruction alignment, microarchitectural generation, and the instructions around the replacement. Throughput asks a different question from dependency-chain latency. A model for either must name the processor data, scheduling assumptions, and context used to combine individual costs.

Measured performance introduces noise and environmental state. A benchmark can compare two sequences on a named machine under a named protocol. It cannot by itself establish a universal minimum over all programs, and a minimum in an abstract schedule model need not be the fastest sequence on silicon. The honest result is often a pair: a theorem about the model and a measurement about the machine.

13.4.2 Orders, ties, and several objectives

Costs need not be single numbers. The adjective **lexicographic** means that we compare the first component and consult the next one only to break a tie. A cost with this order might minimize bytes first and then instructions among equal-byte candidates:

$$C(P) = (C_{\text{bytes}}(P), C_{\text{inst}}(P)).$$

Lexicographic order makes the first component decisive. A Pareto order instead keeps every candidate for which no other candidate is at least as good in all components and strictly better in one. The result is a frontier, not one winner.

Ties matter. Two different programs can have the same minimum cost. A search for *a* minimum witness may return either. A search claiming uniqueness must rule out every other equivalent candidate at the same cost after stating which syntactic differences count. Register renaming, commutative operands, and duplicate encodings can make uniqueness a much stronger claim than minimality.

Well-founded cost orders

A lower-bound search works by exhausting costs below a witness. This argument needs a cost order with no infinite descent in the searched region. Natural numbers and finite lexicographic tuples of natural numbers have that property. Arbitrary real-valued estimates or partially ordered profiles need a more careful statement of what "all cheaper candidates" means.

13.5 An upper bound and a lower bound must meet

Let (L) be the candidate language, (E(P,S)) the equivalence condition, and $C(P)$ the cost. A witness $P^* \in L$ with $E(P^*, S)$ supplies an upper bound $C(P^*)$. Minimality also needs

$$\neg\exists P \in L, \quad C(P) < C(P^*) \ \wedge \ E(P, S).$$

That second formula is the lower bound. It says that the cheaper region is empty of working programs. The two statements do different jobs and deserve different checks.

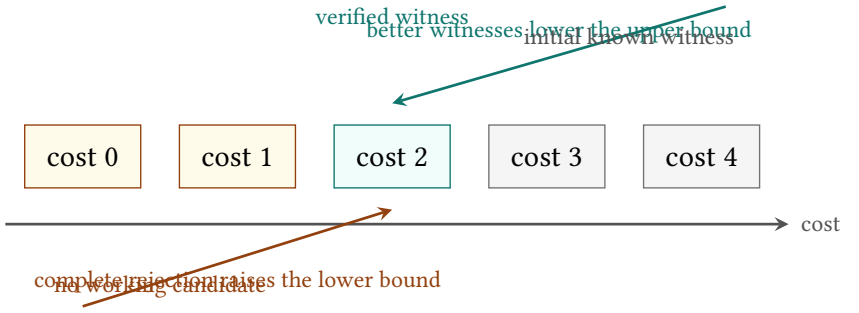


Figure 13.2: A witness lowers the best known upper bound. Exhausting or refuting cheaper cost strata raises the lower bound. Minimality follows only when the two meet under one language and equivalence contract.

A common search alternates the two sides. It begins with a reference program as an upper bound, searches lower costs, and replaces the witness whenever it finds an improvement. When every smaller stratum has been rejected, the last

witness is minimal. Another search asks for cost zero, then one, then two, and stops at the first satisfiable cost. That method is complete only if each earlier query covered its entire cost stratum.

Timeouts do not raise a lower bound. Nor does a solver's failure to find a candidate unless the algorithm's completeness and termination for that query have been established. "No result in one hour" is a measurement of the search, not a property of the language.

13.6 A tiny Ao lower bound

We now make every choice explicit. Fix an Ao word width $w \geq 2$. At entry, $r_0 = x$ is the input and output register, and $r_1 = 1$ is a read-only resource. Other registers and memory are unavailable. Candidates are straight-line sequences over these six decoded instances:

```
mov r0, r0          mov r0, r1
add r0, r0, r0      add r0, r0, r1
add r0, r1, r0      add r0, r1, r1
```

Listing 13.1: The complete tiny candidate alphabet

Every instruction executes normally in a valid code region. The harness stops after the declared number of instructions. The observation contains only the final value of r_0 and normal completion; it ignores pc and the Ao condition bits. Cost is instruction count. The specification is

$$S(x) = x + 2 \pmod{2^w}.$$

The task is to find the least cost among candidates of length at most two.

There is a two-instruction witness:

```
add r0, r0, r1
add r0, r0, r1
```

Listing 13.2: A two-instruction witness for adding two

After the first step, $r_0 = x + 1$. The instruction does not change r_1, so the second step produces $x + 2$, with all arithmetic taken modulo 2^w. This replay establishes the upper bound two.

The lower bound is small enough to see as a table. A zero-instruction program returns (x). The six one-instruction instances return only the following five functions; the two mixed-source additions coincide because word addition is commutative.

Table 13.2: Every program below cost two in the tiny Ao language. A single input separates each observed function from $x + 2$.

Candidate form	Final r_0	Separating input
empty or `mov r0,r0`	(x)	any (x)
`mov r0,r1`	1	$x = 0$
`add r0,r0,r0`	$2x$	$x = 0$
`add r0,r0,r1` or reversed	$x + 1$	any x
`add r0,r1,r1`	2	$x = 1$

Reader proof: no cheaper candidate adds two

At cost zero the output is x, which differs from $x + 2$ because $2 \not\equiv 0 \pmod{2^w}$ when $w \geq 2$. At cost one, the table is a complete split over the six allowed instruction instances. The constant one differs from the specification at $x = 0$. The function $2x$ also differs there. The function $x + 1$ differs for every input because one and two are distinct words. The constant two differs at $x = 1$. Thus no candidate of cost less than two works for every input. The two-step witness works for every input, so the minimum instruction count in this printed language is two.

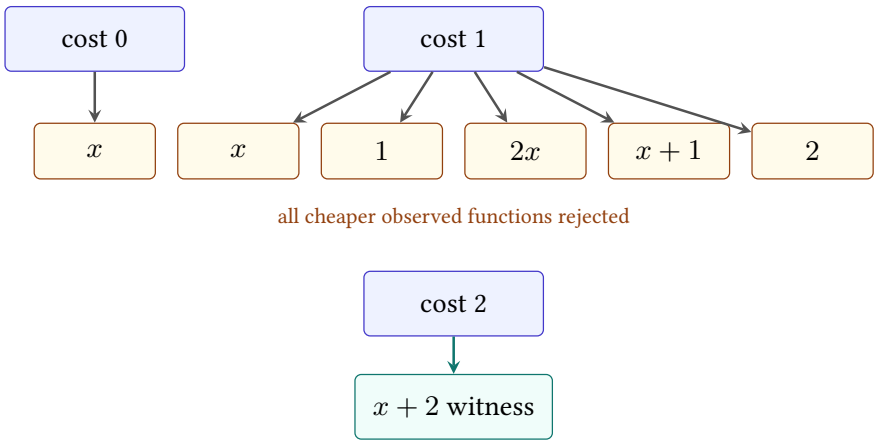


Figure 13.3: The lower-bound argument classifies every observed function below cost two. The witness occupies the first cost stratum that contains the specification.

The case now has a complete reader argument for its teaching language. It is not yet a machine-backed book result: no current object binds this exact grammar, enumeration, checker, and negative control. That distinction matters even when the mathematics fits on half a page. The later evidence route must check the bytes and semantics it claims, not borrow confidence from this table.

13.6.1 Why every clause mattered

The width premise excludes $w = 1$, where $2 \equiv 0 \pmod 2$ and the empty program already implements $x + 2$. Making r_1 read-only prevents the first instruction from destroying the constant used by the second. Ignoring condition bits permits the two-step witness to satisfy a result-only specification; a stronger observation would have to specify the desired final conditions. Banning immediates excludes a one-step `addi` form. Banning memory excludes a table lookup or loaded constant. Limiting the register set keeps the one-step alphabet exactly as printed.

Add just one instruction, `addi r0, r0, 2`, and the old lower bound no longer describes the new language. This is not a counterexample to the argument. It is a reminder that minimality does not float above scope.

Break the claim cleanly

Extend the alphabet with `movi r0, 2`. Does the minimum change? Now extend it with `addi r0, r0, 2`. Explain why the first extension leaves the lower bound intact while the second supplies a cheaper witness. State the modified language before giving either answer.

13.7 Turning enumeration into evidence

The pencil proof used a useful compression: six one-step programs produced only five observed functions. A program search can exploit the same idea. Breadth-first enumeration begins at the empty program, extends each retained prefix by every instruction instance, executes or symbolizes the result, and groups candidates with equal observed behavior. For a deterministic finite machine at a tiny width, a complete truth table can serve as the behavior key.

Suppose two prefixes have the same observed state transformer and the same available resources, while one costs no more than the other. Any common suffix gives the same observed result, so the more expensive prefix can be discarded. This is semantic quotienting. It can shrink a search dramatically, but the equality used for grouping must retain every state component that a future suffix can read. Grouping only by current r_0 would be unsound if a later instruction can read flags, r_1, or memory.

For larger widths, a solver can represent instruction choices and symbolic states. A bounded query asks whether there exist decoded instructions and an allowed input relation such that the candidate meets the universal specification. The quantifier structure needs care. To refute equivalence of a fixed candidate, a solver searches for an input where the outputs differ. To synthesize a candidate that works for all inputs, the conceptual form is $\exists P \, \forall s$. Many practical systems alternate candidate generation with counterexample discovery rather than handing that formula to one solver unchanged.

13.7.1 Counterexample-guided search

A counterexample-guided loop keeps a finite set of test inputs. It finds a candidate that works on those inputs, then asks a **verifier**, the checker for universal agreement, for an allowed input where the candidate fails. A counterexample joins the test set, and the loop repeats. If the verifier finds none under a sound complete route, the candidate meets the universal contract. If no candidate can satisfy the accumulated examples, the current cost stratum may be empty, but the lower bound still depends on how candidate choices and universal correctness were encoded.

This loop turns failures into useful information. The values $x = 0$ and $x = 1$ in Table 13.2 are exactly such separating inputs. They eliminate whole classes of candidate functions. The search is not merely trying programs; it is learning which distinctions the specification requires.

13.7.2 What a checkable lower bound must retain

A durable result needs more than a final line saying that a solver returned an unsatisfiable verdict. The evidence package should retain enough information to reconstruct the claim:

1. a serialized candidate-language contract;
2. the specification, precondition, observation, and cost order;
3. the exact decoder and semantic version or digest;
4. the formula for every cheaper cost stratum, or a reproducible generator;
5. a witness at the claimed minimum cost;
6. a checkable refutation for each cheaper satisfiability query when the solver route can export one; and
7. a negative control that corrupts a witness, formula, or proof and is rejected.

The witness checker and lower-bound checker should fail for different reasons. Change the second add in the Ao witness to `add r0, r0, r0`; replay should find an input where the result differs. Add a legal one-step `addi` candidate without regenerating the cheaper-space formula; a language-closure check should catch the mismatch before a solver result is trusted. Truncate a refutation; the proof checker should reject it. These attacks test three different links.

A solver verdict is not a portable lower bound

An unsatisfiable verdict reports what one solver said about one formula. A checked refutation can support that formula's unsatisfiability. Neither establishes that the formula represents the printed candidate language unless the encoding bridge is also checked.

13.8 Negative controls for search claims

The most revealing negative control gives the checker something it ought to accept but the claimed lower bound says cannot exist. Add a known cheaper witness to the language, rebuild the search space, and require the lower-bound check to fail. This tests candidate coverage and not merely proof parsing.

Other controls target nearby faults:

- omit one operand form from enumeration while leaving it in the printed grammar;
- reverse one subtraction operand in the symbolic semantics;
- observe r_0 but accidentally drop a required flag;
- allow a temporary in synthesis but freeze it in verification;
- compute byte cost from decoded mnemonics instead of actual encodings;
- accept a solver log after deleting the load-bearing proof bytes; or
- change the precondition between the witness and lower-bound queries.

A positive run says the components agree on the intended instance. A negative control shows that at least one nearby wrong instance crosses a failure boundary. It does not prove the checker has no other bugs. It makes the checking claim falsifiable and gives future maintainers a precise alarm.

13.9 From Ao to RV64 and x86-64

Ao makes instruction-count minimality unusually clean. Its instructions have one width, its first scalar model has a small explicit state, and the book owns its definition. The real-ISA slices add obligations before they add glory.

For an RV64I scalar search, name the base ISA version, permitted extensions, architectural integer width, privilege assumptions, and exact operand forms. Decide whether compressed instructions compete. State how illegal encodings, misaligned control flow, loads, stores, and traps enter the observation. If the language includes

only register-register integer operations, say so. “RISC-V program” is much too large a noun.

For x86-64, decode mode and encoding are central. Name 64-bit mode, selected instruction forms, operand and address sizes, allowed prefixes, register classes, memory forms, and the portions of the flags register that matter. Decide how partial-register writes and implicit operands enter the state. If cost is bytes, retain the actual encoding. If cost is instruction count, explain how multiple encodings of one decoded instruction are handled.

Neither slice authorizes a claim about RISC against CISC. One may expose an interesting contrast: a regular fixed-width language makes byte cost easy to relate to instruction count, while a variable-length language makes encoding part of the optimization problem. That is a statement about declared features, not two architectural essences.

Prior systems have claimed instruction-sequence optimality under their own languages and models.(Bansal and Aiken 2006) The contribution sought here is therefore not a priority slogan. It is a per-instance route in which the reader can inspect the language, replay the witness, check the cheaper-space refutations, and break the checker with a negative control.

13.10 The Axeyum route, in the right order

The live Axeyum checkout already supplies useful lower layers: fixed-width bit-vector terms, solver connections, propositional encodings, proof checking components, and kernel-facing evidence machinery. It does not yet supply a reusable Ao decoder and transition system, nor complete RV64 or x86-64 packages. We should not place a Python convenience function over that gap and call the result an ISA proof.

The first implementation route should follow the dependency order already visible in the reader argument.

1. Define the candidate-language data types in Rust: decoded forms, resources, precondition, observation, bound, and cost.
2. Connect those forms to an executable Ao decoder and step semantics.
3. Enumerate the tiny language and replay each candidate concretely at a small width, including the zero- and one-instruction layers above.
4. Translate the same semantics to Axeyum bit-vector expressions and check candidate equivalence by searching for a counterexample input.
5. Emit a canonical record of the language, witness, formula digests, and cheaper-stratum results.
6. Add proof-export and independent checking where the selected solver route supports it, then add mutations that must fail.

7. Expose a small Python interface only after Rust owns these meanings and the evidence record.

Python is useful for teaching and orchestration. A reader should be able to construct the six-form language, request a bounded search, inspect the witness, and print a counterexample without writing Rust. The Python object must serialize to the same canonical contract the Rust checker reads. It must not reimplement instruction semantics or decide what a successful proof means.

The teaching display should expose the search strata as well as the winner. For each cost below the upper bound, it can show the number of syntactic candidates, the number rejected by typing or resource rules, the number eliminated by concrete counterexamples, and the obligations sent to the solver. Those counts make the lower-bound argument visible. They also catch a damaged enumerator: a suspiciously empty stratum is a question, not a gift.

One small witness from every rejection class is useful. A malformed operand explains the grammar. A clobbered scratch register explains the observation. A boundary input explains semantic failure. The retained examples teach why the candidate language and harness are part of the theorem rather than setup around it.

```
# Illustrative API: this interface is not yet implemented.
language = CandidateLanguage.a0(
    width=4,
    forms=["mov.rr", "add.rrr"],
    registers=["r0", "r1"],
    constants={"r1": 1},
    writes=["r0"],
    observation=["normal", "r0"],
    max_instructions=2,
)
result = search_minimal(language, spec="r0_out == r0_in + 2")
result.replay_witness()
result.check_lower_bound()
```

Listing 13.3: The intended shape of a future teaching interface

The comment is part of the lesson: the interface is a design target, not a reported capability. Once it exists, the book should replace illustrative code with an executable example and bind any machine-backed result to the exact Axeyum revision and checker route.

The real-ISA layers come later. First establish that decoded bytes and Ao steps agree, then lift selected RV64 forms, then selected x86-64 forms. Only after those semantic bridges survive their own negative controls should the same minimality engine range over them. The search engine is not the hard foundation. Meaning is.

13.11 Where the model stops

No claim here ranges over all programs on any architecture. The model excludes self-modifying code, concurrency, interrupts, privileged state, caches, speculative execution, and whole-program link layout. It does not infer hardware speed from an instruction count or turn a timeout into a lower bound.

Even within straight-line scalar code, a model may omit a useful instruction, constant route, temporary, or encoding. Such an omission does not make the bounded result false. It makes the result narrower. The remedy is to publish the language so another person can extend it and see whether the minimum changes.

The construction has a modest and surprising reach. Its language is finite because we chose the pieces and the bound. Once those choices are fixed, checking every cheaper program can support a statement about all of them. A small list of instructions becomes a universal negative, but only inside the border we drew. Publishing that border lets the next search enlarge it without misquoting the result already obtained.

13.12 Exercises

1. **Execute.** At width four, replay the two-instruction Ao witness for inputs $0, 1, 7, 14$, and 15. Record the intermediate and final r_0.

2. Enumerate all six one-instruction candidates in the tiny language without quotienting commutative additions. Confirm that their observed functions match Table 13.2.

3. **Prove.** Generalize the reader argument from adding two to adding (k) by repeated addition of the entry constant one. State a language in which (k) instructions are minimal, or explain why the present language admits a shorter construction for some (k).

4. Find every place the width premise $w \geq 2$ enters the lower-bound argument. Give the exact result at width one.

5. **Break.** Add `addi r0, r0, 2` to the language. Write the new one-step function table and identify the failed sentence in the old proof.

6. Add `sub r0, r0, r1`. Does it create a cheaper witness for $x + 2$? Does it create new observed functions? Answer for width two and width three.

7. Change the observation so that it includes Ao's (Z,N,C,V). Define the specification's desired final conditions, then determine whether the old witness still works.

8. Allow r_1 to be overwritten and add `mov r1, r0` plus addition forms writing r_1. State the preservation rule needed if the caller observes r_1 at exit.

9. Give two concrete x86-64 instruction sequences that would be ordered differently by instruction count and encoded byte length. Pin the mode and write the bytes before comparing them.

10. For RV64, compare a language containing only 32-bit RV64I instructions with one that admits a named compressed extension. Which parts of the candidate-language contract must change?

11. Design a lexicographic cost that minimizes memory accesses, then bytes, then instruction count. Give two candidates tied on the first component and separated by the second.

12. Construct three abstract candidates whose costs form a Pareto frontier under bytes and latency. Explain why the frontier has no single minimum without another policy choice.

13. **Act as the checker.** Mutate the second instruction of the Ao witness in three ways. For each mutation, give the smallest separating input you can find and the observed wrong result.

14. Design a negative control that detects omission of one legal operand form from an enumerator. State which component must fail before the solver is called.

15. A search reports no candidate of cost at most three after a one-hour timeout. List the additional evidence needed before this observation can support a lower bound.

16. State the semantic condition under which two search prefixes may be grouped by their current state transformer. Give a counterexample to grouping only by r_0 when later instructions can read Z.

17. **Transfer.** Write the candidate-language contract for replacing a two-instruction RV64I integer sequence at one compiler site. Include live registers, extensions, memory, traps, observation, and cost.

18. Design the first Axeyum evidence record for the tiny Ao search. Include canonical language bytes, semantic revision, witness, formula digests, checker result, and one negative-control mutation. Mark which fields the Python layer may display and which meanings Rust must own.

CHAPTER FOURTEEN

Evidence That Can Fail

A command prints the word we hoped to see. The terminal returns success. We save the output and call it evidence. Six months later, someone changes one byte of the claimed formula and the same command still succeeds.

The first run told us less than we thought. Perhaps the checker ignored its input. Perhaps the command checked a proof against a different formula. Perhaps the formula did not encode the program claim. Perhaps a shell pipeline discarded the checker's failure code. The desired word on a screen was a fact about one execution. It was not yet a reason to believe the theorem.

Evidence becomes useful when the route can lose. A witness checker must reject a wrong witness. A proof checker must reject a damaged proof. A manifest checker must notice a changed semantic package. The failure need not be dramatic. A nonzero exit code at the right boundary is enough. What matters is that the claimed result controls the outcome.

This chapter builds that route from first principles. We will separate tests, complete computations, solver verdicts, certificates, and kernel reconstruction. We will bind each result to its statement and semantics. Then we will examine an awkward case found in an earlier generation of this very book: a proof file almost a megabyte long that carried no load-bearing information.

Evidence must discriminate

Evidence supports a claim only when a relevant change to the claim, input, or supporting object can make its check fail. A saved success message has no such power by itself.

14.1 Why solvers began writing proofs

Boolean satisfiability solvers answer whether a propositional formula has an assignment that makes it true. A satisfying answer can be accompanied by the assignment. Another program can substitute the values and check every clause. An unsatisfiable answer is harder: there is no assignment to hand back. The claim

ranges over all assignments, and a bare verdict asks the reader to trust the solver's search, implementation, memory, and output path.

Proof logging changed that bargain. Instead of returning only an unsatisfiable verdict, a solver can emit a sequence of additions and deletions that leads to contradiction. The DRAT format became practical because it could express the reasoning used by modern SAT solving and preprocessing, while a separate checker could validate the log. The accompanying `drat-trim` system also used backward checking and proof trimming to reduce the portion that must be retained.(Wetzler et al. 2014)

The next question was how much checker code the result should trust. The linear resolution asymmetric tautology (LRAT) format adds explicit hints to clausal steps so a simpler checker can validate them. Its authors demonstrated certified checkers in Coq and ACL2, moving the final decision into existing theorem-prover environments.(Cruz-Filipe et al. 2017) For satisfiability modulo theories, proof formats must also describe reasoning about equality, arithmetic, arrays, bit vectors, and other theories. Alethe grew from the proof language used by veriT toward a common interchange format for such SMT reasoning.(Schurr et al. 2021)

The historical movement is not from untrustworthy software to perfect software. It is from one opaque decision to a chain of smaller, inspectable decisions. A proof-producing solver may be enormous and aggressively optimized. If a much smaller checker rejects invalid proof steps, the solver can leave the trusted base. Formula generation and semantic translation do not disappear, however. A flawless proof of the wrong formula remains a flawless proof of the wrong formula.

14.2 Six forms of support

The book uses six **trust classes**: definition, trace, computation, verdict, certificate, and kernel reconstruction. Object `EVID.def.trust-class` records these names. They describe how a result is supported and where trust remains. They are not medals and do not form one simple ranking for every question.

14.2.1 Definition

A definition fixes meaning. It may introduce a word, machine state, decoder, observation, cost, or relation. Definitions need internal consistency and clear dependencies, but they do not become empirical facts by being executed. The Ao word set and state tuple are examples. A checker can confirm that their records satisfy a schema; it cannot discover whether the author chose the most useful teaching machine.

Definitions still carry a trust boundary. If a later formula implements a different addition rule, the formula does not prove a statement about the defined Ao instruction. Versioning and semantic digests help expose that mismatch. They

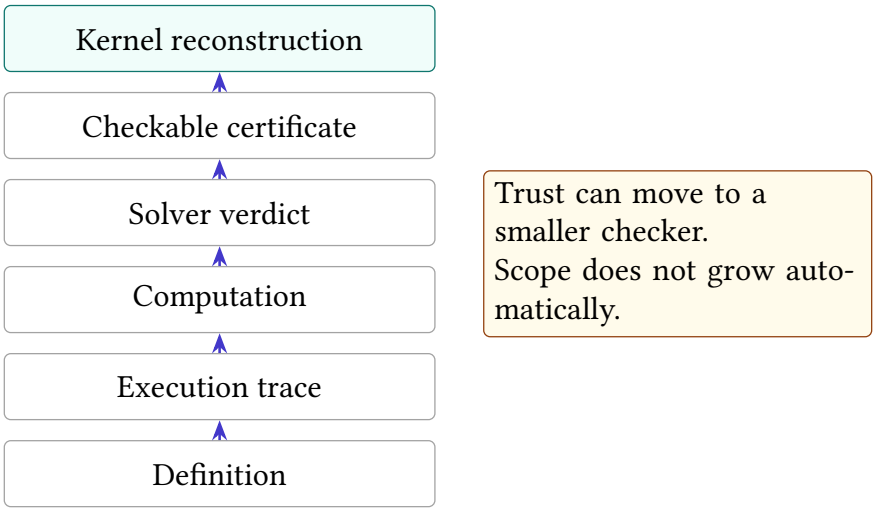


Figure 14.1: Six trust classes used by the book. Moving upward can reduce or relocate trust, but no rung repairs a bad statement, semantic model, or scope.

cannot decide whether the prose and formal definition express the same intended idea. The reader remains part of the route.

14.2.2 Trace

A trace replays a particular execution. Given initial state, program bytes, and steps, a checker can decode each instruction and confirm the next state. This can strongly support an example or a counterexample. It does not cover inputs absent from the trace.

Trace evidence is often the right support for an existence claim. One failing input refutes universal equivalence. One valid two-instruction sequence establishes an upper bound. In both cases, a small concrete object carries the claim. The checker should still compare the complete observed state, reject an illegal decode, and fail if any step changes.

14.2.3 Computation

A complete finite computation covers a declared domain by enumeration or a checked recurrence. At width four, evaluating a function on all sixteen words is complete for that width. A breadth-first search that visits every state in a finite graph can establish reachability or its absence, provided the transition generator and coverage test are correct.

Computation differs from testing by coverage, not by how many cases happened to run. Ten million random tests remain samples. Sixteen cases are complete when the domain contains exactly sixteen. A durable computation records the domain,

enumeration order or invariant, program revision, result digest, and the condition that makes the runner fail.

14.2.4 Verdict

A solver verdict reports the result of a symbolic decision procedure. A satisfying verdict should carry a model that can be decoded and replayed. An unsatisfiable verdict may be reproducible by running the same or another solver, but without a portable proof it still asks us to trust the solver and translation route.

Agreement between two independent solvers is useful. It catches many ordinary implementation errors. It is not the same as checking a certificate, because the solvers may share an encoding bug, a library, or an unsupported-theory interpretation. Report solver agreement as corroboration, not as a changed kind of object.

14.2.5 Certificate

A **certificate** is a finite object whose checker can validate the claimed result without repeating the original search. A satisfying model is a positive certificate when replay checks it. A DRAT or LRAT refutation can certify that a propositional formula has no satisfying assignment. An Alethe proof can record supported SMT reasoning. A rational equality or inequality certificate can be checked with exact arithmetic.

Certificates shift trust from the producer to the checker, but only for the statement the certificate checker receives. The checker must parse the same formula bytes, apply sound rules, and require the claimed conclusion. A proof format is not magic dust sprinkled on a solver log.

14.2.6 Kernel reconstruction

Kernel reconstruction translates the result into a term checked by a small logical kernel. This can produce a theorem whose dependency footprint is inspectable. The route is strongest when the term contains the mathematical reasoning rather than an axiom that merely repeats the desired conclusion.

Reconstruction can also be structural. A module may import a result as an assumption, attach provenance, and show where it is used. That is useful attestation, but the kernel has not checked the external search. The trusted footprint distinguishes these cases. A kernel label without that measurement is too coarse.

> **Higher on the diagram does not mean broader**
>
> A kernel term for a width-four lemma may have a smaller trusted checker and a narrower mathematical scope than a complete width-64 computation. Trust class and claim scope answer different questions. Print both.

14.3 The claim must reach the checker

An evidence route is a chain. The claim names an ISA slice, precondition, observation, and cost. Semantic code turns instructions into state changes. A generator translates the negated claim into a formula. A solver produces a model or refutation. A checker validates that object. The report must bind the result back to the original claim.

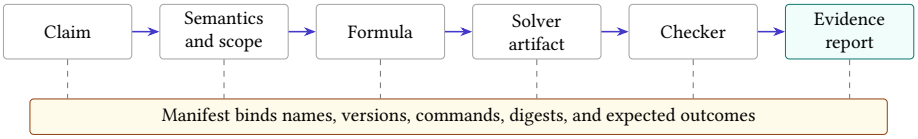


Figure 14.2: A proof checker validates one link near the end. Digests and a manifest bind the checked bytes to the semantics, scope, and claim that give those bytes meaning.

Every arrow admits a characteristic error.

- The prose may describe full architectural state while the observation compares only one register.
- The semantic package may reverse subtraction operands or mishandle a partial-register write.
- The formula generator may omit one instruction form or one cost layer.
- The solver may return a wrong model or verdict.
- The proof checker may accept a malformed step or never require the final contradiction.
- The report may point to stale bytes after a rebuild.

Testing only the final checker cannot catch all six errors. The route needs local controls and end-to-end controls. A semantic mutation should change a trace. A missing instruction should change the candidate-language digest. A damaged proof should fail proof checking. A stale report should fail its hash comparison.

14.4 The evidence manifest

An **evidence manifest** is a small record that binds one claim to the objects and commands used to support it. Object `EVID.def.manifest` records the book's contract. A complete manifest names:

1. the claim identifier and exact scope;

2. semantic package names, versions, and digests;
3. every input, witness, formula, proof, and result file with raw digests;
4. the producer and its configuration;
5. the checker command and expected success condition;
6. the negative control and expected failure class;
7. the trust class and any kernel footprint; and
8. limitations that remain outside the route.

The manifest is not the evidence itself. A manifest file asserting that a proof was checked can lie as easily as prose. Its value is binding: it tells an independent runner which bytes to hash, which command to run, and what outcome must change under the control.

14.4.1 Raw bytes before friendly files

Cryptographic hashes identify byte sequences. They do not identify meanings. The book records SHA-256 digests of raw files before packaging because archive metadata, compression headers, or regenerated line endings can change a container without changing its mathematical content. When a compressed file is the distributed object, record both the distributed bytes and the raw payload when practical.

A matching hash establishes identity with the recorded bytes, not correctness. If the original formula encoded the wrong semantics, preserving it perfectly preserves the error. Hashes close the stale-file and substitution gaps. They do not close the semantic gap.

14.4.2 Commands need outcome contracts

A checker command needs more than a transcript. Its process exit status must depend on the finding. A shell command such as "run checker, then print the previous exit code" is easy to misread when pipelines, timeouts, or later commands overwrite the status. The route should execute one focused checker and treat any unexpected result, timeout, resource exhaustion, or parse error as failure.

Text matching deserves the same care. Searching for the substring `unsat` can accept `not-unsat`, a diagnostic that quotes the input, or two contradictory verdict lines. A wrapper should parse the expected record or count exactly one anchored verdict, then test that count. The checker, not the surrounding prose, must decide success.

Four outcomes, only one success

A lower-bound check may return: the refutation is valid; the formula is satisfiable; the proof is malformed; or the checker exceeded its resource budget. Only the first supports the lower bound. Treating every non-satisfying outcome as equivalent would turn a crash into mathematics.

14.5 Negative controls

A **negative control** is a nearby false claim, damaged object, or altered route that the checker must reject for a named reason. Object `EVID.prin.negative-control` records the rule that every checked route needs one. The label *negative* describes the expected result, not the control's importance.

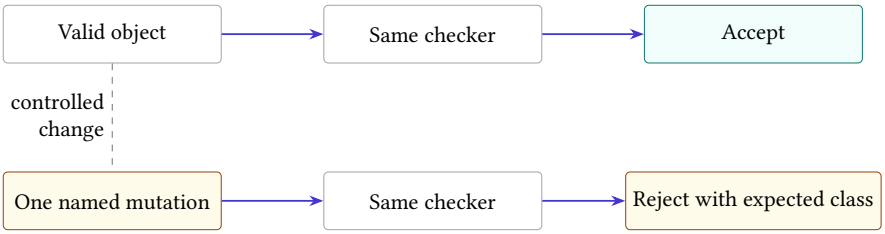


Figure 14.3: The production object and one deliberate mutation travel through the same checker. Acceptance on the left and the named rejection on the right show that the route distinguishes this fault.

A useful control targets a plausible failure at the same layer as the positive claim. For a trace, change one next-state value. For a model, flip one output bit and replay it. For a formula, add a known satisfying witness to a language claimed empty. For a proof, delete a load-bearing step. For a manifest, change one semantic digest. For kernel reconstruction, add one assumed law and require the dependency footprint to contain at least one entry.

The control must share the production path. A hand-written test of a toy parser says little about the command that checks the published artifact. Use the same decoder, semantics, formula reader, proof checker, and wrapper wherever the fault is meant to travel.

14.5.1 Expected failure classes

"It failed" is not precise enough. Suppose a damaged proof also has a wrong hash. If the manifest rejects the hash before the proof checker runs, the test has not shown that proof checking can reject the damage. Both checks may be valuable, but they answer different questions.

Name the expected failure class: digest mismatch, parse rejection, semantic counterexample, illegal decode, unsupported proof rule, missing contradiction, resource exhaustion, or nonempty kernel footprint. Then assert that the control reached that boundary. This prevents a broken setup from impersonating a successful control.

A negative control never supports the main claim

Correct rejection shows that the route detects one selected fault. It does not show that the positive object is correct, that every fault is detectable, or that the theorem's scope is right. Keep the control beside the evidence, but do not count it as a second proof.

14.5.2 Positive controls matter too

Some checkers reject everything. A negative control alone would make such a checker look excellent. Run the production object or a known valid miniature through the same path and require acceptance. The pair establishes one small discrimination: this valid instance crosses the boundary and this invalid neighbor does not.

When testing a resource limit, include a case that completes inside the budget. When testing an unsupported proof rule, include a supported rule that passes. When testing a semantic mutation, replay an unmutated instruction as well. Controls work in pairs because a useful checker must distinguish, not merely approve or object.

14.6 Designing a checker as a small mathematical machine

A checker is easiest to trust when its job can be stated as a short transition system. Its input is a formula and a sequence of certificate steps. Its state contains only the facts needed to judge the next step. Each accepted step must preserve a named invariant. The final state must contain the particular fact that closes the claim. This description is more useful than saying that the checker is "simple." A thousand lines can implement a small relation badly; ten thousand lines can implement it with unusually clear boundaries.

For a clause proof, a **literal** is a Boolean variable or its negation. One useful invariant is that every clause in the active database is either an original clause or has passed the format's admissibility test. Deletion may remove a clause from the active database, but it must not invent a new one. Parsing must reject malformed literals and out-of-range identifiers. The accepting state must contain the empty clause, not merely an end-of-file marker or a line containing a familiar word. These requirements turn implementation review into questions with yes-or-no answers.

The same pattern applies beyond propositional logic. An arithmetic certificate checker may accumulate polynomial identities or inequalities. A program trace checker may update architectural state one instruction at a time. A kernel reconstruction route may translate each external step into a term whose type states the local obligation. In every case, ask four questions:

1. What is the checker's state?
2. Which invariant must every accepted transition preserve?
3. Which malformed or unsupported inputs cause rejection?
4. What exact final state licenses the reported conclusion?

These questions also expose dangerous convenience behavior. A parser that silently skips an unknown instruction has changed the certificate language. A checker that replaces an unsupported theory step with a solver call has moved the trust boundary. A wrapper that catches every exception and returns success has erased the distinction between evidence and failure. Fail-closed behavior means that absence, ambiguity, and unsupported cases cannot cross the acceptance boundary.

The accepting state is part of the theorem

Do not define success as "the checker finished." Define the exact state that the checker must reach, and make every other termination mode distinct.

There is a useful engineering consequence. Keep parsing, transition checking, and reporting separate. The parser produces a typed object or a precise parse error. The transition checker consumes only typed objects. The reporter serializes the outcome without deciding it. This division does not establish implementation correctness, but it makes accidental acceptance paths easier to find and makes negative controls easier to aim.

14.6.1 A hand audit before automation

Before trusting a new checker on a large artifact, construct the smallest example that exercises each rule. Write the expected state after every step. Then alter one premise, one identifier, and the final step in separate copies. The valid example should pass; each altered copy should fail for the predicted reason. This hand audit is the certificate analogue of executing a short instruction trace before running a whole program.

Large benchmarks test scale. They rarely isolate meaning. A million-step proof can pass even when a rarely used rule is wrong, and its size makes diagnosis hard. Tiny rule-directed examples supply semantic resolution; large examples supply operational stress. A serious evidence route needs both.

14.7 A proof file that carried no information

An earlier evidence-led version of this repository contained a DRAT file of 957,982 bytes for a one-step bounded search. Axeyum's checker accepted it. The same checker also accepted the file after its first half was deleted. The checker was not unsound. The underlying formula already contradicted itself under unit propagation, so the last empty-clause step was enough. Everything before it was decoration.

To see the mechanism, begin with a tiny conjunctive normal form formula, a conjunction of clauses in which each clause is a disjunction of literals:

$$F = (a) \wedge (\neg a).$$

The first unit clause forces a to true. The second forces it to false. **Unit propagation** repeatedly applies forced single-literal clauses; here it reaches conflict without any search or added lemma.

One common DRAT check asks whether an added clause has the reverse unit propagation property: temporarily negate that clause, add those negated literals as units, and see whether propagation reaches conflict. For the empty clause there are no literals to negate. If the original formula already conflicts under propagation, the empty clause passes immediately.

Reader argument: why the prefix is irrelevant

Assume a conjunctive normal form formula (F) reaches conflict by unit propagation alone. Consider any candidate DRAT log whose final addition is the empty clause. A backward checker tests whether that empty clause follows by reverse unit propagation. Negating the empty clause adds no assumptions, so the check runs unit propagation on (F) itself. By the premise, that run conflicts. The final step is therefore accepted without using any earlier proof step. Delete or replace the prefix and the same reason remains.

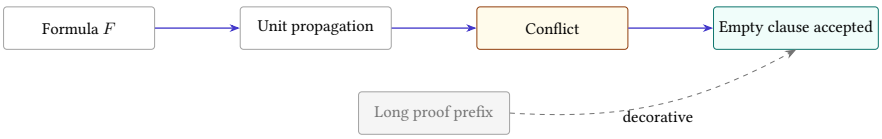


Figure 14.4: When the original formula already conflicts under unit propagation, the final empty clause is accepted without support from the preceding proof steps. The formula, not the long prefix, carries the refutation.

The archived formula contained 2,663 variables, not merely the two clauses above. The structural fact was the same. A small repository script, `scripts/up_refutes.py`, checks whether a standard CNF formula reaches a

unit conflict before a proof file is credited as load-bearing. The archived control used another formula that reached a propagation fixpoint without conflict; its actual refutation was then needed and a truncated proof was rejected.

The lesson is subtler than "large proofs can be fake." The accepted proof file was syntactically valid, and the formula really was unsatisfiable. The mistake was attribution. The repository said the megabyte proof supported the claim when the original clauses already contained a cheaply checkable contradiction. The better evidence route records the unit-propagation check and says that the proof prefix contributes nothing.

Perform the vacuity check by hand

Run unit propagation on $F_1 = (a) \wedge (\neg a \vee b) \wedge (\neg b)$. Then run it on $F_2 = (a \vee b) \wedge (\neg a \vee b) \wedge (a \vee \neg b)$. Which formula reaches conflict without a decision? For the other, give a satisfying assignment or identify the first decision needed.

14.8 Formula generation is part of the proof

A checked refutation establishes that one concrete formula is unsatisfiable. For a program claim, we also need to know what the variables and clauses mean. The generator must cover every allowed candidate, implement the instruction semantics, negate the desired equivalence correctly, and encode the chosen cost bound.

A useful translation argument has two directions. **Soundness of the encoding** says that any satisfying formula assignment decodes to a real counterexample or candidate in the declared language. **Completeness of the encoding** says that every real counterexample or candidate in that language can be represented by a satisfying assignment. For a lower bound, both directions matter. Soundness prevents spurious models; completeness prevents the formula from silently omitting the cheaper program that defeats the claim.

The reader proof can use a miniature. At Ao width two and program length one, enumerate the instruction selectors and show how each selected operation determines the next-state bits. Decode a satisfying assignment back to one instruction and one separating input. Then explain how the production generator repeats the same construction at the target width and bound.

Cross-checks help at this bridge. Generate tiny instances and compare the formula's models with direct enumeration. Mutate one instruction semantic rule and require disagreement. Count the candidate forms produced by the grammar and compare that count with an independent expansion. These checks do not replace a general translation theorem, but they expose the exact surface the theorem must cover.

14.9 Independent checking is a relation, not a brand

Calling a checker independent requires a concrete account. A checker in the same binary may still use different algorithms and parsers. A separate binary may share the same library that contains the bug. Two tools may implement one format from separate specifications. Independence has dimensions: codebase, algorithm, parser, runtime, proof representation, semantic implementation, and institutional origin.

State which dimensions differ. Axeyum's proof-producing SAT core and its DRAT checker are separate implementations in the same repository. Its in-memory, streaming, and file-backed checking paths can be compared, but shared clause types and rule definitions remain part of the trust story. LRAT can move the result to a simpler hint-following checker. Alethe output can be checked by an external implementation only for the rules that implementation supports. Unsupported rules must remain visible refusals, not silently trusted steps.

> **Agreement can still be diagnostically valuable**
>
> Suppose direct enumeration, a bit-vector solver, and a SAT encoding all return the same width-four counterexample. Their agreement does not create a formal certificate. It does sharply localize a later disagreement at width 64: the shared mathematical specification is less suspect than the widened encoding or solver route.

14.10 Kernel reconstruction and its footprint

A kernel check is only as informative as the term and environment supplied to the kernel. If reconstruction creates an axiom named after the desired result and then returns that axiom, the kernel has confirmed type consistency, not the external reasoning. The dependency footprint should list every assumption reached by the final term.

Theory reconstruction can be stronger. A linear-arithmetic certificate may be translated into exact rational equalities and order steps, then checked from the kernel's arithmetic laws. A propositional LRAT proof may be replayed by a verified checker whose soundness theorem lives in the prover. The resulting term can have a small, measured trusted surface.

Search-based program minimality poses a further composition problem. The kernel needs the final propositional contradiction and a link from decoded machine programs to the generated clauses. Importing the SAT result as an assumption does not create that link. A future Ao route may use proof by reflection: define a small checker in the logic, run it on the serialized language and certificate, and apply a kernel-checked soundness theorem. Until that route exists, the manifest must distinguish checked external evidence from kernel reconstruction with mathematical content.

14.11 Resource failures are evidence outcomes

Proof checking consumes time, memory, file descriptors, and disk space. A certificate can be much larger than the formula and more expensive to check than the original search. If the operating system kills the checker, no logical verdict has been produced. Treating absence of a counterexample as a lower bound would confuse resource exhaustion with contradiction.

The checker contract should distinguish verified, rejected, unsupported, timed out, and resource exhausted outcomes. Only the first supports the positive evidence row. Record budgets and peak use when they determine whether another person can reproduce the check. For large proof streams, record whether the checker retained the whole proof, traversed a backward core, or used a file-backed representation.

Resource controls can fail too. A timeout wrapper must deliver termination to the actual checker process and propagate its outcome. A disk-space preflight must inspect the filesystem that will hold the proof. A memory estimate is not a measurement. The workbench is part of the epistemology when its limits can change which answers survive.

Evidence also has a maintenance cost. A certificate format may remain stable while its checker, build system, or runtime disappears. Keep the smallest portable artifacts that preserve the route: plain formula and certificate files, a machine-readable manifest, checker source or a durable source reference, and tiny controls. Containers can make reproduction convenient, but they should package this material rather than become its only description.

When a route is rerun years later, preserve the old report beside the new one. Do not silently replace digests or normalize away a changed outcome. The difference may reveal a repaired checker, a changed resource limit, or an unstable dependency. Evidence is a record of a particular discrimination at a particular boundary; its history can itself become diagnostic evidence.

14.11.1 Reproduction is a second experiment

Reproduction should begin from the manifest, not from the author's shell history. The person repeating the experiment obtains the named inputs and checks their raw digests. That person then builds or identifies the checker version, runs the declared command, and compares the structured outcome with the contract. Only then should the person run the negative control. This order separates an identity failure from a checking failure and both from a control failure.

Environment capture needs judgment. Recording every installed package can bury the dependencies that matter. Recording none can make the result unrepeatable. Prefer a narrow executable description: operating-system and processor assumptions when relevant, compiler or interpreter version, locked dependencies, build command, checker digest, limits, and the exact input digests. If nondeterminism remains, record the seed and say which outputs may vary.

Agreement after reproduction strengthens the operational claim: another run with the bound inputs reached the same outcome. It does not repair a wrong formula or broaden a finite theorem. Reproduction repeats the route; independent validation changes some part of it. The evidence report should say which one occurred.

14.12 Using Axeyum for evidence

The live Axeyum checkout contains substantial evidence machinery. Its propositional layer can produce and check DRAT, stream proofs, perform backward and file-backed checking, and elaborate supported reverse-unit steps to LRAT. Its SMT routes include Alethe production and checking for selected theory rules. Other routes reconstruct arithmetic reasoning or report a kernel dependency footprint. These capabilities are route-specific; no one command turns an arbitrary ISA statement into a kernel theorem.

For the book, Rust should own evidence meaning. It should define the canonical manifest, hash raw inputs, classify checker outcomes, run controls, and reject unsupported trust claims. A Python interface may make the route pleasant to teach:

```
# Illustrative API: the book manifest interface is not yet
  ↪ implemented.
record = EvidenceManifest.load("a0-add-two.json")
record.verify_digests()
record.replay_witness()
record.check_certificate()
record.run_negative_control()
print(record.trust_boundary())
```

Listing 14.1: The intended shape of a future evidence inspection interface

The interface should return typed outcomes rather than booleans that collapse rejection, timeout, and unsupported proof rules. It should display the exact claim, semantic digests, checker version, and failure class. Python may orchestrate and explain; Rust remains the authority for parsing, checking, and canonical serialization.

The present book repository is not at that endpoint. Object `OP.evid.manifest-checker` records the missing implementation obligation. The active artifact tree has the schema design but no completed A0 or real-ISA evidence package to run through it. Existing Axeyum solver machinery is a foundation. It does not fill the semantic and manifest gaps automatically.

14.13 Where the route stops

This chapter does not claim that a negative control proves a checker sound. It does not claim that a certificate validates its formula generator. It does not treat matching hashes as semantic correctness, solver agreement as a proof, or kernel import as reconstruction.

The archived vacuous-proof case concerns one former formula and checker route. It does not impeach DRAT. In fact, the checker behaved according to the proof rule; the repository's interpretation was too generous. The repair is better classification and a propagation precheck, not distrust of every short proof.

No finite collection of controls anticipates every fault. The aim is narrower and more useful: bind the claim to exact bytes, make each checker outcome depend on the finding, attack the likely failure boundaries, and state what remains trusted. A machine-checked result is not the end of human judgment. It is human judgment arranged so that more of its mistakes become visible.

14.14 Exercises

1. **Classify.** For each of the following, name the trust class and nearest remaining trust boundary: five random tests, all sixteen width-four inputs, one satisfying model, one solver verdict, one checked DRAT file, and a kernel term with a nonempty assumption footprint.

2. **Execute.** Replay a three-step Ao trace. Change only the final program counter and state the exact failure a complete trace checker should report.

3. Give a satisfying assignment for $(a \lor b) \land (\neg a \lor c) \land (\neg c)$. Explain why replaying the assignment is easier than supporting an unsatisfiable verdict.

4. Run unit propagation by hand on $(a) \land (\neg a \lor b) \land (\neg b \lor c) \land (\neg c)$. List the forced literals in order and identify the conflicting clause.

5. **Prove.** Generalize the reader argument about a propagation- refutable formula and a final empty clause. State every premise the conclusion needs about the backward checker.

6. Construct a formula that is unsatisfiable but does not conflict under unit propagation from the empty assignment. Explain why an actual proof or search is needed.

7. **Break.** Remove one load-bearing clause from a tiny refutation. Distinguish parse failure, invalid proof step, and failure to derive the empty clause.

8. Design a negative control for a model replayer that accidentally ignores the most significant output bit. Give a positive model and its one-bit mutation.

9. Design a control for an Ao trace checker that updates registers correctly but ignores instruction decoding. Ensure the control reaches the decoder boundary rather than failing a hash check first.

10. Explain why a proof of a byte-identical formula can still fail to support the intended theorem when the semantic package digest changes.

11. Write the minimal fields of an evidence manifest for the two-instruction Ao witness from Chapter 13. Include scope, semantics, observation, cost, witness, checker, and one control.

12. Two solvers return the same verdict, but both consume a formula from one generator. Draw the shared trust boundary and name one check that adds genuine independence.

13. A checker returns code 137 after using all available memory. Explain why the outcome supports neither satisfiability nor unsatisfiability. Specify how the manifest should record it.

14. **Act as the checker.** Review a command that pipes solver output through a text search. List three ways the pipeline can return success without establishing the intended verdict, then design a stricter wrapper contract.

15. Give separate negative controls for formula generation and proof checking. Explain why corrupting a file hash tests neither semantic layer.

16. State soundness and completeness obligations for translating the tiny Ao one-instruction language into a Boolean formula. Give one bug violating each direction.

17. **Transfer.** For one selected RV64 instruction equivalence, list the source pin, decoder, semantics, formula, model or proof, checker, and negative controls that a complete route would bind.

18. Design the Rust and Python boundary for the future book manifest API. Identify which layer owns canonical bytes, typed outcomes, hash checking, display, and user convenience.

CHAPTER FIFTEEN

One Algorithm, Three Machines

A memory region contains words. We want one word that summarizes them: load each word, combine it with an accumulator by exclusive OR, and return the accumulator. The routine is small enough to trace on paper. It is large enough to bring together almost every distinction in this book: words and their readings, byte-addressed memory, effective addresses, loops, observations, calling conventions, cross-machine relations, and evidence that can fail.

The three programs will run on Ao, RV64I, and x86-64. "The same algorithm" will not mean that their registers have the same names or that their traces have the same number of steps. It will mean that each program satisfies one mathematical contract and that explicit relations connect their states to that contract.

One specification, three different movements

We do not prove that the instruction sequences look alike. We prove that each sequence consumes the same logical input region and returns the same XOR reduction, while respecting its own machine and calling contract.

15.1 Why reduction algorithms matter

A **reduction** combines a sequence into one result. Addition produces a sum. Minimum keeps the least element. Conjunction asks whether every Boolean value is true. Exclusive OR records whether each bit position appeared an odd or even number of times. Reductions appear in checksums, statistics, database aggregation, parallel programs, cryptographic constructions, and the inner loops of scientific codes.

Long before a compiler could translate such a loop automatically, programmers had to decide where the accumulator lived, how a pointer advanced, and how an end condition was represented. Modern processors make the same movement at a larger scale. Vector and parallel reductions combine many elements at once, then merge partial accumulators. Those faster forms raise questions about ordering,

associativity, floating-point rounding, and optional extensions. We begin with the scalar integer case because its complete meaning fits on the page.

Exclusive OR is a particularly clean teaching operation. For words of one fixed width it is associative and commutative:

$$(x \operatorname{xor} y) \operatorname{xor} z = x \operatorname{xor} (y \operatorname{xor} z), \qquad x \operatorname{xor} y = y \operatorname{xor} x.$$

Zero is its identity, and every word cancels itself:

$$x \operatorname{xor} 0 = x, \qquad x \operatorname{xor} x = 0.$$

These laws explain the loop invariant without hiding any overflow convention. XOR acts independently at every bit position and always returns another word of the same width.

This routine is not a cryptographic hash. Reordering the input words leaves the result unchanged, and equal words cancel in pairs. The example is useful because its weakness is visible: it teaches a machine proof, not a security claim.

That distinction has a historical analogue. Simple parity and longitudinal redundancy checks were attractive because a human or a small circuit could update them while data moved. They detected selected transmission faults with little state. Their algebra also admitted undetected patterns. Stronger cyclic codes and later cryptographic hashes were developed for stronger fault and adversary models. Hamming's foundational treatment makes the central design question explicit: which error patterns a code can detect or correct (Hamming 1950). The lesson is not that XOR is obsolete. It is that an operation earns meaning from its contract: the same reduction can be a useful diagnostic checksum, a reversible mask component, or a dangerously weak authenticator.

Our contract asks only for the reduction value. It does not say that unequal regions have unequal reductions, that a changed word is always detected, or that an attacker cannot preserve the result. Keeping those tempting claims out of the theorem is part of formal precision. It also demonstrates why proof does not rescue a poor specification: a flawless implementation proof can establish exactly the weak property that was requested.

15.2 The contract before the code

Fix width 64. Let M be byte-addressed memory, a the starting address, and n an unsigned count. Define the little-endian word load from Chapter 4 as $L(M, q)$. The desired result is

$$\mathsf{foldxor}(M, a, n) = L(M, a) \operatorname{xor} L(M, a + 8) \operatorname{xor} \cdots \operatorname{xor} L(M, a + 8(n - 1)),$$

with value zero when $n = 0$.

Ellipsis is convenient prose but a poor base definition. The recursive form names both cases exactly:

$$\begin{aligned} \mathsf{foldxor}(M, a, 0) &= 0, \\ \mathsf{foldxor}(M, a, n+1) &= L(M, a) \text{ xor } \mathsf{foldxor}(M, a+8, n). \end{aligned}$$

The program contract needs more than this equation. Each of the n loads must be valid. Address formation must not wrap around the 64-bit address space. The code region must be executable under the selected machine model. The calling convention must place the pointer and count where the listing expects them. On normal return, the result register must equal $\mathsf{foldxor}(M, a, n)$, and memory must be unchanged.

XOR-reduction call contract

The input consists of an initial memory M_0, address a, and count n. The precondition requires every range $[a + 8i, a + 8i + 8)$, for $0 \le i < n$, to be readable and requires the address calculations not to wrap. Normal return must preserve M_0 and place $\mathsf{foldxor}(M_0, a, n)$ in the declared result register. The contract observes normal return, the result word, and memory preservation.

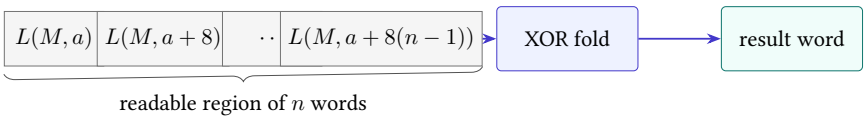


Figure 15.1: The contract connects a logical region of n words to one result. Pointer motion and accumulator updates are implementation details.

The contract deliberately permits $n = 0$. In that case the readable region is empty, no load occurs, and the result is zero. The address a need not be readable because no byte at that address is accessed. A precondition that always demanded eight readable bytes would be stronger than the routine needs.

Alignment is separate. Core Ao permits a word access to begin at any address provided the full byte range exists. The selected RV64I environment may trap or handle a misaligned load outside the base instruction semantics, depending on its execution environment. The selected x86-64 form permits an unaligned ordinary load in this model. For one clean three-way theorem, we require a to be divisible by eight. This is a theorem premise chosen for the relation, not a universal statement about all three machines.

A valid first address does not validate the region

Checking a alone is insufficient. The final access begins at $a + 8(n - 1)$, and all eight of its bytes must exist. The no-wrap and range premises quantify over every iteration.

15.3 The language-independent loop

The algorithm carries four logical values: the original memory M_0, the number i of words already consumed, the current pointer $p = a + 8i$, and the accumulator

$$x = \mathsf{foldxor}(M_0, a, i).$$

It also carries a remaining count $c = n - i$. The following **pseudocode** is a readable algorithm sketch rather than an executable language:

```
p := a
c := n
x := 0
while c != 0:
    x := x xor load64(M, p)
    p := p + 8
    c := c - 1
return x
```

Listing 15.1: The logical XOR-reduction loop

The order of the three body updates matters to a step-by-step proof even though the final mathematics is simple. The load uses the old pointer. The accumulator gains word i. The pointer then names word $i+1$, and the remaining count becomes $n - (i + 1)$.

Loop invariant

At the test before iteration i, where $0 \le i \le n$, memory equals M_0, the pointer equals $a + 8i$, the remaining count equals $n - i$, and the accumulator equals $\mathsf{foldxor}(M_0, a, i)$.

Reader proof of the logical loop

Initialization. Before the first test, $i = 0$. The pointer is a, the count is n, memory is unchanged, and the accumulator is zero. These are exactly the four invariant clauses because the reduction of an empty prefix is zero.

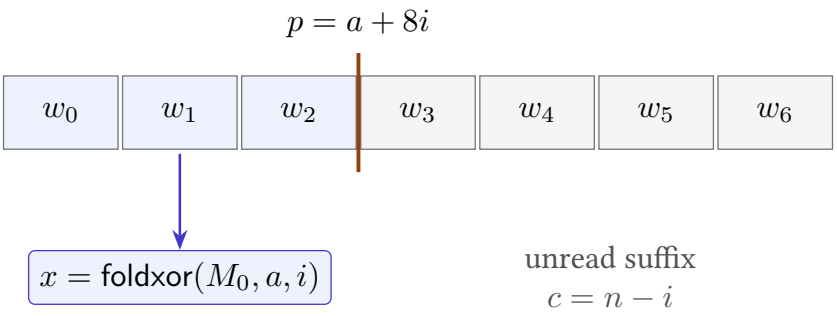


Figure 15.2: The invariant cuts memory into a consumed prefix and an unread suffix. The accumulator summarizes exactly the prefix.

Preservation. Assume the invariant at a test with $c \neq 0$. Then $i < n$, so the precondition makes $L(M_0, a + 8i)$ valid. The load reads that word without changing memory. XOR changes the accumulator to

$$\mathsf{foldxor}(M_0, a, i) \text{ xor } L(M_0, a + 8i) = \mathsf{foldxor}(M_0, a, i + 1).$$

The pointer addition produces $a+8(i+1)$, and decrement produces $n-(i+1)$. Thus the invariant holds for the next test.
Exit. If the test sees $c = 0$, the invariant gives $n - i = 0$, hence $i = n$. The accumulator is therefore $\mathsf{foldxor}(M_0, a, n)$, memory still equals M_0, and the return value satisfies the contract.
Termination. The remaining count is a natural number. Every body execution begins with a positive count and decreases it by one without wraparound. It therefore reaches zero after exactly n iterations.

This proof has two parts that bounded testing cannot replace. Preservation is quantified over every legal iteration, and termination covers every natural input count admitted by the contract. Testing can still find mistakes in an implementation or in the connection between its registers and these logical variables.

15.4 From an algorithm to a machine theorem

The pseudocode is not an instruction-set-neutral executable. Its word `load64` already assumes an address unit, byte order, access width, and failure policy. Its subtraction assumes a positive remaining count. Its return assumes an interface with a caller. Lowering the loop means discharging obligations, not replacing words with mnemonics.

First comes **representation**. Each logical variable needs a machine location at each cut point. The accumulator may remain in one register. The pointer and

count may be overwritten because the contract observes their initial values and final result, not their final register contents.

Second comes **operation refinement**. A machine load must reconstruct $L(M, p)$. XOR must implement bitwise exclusive OR at width 64. The pointer update must represent addition by eight without wrapping on admitted inputs. The count update must represent natural-number subtraction by one. The branch must continue exactly when the new count is nonzero.

Third comes **control refinement**. Setup reaches the first loop cut point. One body path reaches the next cut point. The zero-count path reaches a successful terminal observation. Invalid fetches, loads, or returns must not be relabeled as success. Program-counter equations differ, but this shape is shared.

Fourth comes **frame preservation**: state outside the allowed write set remains unchanged. Memory is the main frame component here. ABI-preserved registers and stack state join it for the real-machine versions that a caller can invoke as procedures. A result equation alone would miss both forms of damage.

Table 15.1: The lowering obligations carried by every listing.

Obligation	Required connection
Representation	registers and memory correspond to p, c, x, M_0
Operation	load, XOR, add, and decrement implement the logical body
Control	setup, body, exit, and outcome follow the loop structure
Frame	memory and protected procedure state remain unchanged
Encoding	fetched bytes decode to the instruction instances in the proof

The last row is easy to omit when reasoning from assembly text. A proof of the mnemonic sequence is not yet a proof about executable bytes. The final artifact must bind assembled bytes, instruction boundaries, decoded operands, and the semantic instances used by the step relation. This chapter prints readable assembly because those decoders do not yet exist in Axeyum; that absence stays visible in the evidence boundary.

Build the cut-point map first

For each listing below, mark entry, the first state satisfying B_0, the load state, the next loop-head state, and the successful terminal state. Write which logical variables are meaningful at each mark.

15.5 Ao makes the proof state visible

Use width-64 Ao with the following entry convention: $r_1 = a, r_2 = n$, and memory M_0. Register r_0 is the result. Registers r_3, r_4, r_5 are scratch. The listing uses labels

for reading; encoding replaces each label with the signed displacement defined in Chapter 9.

```
00: movi      r0, 0
04: movi      r4, 8
08: movi      r5, 1
0c: cmp       r2, r0
10: branch.eq done
loop:
14: load      r3, [r1+0]
18: xor       r0, r0, r3
1c: add       r1, r1, r4
20: sub       r2, r2, r5
24: branch.ne loop
done:
28: halt
```

Listing 15.2: Ao XOR reduction

The first comparison handles the empty input without loading. The subtraction sets Ao's zero condition from the new count, so the backward branch can reuse that result. The XOR also changes conditions, but the later subtraction overwrites them before `branch.ne`. This ordering is a small live-state argument: the branch depends on conditions from `sub`, not conditions from `xor`.

At the loop head, relate r_1 to p, r_2 to c, and r_0 to x. Registers $r_4 = 8$ and $r_5 = 1$ are fixed helpers. The Ao state also contains the program counter, condition bits, outcome, program bytes, and memory. The invariant does not erase these components. It constrains those needed for the proof and permits the current condition bits to have whatever values the preceding legal path produces.

The backward branch begins at byte address 36 and targets byte address 20. Ao's rule is $p + 4 + 4d$, so

$$36 + 4 + 4d = 20,$$

which gives $d = -5$. The forward branch at address 16 targets address 40 and therefore uses $d = 5$. Both fit the signed byte field.

A complete two-word A0 trace at loop granularity

Let $n = 2$, let the first word be u, and let the second be v. At the first loop head, $(r_1, r_2, r_0) = (a, 2, 0)$. After one body, $(r_1, r_2, r_0) = (a + 8, 1, u)$. After the second body, $(r_1, r_2, r_0) = (a + 16, 0, u \operatorname{xor} v)$. The second backward branch is not taken. Execution reaches `halt` with the required result.

This summary suppresses the instruction-level intermediate states. A machine trace must retain them: the state after the load, the state after XOR, both arithmetic

updates, and the branch successor. The loop-granularity relation holds at selected cut points, not after every instruction.

15.5.1 Ao failure controls

Four small mutations test different obligations. Change the load displacement from zero to eight; the first iteration skips the first word. Change the pointer increment from eight to one; the next load overlaps the first word. Change the decrement helper from one to zero; termination fails. Change the backward displacement from -5 to -4; control returns to the XOR rather than the load, reusing stale r_3. Each mutation should produce a distinct counterexample or control-flow failure.

For the empty case, poison the bytes at a or make the address unreadable. The valid program must still halt normally with zero because it never loads. That control distinguishes a true zero-iteration path from an implementation that reads once.

15.6 RV64I carries the predicate in registers

Use the standard integer calling names for exposition: `a0` initially holds a, `a1` holds n, and `a0` carries the return value. Because the pointer and result share `a0` at different times, copy the pointer to temporary `t0` before clearing the accumulator.

```
    addi t0, a0, 0
    addi a0, x0, 0
    beq  a1, x0, done
loop:
    ld   t1, 0(t0)
    xor  a0, a0, t1
    addi t0, t0, 8
    addi a1, a1, -1
    bne  a1, x0, loop
done:
    jalr x0, 0(ra)
```

Listing 15.3: RV64I XOR reduction

Every instruction belongs to the base integer slice used by this book (RISC-V International 2026b). The `addi` forms perform copying, zero construction, pointer advance, and count decrement. The conditional branches compare `a1` directly with the architectural zero register. No arithmetic flags exist in the selected state. The final `jalr` returns through the incoming link register under the calling convention discussed in Chapter 10.

At the RV64 loop head, `t0` represents p, `a1` represents c, and `a0` represents x. Temporary `t1` is unconstrained before the load and equals the current word afterward. The state relation must also say that the address translation used by memory is identity on the input region, or state another explicit mapping.

The instruction `ld` forms an effective address from `t0` plus the sign-extended immediate zero and reconstructs a 64-bit little-endian value. The pointer increment is modulo 2^{64}, but the contract's no-wrap premise makes its mathematical reading equal to $a + 8(i + 1)$. Likewise, decrement is modular machine arithmetic; the loop guard and invariant ensure it is applied only to a positive count.

The proof uses the ABI without confusing it with the ISA

The ISA gives meanings to integer-register numbers, `ld`, branches, and `jalr`. The ABI gives the names `a0`, `a1`, `t0`, `t1`, and `ra`, and states which registers a caller may expect to survive. Both layers are needed for a callable routine. They are different sources of obligations.

The listing changes `a1`, `t0`, and `t1`. Under the selected calling convention these are caller-saved registers. It does not adjust the stack or call another routine. Its memory action is read-only. A correctness statement at ISA level can ignore ABI preservation; the callable-routine theorem cannot.

15.7 x86-64 lets the loop reuse status flags

Under the selected System V integer convention, `rdi` holds a, `rsi` holds n, and `rax` carries the result. This Intel-syntax listing uses only scalar integer instructions.

```
    xor  eax, eax
    test rsi, rsi
    je   done
loop:
    xor  rax, qword ptr [rdi]
    add  rdi, 8
    sub  rsi, 1
    jne  loop
done:
    ret
```

Listing 15.4: x86-64 XOR reduction

Writing `eax` clears all of `rax` in 64-bit mode (Intel Corporation 2026b). This is a selected x86-64 subregister rule, not a generic property of narrow writes. The `test` instruction computes a logical conjunction for flags without storing the result. Here it sets the zero flag exactly when the count is zero. The memory-source XOR loads a 64-bit word and combines it with the accumulator. The subtraction sets the zero flag from the new count, and `jne` consumes that flag.

At the loop head, `rdi` represents p, `rsi` represents c, and `rax` represents x. The relevant status-flag values are those created by the path into the head. The cross-machine relation need not equate x86 status flags with RV64 state because

RV64 has no corresponding component. It must ensure that each machine chooses the same logical exit condition at the compared cut points.

The listing mutates `rdi` and `rsi`. They are caller-saved under the selected ABI, as is `rax`. The routine preserves the stack pointer and callee-saved registers and returns through the address already on the stack. These facts are not visible in the mathematical fold equation, but they are part of the procedure contract.

Equivalent results can hide unequal fault behavior

If the readable-region premise is removed, the three programs need not fail in the same way. Alignment policy, address translation, and fault delivery differ. Our theorem relates normal executions under the printed precondition. It does not assert universal agreement for invalid memory.

15.8 Local lemmas beneath the simulation

The three-machine proof kit below compresses many instruction facts into one body paragraph. A formal development should expose them as small lemmas. The load lemma starts from related pointers and memory and ends with equal loaded words. It depends on effective-address formation, range validity, access width, and little-endian reconstruction. It says nothing yet about the accumulator.

The combine lemma starts from equal logical accumulator values and equal loaded words. It concludes that the three destination words agree after XOR. Ao and x86 also update condition state; RV64I does not. Those side effects are allowed by the relation but must be described because later control may observe them.

The pointer lemma connects three modular additions to the ordinary equality $p' = p + 8$. Its proof uses the no-wrap premise. The decrement lemma connects machine subtraction to $c' = c - 1$ and uses $c > 0$. The control lemma then shows that the Ao zero condition, RV64 register comparison, and x86 zero flag select continuation exactly when $c' \neq 0$.

These lemmas meet in a deliberate order. Proving the branch before showing which instruction last wrote its condition state would leave a gap. Proving the load without the address relation would compare bytes from different locations. Proving arithmetic only as a word equality would not justify the natural-number decrease used for termination.

Small lemmas localize a counterexample

A load mismatch points toward address or memory semantics. An equal load followed by unequal accumulators points toward XOR or register writes. Equal body values followed by unequal paths points toward conditions or targets.

One may also prove each machine correct against the logical loop separately and derive pairwise agreement by transitivity. That structure often scales better than one ternary relation. The direct three-way presentation keeps the shared cut points visible. An Axeyum implementation may store one machine-to-logical simulation per architecture and construct its comparison report from their common logical observations.

15.9 Relating three traces

The listings have different setup lengths and different numbers of instructions per iteration. A lockstep relation would fail immediately. We instead compare **cut points**: entry, loop head, and normal return. One logical body corresponds to several machine steps on every architecture, but the number and placement of those steps need not agree.

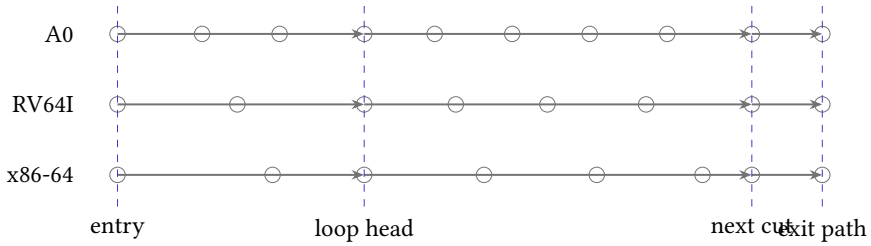


Figure 15.3: Different instruction traces meet at common logical cut points. The dashed bands represent initialization, one body, and exit.

Define relation B_i at the loop head. It requires all three memories to agree on the input region and to remain equal to M_0. It maps Ao r_1, RV64 `t0`, and x86 `rdi` to $a + 8i$. It maps Ao r_2, RV64 `a1`, and x86 `rsi` to $n - i$. It maps Ao r_0, RV64 `a0`, and x86 `rax` to the reduction of the first i words. It also places each program counter at its own loop test or body cut point and requires each machine to remain in normal execution.

Three-machine forward simulation

Entry to first cut point. Each setup sequence preserves memory, copies or retains the input pointer, constructs the zero accumulator, and retains the count. Therefore B_0 holds.
Body correspondence. Assume B_i and $i < n$. The three effective addresses all denote $a + 8i$. By memory agreement and common little-endian reconstruction, the loads return one word $u = L(M_0, a + 8i)$. Each accumulator becomes its old value XOR u. Each pointer advances by eight, and each count decreases by one. The no-wrap and positive-count premises connect the

modular updates to ordinary arithmetic. Thus B_{i+1} holds at the next cut point.
Exit correspondence. Under B_i, all three loop tests see zero exactly when $n - i = 0$. They therefore exit together at the logical level, although through different instructions. Then $i = n$, and all three result registers equal foldxor(M_0, a, n). Their return mechanisms establish the selected procedure observations.

The proof composes two ideas. The loop invariant connects each implementation to the mathematical fold. The cross-state relation connects corresponding implementation cut points. Either route can be used alone, but together they explain both correctness and agreement.

15.9.1 What the relation intentionally forgets

Ao condition bits, x86 status flags, and RV64 temporary values need not agree. Instruction bytes and program counters differ. Ao ends through `halt` in the pedagogical listing, whereas the two callable real-ISA listings return to a continuation. A direct theorem between those terminal states must either map Ao halt to a harness-defined return observation or give Ao a fixed-return wrapper as described in Chapter 10.

The distinction is substantive. Without the harness, "all three return" is false for the printed Ao program. The clean statement is that all three reach their declared successful terminal observation with the same result. A later Axeyum package should encode that observation rather than force unlike control states to be equal.

15.10 Execute a concrete instance

Let $a = 256$, $n = 3$, and let the three loaded words be

$$u = \texttt{0f0f}, \qquad v = \texttt{00ff}, \qquad z = \texttt{0ff0},$$

shown with leading zeroes omitted. The successive accumulator values are

$$0, quad\texttt{0f0f}, quad\texttt{0ff0}, quad\texttt{0000}.$$

The pointer values at loop heads are 256, 264, 272, and 280. The remaining counts are 3, 2, 1, and 0. The final zero is not evidence that no work occurred; these particular words cancel.

A useful test suite includes more than this attractive cancellation. Test an empty input, one word, two unequal words, two equal words, a region near the highest permitted address, and a pattern with high bits set. For each case, compare the complete outcome class and memory, not only the result register.

Table 15.2: The logical trace for the three-word example.

i	pointer	remaining	accumulator
0	256	3	0000
1	264	2	0f0f
2	272	1	0ff0
3	280	0	0000

15.11 Breaking the proof on purpose

Negative controls should target the binding chain from Chapter 14. At the specification layer, change the fold identity from zero to all ones. At the memory layer, reverse the byte order of one decoder. At the instruction layer, change one pointer increment from eight to four. At the control layer, branch on zero rather than nonzero after decrement. At the ABI layer, place the count in the wrong entry register. At the observation layer, forget memory preservation and introduce a store that the result-only comparison ignores.

Each control needs a small separating input. The empty input directly separates the wrong identity. A two-word region with distinct halves exposes a four-byte pointer increment. A count of one exposes an inverted continuation branch. A count of zero exposes an implementation that loads before checking. A memory snapshot before and after exposes the hidden store.

Find the first separating iteration

Change the RV64 pointer update to `addi t0,t0,16`. For $n = 3$, write the addresses loaded by the valid and mutated programs. Choose three word values so that the first result disagreement occurs only after the second load.

Counterexamples should replay as traces. A solver assignment saying $n = 2$ and giving two memory words is not yet an explanation. Decode the program, execute both versions from the assigned state, and print the first cut point where pointer, count, accumulator, outcome, or memory agreement fails.

15.12 Cost questions come after correctness

The three listings invite counting, but one number cannot settle which is best. Ao uses fixed four-byte instructions and materializes constants in registers. RV64I also uses fixed-width base instructions but has immediate forms and a zero register. x86-64 uses variable-length encodings and a memory-source XOR. Instruction counts, encoded bytes, decoded operations, loads, branches, and estimated cycles are different costs.

For a positive count, every listing executes its setup and entry test, one load and combine per word, pointer and count updates, and a continuation test. The final not-taken branch is still an executed instruction. Symbolic dynamic counts can be derived from the listings, but they describe abstract paths, not processor timing. Caches, prediction, instruction fusion, and other microarchitectural facts are outside the scalar ISA model.

Code-byte comparison requires actual encodings. The x86 memory-source form may save a separately decoded load while using more bytes for other instructions. RV64 compressed encodings would change byte cost, but the optional compressed extension is outside this chapter's language. Admitting it for one candidate and not another would change the optimization question.

The loop is not a minimality theorem. Another implementation might count up toward an end pointer, unroll two loads, use a different branch layout, or return through a harness-specific convention. Chapter 13 requires a finite candidate grammar, cost, precondition, observation, and lower-bound argument. These listings are transparent correctness witnesses, not globally optimal programs.

There is nevertheless a useful comparison exercise. Count static instructions in each printed body, then count dynamic instructions for $n = 0$, $n = 1$, and general positive n. Next assemble only after fixing exact syntax, profiles, and entry addresses, because branch displacements and x86 instruction lengths depend on that layout. Finally, keep the resulting table descriptive. No column should be silently promoted into "the cost."

On real hardware, even a fixed byte sequence has no single context-free time. The first load may miss in cache; later loads may stream. A branch predictor may learn the continuation pattern. Front-end and execution resources may overlap instructions. Those phenomena are important, but they demand a microarchitectural model and measurement method. The ISA proof establishes functional movement across architectural states, not a cycle count.

The scalar proof prepares a parallel proof

Because integer XOR is associative, a vector or tree reduction may partition the region, reduce each part, and combine partial results. Its proof adds a partition lemma and a relation for vector lanes or worker states. It also adds alignment, tail, and extension-specific obligations. Chapter 16 places those states beyond the present scalar core.

15.13 How Axeyum should carry this chapter

Axeyum already supplies bit-vector expressions, solver routes, evidence reports, propositional proof checking, selected SMT proof support, and kernel reconstruction machinery. Those general capabilities are useful ingredients. The live checkout does not yet supply the reusable Ao, RV64I, and x86-64 semantic packages required

to execute these three listings as book artifacts. This chapter therefore separates the reader proof above from a future machine route below.

The first implementation layer should be Rust. Instruction decoding, typed machine states, step relations, trace replay, memory-range checks, and cross-state relations are trust-bearing core logic. They belong in small Axeyum crates with exhaustive unit tests and negative controls. Python bindings should then expose those checked operations for lessons, notebooks, and artifact runners. Python is the workbench; it should not become a second, quietly different semantics.

An introductory Python session should eventually have this shape:

```
from axeyum.machines import a0, rv64, x86_64
from axeyum.examples import xor_reduce

case = xor_reduce.case(address=256, words=[0x0f0f, 0x00ff,
  ↪ 0x0ff0])
a0_run = a0.run(xor_reduce.a0_bytes(), case.a0_state())
rv_run = rv64.run(xor_reduce.rv64_bytes(), case.rv64_state())
x86_run = x86_64.run(xor_reduce.x86_bytes(), case.x86_state())

report = xor_reduce.compare(a0_run, rv_run, x86_run)
report.require_controls()
print(report.boundary())
```

Listing 15.5: Illustrative future Axeyum lesson interface

The listing sketches a future design; the import is not currently supported. The eventual API must return typed outcomes, decoded traces, first-divergence information, semantic-package digests, and control results. It must reject unsupported instructions and malformed bytes. It must not turn a timeout, missing checker, or decode error into successful evidence.

required executable evidence route

	decoder	step	trace	relation	report
A0	to build	to build	to build	to build	to build
RV64I	to build	to build	to build	to build	to build
x86-64	to build	to build	to build	to build	to build

Figure 15.4: The case study needs separate support at the definition, execution, relation, and certificate layers. A higher layer cannot repair a missing lower binding.

15.13.1 A staged artifact route

The Ao package comes first because its semantics are defined by this book. A complete package needs state types, encoder and decoder, single-step behavior, bounded traces, range-checked little-endian memory, and controls for every instruction used here. The XOR-reduction artifact can then execute the Ao listing and compare its terminal state with a direct fold computation.

Each real-machine package needs a pinned source boundary, decoder coverage for the exact byte forms in its listing, and executable step semantics. The first cross-ISA milestone is not the whole loop. It is one load, one XOR, one pointer update, and one branch under explicit state relations. Only after those local steps pass positive and negative controls should the route compose them into the loop invariant.

A bounded solver query can search for a disagreement up to a chosen maximum count and finite memory shape. If it finds one, replay it through all three executors. If it finds none, the report supports only that bound and encoding. The universal reader proof still depends on the stated semantic lemmas. A future certificate route may reduce the bounded query to a checkable formula, but it must bind formula generation to the machine definitions as Chapter 14 requires.

The evidence manifest for this example should name more than three program files. It should bind the fold specification, precondition, observation, machine-package versions, ABI profiles, assembled bytes, decoded listings, initial-state constructors, trace bounds, comparison command, and controls. The report must distinguish successful normal comparison from a trap, bound exhaustion, decode refusal, unsupported instruction, digest mismatch, and control failure.

Two reproduction routes are useful. The lesson route takes a short word list and prints a human-sized trace like the table above. The artifact route consumes raw program and memory bytes and emits a structured report. Both must invoke the same Rust semantics. If the lesson reimplements the machines directly in Python, agreement with the artifact might compare two subtly different definitions.

Table 15.3: Minimum manifest bindings for the three-machine case study.

Binding	Question it answers
Claim and scope	Which width, inputs, outcomes, and exclusions are asserted?
Semantics	Which A0 definition and pinned real-ISA slices interpret bytes?
Programs	Which raw bytes, entry addresses, and decoded instructions run?
Harness	How do logical inputs and successful outputs map to machine states?
Checker	Which command and version decide the comparison?
Controls	Which named mutations must the same route reject?

The empty case is the best first end-to-end control because it crosses every setup and return layer without accessing data memory. The one-word case then fires the load and XOR paths. The two-word case fires the backward edge and pointer update. A test plan built in this order diagnoses more than a single large example: when a stage first fails, the newly exercised movement is small enough to inspect.

After those concrete cases, symbolic execution can replace the word contents with bit-vector variables. The expected result is the symbolic XOR of the loaded variables. Symbolic addresses should wait until the range model and alias policy are explicit; otherwise one query silently combines instruction correctness with a much harder memory-model question.

The concrete ladder should also include values chosen for semantic edges, not only convenient arithmetic. An all-zero word checks that XOR itself does not manufacture bits. Repeating the same word twice checks cancellation. A word with only the high bit set catches accidental signed reasoning. Alternating bytes make an endian reversal visible. Placing the array at the highest valid starting address exercises the range premise without crossing it. None of these cases replaces the invariant, but each attacks a different connection between the logical fold and a machine trace.

From test ladder to proof obligations

For each concrete case, record the semantic feature it exercises. Then replace the particular values with variables while keeping the same precondition and observation. The resulting obligation should explain why the case generalizes: XOR identity, cancellation, byte reconstruction, range validity, pointer advance, or loop progress. A test with no named obligation is useful for fault finding but carries no clear part of the proof.

The same ladder gives controls a deliberate order. Reverse one load's byte join for the alternating-byte case. Change one branch predicate for the empty case. Advance the pointer by the wrong word size for the two-word case. Remove the memory-frame clause and show that an injected store escapes detection. Each mutation should fail first at the relation component designed to catch it. If a later, unrelated check fails instead, the report is too coarse or an earlier binding is missing.

15.14 What this chapter establishes, and what it does not

The mathematical argument establishes the XOR-fold loop for the printed logical algorithm. The instruction-level arguments explain how the selected listings instantiate its variables and control. The cross-machine simulation identifies the

cut-point relation needed for agreement. These are reader proofs over the semantics stated in this book and the pinned real-ISA sources.

The chapter does not yet present machine-produced Axeyum evidence for the real listings. It does not cover optional compressed or vector instructions, different ABIs, arbitrary misaligned accesses, concurrent memory changes, memory-mapped input, faults outside the precondition, or timing. It does not claim the listings are minimal or fastest. Those exclusions are part of the result's meaning.

The example also marks a transition in scale. We have moved from one instruction to a loop and from one state relation to three executions. Yet the method did not change: define the words, inventory the state, state the observation, prove a local movement, compose movements at named cut points, and bind any machine-derived evidence to the exact claim.

15.15 Exercises

1. **Execute.** Trace all three logical iterations in the table from the byte contents of memory, showing each little-endian reconstruction.

2. Trace the Ao listing for $n = 0$. Name every executed address and explain why no data-memory read occurs.

3. Compute the two Ao branch displacements from the printed addresses and reconstruct both targets from the encoded values.

4. State the loop invariant after the load but before XOR. Which additional temporary value must it name?

5. Prove that XOR reduction is unchanged by swapping two adjacent input words. Explain why that is a weakness for a security checksum.

6. Prove that appending the current reduction value to a region makes the new reduction zero.

7. **Break.** Change the pointer increment from eight to four. Give the smallest count and memory bytes that separate the programs.

8. Change the Ao backward branch displacement from -5 to -4. Trace the first two visits to the resulting target.

9. Delete the RV64 empty-input branch. Give an input allowed by the original contract that now fails, and classify the failure.

10. Replace x86 `xor eax, eax` with a write to `ax`. Explain which initial register values expose the subregister error.

11. Write the complete read and write sets for one loop body on each machine, including program counter and condition state.

12. Define a relation between the A0 state after `load` and the RV64 state after `ld`. Include the temporary loaded word.

13. **Prove.** Expand the body-correspondence paragraph into one lemma per instruction group, with precondition and postcondition for each lemma.

14. Strengthen the observation to require scratch registers to agree. Show why the printed three-way theorem then fails or becomes unnatural.

15. Weaken the observation by dropping memory preservation. Construct a program that returns the right result while corrupting one input byte.

16. Change the specification from XOR reduction to modular sum. Identify the parts of the invariant and listings that change, and the parts that remain.

17. Add a stride s to the contract. State sufficient no-wrap and readable- range premises for addresses $a + si$.

18. Permit n to use every 64-bit unsigned value. Explain why termination in natural-number arithmetic still needs an address and resource story.

19. Design one positive and four negative controls for the future Axeyum A0 artifact. Name the expected outcome class of each.

20. Design a bounded disagreement query for counts at most three. List its symbolic inputs, transition constraints, observation, and replay requirement.

21. **Transfer.** Rewrite the routine for a different calling convention. Separate instruction semantics from the new register and preservation rules.

22. Compare a scalar XOR reduction with a vector version conceptually. List the additional state, instruction, and reduction-order questions that move the vector proof beyond this chapter's model.

CHAPTER SIXTEEN

The Edge of the Model

Ao and the two scalar slices make many program claims precise. They do not contain every state that a modern processor exposes or every behavior that software can observe. The boundary is not a miscellaneous list of hard problems. Each item in this chapter identifies structure absent from the model we built.

That way of organizing omissions matters. "The model does not support floating point" sounds like a missing feature checkbox. "The state has no rounding mode, floating-point exceptions, or representation for NaN" names what a definition must add. One description invites a larger instruction table. The other begins a semantic design.

Every boundary names missing state or missing observation

When a claim falls outside the scalar model, ask what new state can affect the movement and what new observation can distinguish outcomes. Extend those definitions before extending the proof machinery.

16.1 A model is an instrument, not a miniature universe

A scientific model is built for questions. A map that shows streets may omit soil composition; a geological map may omit turn restrictions. Neither is false merely because it cannot answer the other's question. It becomes misleading when its user forgets the purpose of the abstraction or treats an omitted distinction as an established equality.

Our scalar architectural model answers questions about decoded instructions, word and memory effects, control transfer, normal and trapped outcomes, selected calling conventions, and observations over those states. It can support a theorem that three loops return the same XOR reduction. It cannot, without extension, support a theorem that they consume the same energy, reveal the same information through a cache, behave alike under another thread's writes, or survive a page-table change.

The recurring error is to ask a strong question of a weak observation. If timing is absent, every timing difference is invisible. If speculative state is absent, a proof cannot constrain speculative accesses. If the program map is immutable, a theorem cannot describe a store that changes future decoded instructions. Silence in the model is not evidence of absence in the machine.

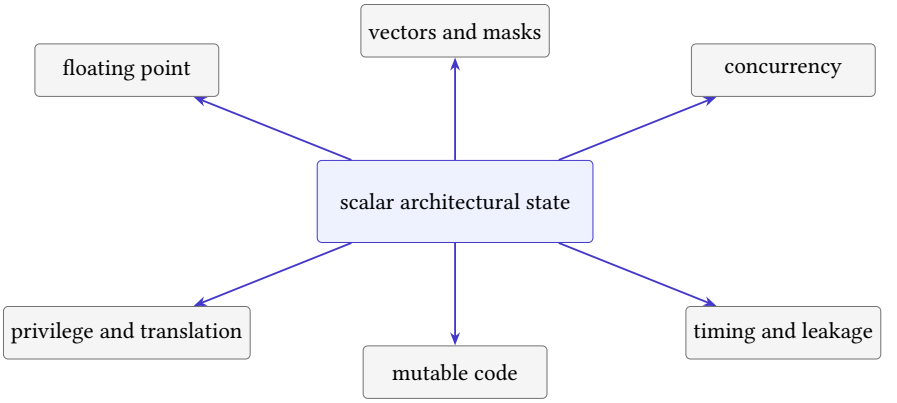


Figure 16.1: The scalar architectural core is surrounded by extensions, each of which adds state, transitions, observations, or all three.

Honest model extension

An extension states the new values and state components, defines how transitions read and write them, enlarges observations only as required, and revisits every preservation or refinement theorem whose premises mention the old state. It also supplies a conservative-embedding argument when old programs are expected to retain their old meaning.

The final clause protects earlier work. Adding vector registers should not quietly change scalar addition. Adding virtual memory should explain when the old flat-address model embeds into the new model, perhaps under an identity translation and stable permissions. The extension may reveal that an earlier claim needed a stronger premise. That is a correction, not a failure of the method.

16.2 Floating point adds an arithmetic environment

An integer word represents one residue modulo a power of two, with optional signed and unsigned readings. A floating-point datum has a different structure: sign, significand, exponent, finite and infinite values, signed zero, and values that represent “not a number.” Operations depend on a destination format and a rounding direction. They may also report exception conditions. IEEE 754 specifies

formats, operations, exception conditions, and default handling for binary and decimal arithmetic (IEEE Computer Society 2019).

The familiar algebraic laws need review. Floating-point addition is generally not associative because intermediate rounding depends on grouping. Two mathematical expressions that are equal over real numbers may produce different floating-point values. NaN comparisons do not behave like an ordinary total order. Positive and negative zero compare equal in selected operations while retaining distinguishable encodings and behavior elsewhere.

A regrouping can change a result

Choose a format and finite values where adding a small y to a large x rounds back to x, but adding y to $-x$ first leaves a representable small result. Then $(x + y) + (-x)$ can differ from $x + (y + (-x))$. A proof that rewrites by real-number associativity would be invalid for that floating-point operation.

An Ao floating-point extension would need floating registers or a declared reuse of integer registers, supported formats, operation semantics, rounding state, exception flags, and conversion rules. The observation must say whether it compares exact encodings, numerical classes, exception state, or some combination. A solver route needs floating-point theories or a sound reduction to bit vectors; a certificate route must cover whichever semantics it uses.

Bit-pattern equality and numerical agreement differ

Two floating-point results may be numerically interchangeable for one client yet have different zero signs, NaN payloads, or exception histories. State the observation before calling an optimization correct.

16.3 Vectors add shape, masks, and partial activity

A packed or vector instruction does not merely apply a scalar operation to a wider word. It introduces lanes, element widths, lane ordering, masks, tail policy, and wider register state. Scalable vector designs may make the active length a state-dependent quantity rather than a constant in the program text. Loads and stores add stride, grouping, fault, and partial-completion questions.

Chapter 15's XOR reduction suggests the appeal. Several words can be loaded and combined in parallel, then the lanes can be reduced to one word. The correctness proof needs a partition of the input region, a mapping from memory indices to lanes, an invariant for partial lane accumulators, and a final horizontal reduction lemma. A tail whose length is not a multiple of the vector width needs masking or a scalar remainder path.

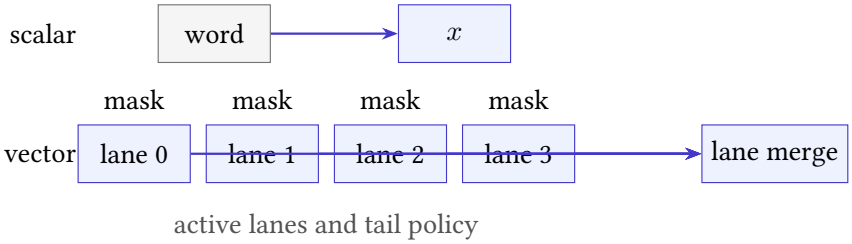


Figure 16.2: A scalar reduction carries one accumulator. A vector reduction adds lane state, an activity mask, a tail policy, and a final lane merge.

The existing scalar proof is still useful. It supplies the specification and the algebraic XOR laws. It does not supply lane semantics or the partition proof. Optional vector extensions belong here rather than at the book's center because the scalar core is the shared foundation for both RISC and CISC architectures; a particular vector extension is one elaboration of that core.

An honest Axeyum vector package should begin with a small lane model and controls for inactive lanes, tail elements, lane order, and masked faults. Old archived shuffle or bit-manipulation artifacts may inform tests, but they do not define the new package's semantics or establish its claims.

16.4 Concurrency replaces one trace with an event structure

The book's executions are single-threaded sequences. At each step one instruction reads one state and produces one successor. With multiple agents, memory actions can overlap in time, observations can depend on ordering, and a global sequence may be too strong or too weak a description.

Lamport's formulation of sequential consistency asks for results equivalent to one sequential order that preserves each processor's program order (Lamport 1979). Modern weak-memory models permit selected reorderings while constraining them through relations such as program order, reads-from, coherence, and synchronization. The object of study becomes a set or graph of events plus consistency conditions.

Consider two threads that each write one location and read the other. A single-thread proof can establish each thread's local instruction effects. It cannot determine which paired read results the whole system permits. That question needs shared-memory events, an architectural memory model, and an observation containing both threads' outcomes.

Atomics add indivisibility and ordering contracts. Fences constrain event relations. Read-modify-write operations combine a read and conditional write under special rules. A state extension with merely two program counters is therefore

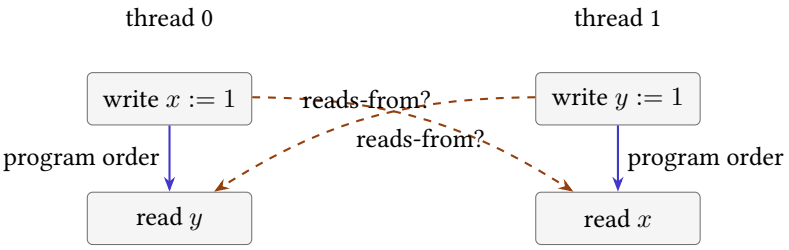


Figure 16.3: Concurrency adds relations across local traces. Program order alone does not determine reads-from or the allowed global observation.

insufficient; it would still lack the memory-event semantics that make the extra execution meaningful.

The proof style changes as well. Invariants may describe shared resources and interference. Refinement may compare sets of allowed executions rather than one successor at a time. A counterexample is an event graph with a forbidden or surprising outcome, and replay means validating every event and relation against the memory model.

16.5 Privilege and virtual memory put addresses in context

Our scalar memory maps addresses directly to bytes and checks whether a range exists. An operating system and privileged architecture add translation, permissions, modes, exceptions, and control registers. A virtual address may name different physical storage in different address spaces. The same address may be readable, executable, or forbidden depending on privilege and page attributes.

Address translation is itself a movement through state. Page-table entries are read, permissions are checked, and translation caches may retain derived information. A load theorem that begins with "effective address a" no longer reaches memory directly. It needs a translation result or fault. An instruction fetch and a data load may use related but differently constrained translation paths.

The smallest clean extension can treat translation as an abstract function of virtual address, access kind, and privileged state. A deeper model can expose page walks and translation caches. The right choice depends on the theorem. A user-level functional proof may assume a stable valid mapping. A page-table update proof cannot hide the walk. A side-channel proof may need translation- cache observations that both functional models omit.

Contextual address meaning

In a translated machine, an address does not identify storage by itself. Its meaning depends at least on the address space, access kind, privilege state, translation structures, and permission policy selected by the model.

Interrupts and exceptions add another control source. The next instruction may not be the architectural successor of the current user instruction; execution may enter a handler with saved state. Returning from that handler has its own privilege and restoration rules. A proof that assumes uninterrupted procedure execution must state that premise or model the interference.

16.6 Self-modifying code joins program and data memory

Ao deliberately keeps an immutable program map separate from mutable data memory. That design makes decoding stable: the instruction at a program counter cannot change because of an ordinary store. Real systems can generate, load, patch, or overwrite code. Once data writes can affect future fetches, the program is part of evolving state.

The naive extension is to use one byte map for both fetch and data access. That is necessary but not always sufficient. Instruction and data views may require architectural synchronization before new bytes are guaranteed to be fetched. Decoders with variable-length instructions may see changed boundaries. Another thread may race with the modification. Permissions may prohibit simultaneous write and execute access to a page.

A store can invalidate the next proof step

Suppose a proof decodes the instruction at address p, then reasons about a store, then applies the saved decoded instruction at p. If the store can alter bytes in that instruction, the proof has reused stale syntax. It must show that the written range is disjoint from the fetched range or perform the architecture's required synchronization and decode again.

This boundary reaches ordinary tooling. Linkers relocate instructions and loaders place segments before execution begins. Just-in-time compilers create new code while a process runs. Dynamic patching changes code already in use. Each activity needs a phase or transition model that binds bytes to the moment at which their semantics is claimed.

An Axeyum extension should therefore make decode results depend on a code-memory version or digest. A negative control can modify one instruction byte

after a cached decode and require the stale step to be rejected. Another can omit a required synchronization event and require the execution route to refuse the stronger claim.

16.7 Timing and leakage require richer observations

Two programs can be functionally equivalent and still take different time, touch different cache sets, contend for different resources, or draw different power. The scalar model projects all of those distinctions away. It cannot establish constant-time behavior because it contains neither a time measure nor the events through which time varies.

The Spectre work demonstrated that speculative execution can affect microarchitectural state and reveal information even when architectural control later discards the speculative path (Kocher et al. 2019). A purely architectural trace can show that the discarded instructions wrote no committed register or memory value. That fact does not describe their cache effects or the time needed by a later probe.

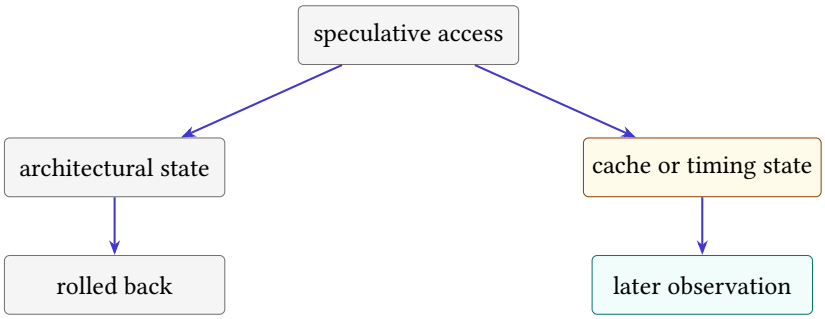


Figure 16.4: Architectural rollback can erase committed effects while a richer observation retains timing or cache consequences.

A constant-time model might record an abstract leakage trace containing branch directions and memory addresses while omitting actual cycle counts. A cache model might record sets, tags, replacement state, and hits. A contention model might add execution ports or shared queues. A power model needs a different abstraction again. There is no single side-channel state that answers every leakage question.

Functional equivalence is not a security theorem

Equal architectural outputs under equal public inputs do not imply equal timing, equal memory-access patterns, or equal speculative effects. A security claim must name secrets, public observations, attacker capabilities, and the leakage model.

The proof relation also changes. A **noninterference** statement compares two runs that differ in secret data and requires their public observations to agree. Negative controls should change a secret-dependent address or branch and require the leakage checker to notice. A result-only checker would accept the mutation and thereby expose its own inadequate observation.

16.8 Verified compilation adds a tower of languages

Our cross-ISA proofs begin with machine programs on both sides. A compiler starts higher: source syntax, types, undefined or unspecified behavior, intermediate representations, optimization passes, assembly, object files, linking, loading, and target execution. Each level has states and observations, and each translation needs a preservation statement.

CompCert showed that a realistic compiler could be developed with a proof of semantic preservation from a substantial C-like source language to assembly (Leroy 2009). Its importance here is methodological. The theorem does not say "the compiler is correct" without qualification. It names source and target semantics and a relation between their behaviors.

Undefined source behavior is a sharp boundary. If a source execution has no promised meaning after an invalid operation, a target behavior need not refine a result the source never specified. Conversely, machine details such as finite memory or resource exhaustion may introduce target outcomes that a source model must accommodate. The exact direction and premises of refinement matter.

Separate compilation adds interfaces between units. Linking adds symbol resolution and relocation. Loading adds addresses, permissions, and initial machine state. A verified compiler theorem cannot by itself establish a claim about a final executable if tools outside the proof or unchecked assumptions break the chain. The evidence manifest must bind the whole route actually used.

Axeyum can contribute solvers, certificates, and kernel reconstruction to local obligations in such a tower. It does not currently provide this book with a verified source language, compiler, linker, loader, and complete real-machine semantics. The right introductory path is to verify one small translation pass or one instruction-selection rule, then compose only after the interfaces and controls are stable.

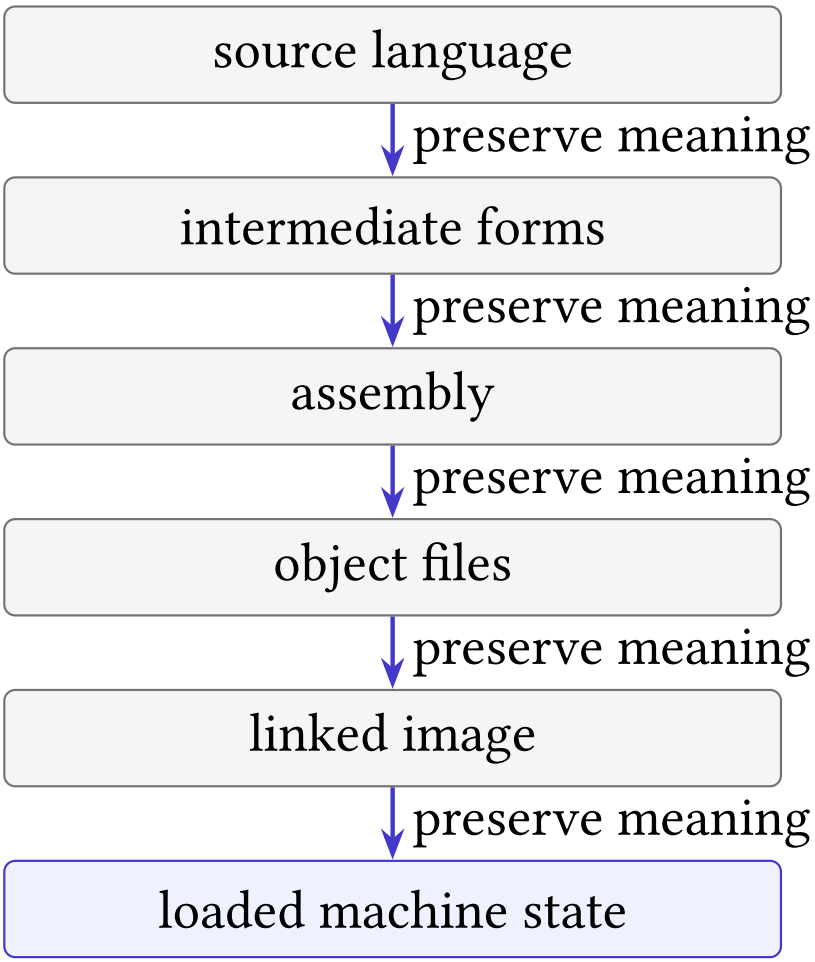


Figure 16.5: A source-to-machine claim crosses several semantic interfaces. Each arrow needs a preservation theorem or an explicitly trusted boundary.

16.9 How to extend the workbench without blurring it

Every extension should pass four gates. The **state gate** inventories new components and preservation rules. The **step gate** defines legal transitions and failure classes. The **observation gate** states which new distinctions the theorem exposes. The **evidence gate** binds executors, formulas, certificates, and controls to those definitions.

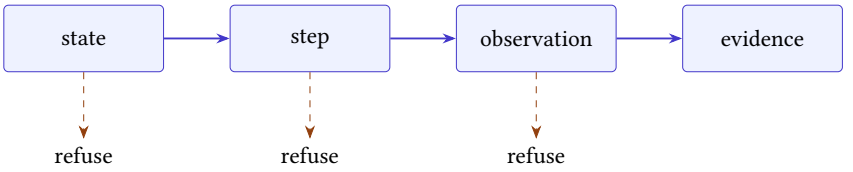


Figure 16.6: An extension reaches evidence only after state, transition, and observation contracts are explicit.

Start with a conservative miniature. A floating-point lesson might support one format, one rounding mode, and addition, while rejecting every other form. A concurrency lesson might support two threads and a tiny event language. A virtual-memory lesson might use one-level translation. Small scope is honest when unsupported cases are visible refusals rather than silent defaults.

The Rust layer should own the typed semantic core and evidence-critical checking. Python should construct examples, call that core, display traces, and help readers mutate controls. This repeats Chapter 15's division. As the model grows, a single implementation boundary becomes more valuable, not less.

Machine evidence cannot replace the conservative-extension argument. Before reusing an old scalar theorem, show that executing an old instruction from an embedded old state produces the embedded old successor and does not touch new state in an observable way. A counterexample to that property identifies an interaction that the extension must expose.

Template for a conservative extension

Let E embed old states into new states, s be an old legal state, and I an old instruction. Show that the new decoder recognizes the same instruction and that one new step from $E(s)$ reaches $E(s')$ exactly when the old step from s reaches s'. State separately how new components are initialized and preserved. Then prove that the new observation of $E(s)$ agrees with the old observation of s.

16.10 Three ways boundary claims go wrong

The first failure is **silent completion**: an unsupported case is given a convenient default. A floating-point decoder treats an unknown format as binary64. A vector checker treats absent mask semantics as "all lanes active." A memory model orders events that it does not know how to relate. The tool returns an answer, but the answer belongs to an invented model.

The repair is refusal. Unsupported instructions, formats, privilege modes, events, and observations must have typed failure outcomes. A report can then distinguish "the claim is false" from "this semantic route does not cover the claim." That distinction is as important at the model edge as it was for solver evidence in Chapter 14.

The second failure is **decorative state**. New fields appear in a state record but no transition reads or writes them. Adding a rounding-mode field does not create floating-point semantics if every operation still uses one hard-coded rounding rule. Adding a cache map does not create a leakage model if speculative and committed accesses never update it. State earns its place by participating in defined movements and observations.

The third failure is **accidental theorem transport**. An earlier proof is reused because the old syntax still parses. Yet a new transition may change an old instruction's effect, a new observation may reveal an old hidden write, or a new context may distinguish programs that were interchangeable before. Conservative embedding must be established; textual inclusion is not enough.

Table 16.1: Typical boundary mistakes and controls that expose them.

Mistake	Hidden assumption	Negative control
Silent completion	unknown means default	supply one unsupported form
Decorative state	new field cannot matter	vary it while holding old state fixed
Accidental transport	old theorem survives automatically	expose a new observation or context
Layer collapse	one checker covers every layer	mutate the unbound translation step

A fourth failure, layer collapse, appears when one successful check is allowed to stand for several missing connections. A verified event-graph checker does not show that machine bytes generated that graph. A floating-point solver does not show that an instruction decoder selected the intended format. A compiler proof does not verify an unmodeled loader. The claim-to-evidence chain grows as the model grows; no link becomes optional because another link is strong.

16.11 A research question for every new component

Adding state should immediately produce proof questions. For each new component q, ask how it is initialized, which transitions can read it, which can write it, and which transitions must preserve it. Ask whether it affects legality or only results. Ask whether the observation exposes it directly or through later behavior. Finally ask how a malformed or adversarial value of q reaches a refusal.

For floating point, q might be the rounding mode. Initialization selects a mode; arithmetic reads it; a control-register write may change it; integer addition preserves it. The result observation may reveal it only indirectly through rounded values, while a full-state observation sees it directly. A control changes the mode on a halfway case and requires a changed result.

For concurrency, the new object may be a reads-from relation rather than a register field. Its endpoints must be compatible read and write events, values must agree, and the whole graph must meet the architectural consistency rules. A control redirects one read to an incompatible write and requires rejection. This shows why "state component" should be read broadly: event relations and histories can be semantic state even when no hardware register stores them.

For leakage, the component might be an address trace. Every selected load and store appends an address; non-memory instructions preserve the trace; the security observation projects it. A control changes a secret-dependent index while preserving the final architectural result. The functional checker still accepts, but the leakage comparison must reject.

Write the five questions before the semantics

Choose a boundary component and answer: how is it initialized, who reads it, who writes it, who preserves it, and which observation can distinguish it? Then design one unsupported input and one semantic mutation for the checker.

This worksheet prevents feature lists from outrunning definitions. It also creates the first test plan. Each answer suggests a positive path, a preservation check, and a negative control. When no observation can distinguish the new state, ask whether the theorem truly needs it or whether its effect is only latent in a larger context.

16.12 An Axeyum learning sequence beyond the scalar core

The next Axeyum lessons should grow one semantic dimension at a time. A useful sequence is: one binary floating-point addition with a fixed format; the same operation under two rounding modes; a four-lane integer operation with a

mask; two single-event threads under sequential consistency; then one permitted weak-memory outcome. Each lesson adds one state idea and one checker attack.

The Rust package for a lesson should expose constructors that make unsupported cases impossible or explicit. The Python layer should show the state before and after a step, run a positive miniature, fire a named mutation, and print the trust boundary. Only after the miniature is stable should symbolic inputs and solver routes be added.

Kernel reconstruction belongs last in this sequence, not because it is unimportant, but because it can only reconstruct the proposition it receives. First bind bytes or events to semantics. Then validate the semantic relation. Then translate a bounded obligation and check its certificate. A kernel proof over an unbound formula would certify the wrong layer with great confidence.

16.13 Choosing the next boundary

The most attractive extension is not always the best next one. Choose a claim whose missing state is small enough to define completely, whose official sources can be pinned, whose examples fit in a reader trace, and whose controls can fail for recognizable reasons. A fashionable topic with an ambiguous semantic boundary will produce impressive artifacts and weak conclusions.

For this book's workbench, a narrow floating-point operation or a two-thread memory litmus test would be plausible next lessons after the scalar packages exist. A full speculative out-of-order processor or verified optimizing compiler would be a different project. The distinction is one of semantic surface, not ambition.

There is also a maintenance test. A good extension makes its new state visible in definitions, traces, observations, serialization, and controls. If adding one rounding mode or mask bit requires unexplained special cases throughout the old scalar executor, the boundary has not been isolated cleanly. The design should let old scalar claims retain their former meaning while new claims opt into the richer state.

The scalar core remains valuable at every edge. Words still encode fields. States still need inventories. Instructions still produce movements. Proofs still depend on observations. Counterexamples still need replay. Evidence still needs a claim, a checker, and a control. The universal method is not one instruction set; it is the discipline of naming meaning before trusting agreement.

The boundary is therefore an invitation, not an apology. Each omitted concept opens a new version of the book's central mystery: how a finite inscription can govern a movement, how different inscriptions can share meaning, and how a checker can know. The answers become harder as state grows, but they remain approachable when the new distinctions are named one at a time. Precision does not drain the wonder from computation. It lets us see exactly where the wonder has moved.

16.14 Exercises

1. **Execute.** Choose one floating-point format and classify positive zero, negative zero, infinity, and one NaN by their fields.

2. Give a concrete finite floating-point example in which reassociating three additions changes the result. Name the rounding mode.

3. Define an observation that treats all NaNs alike and another that compares their encodings. Give a pair the two observations classify differently.

4. **Break.** Add vector lanes but omit the activity mask from state. Construct two executions that the incomplete model confuses.

5. State a loop invariant for a four-lane XOR reduction with a masked tail. Name the partition represented by each lane.

6. Explain why a scalar proof of XOR associativity is useful but insufficient for a vector memory-access proof.

7. Draw the events for the two-thread store-and-load example. Add program-order and two possible reads-from relations.

8. State Lamport's sequential-consistency condition in your own words. Which part cannot be expressed by two unrelated local traces?

9. Design a negative control that removes one fence from a concurrent program and changes an allowed observation.

10. Add an abstract address-translation function to Ao. List its inputs and its success and fault outcomes.

11. Give premises under which flat Ao memory embeds into that translated model as an identity mapping.

12. **Break.** Allow a store to change program bytes while retaining a cached decode. Give the shortest stale-instruction trace you can.

13. Define a code-memory version check that would reject the stale trace.

14. Give two programs with equal return values and different address traces. Explain why result equivalence does not establish constant-time behavior.

15. Define public and secret inputs for a two-run noninterference claim. State the observation that must agree.

16. Add a speculative cache event to one path and roll back architectural state. Which observation detects the remaining difference?

17. List the semantic interfaces between a C-like expression and a loaded machine instruction. Mark one trusted boundary at each unverified tool.

18. Explain why compiler correctness cannot supply meaning to a source program after undefined behavior.

19. Draft a conservative-extension theorem for adding one floating-point register to Ao while running an old integer instruction.

20. Design positive and negative controls for the four extension gates.

21. **Transfer.** Choose one boundary from this chapter and write a one-page Axeyum package plan: types, transitions, observation, source pins, checker route, and refusal cases.

22. Compare two possible next projects—a floating-point addition miniature and a complete weak-memory model—by semantic surface and evidence burden.

Acknowledgments

Every book is a relay. Thanks are owed to the maintainers of the open tools this template leans on—LaTeX and its remarkable package ecosystem, the font designers whose work gives these pages a voice, and the print-on-demand platforms that turned publishing into something one person can finish at a kitchen table.

Thanks also to the readers of the earlier books whose margins, literally and figuratively, taught us which of these conventions survive contact with real paper.

* * *

Any errors that remain are, of course, the author's alone—the template can enforce widow penalties, but not good judgment.

About the Author

Michael J. Bommarito II writes about the craft of making books with software—and makes the occasional book to prove the tooling works. This volume was produced end-to-end with the template it describes: one YAML file of metadata, three font profiles, and a build system that treats a manuscript the way good engineers treat code.

Publisher inquiries may be directed to Bommarito Press.

Sources

Bansal, Sorav and Alex Aiken (2006). "Automatic Generation of Peephole Superoptimizers." In: *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 394–403. DOI: 10.1145/1168917.1168906.

Cruz-Filipe, Luís, Marijn Heule, Jr. Hunt Warren, Matt Kaufmann, and Peter Schneider-Kamp (2017). "Efficient Certified RAT Verification." In: *Automated Deduction – CADE 26*. Vol. 10395. Lecture Notes in Computer Science. Springer, pp. 220–236. DOI: 10.1007/978-3-319-63046-5_14. URL: https://www.cs.utexas.edu/~marijn/publications/lrat.pdf (visited on 08/29/2026).

Hamming, Richard W. (1950). "Error Detecting and Error Correcting Codes." In: *The Bell System Technical Journal* 29.2, pp. 147–160. DOI: 10.1002/j.1538-7305.1950.tb00463.x. URL: https://www.nokia.com/bell-labs/publications-and-media/publications/error-detecting-and-error-correcting-codes/.

IEEE Computer Society (2019). *IEEE Standard for Floating-Point Arithmetic*. IEEE. DOI: 10.1109/IEEESTD.2019.8766229. URL: https://standards.ieee.org/ieee/754/6210/ (visited on 08/29/2026).

Intel Corporation (Aug. 2026a). *Intel 64 and IA-32 Architectures Software Developer's Manual. Combined Volumes 2A, 2B, 2C, and 2D: Instruction Set Reference, A–Z*. 325383-092. Version 092. Intel Corporation.

— (Aug. 2026b). *Intel 64 and IA-32 Architectures Software Developer's Manual. Volume 1: Basic Architecture*. 253665-092. Version 092. Intel Corporation.

Kocher, Paul, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom (2019). "Spectre Attacks: Exploiting Speculative Execution." In: *2019 IEEE Symposium on Security and Privacy*, pp. 1–19. DOI: 10.1109/SP.2019.00002. URL: https://www.cs.cmu.edu/~18742/papers/spectre.pdf.

Lamport, Leslie (1979). "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs." In: *IEEE Transactions on Computers* C-28.9, pp. 690–691. DOI: 10.1109/TC.1979.1675439. URL: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/How-to-Make-a-Multiprocessor-Computer-That-Correctly-Executes-Multiprocess-Programs.pdf.

Leibniz, Gottfried Wilhelm (1703). *Explanation of Binary Arithmetic*. English translation of "Explication de l'arithmétique binaire". URL: https://www.leibniz-translations.com/binary (visited on 08/27/2026).

Leroy, Xavier (2009). "Formal Verification of a Realistic Compiler." In: *Communications of the ACM* 52.7, pp. 107–115. DOI: 10.1145/1538788.1538814. URL: https://inria.hal.science/inria-00415861v1/file/compcert-CACM.pdf.

Lu, Sirui and Rastislav Bodík (Oct. 2025). "HieraSynth: A Parallel Framework for Complete Super-Optimization with Hierarchical Space Decomposition." In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2, 384. DOI: 10.1145/3763162.

Massalin, Henry (1987). "Superoptimizer: A Look at the Smallest Program." In: *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 122–126. DOI: 10.1145/36206.36194.

Neumann, John von (June 30, 1945). *First Draft of a Report on the EDVAC*. Philadelphia: Moore School of Electrical Engineering, University of Pennsylvania.

RISC-V International (2026a). *RISC-V ABIs Specification*. Version 1.0. RISC-V International. URL: https://docs.riscv.org/reference/abi/ (visited on 08/29/2026).

— (Jan. 20, 2026b). *The RISC-V Instruction Set Manual, Volume I. Unprivileged Architecture*. Version 20260120. RV64I base integer instruction set, version 2.1. RISC-V International.

Schurr, Hans-Jörg, Mathias Fleury, Haniel Barbosa, and Pascal Fontaine (2021). "Alethe: Towards a Generic SMT Proof Format." In: *Electronic Proceedings in Theoretical Computer Science* 336, pp. 49–54. DOI: 10.4204/EPTCS.336.6. URL: https://www.verit-solver.org/papers/pxtp2021.pdf (visited on 08/29/2026).

Wetzler, Nathan, Marijn J. H. Heule, and Jr. Hunt Warren A. (2014). "DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs." In: *Theory and Applications of Satisfiability Testing – SAT 2014*. Vol. 8561. Lecture Notes in Computer Science. Springer, pp. 422–429. DOI: 10.1007/978-3-319-09284-3_31. URL: https://www.cs.cmu.edu/~mheule/publications/drat-trim.pdf (visited on 08/29/2026).

x86 psABIs Project (Sept. 26, 2023). *System V Application Binary Interface. AMD64 Architecture Processor Supplement*. Version 1.0. x86 psABIs Project. URL: https://gitlab.com/x86-psABIs/x86-64-ABI/-/wikis/uploads/221b09355dd540efcbe61b783b6c0ece/x86-64-psABI-2023-09-26.pdf (visited on 08/29/2026).

Colophon

This book was designed and typeset with LuaLaTeX from a single-source template: metadata, trim, typography, and edition structure all derive from one configuration file, and every output format—print interior, bleed variant, and ebook—builds from the same LaTeX sources.

The text is set in Libertinus Serif, with Libertinus Sans for titling accents and Libertinus Mono for code, at a base size of 11pt on a 7.0in by 10.0in trim.

Interior architecture: a modular preamble with a four-layer color system, semantic callout boxes, and micro-typographic protrusion tuned per typeface. Citations are managed with biblatex and biber.

Bommarito Press · 2026